

Curso Profesional de Entity Framework Core 8

Módulo 2 - Modelado de datos con Entity Framework Core

PRÁCTICAS

AUTOR: JAIME GALLO

.NET 8 · Entity Framework Core 8 · Visual Studio Community · SQL Server LocalDB · Proyecto AceriaData

Índice

Punto 2.1 - Convenciones de modelado en Entity Framework Core	3
Punto 2.2 - Entidades y propiedades	12
Punto 2.3 - Relaciones uno a muchos	26
Punto 2.4 - Relaciones uno a uno	39
Punto 2.5 - Relaciones muchos a muchos	55
Punto 2.6 - Configuración mediante Data Annotations	74
Punto 2.7 - Configuración mediante Fluent API	94
Punto 2.8 - Claves primarias, alternativas y compuestas	115
Punto 2.9 - Índices y restricciones	136
Punto 2.10 - Filtros globales de consulta	158
Punto 2.11 - Migraciones en el modelado: ciclo completo con Soft Delete	181
Punto 2.12 - Clean Architecture y Arquitectura Hexagonal	210

Módulo 2 - Prácticas de modelado de datos con Entity Framework Core 8

Estas prácticas continúan exactamente desde `M01/PROYECTO/1.12`. Cada punto dispone de una solución autónoma y completa en `M02/PROYECTO/2.x/AceriaData.sln`. El estado de un punto se construye sobre el punto anterior y las migraciones evolucionan el mismo esquema de `AceriaDB`.

Punto 2.1 - Convenciones de modelado en Entity Framework Core

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL que trabajan con .NET 8, EF Core 8 y SQL Server LocalDB.

Proyecto: El estado 2.1 parte de `M01/PROYECTO/1.12` y tiene como objetivo inspeccionar el modelo heredado sin modificar todavía su estructura.

Objetivos de aprendizaje

- Reconocer qué descubre EF Core por convención.
- Inspeccionar entidades, tablas, claves primarias y claves foráneas.
- Comprobar las navegaciones ya existentes.
- Diferenciar convención de configuración explícita.
- Trabajar con el ServiceProvider local sin campos estáticos.
- Conservar intacto el esquema heredado de M1.

Paso 1: Abrir la solución autónoma del punto

```
cd M02/PROYECTO/2.1
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

No se abre una solución global situada en la raíz del repositorio. `AceriaData.sln` está dentro de la carpeta del punto y referencia únicamente los proyectos de ese estado.

Paso 2: Identificar el cambio respecto al estado anterior

El proyecto conserva todo lo terminado en `M01/PROYECTO/1.12`. En 2.1 se introduce exclusivamente el contenido que corresponde a **Convenciones de modelado en Entity Framework Core**. La práctica no reinicia `AceriaData` ni crea un ejemplo paralelo.

Paso 3: Implementar y comprender la configuración principal

```
using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();

foreach (var entity in context.Model.GetEntityTypes().OrderBy(e => e.ClrType.Name))
{
    var pk = entity.FindPrimaryKey();
    global::System.Console.WriteLine(
        $"Entidad: {entity.ClrType.Name} | Tabla: {entity.GetTableName()} | " +
        $"PK: {string.Join(", ", pk?.Properties.Select(x => x.Name) ?? Array.Empty<string>())}");
}
```

CSHARP

Línea 1	<code>using var provider = services.BuildServiceProvider(...);</code> → construye el contenedor ya configurado en el estado heredado de M1.
Línea 2	<code>using var scope = provider.CreateScope();</code> → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.
Línea 3	<code>GetRequiredService<AceriaDbContext>()</code> → obtiene el DbContext sin guardar el proveedor en un campo estático.
Línea 4	<code>context.Model.GetEntityTypes()</code> → recorre el modelo que EF Core ha construido realmente.
Línea 5	<code>entity.FindPrimaryKey()</code> → permite comprobar qué propiedad o propiedades forman la clave primaria.

Paso 4: Confirmar que 2.1 no crea una migración

Este punto no cambia el modelo. Se conservan las migraciones heredadas de M1 y **no** se ejecuta `EnsureCreated()`. La práctica se limita a inspeccionar metadatos.

```
dotnet ef migrations list
```

POWERSHELL

El resultado debe mostrar el historial heredado, sin una migración nueva de 2.1.

Paso 5: Compilar y ejecutar el estado

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

POWERSHELL

Resultado esperado: la ejecución termina sin excepción y contiene la evidencia `--- MODELO EF CORE 2.1 ---`. GitHub Actions repite esta compilación y ejecución sobre Windows y SQL Server LocalDB.

Paso 6: Verificar el estado acumulativo

Además de ejecutar el ejemplo, conviene revisar el modelo y la base:

```
dotnet ef migrations list
```

POWERSHELL

Cuando el punto modifica el esquema, la lista debe contener la migración acumulativa correspondiente. En SQL Server Object Explorer se puede comprobar que las tablas, claves, relaciones, índices o restricciones coinciden con el modelo del punto.

Errores comunes

Error	Causa	Solución
Se crea un ServiceProvider estático	Se copia una versión antigua del ejercicio	Usar el provider local de Main y CreateScope.
Se recrea la base	Se usa EnsureCreated/EnsureDeleted innecesariamente	2.1 sólo inspecciona metadatos; no cambia el esquema.
Se introduce una relación nueva	Se adelanta contenido	Reservar 1:1 para 2.4 y N:M para 2.5.

Reto resuelto

Reto: Enumerar las claves foráneas de cada entidad y mostrar también DeleteBehavior sin modificar el modelo.

Solución: partir del código de `M02/PROYECTO/2.1`, realizar únicamente el cambio descrito y volver a ejecutar build, migraciones y el programa. El reto no introduce conceptos reservados a un punto posterior.

Analogía final

Inspeccionar el modelo es leer el plano de una instalación antes de modificarla: primero se identifica qué está ya construido.

Resultado esperado

Al terminar 2.1, la solución local compila, el proyecto se ejecuta, el esquema se actualiza mediante Migrations cuando corresponde y la salida contiene `--- MODELO EF CORE 2.1 ---`. El código conserva todos los cambios válidos de los puntos anteriores.

Conexión con el siguiente punto

El siguiente estado es 2.2. Se parte del proyecto completo de 2.1; no se vuelve a crear AceriaData desde cero.

Código acumulativo completo del estado 2.1

El siguiente archivo es el `Program.cs` real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

CSHARP

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
}

public interface IOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
}
```

```

OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
void Agregar(OrdenFabricacion orden);
void Eliminar(OrdenFabricacion orden);
int Guardar();
}

public sealed class OrdenRepositorio : IOOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                    {
                        sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                        sql.CommandTimeout(60);
                    })
                .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        global::System.Console.WriteLine("--- MODELO EF CORE 2.1 ---");
        foreach (var entity in context.Model.GetEntityTypes().OrderBy(e => e.ClrType.Name))
        {
            var pk = entity.FindPrimaryKey();
            global::System.Console.WriteLine($"Entidad: {entity.ClrType.Name} | Tabla: {entity.GetTableName()} | PK:

```

```

{string.Join(",", pk?.Properties.Select(x => x.Name) ?? Array.Empty<string>())});
    foreach (var fk in entity.GetForeignKeys())
    {
        global::System.Console.WriteLine($" FK: {string.Join(",", fk.Properties.Select(x => x.Name))} ->
{fk.PrincipalEntityType.ClrType.Name}");
    }
}
}
}
}

```

Explicación línea a línea del código acumulativo 2.1

Las referencias siguientes usan la numeración real de `M02/PROYECTO/2.1/Program.cs`. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1	<code>using System.ComponentModel.DataAnnotations;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 2	<code>using System.ComponentModel.DataAnnotations.Schema;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 3	<code>using Microsoft.EntityFrameworkCore;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 4	<code>using Microsoft.Extensions.Configuration;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 5	<code>using Microsoft.Extensions.DependencyInjection;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 6	<code>using Microsoft.Extensions.Logging;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 8	<code>namespace AceriaData.ConsoleApp;</code> → declara el espacio de nombres del estado acumulativo.
Línea 10	<code>public class OrdenFabricacion</code> → declara la clase <code>OrdenFabricacion</code> que forma parte del estado acumulativo del ejercicio.
Línea 11	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 12	<code>public int Id { get; set; }</code> → declara la propiedad <code>Id</code> de tipo <code>int</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 13	<code>public string NumeroOrden { get; set; } = string.Empty;</code> → declara la propiedad <code>NumeroOrden</code> de tipo <code>string</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 14	<code>public string Cliente { get; set; } = string.Empty;</code> → declara la propiedad <code>Cliente</code> de tipo <code>string</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 15	<code>public DateTime FechaCreacion { get; set; }</code> → declara la propiedad <code>FechaCreacion</code> de tipo <code>DateTime</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 16	<code>public List<PlanchaAcero> Planchas { get; set; } = new();</code> → declara la propiedad <code>Planchas</code> de tipo <code>List<PlanchaAcero></code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 17	<code>}</code> → cierra el bloque de código actual.
Línea 19	<code>public class PlanchaAcero</code> → declara la clase <code>PlanchaAcero</code> que forma parte del estado acumulativo del ejercicio.
Línea 20	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 21	<code>public int Id { get; set; }</code> → declara la propiedad <code>Id</code> de tipo <code>int</code> que EF Core o la lógica de negocio utiliza en este modelo.

Línea 22	<code>public int OrdenId { get; set; } → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 23	<code>public double Espesor { get; set; } → declara la propiedad Espesor de tipo double que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 24	<code>public double Ancho { get; set; } → declara la propiedad Ancho de tipo double que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 25	<code>public double Largo { get; set; } → declara la propiedad Largo de tipo double que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 26	<code>public OrdenFabricacion Orden { get; set; } = null!; → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 27	<code>} → cierra el bloque de código actual.</code>
Línea 29	<code>public class Aleacion → declara la clase Aleacion que forma parte del estado acumulativo del ejercicio.</code>
Línea 30	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 31	<code>public int Id { get; set; } → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 32	<code>public string Nombre { get; set; } = string.Empty; → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 33	<code>public double PorcentajeCarbono { get; set; } → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 34	<code>public double PorcentajeManganeso { get; set; } → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 35	<code>} → cierra el bloque de código actual.</code>
Línea 37	<code>public class EstadoOrden → declara la clase EstadoOrden que forma parte del estado acumulativo del ejercicio.</code>
Línea 38	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 39	<code>public int Id { get; set; } → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 40	<code>public string Nombre { get; set; } = string.Empty; → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 41	<code>public string Descripcion { get; set; } = string.Empty; → declara la propiedad Descripcion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 42	<code>} → cierra el bloque de código actual.</code>
Línea 44	<code>public class AceriaDbContext : DbContext → declara la clase AceriaDbContext que forma parte del estado acumulativo del ejercicio.</code>
Línea 45	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 46	<code>public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { } → declara el constructor o método AceriaDbContext con los parámetros necesarios para este componente.</code>
Línea 48	<code>public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>(); → expone un conjunto de entidades para consultar y persistir mediante EF Core.</code>

Línea 49	<code>public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 50	<code>public DbSet<Aleacion> Aleaciones => Set<Aleacion>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 51	<code>public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 53	<code>}</code> → cierra el bloque de código actual.
Línea 55	<code>public interface IOrdenRepositorio</code> → declara el contrato IOrdenRepositorio para desacoplar consumidores e implementación.
Línea 56	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 57	<code>List<OrdenFabricacion> ObtenerTodas();</code> → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.
Línea 58	<code>OrdenFabricacion? ObtenerPorId(int id);</code> → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.
Línea 59	<code>OrdenFabricacion? ObtenerPorNumero(string numeroOrden);</code> → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.
Línea 60	<code>void Agregar(OrdenFabricacion orden);</code> → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.
Línea 61	<code>void Eliminar(OrdenFabricacion orden);</code> → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.
Línea 62	<code>int Guardar();</code> → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.
Línea 63	<code>}</code> → cierra el bloque de código actual.
Línea 65	<code>public sealed class OrdenRepositorio : IOrdenRepositorio</code> → declara la clase OrdenRepositorio que forma parte del estado acumulativo del ejercicio.
Línea 66	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 67	<code>private readonly AceriaDbContext _context;</code> → declara el campo privado _context de tipo AceriaDbContext que conserva una dependencia de la clase.
Línea 68	<code>public OrdenRepositorio(AceriaDbContext context) => _context = context;</code> → declara el constructor o método OrdenRepositorio con los parámetros necesarios para este componente.
Línea 70	<code>public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();</code> → ordena el resultado de la consulta antes de materializarlo.
Línea 71	<code>public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);</code> → implementa la búsqueda por clave primaria mediante Find, aprovechando primero el ChangeTracker y después la base de datos.
Línea 72	<code>public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);</code> → recupera la primera coincidencia o null cuando no existe.
Línea 73	<code>public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);</code> → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.

Línea 74	<code>public void Eliminar(OrdenFabricacion orden) =></code> <code>_context.OrdenesFabricacion.Remove(orden);</code> → marca la entidad indicada para borrado al confirmar los cambios.
Línea 75	<code>public int Guardar() => _context.SaveChanges();</code> → persiste en SQL Server los cambios seguidos por el DbContext.
Línea 76	<code>}</code> → cierra el bloque de código actual.
Línea 78	<code>public interface IServicioOrdenes</code> → declara el contrato IServicioOrdenes para desacoplar consumidores e implementación.
Línea 79	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 80	<code>string ObtenerResumen(int ordenId);</code> → declara el servicio que construye un resumen de una orden a partir de su identificador.
Línea 81	<code>}</code> → cierra el bloque de código actual.
Línea 83	<code>public sealed class ServicioOrdenes : IServicioOrdenes</code> → declara la clase ServicioOrdenes que forma parte del estado acumulativo del ejercicio.
Línea 84	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 85	<code>private readonly IOrdenRepositorio _repositorio;</code> → declara el campo privado _repositorio de tipo IOrdenRepositorio que conserva una dependencia de la clase.
Línea 86	<code>private readonly AceriaDbContext _context;</code> → declara el campo privado _context de tipo AceriaDbContext que conserva una dependencia de la clase.
Línea 87	<code>public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)</code> → declara el constructor o método ServicioOrdenes con los parámetros necesarios para este componente.
Línea 88	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 89	<code>_repositorio = repositorio;</code> → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.
Línea 90	<code>_context = context;</code> → guarda el DbContext inyectado para consultar los datos necesarios durante el servicio.
Línea 91	<code>}</code> → cierra el bloque de código actual.
Línea 93	<code>public string ObtenerResumen(int ordenId)</code> → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.
Línea 94	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 95	<code>var orden = _repositorio.ObtenerPorId(ordenId);</code> → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.
Línea 96	<code>if (orden is null) return \$"Orden {ordenId} no encontrada";</code> → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.
Línea 97	<code>var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);</code> → cuenta las filas que cumplen la consulta construida.
Línea 98	<code>return \$"Orden {orden.NumeroOrden} Cliente: {orden.Cliente} Planchas: {totalPlanchas}";</code> → devuelve el resultado calculado al código que invocó el método.
Línea 99	<code>}</code> → cierra el bloque de código actual.
Línea 100	<code>}</code> → cierra el bloque de código actual.

Línea 102	<code>public static class Program</code> → declara la clase Program que forma parte del estado acumulativo del ejercicio.
Línea 103	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 104	<code>public static void Main()</code> → declara el punto de entrada ejecutable de la aplicación de consola.
Línea 105	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 106	<code>var configuration = new ConfigurationBuilder()</code> → inicia la construcción de la configuración de la aplicación.
Línea 107	<code>.SetBasePath(AppContext.BaseDirectory)</code> → establece la ruta base usada para localizar los archivos de configuración.
Línea 108	<code>.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)</code> → incorpora appsettings.json a la configuración.
Línea 109	<code>.AddEnvironmentVariables()</code> → incorpora variables de entorno como fuente adicional de configuración.
Línea 110	<code>.Build();</code> → materializa la configuración o el objeto encadenado construido en las líneas anteriores.
Línea 112	<code>var connectionString = configuration.GetConnectionString("AceriaDB")</code> → obtiene la cadena de conexión AceriaDB desde la configuración.
Línea 113	<code>?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");</code> → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.
Línea 115	<code>var services = new ServiceCollection();</code> → crea la colección donde se registran las dependencias de la aplicación.
Línea 116	<code>services.AddDbContext<AceriaDbContext>(options =></code> → registra el DbContext en el contenedor de inyección de dependencias.
Línea 117	<code>options</code> → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.
Línea 118	<code>.UseSqlServer(connectionString, sql =></code> → selecciona SQL Server como proveedor de EF Core.
Línea 119	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 120	<code>sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);</code> → habilita reintentos ante fallos transitorios del proveedor SQL Server.
Línea 121	<code>sql.CommandTimeout(60);</code> → establece el tiempo máximo permitido para los comandos SQL.
Línea 122	<code>})</code> → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.
Línea 123	<code>.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)</code> → configura la salida del logging de EF Core.
Línea 124	<code>.EnableDetailedErrors();</code> → activa errores detallados para facilitar el diagnóstico del laboratorio.
Línea 126	<code>services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();</code> → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.
Línea 127	<code>services.AddScoped<IServicioOrdenes, ServicioOrdenes>();</code> → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 129	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions → construye el ServiceProvider con validación de servicios y garantiza su liberación al salir del ámbito mediante using var.</code>
Línea 130	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 131	<code>ValidateScopes = true, → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.</code>
Línea 132	<code>ValidateOnBuild = true → obliga al contenedor a validar las resoluciones registradas al construir el ServiceProvider.</code>
Línea 133	<code>}); → cierra la lambda o configuración encadenada y finaliza la llamada.</code>
Línea 135	<code>using var scope = provider.CreateScope(); → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.</code>
Línea 136	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>(); → resuelve un servicio obligatorio desde el contenedor de dependencias.</code>
Línea 137	<code>global::System.Console.WriteLine("--- MODELO EF CORE 2.1 ---"); → escribe en consola la evidencia usada para verificar el comportamiento del estado.</code>
Línea 138	<code>foreach (var entity in context.Model.GetEntityTypes().OrderBy(e => e.ClrType.Name)) → ordena el resultado de la consulta antes de materializarlo.</code>
Línea 139	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 140	<code>var pk = entity.FindPrimaryKey(); → consulta los metadatos de EF Core para obtener la clave primaria de la entidad.</code>
Línea 141	<code>global::System.Console.WriteLine(\$"Entidad: {entity.ClrType.Name} Tabla: {entity.GetTableName()} PK: {string.Join(", ", pk?.Properties.Select(x => x.Name) ?? Array.Empty<string>())}"); → escribe en consola la evidencia usada para verificar el comportamiento del estado.</code>
Línea 142	<code>foreach (var fk in entity.GetForeignKeys()) → enumera las claves foráneas configuradas para la entidad.</code>
Línea 143	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 144	<code>global::System.Console.WriteLine(\$" FK: {string.Join(", ", fk.Properties.Select(x => x.Name))} -> {fk.PrincipalEntityType.ClrType.Name}"); → escribe en consola la evidencia usada para verificar el comportamiento del estado.</code>
Línea 145	<code>} → cierra el bloque de código actual.</code>
Línea 146	<code>} → cierra el bloque de código actual.</code>
Línea 147	<code>} → cierra el bloque de código actual.</code>
Línea 148	<code>} → cierra el bloque de código actual.</code>

Punto 2.2 - Entidades y propiedades

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL que trabajan con .NET 8, EF Core 8 y SQL Server LocalDB.

Proyecto: El estado 2.2 parte de 2.1 y tiene como objetivo enriquecer las entidades y configurar tipos, longitudes, requeridos, precisión y valores por defecto.

Objetivos de aprendizaje

- Añadir propiedades de negocio al modelo existente.
- Distinguir propiedades requeridas y opcionales.
- Configurar longitudes máximas.
- Configurar precisión decimal.
- Configurar valores por defecto.
- Generar una migración acumulativa sin introducir todavía índices del punto 2.9.

Paso 1: Abrir la solución autónoma del punto

```
cd M02/PROYECTO/2.2
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

No se abre una solución global situada en la raíz del repositorio. `AceriaData.sln` está dentro de la carpeta del punto y referencia únicamente los proyectos de ese estado.

Paso 2: Identificar el cambio respecto al estado anterior

El proyecto conserva todo lo terminado en 2.1. En 2.2 se introduce exclusivamente el contenido que corresponde a **Entidades y propiedades**. La práctica no reinicia `AceriaData` ni crea un ejemplo paralelo.

Paso 3: Implementar y comprender la configuración principal

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.Property(x => x.Peso).HasPrecision(18, 3);
    entity.Property(x => x.Activa).HasDefaultValue(true);
});
```

CSHARP

Línea 1	<code>DateTime? FechaEntrega</code> → declara una fecha opcional; la columna puede admitir NULL.
Línea 2	<code>string? Observaciones</code> → declara texto opcional.
Línea 3	<code>HasPrecision(18, 3)</code> → fija precisión y escala del peso.
Línea 4	<code>HasDefaultValue(true)</code> → establece el valor por defecto de Activa en la base de datos.
Línea 5	<code>HasMaxLength(...)</code> → evita que las cadenas queden como <code>nvarchar(max)</code> cuando el dominio conoce su tamaño.

Paso 4: Generar y aplicar la migración acumulativa

El repositorio contiene una migración real generada para este estado. Si se reproduce el ejercicio desde el estado anterior, los comandos son:

```
dotnet ef migrations add M2_2_2
dotnet ef database update
```

POWERSHELL

`dotnet ef migrations add` compara el modelo actual con el snapshot heredado. `dotnet ef database update` aplica únicamente los cambios pendientes.

En los estados ejecutables del curso se usa `Database.Migrate()` para aplicar el historial durante la demostración. El laboratorio puede usar `EnsureDeleted()` para empezar desde una base limpia, pero **no usa `EnsureCreated()`**, porque el esquema está gobernado por Migrations.

Paso 5: Compilar y ejecutar el estado

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

POWERSHELL

Resultado esperado: la ejecución termina sin excepción y contiene la evidencia `2.2 OK`. GitHub Actions repite esta compilación y ejecución sobre Windows y SQL Server LocalDB.

Paso 6: Verificar el estado acumulativo

Además de ejecutar el ejemplo, conviene revisar el modelo y la base:

```
dotnet ef migrations list
```

POWERSHELL

Cuando el punto modifica el esquema, la lista debe contener la migración acumulativa correspondiente. En SQL Server Object Explorer se puede comprobar que las tablas, claves, relaciones, índices o restricciones coinciden con el modelo del punto.

Errores comunes

Error	Causa	Solución
La migración contiene índices de negocio	Se adelantó contenido de 2.9	En 2.2 configurar sólo propiedades; la unicidad se formaliza después.
FechaEntrega no admite NULL	Se declaró DateTime en vez de DateTimeOffset?	Usar DateTimeOffset?.
Peso pierde escala	Se dejó el mapping por defecto	Configurar HasPrecision(18,3).

Reto resuelto

Reto: Añadir y comprobar una propiedad opcional Observaciones y una propiedad decimal Peso con escala 3, generando la migración acumulativa.

Solución: partir del código de `M02/PROYECTO/2.2`, realizar únicamente el cambio descrito y volver a ejecutar build, migraciones y el programa. El reto no introduce conceptos reservados a un punto posterior.

Analogía final

Configurar propiedades equivale a fijar tolerancias dimensionales y formatos antes de fabricar una pieza.

Resultado esperado

Al terminar 2.2, la solución local compila, el proyecto se ejecuta, el esquema se actualiza mediante Migrations cuando corresponde y la salida contiene `2.2 OK`. El código conserva todos los cambios válidos de los puntos anteriores.

Conexión con el siguiente punto

El siguiente estado es `2.3`. Se parte del proyecto completo de 2.2; no se vuelve a crear AceriaData desde cero.

Código acumulativo completo del estado 2.2

El siguiente archivo es el `Program.cs` real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public stringCodigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
            entity.Property(o => o.Observaciones).HasMaxLength(500);
        });
        modelBuilder.Entity<PlanchaAcero>(entity =>
        {
            entity.ToTable("PlanchasAcero");
            entity.HasKey(x => x.Id);
            entity.Property(x => x.Peso).HasPrecision(18, 3);
            entity.Property(x => x.Activa).HasDefaultValue(true);
        });
        modelBuilder.Entity<Aleacion>(entity =>
        {
            entity.ToTable("Aleaciones");

```

```

        entity.HasKey(a => a.Id);
        entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
        entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
        entity.Property(a => a.Descripcion).HasMaxLength(500);
    });
    modelBuilder.Entity<EstadoOrden>(entity =>
    {
        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
        entity.Property(e => e.Descripcion).HasMaxLength(250);
        entity.Property(e => e.Activo).HasDefaultValue(true);
    });
}

}

public interface IOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                {
                    sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                }
            )
        );
    }
}

```

```

        sql.CommandTimeout(60);
    })
    .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
    LogLevel.Information)
    .EnableDetailedErrors());

services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
};

context.OrdenesFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo =
true });
context.SaveChanges();

global::System.Console.WriteLine($"2.2 OK | Órdenes: {context.OrdenesFabricacion.Count()} | Planchas:
{context.PlanchasAcero.Count()}");
}
}

```

Explicación línea a línea del código acumulativo 2.2

Las referencias siguientes usan la numeración real de M02/PROYECTO/2.2/Program.cs. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1	using System.ComponentModel.DataAnnotations; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 2	using System.ComponentModel.DataAnnotations.Schema; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 3	using Microsoft.EntityFrameworkCore; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 4	using Microsoft.Extensions.Configuration; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 5	using Microsoft.Extensions.DependencyInjection; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 6	using Microsoft.Extensions.Logging; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 8	namespace AceriaData.ConsoleApp; → declara el espacio de nombres del estado acumulativo.

Línea 10	<code>public class OrdenFabricacion</code> → declara la clase OrdenFabricacion que forma parte del estado acumulativo del ejercicio.
Línea 11	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 12	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 13	<code>public string NumeroOrden { get; set; } = string.Empty;</code> → declara la propiedad NumeroOrden de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 14	<code>public string Cliente { get; set; } = string.Empty;</code> → declara la propiedad Cliente de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 15	<code>public DateTime FechaCreacion { get; set; }</code> → declara la propiedad FechaCreacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.
Línea 16	<code>public DateTime? FechaEntrega { get; set; }</code> → declara la propiedad FechaEntrega de tipo DateTime? que EF Core o la lógica de negocio utiliza en este modelo.
Línea 17	<code>public string Estado { get; set; } = "Pendiente";</code> → declara la propiedad Estado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 18	<code>public string? Observaciones { get; set; }</code> → declara la propiedad Observaciones de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.
Línea 19	<code>public List<PlanchaAcero> Planchas { get; set; } = new();</code> → declara la propiedad Planchas de tipo List<PlanchaAcero> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 20	<code>}</code> → cierra el bloque de código actual.
Línea 22	<code>public class PlanchaAcero</code> → declara la clase PlanchaAcero que forma parte del estado acumulativo del ejercicio.
Línea 23	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 24	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 25	<code>public int OrdenId { get; set; }</code> → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 26	<code>public double Espesor { get; set; }</code> → declara la propiedad Espesor de tipo double que EF Core o la lógica de negocio utiliza en este modelo.
Línea 27	<code>public double Ancho { get; set; }</code> → declara la propiedad Ancho de tipo double que EF Core o la lógica de negocio utiliza en este modelo.
Línea 28	<code>public double Largo { get; set; }</code> → declara la propiedad Largo de tipo double que EF Core o la lógica de negocio utiliza en este modelo.
Línea 29	<code>public decimal Peso { get; set; }</code> → declara la propiedad Peso de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.
Línea 30	<code>public bool Activa { get; set; } = true;</code> → declara la propiedad Activa de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.
Línea 31	<code>public OrdenFabricacion Orden { get; set; } = null!;</code> → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.
Línea 32	<code>}</code> → cierra el bloque de código actual.

Línea 34	<code>public class Aleacion</code> → declara la clase Aleacion que forma parte del estado acumulativo del ejercicio.
Línea 35	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 36	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 37	<code>public string Nombre { get; set; } = string.Empty;</code> → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 38	<code>public string Codigo { get; set; } = string.Empty;</code> → declara la propiedad Codigo de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 39	<code>public double PorcentajeCarbono { get; set; }</code> → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.
Línea 40	<code>public double PorcentajeManganeso { get; set; }</code> → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.
Línea 41	<code>public string? Descripcion { get; set; }</code> → declara la propiedad Descripcion de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.
Línea 42	<code>}</code> → cierra el bloque de código actual.
Línea 44	<code>public class EstadoOrden</code> → declara la clase EstadoOrden que forma parte del estado acumulativo del ejercicio.
Línea 45	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 46	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 47	<code>public string Nombre { get; set; } = string.Empty;</code> → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 48	<code>public string Descripcion { get; set; } = string.Empty;</code> → declara la propiedad Descripcion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 49	<code>public bool Activo { get; set; } = true;</code> → declara la propiedad Activo de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.
Línea 50	<code>}</code> → cierra el bloque de código actual.
Línea 52	<code>public class AceriaDbContext : DbContext</code> → declara la clase AceriaDbContext que forma parte del estado acumulativo del ejercicio.
Línea 53	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 54	<code>public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }</code> → declara el constructor o método AceriaDbContext con los parámetros necesarios para este componente.
Línea 56	<code>public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 57	<code>public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 58	<code>public DbSet<Aleacion> Aleaciones => Set<Aleacion>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 59	<code>public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 62	<code>protected override void OnModelCreating(ModelBuilder modelBuilder)</code> → sobrescribe <code>OnModelCreating</code> para definir configuración explícita del modelo.
Línea 63	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 64	<code>base.OnModelCreating(modelBuilder);</code> → sobrescribe <code>OnModelCreating</code> para definir configuración explícita del modelo.
Línea 65	<code>modelBuilder.Entity<OrdenFabricacion>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 66	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 67	<code>entity.ToTable("OrdenesFabricacion");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 68	<code>entity.HasKey(o => o.Id);</code> → define la clave primaria de la entidad.
Línea 69	<code>entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 70	<code>entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 71	<code>entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 72	<code>entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 73	<code>entity.Property(o => o.Observaciones).HasMaxLength(500);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 74	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 75	<code>modelBuilder.Entity<PlanchaAcero>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 76	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 77	<code>entity.ToTable("PlanchasAcero");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 78	<code>entity.HasKey(x => x.Id);</code> → define la clave primaria de la entidad.
Línea 79	<code>entity.Property(x => x.Peso).HasPrecision(18, 3);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 80	<code>entity.Property(x => x.Activa).HasDefaultValue(true);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 81	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 82	<code>modelBuilder.Entity<Aleacion>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 83	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 84	<code>entity.ToTable("Aleaciones");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 85	<code>entity.HasKey(a => a.Id);</code> → define la clave primaria de la entidad.
Línea 86	<code>entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 87	<code>entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 88	<code>entity.Property(a => a.Descripcion).HasMaxLength(500);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 89	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 90	<code>modelBuilder.Entity<EstadoOrden>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 91	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 92	<code>entity.ToTable("EstadosOrden");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 93	<code>entity.HasKey(e => e.Id);</code> → define la clave primaria de la entidad.
Línea 94	<code>entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 95	<code>entity.Property(e => e.Descripcion).HasMaxLength(250);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 96	<code>entity.Property(e => e.Activo).HasDefaultValue(true);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 97	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 100	<code>}</code> → cierra el bloque de código actual.
Línea 101	<code>}</code> → cierra el bloque de código actual.
Línea 103	<code>public interface IOrdenRepositorio</code> → declara el contrato IOrdenRepositorio para desacoplar consumidores e implementación.
Línea 104	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 105	<code>List<OrdenFabricacion> ObtenerTodas();</code> → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.
Línea 106	<code>OrdenFabricacion? ObtenerPorId(int id);</code> → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.
Línea 107	<code>OrdenFabricacion? ObtenerPorNumero(string numeroOrden);</code> → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.
Línea 108	<code>void Agregar(OrdenFabricacion orden);</code> → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.
Línea 109	<code>void Eliminar(OrdenFabricacion orden);</code> → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.
Línea 110	<code>int Guardar();</code> → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 111	} → cierra el bloque de código actual.
Línea 113	public sealed class OrdenRepositorio : IOrdenRepositorio → declara la clase OrdenRepositorio que forma parte del estado acumulativo del ejercicio.
Línea 114	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 115	private readonly AceriaDbContext _context; → declara el campo privado _context de tipo AceriaDbContext que conserva una dependencia de la clase.
Línea 116	public OrdenRepositorio(AceriaDbContext context) => _context = context; → declara el constructor o método OrdenRepositorio con los parámetros necesarios para este componente.
Línea 118	public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList(); → ordena el resultado de la consulta antes de materializarlo.
Línea 119	public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id); → implementa la búsqueda por clave primaria mediante Find, aprovechando primero el ChangeTracker y después la base de datos.
Línea 120	public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden); → recupera la primera coincidencia o null cuando no existe.
Línea 121	public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden); → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.
Línea 122	public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden); → marca la entidad indicada para borrado al confirmar los cambios.
Línea 123	public int Guardar() => _context.SaveChanges(); → persiste en SQL Server los cambios seguidos por el DbContext.
Línea 124	} → cierra el bloque de código actual.
Línea 126	public interface IServicioOrdenes → declara el contrato IServicioOrdenes para desacoplar consumidores e implementación.
Línea 127	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 128	string ObtenerResumen(int ordenId); → declara el servicio que construye un resumen de una orden a partir de su identificador.
Línea 129	} → cierra el bloque de código actual.
Línea 131	public sealed class ServicioOrdenes : IServicioOrdenes → declara la clase ServicioOrdenes que forma parte del estado acumulativo del ejercicio.
Línea 132	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 133	private readonly IOrdenRepositorio _repositorio; → declara el campo privado _repositorio de tipo IOrdenRepositorio que conserva una dependencia de la clase.
Línea 134	private readonly AceriaDbContext _context; → declara el campo privado _context de tipo AceriaDbContext que conserva una dependencia de la clase.
Línea 135	public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context) → declara el constructor o método ServicioOrdenes con los parámetros necesarios para este componente.
Línea 136	{ → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 137	<code>_repositorio = repositorio;</code> → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.
Línea 138	<code>_context = context;</code> → guarda el DbContext inyectado para consultar los datos necesarios durante el servicio.
Línea 139	<code>}</code> → cierra el bloque de código actual.
Línea 141	<code>public string ObtenerResumen(int ordenId)</code> → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.
Línea 142	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 143	<code>var orden = _repositorio.ObtenerPorId(ordenId);</code> → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.
Línea 144	<code>if (orden is null) return \$"Orden {ordenId} no encontrada";</code> → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.
Línea 145	<code>var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);</code> → cuenta las filas que cumplen la consulta construida.
Línea 146	<code>return \$"Orden {orden.NumeroOrden} Cliente: {orden.Cliente} Planchas: {totalPlanchas}";</code> → devuelve el resultado calculado al código que invocó el método.
Línea 147	<code>}</code> → cierra el bloque de código actual.
Línea 148	<code>}</code> → cierra el bloque de código actual.
Línea 150	<code>public static class Program</code> → declara la clase Program que forma parte del estado acumulativo del ejercicio.
Línea 151	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 152	<code>public static void Main()</code> → declara el punto de entrada ejecutable de la aplicación de consola.
Línea 153	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 154	<code>var configuration = new ConfigurationBuilder()</code> → inicia la construcción de la configuración de la aplicación.
Línea 155	<code>.SetBasePath(AppContext.BaseDirectory)</code> → establece la ruta base usada para localizar los archivos de configuración.
Línea 156	<code>.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)</code> → incorpora appsettings.json a la configuración.
Línea 157	<code>.AddEnvironmentVariables()</code> → incorpora variables de entorno como fuente adicional de configuración.
Línea 158	<code>.Build();</code> → materializa la configuración o el objeto encadenado construido en las líneas anteriores.
Línea 160	<code>var connectionString = configuration.GetConnectionString("AceriaDB")</code> → obtiene la cadena de conexión AceriaDB desde la configuración.
Línea 161	<code>?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");</code> → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.
Línea 163	<code>var services = new ServiceCollection();</code> → crea la colección donde se registran las dependencias de la aplicación.

Línea 164	<code>services.AddDbContext<AceriaDbContext>(options => → registra el DbContext en el contenedor de inyección de dependencias.</code>
Línea 165	<code>options → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.</code>
Línea 166	<code>.UseSqlServer(connectionString, sql => → selecciona SQL Server como proveedor de EF Core.</code>
Línea 167	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 168	<code>sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null); → habilita reintentos ante fallos transitorios del proveedor SQL Server.</code>
Línea 169	<code>sql.CommandTimeout(60); → establece el tiempo máximo permitido para los comandos SQL.</code>
Línea 170	<code>}) → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.</code>
Línea 171	<code>.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information) → configura la salida del logging de EF Core.</code>
Línea 172	<code>.EnableDetailedErrors()); → activa errores detallados para facilitar el diagnóstico del laboratorio.</code>
Línea 174	<code>services.AddScoped<IOrdenRepositorio, OrdenRepositorio>(); → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.</code>
Línea 175	<code>services.AddScoped<IServicioOrdenes, ServicioOrdenes>(); → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.</code>
Línea 177	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions → construye el ServiceProvider con validación de servicios y garantiza su liberación al salir del ámbito mediante using var.</code>
Línea 178	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 179	<code>ValidateScopes = true, → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.</code>
Línea 180	<code>ValidateOnBuild = true → obliga al contenedor a validar las resoluciones registradas al construir el ServiceProvider.</code>
Línea 181	<code>}); → cierra la lambda o configuración encadenada y finaliza la llamada.</code>
Línea 183	<code>using var scope = provider.CreateScope(); → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.</code>
Línea 184	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>(); → resuelve un servicio obligatorio desde el contenedor de dependencias.</code>
Línea 185	<code>context.Database.EnsureDeleted(); → elimina la base del laboratorio para reproducir el escenario desde un estado limpio.</code>
Línea 186	<code>context.Database.Migrate(); → aplica las migraciones pendientes y deja AceriaDB en el esquema del punto.</code>
Línea 188	<code>var orden = new OrdenFabricacion → crea la entidad OrdenFabricacion que concentra el grafo de datos del escenario acumulativo.</code>
Línea 189	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 190	<code>NumeroOrden = "OF-M2-0001", → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.</code>

Línea 191	<code>Cliente = "Constructora del Norte",</code> → asigna el cliente de la orden utilizada en el escenario E2E del punto.
Línea 192	<code>FechaCreacion = DateTime.UtcNow,</code> → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.
Línea 193	<code>FechaEntrega = DateTime.Today.AddDays(14),</code> → establece una fecha de entrega prevista catorce días después de la fecha local actual.
Línea 194	<code>Estado = "Pendiente",</code> → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.
Línea 195	<code>Observaciones = "Orden de validación M2",</code> → añade texto opcional para comprobar el mapeo de Observaciones.
Línea 196	<code>Planchas =</code> → inicia el inicializador de la colección de planchas asociadas a la orden.
Línea 197	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 198	<code>new PlanchaAcero</code> → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.
Línea 199	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 200	<code>Espesor = 10.5,</code> → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.
Línea 201	<code>Ancho = 1500,</code> → asigna el ancho de la plancha usada en el escenario de persistencia.
Línea 202	<code>Largo = 3000,</code> → asigna el largo de la plancha usada en el escenario de persistencia.
Línea 203	<code>Peso = 371.250m,</code> → asigna un peso decimal con tres posiciones para validar la precisión configurada.
Línea 204	<code>Activa = true,</code> → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.
Línea 205	<code>}</code> → cierra el bloque de código actual.
Línea 206	<code>},</code> → continúa el inicializador o la llamada iniciada en el bloque actual.
Línea 207	<code>};</code> → cierra el inicializador del objeto y finaliza la instrucción.
Línea 209	<code>context.OrdenesFabricacion.Add(orden);</code> → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.
Línea 210	<code>context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });</code> → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.
Línea 211	<code>context.SaveChanges();</code> → persiste en SQL Server los cambios seguidos por el DbContext.
Línea 213	<code>global::System.Console.WriteLine(\$"2.2 OK Órdenes: {context.OrdenesFabricacion.Count()} Planchas: {context.PlanchasAcero.Count()}");</code> → imprime el marcador E2E de 2.2 junto con los recuentos persistidos de órdenes y planchas.
Línea 214	<code>}</code> → cierra el bloque de código actual.
Línea 215	<code>}</code> → cierra el bloque de código actual.

Punto 2.3 - Relaciones uno a muchos

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL que trabajan con .NET 8, EF Core 8 y SQL Server LocalDB.

Proyecto: El estado 2.3 parte de 2.2 y tiene como objetivo hacer explícita la relación OrdenFabricacion 1 -> N PlanchaAcero.

Objetivos de aprendizaje

- Identificar principal y dependiente.
- Configurar HasOne/WithMany.
- Definir la clave foránea OrdenId.
- Configurar DeleteBehavior.Cascade.
- Cargar la colección con Include.
- No adelantar relaciones uno-a-uno ni muchos-a-muchos.

Paso 1: Abrir la solución autónoma del punto

```
cd M02/PROYECTO/2.3
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

No se abre una solución global situada en la raíz del repositorio. AceriaData.sln está dentro de la carpeta del punto y referencia únicamente los proyectos de ese estado.

Paso 2: Identificar el cambio respecto al estado anterior

El proyecto conserva todo lo terminado en 2.2. En 2.3 se introduce exclusivamente el contenido que corresponde a **Relaciones uno a muchos**. La práctica no reinicia AceriaData ni crea un ejemplo paralelo.

Paso 3: Implementar y comprender la configuración principal

```
modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.HasOne(x => x.Orden)
        .WithMany(o => o.Planchas)
        .HasForeignKey(x => x.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
```

CSHARP

Línea 1	HasOne(x => x.Orden) → selecciona la navegación de referencia del dependiente.
Línea 2	WithMany(o => o.Planchas) → selecciona la colección del principal.
Línea 3	HasForeignKey(x => x.OrdenId) → declara la FK de PlanchaAcero.
Línea 4	OnDelete(DeleteBehavior.Cascade) → define el comportamiento cuando se elimina la orden.
Línea 5	IsRequired() → confirma que una plancha persistida debe pertenecer a una orden.

Paso 4: Generar y aplicar la migración acumulativa

El repositorio contiene una migración real generada para este estado. Si se reproduce el ejercicio desde el estado anterior, los comandos son:

```
dotnet ef migrations add M2_2_3
dotnet ef database update
```

POWERSHELL

`dotnet ef migrations add` compara el modelo actual con el snapshot heredado. `dotnet ef database update` aplica únicamente los cambios pendientes.

En los estados ejecutables del curso se usa `Database.Migrate()` para aplicar el historial durante la demostración. El laboratorio puede usar `EnsureDeleted()` para empezar desde una base limpia, pero **no usa `EnsureCreated()`**, porque el esquema está gobernado por Migrations.

Paso 5: Compilar y ejecutar el estado

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

POWERSHELL

Resultado esperado: la ejecución termina sin excepción y contiene la evidencia 2.3 OK. GitHub Actions repite esta compilación y ejecución sobre Windows y SQL Server LocalDB.

Paso 6: Verificar el estado acumulativo

Además de ejecutar el ejemplo, conviene revisar el modelo y la base:

```
dotnet ef migrations list
```

POWERSHELL

Cuando el punto modifica el esquema, la lista debe contener la migración acumulativa correspondiente. En SQL Server Object Explorer se puede comprobar que las tablas, claves, relaciones, índices o restricciones coinciden con el modelo del punto.

Errores comunes

Error	Causa	Solución
Include no carga planchas	Se olvidó Include	Usar <code>Include(o => o.Planchas)</code> .
FK inválida	La plancha apunta a una orden inexistente	Insertar el agregado o asignar una <code>OrdenId</code> válida.
Se añade <code>OrdenAleacion</code>	Se adelantó 2.5	Mantener 2.3 exclusivamente en la relación 1:N.

Reto resuelto

Reto: Crear una orden con dos planchas, recuperarla con `Include` y comprobar que la colección contiene exactamente dos elementos.

Solución: partir del código de `M02/PROYECTO/2.3`, realizar únicamente el cambio descrito y volver a ejecutar `build`, migraciones y el programa. El reto no introduce conceptos reservados a un punto posterior.

Analogía final

Una relación uno-a-muchos se parece a una orden de producción que agrupa varias planchas, mientras cada plancha pertenece a una sola orden.

Resultado esperado

Al terminar 2.3, la solución local compila, el proyecto se ejecuta, el esquema se actualiza mediante Migrations cuando corresponde y la salida contiene 2.3 OK. El código conserva todos los cambios válidos de los puntos anteriores.

Conexión con el siguiente punto

El siguiente estado es 2.4. Se parte del proyecto completo de 2.3; no se vuelve a crear `AceriaData` desde cero.

Código acumulativo completo del estado 2.3

El siguiente archivo es el `Program.cs` real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public stringCodigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
            entity.Property(o => o.Observaciones).HasMaxLength(500);
        });
        modelBuilder.Entity<PlanchaAcero>(entity =>
        {
            entity.ToTable("PlanchasAcero");
            entity.HasKey(x => x.Id);
            entity.Property(x => x.Peso).HasPrecision(18, 3);
            entity.Property(x => x.Activa).HasDefaultValue(true);
            entity.HasOne(x => x.Orden)
                .WithMany(o => o.Planchas)
                .HasForeignKey(x => x.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
    
```

```

        .IsRequired();
    });
    modelBuilder.Entity<Aleacion>(entity =>
    {
        entity.ToTable("Aleaciones");
        entity.HasKey(a => a.Id);
        entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
        entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
        entity.Property(a => a.Descripcion).HasMaxLength(500);
    });
    modelBuilder.Entity<EstadoOrden>(entity =>
    {
        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
        entity.Property(e => e.Descripcion).HasMaxLength(250);
        entity.Property(e => e.Activo).HasDefaultValue(true);
    });
}

}

public interface IOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();

```

```

services.AddDbContext<AceriaDbContext>(options =>
    options
        .UseSqlServer(connectionString, sql =>
            {
                sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                sql.CommandTimeout(60);
            })
        .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
            LogLevel.Information)
        .EnableDetailedErrors());

services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
};

context.OrdenesFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo =
true });
context.SaveChanges();

var cargada = context.OrdenesFabricacion.Include(o => o.Planchas).Single(o => o.NumeroOrden == "OF-M2-0001");
global::System.Console.WriteLine($"2.3 OK | {cargada.NumeroOrden} | Planchas: {cargada.Planchas.Count}");
}

```

Explicación línea a línea del código acumulativo 2.3

Las referencias siguientes usan la numeración real de M02/PROYECTO/2.3/Program.cs. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1	using System.ComponentModel.DataAnnotations; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 2	using System.ComponentModel.DataAnnotations.Schema; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 3	using Microsoft.EntityFrameworkCore; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 4	using Microsoft.Extensions.Configuration; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 5	using Microsoft.Extensions.DependencyInjection; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 6	<code>using Microsoft.Extensions.Logging;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 8	<code>namespace AceriaData.ConsoleApp;</code> → declara el espacio de nombres del estado acumulativo.
Línea 10	<code>public class OrdenFabricacion</code> → declara la clase <code>OrdenFabricacion</code> que forma parte del estado acumulativo del ejercicio.
Línea 11	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 12	<code>public int Id { get; set; }</code> → declara la propiedad <code>Id</code> de tipo <code>int</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 13	<code>public string NumeroOrden { get; set; } = string.Empty;</code> → declara la propiedad <code>NumeroOrden</code> de tipo <code>string</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 14	<code>public string Cliente { get; set; } = string.Empty;</code> → declara la propiedad <code>Cliente</code> de tipo <code>string</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 15	<code>public DateTime FechaCreacion { get; set; }</code> → declara la propiedad <code>FechaCreacion</code> de tipo <code>DateTime</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 16	<code>public DateTime? FechaEntrega { get; set; }</code> → declara la propiedad <code>FechaEntrega</code> de tipo <code>DateTime?</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 17	<code>public string Estado { get; set; } = "Pendiente";</code> → declara la propiedad <code>Estado</code> de tipo <code>string</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 18	<code>public string? Observaciones { get; set; }</code> → declara la propiedad <code>Observaciones</code> de tipo <code>string?</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 19	<code>public List<PlanchaAcero> Planchas { get; set; } = new();</code> → declara la propiedad <code>Planchas</code> de tipo <code>List<PlanchaAcero></code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 20	<code>}</code> → cierra el bloque de código actual.
Línea 22	<code>public class PlanchaAcero</code> → declara la clase <code>PlanchaAcero</code> que forma parte del estado acumulativo del ejercicio.
Línea 23	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 24	<code>public int Id { get; set; }</code> → declara la propiedad <code>Id</code> de tipo <code>int</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 25	<code>public int OrdenId { get; set; }</code> → declara la propiedad <code>OrdenId</code> de tipo <code>int</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 26	<code>public double Espesor { get; set; }</code> → declara la propiedad <code>Espesor</code> de tipo <code>double</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 27	<code>public double Ancho { get; set; }</code> → declara la propiedad <code>Ancho</code> de tipo <code>double</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 28	<code>public double Largo { get; set; }</code> → declara la propiedad <code>Largo</code> de tipo <code>double</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 29	<code>public decimal Peso { get; set; }</code> → declara la propiedad <code>Peso</code> de tipo <code>decimal</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 30	<code>public bool Activa { get; set; } = true;</code> → declara la propiedad <code>Activa</code> de tipo <code>bool</code> que EF Core o la lógica de negocio utiliza en este modelo.

Línea 31	<code>public OrdenFabricacion Orden { get; set; } = null!;</code> → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.
Línea 32	<code>}</code> → cierra el bloque de código actual.
Línea 34	<code>public class Aleacion</code> → declara la clase Aleacion que forma parte del estado acumulativo del ejercicio.
Línea 35	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 36	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 37	<code>public string Nombre { get; set; } = string.Empty;</code> → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 38	<code>public stringCodigo { get; set; } = string.Empty;</code> → declara la propiedad Codigo de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 39	<code>public double PorcentajeCarbono { get; set; }</code> → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.
Línea 40	<code>public double PorcentajeManganeso { get; set; }</code> → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.
Línea 41	<code>public string? Descripcion { get; set; }</code> → declara la propiedad Descripcion de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.
Línea 42	<code>}</code> → cierra el bloque de código actual.
Línea 44	<code>public class EstadoOrden</code> → declara la clase EstadoOrden que forma parte del estado acumulativo del ejercicio.
Línea 45	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 46	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 47	<code>public string Nombre { get; set; } = string.Empty;</code> → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 48	<code>public string Descripcion { get; set; } = string.Empty;</code> → declara la propiedad Descripcion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 49	<code>public bool Activo { get; set; } = true;</code> → declara la propiedad Activo de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.
Línea 50	<code>}</code> → cierra el bloque de código actual.
Línea 52	<code>public class AceriaDbContext : DbContext</code> → declara la clase AceriaDbContext que forma parte del estado acumulativo del ejercicio.
Línea 53	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 54	<code>public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }</code> → declara el constructor o método AceriaDbContext con los parámetros necesarios para este componente.
Línea 56	<code>public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 57	<code>public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 58	<code>public DbSet<Aleacion> Aleaciones => Set<Aleacion>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 59	<code>public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 62	<code>protected override void OnModelCreating(ModelBuilder modelBuilder)</code> → sobrescribe <code>OnModelCreating</code> para definir configuración explícita del modelo.
Línea 63	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 64	<code>base.OnModelCreating(modelBuilder);</code> → sobrescribe <code>OnModelCreating</code> para definir configuración explícita del modelo.
Línea 65	<code>modelBuilder.Entity<OrdenFabricacion>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 66	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 67	<code>entity.ToTable("OrdenesFabricacion");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 68	<code>entity.HasKey(o => o.Id);</code> → define la clave primaria de la entidad.
Línea 69	<code>entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 70	<code>entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 71	<code>entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 72	<code>entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 73	<code>entity.Property(o => o.Observaciones).HasMaxLength(500);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 74	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 75	<code>modelBuilder.Entity<PlanchaAcero>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 76	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 77	<code>entity.ToTable("PlanchasAcero");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 78	<code>entity.HasKey(x => x.Id);</code> → define la clave primaria de la entidad.
Línea 79	<code>entity.Property(x => x.Peso).HasPrecision(18, 3);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 80	<code>entity.Property(x => x.Activa).HasDefaultValue(true);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 81	<code>entity.HasOne(x => x.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 82	<code>.WithMany(o => o.Planchas)</code> → establece el extremo de colección de una relación uno-a-muchos.

Línea 83	<code>.HasForeignKey(x => x.OrdenId) →</code> declara explícitamente la propiedad o propiedades de clave foránea.
Línea 84	<code>.OnDelete(DeleteBehavior.Cascade) →</code> define el comportamiento de eliminación de la relación.
Línea 85	<code>.IsRequired(); →</code> marca la propiedad o relación como obligatoria.
Línea 86	<code>}); →</code> cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 87	<code>modelBuilder.Entity<Aleacion>(entity => →</code> selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 88	<code>{ →</code> abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 89	<code>entity.ToTable("Aleaciones"); →</code> asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 90	<code>entity.HasKey(a => a.Id); →</code> define la clave primaria de la entidad.
Línea 91	<code>entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100); →</code> selecciona una propiedad escalar para continuar su configuración.
Línea 92	<code>entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20); →</code> selecciona una propiedad escalar para continuar su configuración.
Línea 93	<code>entity.Property(a => a.Descripcion).HasMaxLength(500); →</code> selecciona una propiedad escalar para continuar su configuración.
Línea 94	<code>}); →</code> cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 95	<code>modelBuilder.Entity<EstadoOrden>(entity => →</code> selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 96	<code>{ →</code> abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 97	<code>entity.ToTable("EstadosOrden"); →</code> asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 98	<code>entity.HasKey(e => e.Id); →</code> define la clave primaria de la entidad.
Línea 99	<code>entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50); →</code> selecciona una propiedad escalar para continuar su configuración.
Línea 100	<code>entity.Property(e => e.Descripcion).HasMaxLength(250); →</code> selecciona una propiedad escalar para continuar su configuración.
Línea 101	<code>entity.Property(e => e.Activo).HasDefaultValue(true); →</code> selecciona una propiedad escalar para continuar su configuración.
Línea 102	<code>}); →</code> cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 105	<code>} →</code> cierra el bloque de código actual.
Línea 106	<code>} →</code> cierra el bloque de código actual.
Línea 108	<code>public interface IOrdenRepositorio →</code> declara el contrato IOrdenRepositorio para desacoplar consumidores e implementación.
Línea 109	<code>{ →</code> abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 110	<code>List<OrdenFabricacion> ObtenerTodas(); →</code> declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 111	<code>OrdenFabricacion? ObtenerPorId(int id);</code> → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.
Línea 112	<code>OrdenFabricacion? ObtenerPorNumero(string numeroOrden);</code> → declara en el contrato la búsqueda por el identificador de negocio <code>NumeroOrden</code> .
Línea 113	<code>void Agregar(OrdenFabricacion orden);</code> → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.
Línea 114	<code>void Eliminar(OrdenFabricacion orden);</code> → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.
Línea 115	<code>int Guardar();</code> → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.
Línea 116	<code>}</code> → cierra el bloque de código actual.
Línea 118	<code>public sealed class OrdenRepositorio : IOrdenRepositorio</code> → declara la clase <code>OrdenRepositorio</code> que forma parte del estado acumulativo del ejercicio.
Línea 119	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 120	<code>private readonly AceriaDbContext _context;</code> → declara el campo privado <code>_context</code> de tipo <code>AceriaDbContext</code> que conserva una dependencia de la clase.
Línea 121	<code>public OrdenRepositorio(AceriaDbContext context) => _context = context;</code> → declara el constructor o método <code>OrdenRepositorio</code> con los parámetros necesarios para este componente.
Línea 123	<code>public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();</code> → ordena el resultado de la consulta antes de materializarlo.
Línea 124	<code>public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);</code> → implementa la búsqueda por clave primaria mediante <code>Find</code> , aprovechando primero el <code>ChangeTracker</code> y después la base de datos.
Línea 125	<code>public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);</code> → recupera la primera coincidencia o null cuando no existe.
Línea 126	<code>public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);</code> → registra una entidad o relación nueva en el <code>ChangeTracker</code> para insertarla al guardar.
Línea 127	<code>public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);</code> → marca la entidad indicada para borrado al confirmar los cambios.
Línea 128	<code>public int Guardar() => _context.SaveChanges();</code> → persiste en SQL Server los cambios seguidos por el <code>DbContext</code> .
Línea 129	<code>}</code> → cierra el bloque de código actual.
Línea 131	<code>public interface IServicioOrdenes</code> → declara el contrato <code>IServicioOrdenes</code> para desacoplar consumidores e implementación.
Línea 132	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 133	<code>string ObtenerResumen(int ordenId);</code> → declara el servicio que construye un resumen de una orden a partir de su identificador.
Línea 134	<code>}</code> → cierra el bloque de código actual.

Línea 136	<code>public sealed class ServicioOrdenes : IServicioOrdenes</code> → declara la clase <code>ServicioOrdenes</code> que forma parte del estado acumulativo del ejercicio.
Línea 137	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 138	<code>private readonly IOrdenRepositorio _repositorio;</code> → declara el campo privado <code>_repositorio</code> de tipo <code>IOrdenRepositorio</code> que conserva una dependencia de la clase.
Línea 139	<code>private readonly AceriaDbContext _context;</code> → declara el campo privado <code>_context</code> de tipo <code>AceriaDbContext</code> que conserva una dependencia de la clase.
Línea 140	<code>public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)</code> → declara el constructor o método <code>ServicioOrdenes</code> con los parámetros necesarios para este componente.
Línea 141	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 142	<code>_repositorio = repositorio;</code> → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.
Línea 143	<code>_context = context;</code> → guarda el <code>DbContext</code> inyectado para consultar los datos necesarios durante el servicio.
Línea 144	<code>}</code> → cierra el bloque de código actual.
Línea 146	<code>public string ObtenerResumen(int ordenId)</code> → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.
Línea 147	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 148	<code>var orden = _repositorio.ObtenerPorId(ordenId);</code> → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.
Línea 149	<code>if (orden is null) return \$"Orden {ordenId} no encontrada";</code> → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.
Línea 150	<code>var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);</code> → cuenta las filas que cumplen la consulta construida.
Línea 151	<code>return \$"Orden {orden.NumeroOrden} Cliente: {orden.Cliente} Planchas: {totalPlanchas}";</code> → devuelve el resultado calculado al código que invocó el método.
Línea 152	<code>}</code> → cierra el bloque de código actual.
Línea 153	<code>}</code> → cierra el bloque de código actual.
Línea 155	<code>public static class Program</code> → declara la clase <code>Program</code> que forma parte del estado acumulativo del ejercicio.
Línea 156	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 157	<code>public static void Main()</code> → declara el punto de entrada ejecutable de la aplicación de consola.
Línea 158	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 159	<code>var configuration = new ConfigurationBuilder()</code> → inicia la construcción de la configuración de la aplicación.
Línea 160	<code>.SetBasePath(AppContext.BaseDirectory)</code> → establece la ruta base usada para localizar los archivos de configuración.
Línea 161	<code>.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)</code> → incorpora <code>appsettings.json</code> a la configuración.

Línea 162	<code>.AddEnvironmentVariables()</code> → incorpora variables de entorno como fuente adicional de configuración.
Línea 163	<code>.Build();</code> → materializa la configuración o el objeto encadenado construido en las líneas anteriores.
Línea 165	<code>var connectionString = configuration.GetConnectionString("AceriaDB")</code> → obtiene la cadena de conexión AceriaDB desde la configuración.
Línea 166	<code>?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");</code> → interrumpe el arranque con un error explícito si la configuración no contiene <code>ConnectionStrings:AceriaDB</code> .
Línea 168	<code>var services = new ServiceCollection();</code> → crea la colección donde se registran las dependencias de la aplicación.
Línea 169	<code>services.AddDbContext<AceriaDbContext>(options =></code> → registra el <code>DbContext</code> en el contenedor de inyección de dependencias.
Línea 170	<code>options</code> → continúa la configuración del <code>DbContext</code> sobre el parámetro <code>options</code> recibido por la lambda de <code>AddDbContext</code> .
Línea 171	<code>.UseSqlServer(connectionString, sql =></code> → selecciona SQL Server como proveedor de EF Core.
Línea 172	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 173	<code>sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);</code> → habilita reintentos ante fallos transitorios del proveedor SQL Server.
Línea 174	<code>sql.CommandTimeout(60);</code> → establece el tiempo máximo permitido para los comandos SQL.
Línea 175	<code>})</code> → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.
Línea 176	<code>.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)</code> → configura la salida del logging de EF Core.
Línea 177	<code>.EnableDetailedErrors();</code> → activa errores detallados para facilitar el diagnóstico del laboratorio.
Línea 179	<code>services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();</code> → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.
Línea 180	<code>services.AddScoped<IServicioOrdenes, ServicioOrdenes>();</code> → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.
Línea 182	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions</code> → construye el <code>ServiceProvider</code> con validación de servicios y garantiza su liberación al salir del ámbito mediante <code>using var</code> .
Línea 183	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 184	<code>ValidateScopes = true,</code> → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.
Línea 185	<code>ValidateOnBuild = true</code> → obliga al contenedor a validar las resoluciones registradas al construir el <code>ServiceProvider</code> .
Línea 186	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 188	<code>using var scope = provider.CreateScope();</code> → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 189	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → resuelve un servicio obligatorio desde el contenedor de dependencias.
Línea 190	<code>context.Database.EnsureDeleted();</code> → elimina la base del laboratorio para reproducir el escenario desde un estado limpio.
Línea 191	<code>context.Database.Migrate();</code> → aplica las migraciones pendientes y deja AceriaDB en el esquema del punto.
Línea 193	<code>var orden = new OrdenFabricacion</code> → crea la entidad OrdenFabricacion que concentra el grafo de datos del escenario acumulativo.
Línea 194	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 195	<code>NumeroOrden = "0F-M2-0001",</code> → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.
Línea 196	<code>Cliente = "Constructora del Norte",</code> → asigna el cliente de la orden utilizada en el escenario E2E del punto.
Línea 197	<code>FechaCreacion = DateTime.UtcNow,</code> → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.
Línea 198	<code>FechaEntrega = DateTime.Today.AddDays(14),</code> → establece una fecha de entrega prevista catorce días después de la fecha local actual.
Línea 199	<code>Estado = "Pendiente",</code> → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.
Línea 200	<code>Observaciones = "Orden de validación M2",</code> → añade texto opcional para comprobar el mapeo de Observaciones.
Línea 201	<code>Planchas =</code> → inicia el inicializador de la colección de planchas asociadas a la orden.
Línea 202	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 203	<code>new PlanchaAcero</code> → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.
Línea 204	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 205	<code>Espesor = 10.5,</code> → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.
Línea 206	<code>Ancho = 1500,</code> → asigna el ancho de la plancha usada en el escenario de persistencia.
Línea 207	<code>Largo = 3000,</code> → asigna el largo de la plancha usada en el escenario de persistencia.
Línea 208	<code>Peso = 371.250m,</code> → asigna un peso decimal con tres posiciones para validar la precisión configurada.
Línea 209	<code>Activa = true,</code> → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.
Línea 210	<code>}</code> → cierra el bloque de código actual.
Línea 211	<code>},</code> → continúa el inicializador o la llamada iniciada en el bloque actual.
Línea 212	<code>};</code> → cierra el inicializador del objeto y finaliza la instrucción.
Línea 214	<code>context.OrdenesFabricacion.Add(orden);</code> → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.

Línea 215	<code>context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });</code> → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.
Línea 216	<code>context.SaveChanges();</code> → persiste en SQL Server los cambios seguidos por el DbContext.
Línea 218	<code>var cargada = context.OrdenesFabricacion.Include(o => o.Planchas).Single(o => o.NumeroOrden == "0F-M2-0001");</code> → incluye de forma eager la navegación relacionada en la consulta.
Línea 219	<code>global::System.Console.WriteLine(\$"2.3 OK {cargada.NumeroOrden} Planchas: {cargada.Planchas.Count}");</code> → escribe en consola la evidencia usada para verificar el comportamiento del estado.
Línea 220	<code>}</code> → cierra el bloque de código actual.
Línea 221	<code>}</code> → cierra el bloque de código actual.

Punto 2.4 - Relaciones uno a uno

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL que trabajan con .NET 8, EF Core 8 y SQL Server LocalDB.

Proyecto: El estado 2.4 parte de 2.3 y tiene como objetivo añadir `DetalleOrden` y `CertificadoCalidad` como relaciones uno-a-uno.

Objetivos de aprendizaje

- Modelar relaciones uno-a-uno.
- Elegir el dependiente mediante `HasForeignKey<T>`.
- Mantener `OrdenId` obligatorio en el dependiente.
- Permitir que una orden exista sin detalle mediante navegación nullable.
- Configurar la relación una sola vez.
- Comprobar índices únicos de las claves foráneas.

Paso 1: Abrir la solución autónoma del punto

```
cd M02/PROYECTO/2.4
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

No se abre una solución global situada en la raíz del repositorio. `AceriaData.sln` está dentro de la carpeta del punto y referencia únicamente los proyectos de ese estado.

Paso 2: Identificar el cambio respecto al estado anterior

El proyecto conserva todo lo terminado en 2.3. En 2.4 se introduce exclusivamente el contenido que corresponde a **Relaciones uno a uno**. La práctica no reinicia `AceriaData` ni crea un ejemplo paralelo.

Paso 3: Implementar y comprender la configuración principal

```
modelBuilder.Entity<DetalleOrden>(entity =>
{
    entity.ToTable("DetallesOrden");
    entity.HasKey(d => d.Id);

    entity.HasOne(d => d.Orden)
        .WithOne(o => o.Detalle)
        .HasForeignKey<DetalleOrden>(d => d.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
```

CSHARP

```
.IsRequired();
});
```

Línea 1	HasOne(d => d.Orden) → configura la relación desde el dependiente.
Línea 2	WithOne(o => o.Detalle) → indica cardinalidad uno-a-uno.
Línea 3	HasForeignKey<DetalleOrden>(d => d.OrdenId) → identifica explícitamente al dependiente y su FK.
Línea 4	IsRequired() → hace obligatorio OrdenId para un DetalleOrden existente.
Línea 5	DetalleOrden? Detalle → permite, al mismo tiempo, que una orden todavía no tenga detalle.

Paso 4: Generar y aplicar la migración acumulativa

El repositorio contiene una migración real generada para este estado. Si se reproduce el ejercicio desde el estado anterior, los comandos son:

```
dotnet ef migrations add M2_2_4
dotnet ef database update
```

POWERSHELL

`dotnet ef migrations add` compara el modelo actual con el snapshot heredado. `dotnet ef database update` aplica únicamente los cambios pendientes.

En los estados ejecutables del curso se usa `Database.Migrate()` para aplicar el historial durante la demostración. El laboratorio puede usar `EnsureDeleted()` para empezar desde una base limpia, pero **no usa `EnsureCreated()`**, porque el esquema está gobernado por Migrations.

Paso 5: Compilar y ejecutar el estado

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

POWERSHELL

Resultado esperado: la ejecución termina sin excepción y contiene la evidencia `2.4 OK`. GitHub Actions repite esta compilación y ejecución sobre Windows y SQL Server LocalDB.

Paso 6: Verificar el estado acumulativo

Además de ejecutar el ejemplo, conviene revisar el modelo y la base:

```
dotnet ef migrations list
```

POWERSHELL

Cuando el punto modifica el esquema, la lista debe contener la migración acumulativa correspondiente. En SQL Server Object Explorer se puede comprobar que las tablas, claves, relaciones, índices o restricciones coinciden con el modelo del punto.

Errores comunes

Error	Causa	Solución
Configuración contradictoria	Se configura la misma relación con <code>IsRequired</code> y <code>IsRequired(false)</code>	Configurar una sola vez desde <code>DetalleOrden</code> .
Dos detalles para la misma orden	La FK tiene índice único	Crear como máximo un dependiente por orden.
Detalle sin orden	<code>OrdenId</code> no apunta a una fila válida	Guardar la orden o asociar correctamente la navegación.

Reto resuelto

Reto: Crear un CertificadoCalidad para una orden y demostrar que no puede existir un segundo certificado con la misma OrdenId.

Solución: partir del código de M02/PROYECTO/2.4, realizar únicamente el cambio descrito y volver a ejecutar build, migraciones y el programa. El reto no introduce conceptos reservados a un punto posterior.

Analogía final

Una relación uno-a-uno representa un expediente técnico único asociado a una orden: puede no existir todavía, pero cuando existe pertenece a esa orden.

Resultado esperado

Al terminar 2.4, la solución local compila, el proyecto se ejecuta, el esquema se actualiza mediante Migrations cuando corresponde y la salida contiene 2.4 OK. El código conserva todos los cambios válidos de los puntos anteriores.

Conexión con el siguiente punto

El siguiente estado es 2.5. Se parte del proyecto completo de 2.4; no se vuelve a crear AceriaData desde cero.

Código acumulativo completo del estado 2.4

El siguiente archivo es el Program.cs real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

CSHARP

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public stringCodigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
```

```

    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    public string? Notas { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class CertificadoCalidad
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    public string OrganismoCertificador { get; set; } = string.Empty;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
            entity.Property(o => o.Observaciones).HasMaxLength(500);
        });
        modelBuilder.Entity<PlanchaAcero>(entity =>
        {
            entity.ToTable("PlanchasAcero");
            entity.HasKey(x => x.Id);
            entity.Property(x => x.Peso).HasPrecision(18, 3);
            entity.Property(x => x.Activa).HasDefaultValue(true);
            entity.HasOne(x => x.Orden)
                .WithMany(o => o.Planchas)
                .HasForeignKey(x => x.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();
        });
        modelBuilder.Entity<Aleacion>(entity =>
        {
            entity.ToTable("Aleaciones");
            entity.HasKey(a => a.Id);
            entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
            entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
            entity.Property(a => a.Descripcion).HasMaxLength(500);
        });
        modelBuilder.Entity<EstadoOrden>(entity =>
        {
            entity.ToTable("EstadosOrden");
            entity.HasKey(e => e.Id);
            entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
            entity.Property(e => e.Descripcion).HasMaxLength(250);
            entity.Property(e => e.Activo).HasDefaultValue(true);
        });
        modelBuilder.Entity<DetalleOrden>(entity =>
        {
            entity.ToTable("DetallesOrden");
            entity.HasKey(d => d.Id);
            entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
            entity.Property(d => d.Notas).HasMaxLength(500);
            entity.HasOne(d => d.Orden)
                .WithOne(o => o.Detalle)

```

```

        .HasForeignKey<DetalleOrden>(d => d.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
    });

    modelBuilder.Entity<CertificadoCalidad>(entity =>
    {
        entity.ToTable("CertificadosCalidad");
        entity.HasKey(c => c.Id);
        entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
        entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
        entity.HasOne(c => c.Orden)
            .WithOne(o => o.Certificado)
            .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
}

}

public interface IOOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options

```

```

        .UseSqlServer(connectionString, sql =>
        {
            sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
            sql.CommandTimeout(60);
        })
        .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
        LogLevel.Information)
        .EnableDetailedErrors());

services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },
    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

context.OrdenesFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo =
true });
context.SaveChanges();

var cargada = context.OrdenesFabricacion.Include(o => o.Detalle).Include(o => o.Certificado).Single(o =>
o.NumeroOrden == "OF-M2-0001");
global::System.Console.WriteLine($"2.4 OK | Detalle: {cargada.Detalle?.ComposicionQuimica} | Certificado:
{cargada.Certificado?.NumeroCertificado}");
}
}

```

Explicación línea a línea del código acumulativo 2.4

Las referencias siguientes usan la numeración real de M02/PROYECTO/2.4/Program.cs. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1	using System.ComponentModel.DataAnnotations; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 2	using System.ComponentModel.DataAnnotations.Schema; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 3	<code>using Microsoft.EntityFrameworkCore; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.</code>
Línea 4	<code>using Microsoft.Extensions.Configuration; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.</code>
Línea 5	<code>using Microsoft.Extensions.DependencyInjection; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.</code>
Línea 6	<code>using Microsoft.Extensions.Logging; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.</code>
Línea 8	<code>namespace AceriaData.ConsoleApp; → declara el espacio de nombres del estado acumulativo.</code>
Línea 10	<code>public class OrdenFabricacion → declara la clase OrdenFabricacion que forma parte del estado acumulativo del ejercicio.</code>
Línea 11	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 12	<code>public int Id { get; set; } → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 13	<code>public string NumeroOrden { get; set; } = string.Empty; → declara la propiedad NumeroOrden de tipo string que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 14	<code>public string Cliente { get; set; } = string.Empty; → declara la propiedad Cliente de tipo string que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 15	<code>public DateTime FechaCreacion { get; set; } → declara la propiedad FechaCreacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 16	<code>public DateTime? FechaEntrega { get; set; } → declara la propiedad FechaEntrega de tipo DateTime? que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 17	<code>public string Estado { get; set; } = "Pendiente"; → declara la propiedad Estado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 18	<code>public string? Observaciones { get; set; } → declara la propiedad Observaciones de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 19	<code>public List<PlanchaAcero> Planchas { get; set; } = new(); → declara la propiedad Planchas de tipo List<PlanchaAcero> que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 20	<code>public DetalleOrden? Detalle { get; set; } → declara la propiedad Detalle de tipo DetalleOrden? que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 21	<code>public CertificadoCalidad? Certificado { get; set; } → declara la propiedad Certificado de tipo CertificadoCalidad? que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 22	<code>} → cierra el bloque de código actual.</code>
Línea 24	<code>public class PlanchaAcero → declara la clase PlanchaAcero que forma parte del estado acumulativo del ejercicio.</code>
Línea 25	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 26	<code>public int Id { get; set; } → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 27	<code>public int OrdenId { get; set; } → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.</code>

Línea 28	<code>public double Espesor { get; set; } → declara la propiedad Espesor de tipo double que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 29	<code>public double Ancho { get; set; } → declara la propiedad Ancho de tipo double que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 30	<code>public double Largo { get; set; } → declara la propiedad Largo de tipo double que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 31	<code>public decimal Peso { get; set; } → declara la propiedad Peso de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 32	<code>public bool Activa { get; set; } = true; → declara la propiedad Activa de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 33	<code>public OrdenFabricacion Orden { get; set; } = null!; → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 34	<code>} → cierra el bloque de código actual.</code>
Línea 36	<code>public class Aleacion → declara la clase Aleacion que forma parte del estado acumulativo del ejercicio.</code>
Línea 37	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 38	<code>public int Id { get; set; } → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 39	<code>public string Nombre { get; set; } = string.Empty; → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 40	<code>public string Codigo { get; set; } = string.Empty; → declara la propiedad Codigo de tipo string que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 41	<code>public double PorcentajeCarbono { get; set; } → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 42	<code>public double PorcentajeManganeso { get; set; } → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 43	<code>public string? Descripcion { get; set; } → declara la propiedad Descripcion de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 44	<code>} → cierra el bloque de código actual.</code>
Línea 46	<code>public class EstadoOrden → declara la clase EstadoOrden que forma parte del estado acumulativo del ejercicio.</code>
Línea 47	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 48	<code>public int Id { get; set; } → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 49	<code>public string Nombre { get; set; } = string.Empty; → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 50	<code>public string Descripcion { get; set; } = string.Empty; → declara la propiedad Descripcion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 51	<code>public bool Activo { get; set; } = true; → declara la propiedad Activo de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 52	<code>} → cierra el bloque de código actual.</code>

Línea 54	<code>public class DetalleOrden</code> → declara la clase <code>DetalleOrden</code> que forma parte del estado acumulativo del ejercicio.
Línea 55	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 56	<code>public int Id { get; set; }</code> → declara la propiedad <code>Id</code> de tipo <code>int</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 57	<code>public int OrdenId { get; set; }</code> → declara la propiedad <code>OrdenId</code> de tipo <code>int</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 58	<code>public string ComposicionQuimica { get; set; } = string.Empty;</code> → declara la propiedad <code>ComposicionQuimica</code> de tipo <code>string</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 59	<code>public double TemperaturaColada { get; set; }</code> → declara la propiedad <code>TemperaturaColada</code> de tipo <code>double</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 60	<code>public string? Notas { get; set; }</code> → declara la propiedad <code>Notas</code> de tipo <code>string?</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 61	<code>public OrdenFabricacion Orden { get; set; } = null!;</code> → declara la propiedad <code>Orden</code> de tipo <code>OrdenFabricacion</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 62	<code>}</code> → cierra el bloque de código actual.
Línea 64	<code>public class CertificadoCalidad</code> → declara la clase <code>CertificadoCalidad</code> que forma parte del estado acumulativo del ejercicio.
Línea 65	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 66	<code>public int Id { get; set; }</code> → declara la propiedad <code>Id</code> de tipo <code>int</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 67	<code>public int OrdenId { get; set; }</code> → declara la propiedad <code>OrdenId</code> de tipo <code>int</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 68	<code>public string NumeroCertificado { get; set; } = string.Empty;</code> → declara la propiedad <code>NumeroCertificado</code> de tipo <code>string</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 69	<code>public DateTime FechaEmision { get; set; }</code> → declara la propiedad <code>FechaEmision</code> de tipo <code>DateTime</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 70	<code>public string OrganismoCertificador { get; set; } = string.Empty;</code> → declara la propiedad <code>OrganismoCertificador</code> de tipo <code>string</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 71	<code>public OrdenFabricacion Orden { get; set; } = null!;</code> → declara la propiedad <code>Orden</code> de tipo <code>OrdenFabricacion</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 72	<code>}</code> → cierra el bloque de código actual.
Línea 74	<code>public class AceriaDbContext : DbContext</code> → declara la clase <code>AceriaDbContext</code> que forma parte del estado acumulativo del ejercicio.
Línea 75	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 76	<code>public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }</code> → declara el constructor o método <code>AceriaDbContext</code> con los parámetros necesarios para este componente.
Línea 78	<code>public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 79	<code>public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 80	<code>public DbSet<Aleacion> Aleaciones => Set<Aleacion>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 81	<code>public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 82	<code>public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 83	<code>public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 86	<code>protected override void OnModelCreating(ModelBuilder modelBuilder)</code> → sobrescribe <code>OnModelCreating</code> para definir configuración explícita del modelo.
Línea 87	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 88	<code>base.OnModelCreating(modelBuilder);</code> → sobrescribe <code>OnModelCreating</code> para definir configuración explícita del modelo.
Línea 89	<code>modelBuilder.Entity<OrdenFabricacion>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 90	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 91	<code>entity.ToTable("OrdenesFabricacion");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 92	<code>entity.HasKey(o => o.Id);</code> → define la clave primaria de la entidad.
Línea 93	<code>entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 94	<code>entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 95	<code>entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 96	<code>entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 97	<code>entity.Property(o => o.Observaciones).HasMaxLength(500);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 98	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 99	<code>modelBuilder.Entity<PlanchaAcero>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 100	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 101	<code>entity.ToTable("PlanchasAcero");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 102	<code>entity.HasKey(x => x.Id);</code> → define la clave primaria de la entidad.
Línea 103	<code>entity.Property(x => x.Peso).HasPrecision(18, 3);</code> → selecciona una propiedad escalar para continuar su configuración.

Línea 104	<code>entity.Property(x => x.Activa).HasDefaultValue(true);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 105	<code>entity.HasOne(x => x.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 106	<code>.WithMany(o => o.Planchas)</code> → establece el extremo de colección de una relación uno-a-muchos.
Línea 107	<code>.HasForeignKey(x => x.OrdenId)</code> → declara explícitamente la propiedad o propiedades de clave foránea.
Línea 108	<code>.onDelete(DeleteBehavior.Cascade)</code> → define el comportamiento de eliminación de la relación.
Línea 109	<code>.IsRequired();</code> → marca la propiedad o relación como obligatoria.
Línea 110	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 111	<code>modelBuilder.Entity<Aleacion>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 112	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 113	<code>entity.ToTable("Aleaciones");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 114	<code>entity.HasKey(a => a.Id);</code> → define la clave primaria de la entidad.
Línea 115	<code>entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 116	<code>entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 117	<code>entity.Property(a => a.Descripcion).HasMaxLength(500);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 118	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 119	<code>modelBuilder.Entity<EstadoOrden>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 120	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 121	<code>entity.ToTable("EstadosOrden");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 122	<code>entity.HasKey(e => e.Id);</code> → define la clave primaria de la entidad.
Línea 123	<code>entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 124	<code>entity.Property(e => e.Descripcion).HasMaxLength(250);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 125	<code>entity.Property(e => e.Activo).HasDefaultValue(true);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 126	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 127	<code>modelBuilder.Entity<DetalleOrden>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 128	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 129	<code>entity.ToTable("DetallesOrden");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 130	<code>entity.HasKey(d => d.Id);</code> → define la clave primaria de la entidad.
Línea 131	<code>entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 132	<code>entity.Property(d => d.Notas).HasMaxLength(500);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 133	<code>entity.HasOne(d => d.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 134	<code>.WithOne(o => o.Detalle)</code> → establece el otro extremo de referencia de una relación uno-a-uno.
Línea 135	<code>.HasForeignKey<DetalleOrden>(d => d.OrdenId)</code> → declara explícitamente la propiedad o propiedades de clave foránea.
Línea 136	<code>.OnDelete(DeleteBehavior.Cascade)</code> → define el comportamiento de eliminación de la relación.
Línea 137	<code>.IsRequired();</code> → marca la propiedad o relación como obligatoria.
Línea 138	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 140	<code>modelBuilder.Entity<CertificadoCalidad>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 141	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 142	<code>entity.ToTable("CertificadosCalidad");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 143	<code>entity.HasKey(c => c.Id);</code> → define la clave primaria de la entidad.
Línea 144	<code>entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 145	<code>entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 146	<code>entity.HasOne(c => c.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 147	<code>.WithOne(o => o.Certificado)</code> → establece el otro extremo de referencia de una relación uno-a-uno.
Línea 148	<code>.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)</code> → declara explícitamente la propiedad o propiedades de clave foránea.
Línea 149	<code>.OnDelete(DeleteBehavior.Cascade)</code> → define el comportamiento de eliminación de la relación.
Línea 150	<code>.IsRequired();</code> → marca la propiedad o relación como obligatoria.
Línea 151	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 153	<code>}</code> → cierra el bloque de código actual.
Línea 154	<code>}</code> → cierra el bloque de código actual.
Línea 156	<code>public interface IOrdenRepositorio</code> → declara el contrato IOrdenRepositorio para desacoplar consumidores e implementación.
Línea 157	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 158	<code>List<OrdenFabricacion> ObtenerTodas();</code> → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.
Línea 159	<code>OrdenFabricacion? ObtenerPorId(int id);</code> → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.
Línea 160	<code>OrdenFabricacion? ObtenerPorNumero(string numeroOrden);</code> → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.
Línea 161	<code>void Agregar(OrdenFabricacion orden);</code> → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.
Línea 162	<code>void Eliminar(OrdenFabricacion orden);</code> → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.
Línea 163	<code>int Guardar();</code> → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.
Línea 164	<code>}</code> → cierra el bloque de código actual.
Línea 166	<code>public sealed class OrdenRepositorio : IOrdenRepositorio</code> → declara la clase OrdenRepositorio que forma parte del estado acumulativo del ejercicio.
Línea 167	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 168	<code>private readonly AceriaDbContext _context;</code> → declara el campo privado _context de tipo AceriaDbContext que conserva una dependencia de la clase.
Línea 169	<code>public OrdenRepositorio(AceriaDbContext context) => _context = context;</code> → declara el constructor o método OrdenRepositorio con los parámetros necesarios para este componente.
Línea 171	<code>public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();</code> → ordena el resultado de la consulta antes de materializarlo.
Línea 172	<code>public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);</code> → implementa la búsqueda por clave primaria mediante Find, aprovechando primero el ChangeTracker y después la base de datos.
Línea 173	<code>public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);</code> → recupera la primera coincidencia o null cuando no existe.
Línea 174	<code>public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);</code> → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.
Línea 175	<code>public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);</code> → marca la entidad indicada para borrado al confirmar los cambios.
Línea 176	<code>public int Guardar() => _context.SaveChanges();</code> → persiste en SQL Server los cambios seguidos por el DbContext.
Línea 177	<code>}</code> → cierra el bloque de código actual.
Línea 179	<code>public interface IServicioOrdenes</code> → declara el contrato IServicioOrdenes para desacoplar consumidores e implementación.
Línea 180	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 181	<code>string ObtenerResumen(int ordenId);</code> → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 182	} → cierra el bloque de código actual.
Línea 184	public sealed class ServicioOrdenes : IServicioOrdenes → declara la clase ServicioOrdenes que forma parte del estado acumulativo del ejercicio.
Línea 185	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 186	private readonly IOrdenRepositorio _repositorio; → declara el campo privado _repositorio de tipo IOrdenRepositorio que conserva una dependencia de la clase.
Línea 187	private readonly AceriaDbContext _context; → declara el campo privado _context de tipo AceriaDbContext que conserva una dependencia de la clase.
Línea 188	public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context) → declara el constructor o método ServicioOrdenes con los parámetros necesarios para este componente.
Línea 189	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 190	_repositorio = repositorio; → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.
Línea 191	_context = context; → guarda el DbContext inyectado para consultar los datos necesarios durante el servicio.
Línea 192	} → cierra el bloque de código actual.
Línea 194	public string ObtenerResumen(int ordenId) → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.
Línea 195	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 196	var orden = _repositorio.ObtenerPorId(ordenId); → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.
Línea 197	if (orden is null) return \$"Orden {ordenId} no encontrada"; → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.
Línea 198	var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId); → cuenta las filas que cumplen la consulta construida.
Línea 199	return \$"Orden {orden.NumeroOrden} Cliente: {orden.Cliente} Planchas: {totalPlanchas}"; → devuelve el resultado calculado al código que invocó el método.
Línea 200	} → cierra el bloque de código actual.
Línea 201	} → cierra el bloque de código actual.
Línea 203	public static class Program → declara la clase Program que forma parte del estado acumulativo del ejercicio.
Línea 204	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 205	public static void Main() → declara el punto de entrada ejecutable de la aplicación de consola.
Línea 206	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 207	var configuration = new ConfigurationBuilder() → inicia la construcción de la configuración de la aplicación.
Línea 208	.SetBasePath(AppContext.BaseDirectory) → establece la ruta base usada para localizar los archivos de configuración.

Línea 209	<code>.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false) → incorpora appsettings.json a la configuración.</code>
Línea 210	<code>.AddEnvironmentVariables() → incorpora variables de entorno como fuente adicional de configuración.</code>
Línea 211	<code>.Build(); → materializa la configuración o el objeto encadenado construido en las líneas anteriores.</code>
Línea 213	<code>var connectionString = configuration.GetConnectionString("AceriaDB") → obtiene la cadena de conexión AceriaDB desde la configuración.</code>
Línea 214	<code>?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'."); → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.</code>
Línea 216	<code>var services = new ServiceCollection(); → crea la colección donde se registran las dependencias de la aplicación.</code>
Línea 217	<code>services.AddDbContext<AceriaDbContext>(options => → registra el DbContext en el contenedor de inyección de dependencias.</code>
Línea 218	<code>options → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.</code>
Línea 219	<code>.UseSqlServer(connectionString, sql => → selecciona SQL Server como proveedor de EF Core.</code>
Línea 220	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 221	<code>sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null); → habilita reintentos ante fallos transitorios del proveedor SQL Server.</code>
Línea 222	<code>sql.CommandTimeout(60); → establece el tiempo máximo permitido para los comandos SQL.</code>
Línea 223	<code>}) → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.</code>
Línea 224	<code>.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information) → configura la salida del logging de EF Core.</code>
Línea 225	<code>.EnableDetailedErrors(); → activa errores detallados para facilitar el diagnóstico del laboratorio.</code>
Línea 227	<code>services.AddScoped<IOrdenRepositorio, OrdenRepositorio>(); → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.</code>
Línea 228	<code>services.AddScoped<IServicioOrdenes, ServicioOrdenes>(); → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.</code>
Línea 230	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions → construye el ServiceProvider con validación de servicios y garantiza su liberación al salir del ámbito mediante using var.</code>
Línea 231	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 232	<code>ValidateScopes = true, → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.</code>
Línea 233	<code>ValidateOnBuild = true → obliga al contenedor a validar las resoluciones registradas al construir el ServiceProvider.</code>
Línea 234	<code>}); → cierra la lambda o configuración encadenada y finaliza la llamada.</code>

Línea 236	<code>using var scope = provider.CreateScope();</code> → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.
Línea 237	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → resuelve un servicio obligatorio desde el contenedor de dependencias.
Línea 238	<code>context.Database.EnsureDeleted();</code> → elimina la base del laboratorio para reproducir el escenario desde un estado limpio.
Línea 239	<code>context.Database.Migrate();</code> → aplica las migraciones pendientes y deja AceriaDB en el esquema del punto.
Línea 241	<code>var orden = new OrdenFabricacion</code> → crea la entidad OrdenFabricacion que concentra el grafo de datos del escenario acumulativo.
Línea 242	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 243	<code>NumeroOrden = "OF-M2-0001",</code> → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.
Línea 244	<code>Cliente = "Constructora del Norte",</code> → asigna el cliente de la orden utilizada en el escenario E2E del punto.
Línea 245	<code>FechaCreacion = DateTime.UtcNow,</code> → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.
Línea 246	<code>FechaEntrega = DateTime.Today.AddDays(14),</code> → establece una fecha de entrega prevista catorce días después de la fecha local actual.
Línea 247	<code>Estado = "Pendiente",</code> → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.
Línea 248	<code>Observaciones = "Orden de validación M2",</code> → añade texto opcional para comprobar el mapeo de Observaciones.
Línea 249	<code>Planchas =</code> → inicia el inicializador de la colección de planchas asociadas a la orden.
Línea 250	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 251	<code>new PlanchaAcero</code> → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.
Línea 252	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 253	<code>Espesor = 10.5,</code> → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.
Línea 254	<code>Ancho = 1500,</code> → asigna el ancho de la plancha usada en el escenario de persistencia.
Línea 255	<code>Largo = 3000,</code> → asigna el largo de la plancha usada en el escenario de persistencia.
Línea 256	<code>Peso = 371.250m,</code> → asigna un peso decimal con tres posiciones para validar la precisión configurada.
Línea 257	<code>Activa = true,</code> → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.
Línea 258	<code>}</code> → cierra el bloque de código actual.
Línea 259	<code>},</code> → continúa el inicializador o la llamada iniciada en el bloque actual.
Línea 260	<code>Detalle = new DetalleOrden</code> → crea el detalle uno-a-uno que se persistirá junto con la orden principal.

Línea 261	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 262	ComposicionQuimica = "C: 0.45%, Mn: 0.75%", → asigna la composición química utilizada para verificar la entidad DetalleOrden.
Línea 263	TemperaturaColada = 1550.5, → asigna la temperatura de colada del detalle de fabricación.
Línea 264	Notas = "Colada principal" → asigna las notas opcionales del detalle asociado a la orden.
Línea 265	}, → continúa el inicializador o la llamada iniciada en el bloque actual.
Línea 266	Certificado = new CertificadoCalidad → crea el certificado uno-a-uno asociado a la orden del escenario.
Línea 267	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 268	NumeroCertificado = "CERT-0001", → asigna el identificador natural del certificado de calidad.
Línea 269	FechaEmision = DateTime.Today, → asigna la fecha de emisión del certificado usado en la prueba.
Línea 270	OrganismoCertificador = "Laboratorio Aceria" → asigna el organismo que emite el certificado de calidad.
Línea 271	}, → continúa el inicializador o la llamada iniciada en el bloque actual.
Línea 272	}; → cierra el inicializador del objeto y finaliza la instrucción.
Línea 274	context.OrdenesFabricacion.Add(orden); → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.
Línea 275	context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true }); → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.
Línea 276	context.SaveChanges(); → persiste en SQL Server los cambios seguidos por el DbContext.
Línea 278	var cargada = context.OrdenesFabricacion.Include(o => o.Detalle).Include(o => o.Certificado).Single(o => o.NumeroOrden == "OF-M2-0001"); → incluye de forma eager la navegación relacionada en la consulta.
Línea 279	global::System.Console.WriteLine(\$"2.4 OK Detalle: {cargada.Detalle?.ComposicionQuimica} Certificado: {cargada.Certificado?.NumeroCertificado}"); → escribe en consola la evidencia usada para verificar el comportamiento del estado.
Línea 280	} → cierra el bloque de código actual.
Línea 281	} → cierra el bloque de código actual.

Punto 2.5 - Relaciones muchos a muchos

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL que trabajan con .NET 8, EF Core 8 y SQL Server LocalDB.

Proyecto: El estado 2.5 parte de 2.4 y tiene como objetivo incorporar una entidad intermedia explícita OrdenAleacion con datos propios.

Objetivos de aprendizaje

- Distinguir many-to-many implícito de entidad intermedia explícita.
- Crear OrdenAleacion.

- Configurar dos relaciones uno-a-muchos.
- Definir una clave primaria compuesta.
- Añadir datos de la relación.
- Cargar la relación con Include/ThenInclude.

Paso 1: Abrir la solución autónoma del punto

```
cd M02/PROYECTO/2.5
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

No se abre una solución global situada en la raíz del repositorio. `AceriaData.sln` está dentro de la carpeta del punto y referencia únicamente los proyectos de ese estado.

Paso 2: Identificar el cambio respecto al estado anterior

El proyecto conserva todo lo terminado en 2.4. En 2.5 se introduce exclusivamente el contenido que corresponde a **Relaciones muchos a muchos**. La práctica no reinicia `AceriaData` ni crea un ejemplo paralelo.

Paso 3: Implementar y comprender la configuración principal

```
modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.ToTable("OrdenesAleaciones");
    entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });

    entity.HasOne(x => x.Orden)
        .WithMany(o => o.OrdenesAleaciones)
        .HasForeignKey(x => x.OrdenFabricacionId)
        .OnDelete(DeleteBehavior.Cascade);

    entity.HasOne(x => x.Aleacion)
        .WithMany(a => a.OrdenesAleaciones)
        .HasForeignKey(x => x.AleacionId)
        .OnDelete(DeleteBehavior.Restrict);
});
```

CSHARP

Línea 1	<code>HasKey(x => new { x.OrdenFabricacionId, x.AleacionId })</code> → define la clave primaria compuesta de la entidad puente.
Línea 2	<code>WithMany(o => o.OrdenesAleaciones)</code> → modela una de las dos relaciones uno-a-muchos que forman el muchos-a-muchos.
Línea 3	<code>DeleteBehavior.Cascade</code> → elimina las filas puente cuando desaparece la orden.
Línea 4	<code>DeleteBehavior.Restrict</code> → evita eliminar una aleación mientras siga referenciada.
Línea 5	<code>CantidadUtilizada</code> → demuestra por qué se usa una entidad puente explícita: la relación tiene datos propios.

Paso 4: Generar y aplicar la migración acumulativa

El repositorio contiene una migración real generada para este estado. Si se reproduce el ejercicio desde el estado anterior, los comandos son:

```
dotnet ef migrations add M2_2_5
dotnet ef database update
```

POWERSHELL

`dotnet ef migrations add` compara el modelo actual con el snapshot heredado. `dotnet ef database update` aplica únicamente los cambios pendientes.

En los estados ejecutables del curso se usa `Database.Migrate()` para aplicar el historial durante la demostración. El laboratorio puede usar `EnsureDeleted()` para empezar desde una base limpia, pero **no usa** `EnsureCreated()`, porque el esquema está gobernado por Migrations.

Paso 5: Compilar y ejecutar el estado

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

POWERSHELL

Resultado esperado: la ejecución termina sin excepción y contiene la evidencia `2.5 OK`. GitHub Actions repite esta compilación y ejecución sobre Windows y SQL Server LocalDB.

Paso 6: Verificar el estado acumulativo

Además de ejecutar el ejemplo, conviene revisar el modelo y la base:

```
dotnet ef migrations list
```

POWERSHELL

Cuando el punto modifica el esquema, la lista debe contener la migración acumulativa correspondiente. En SQL Server Object Explorer se puede comprobar que las tablas, claves, relaciones, índices o restricciones coinciden con el modelo del punto.

Errores comunes

Error	Causa	Solución
Clave duplicada	Se repite el par OrdenFabricacionId/ AleacionId	La PK compuesta impide duplicados.
Borrado de Aleacion falla	DeleteBehavior.Restrict protege referencias	Eliminar primero las relaciones o cambiar conscientemente la estrategia.
Se usa tabla puente implícita	La relación tiene propiedades propias	Usar OrdenAleacion explícita.

Reto resuelto

Reto: Asignar dos aleaciones distintas a una orden, cada una con CantidadUtilizada diferente, y recuperarlas con ThenInclude.

Solución: partir del código de `M02/PROYECTO/2.5`, realizar únicamente el cambio descrito y volver a ejecutar build, migraciones y el programa. El reto no introduce conceptos reservados a un punto posterior.

Analogía final

OrdenAleacion funciona como una hoja de asignación: une dos elementos y además registra datos propios de esa asignación.

Resultado esperado

Al terminar 2.5, la solución local compila, el proyecto se ejecuta, el esquema se actualiza mediante Migrations cuando corresponde y la salida contiene `2.5 OK`. El código conserva todos los cambios válidos de los puntos anteriores.

Conexión con el siguiente punto

El siguiente estado es `2.6`. Se parte del proyecto completo de 2.5; no se vuelve a crear AceriaData desde cero.

Código acumulativo completo del estado 2.5

El siguiente archivo es el `Program.cs` real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
```

CSHARP

```

using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public stringCodigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    public string? Notas { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class CertificadoCalidad
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    public string OrganismoCertificador { get; set; } = string.Empty;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    public decimal CantidadUtilizada { get; set; }
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }
}

```

```

public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
    modelBuilder.Entity<OrdenFabricacion>(entity =>
    {
        entity.ToTable("OrdenesFabricacion");
        entity.HasKey(o => o.Id);
        entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
        entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
        entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
        entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
        entity.Property(o => o.Observaciones).HasMaxLength(500);
    });
    modelBuilder.Entity<PlanchaAcero>(entity =>
    {
        entity.ToTable("PlanchasAcero");
        entity.HasKey(x => x.Id);
        entity.Property(x => x.Peso).HasPrecision(18, 3);
        entity.Property(x => x.Activa).HasDefaultValue(true);
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.Planchas)
            .HasForeignKey(x => x.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<Aleacion>(entity =>
    {
        entity.ToTable("Aleaciones");
        entity.HasKey(a => a.Id);
        entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
        entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
        entity.Property(a => a.Descripcion).HasMaxLength(500);
    });
    modelBuilder.Entity<EstadoOrden>(entity =>
    {
        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
        entity.Property(e => e.Descripcion).HasMaxLength(250);
        entity.Property(e => e.Activo).HasDefaultValue(true);
    });
    modelBuilder.Entity<DetalleOrden>(entity =>
    {
        entity.ToTable("DetallesOrden");
        entity.HasKey(d => d.Id);
        entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
        entity.Property(d => d.Notas).HasMaxLength(500);
        entity.HasOne(d => d.Orden)
            .WithOne(o => o.Detalle)
            .HasForeignKey<DetalleOrden>(d => d.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<CertificadoCalidad>(entity =>
    {
        entity.ToTable("CertificadosCalidad");
        entity.HasKey(c => c.Id);
        entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
        entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
        entity.HasOne(c => c.Orden)
            .WithOne(o => o.Certificado)
            .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<OrdenAleacion>(entity =>
    {
        entity.ToTable("OrdenesAleaciones");
        entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
        entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
        entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.OrdenesAleaciones)
            .HasForeignKey(x => x.OrdenFabricacionId)

```

```

        .OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(x => x.Aleacion)
            .WithMany(a => a.OrdenesAleaciones)
            .HasForeignKey(x => x.AleacionId)
            .OnDelete(DeleteBehavior.Restrict);
    });
}

}

public interface IOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                    {
                        sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                        sql.CommandTimeout(60);
                    })
                .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();
    }
}

```

```

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },
    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});
context.OrdenesFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo =
true });
context.SaveChanges();

var cargada = context.OrdenesFabricacion.Include(o => o.OrdenesAleaciones).ThenInclude(x =>
x.Aleacion).Single(o => o.NumeroOrden == "OF-M2-0001");
global::System.Console.WriteLine($"2.5 OK | Aleaciones: {cargada.OrdenesAleaciones.Count} | Primera:
{cargada.OrdenesAleaciones[0].Aleacion.Codigo}");
}
}

```

Explicación línea a línea del código acumulativo 2.5

Las referencias siguientes usan la numeración real de M02/PROYECTO/2.5/Program.cs. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1	using System.ComponentModel.DataAnnotations; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 2	using System.ComponentModel.DataAnnotations.Schema; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 3	<code>using Microsoft.EntityFrameworkCore;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 4	<code>using Microsoft.Extensions.Configuration;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 5	<code>using Microsoft.Extensions.DependencyInjection;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 6	<code>using Microsoft.Extensions.Logging;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 8	<code>namespace AceriaData.ConsoleApp;</code> → declara el espacio de nombres del estado acumulativo.
Línea 10	<code>public class OrdenFabricacion</code> → declara la clase <code>OrdenFabricacion</code> que forma parte del estado acumulativo del ejercicio.
Línea 11	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 12	<code>public int Id { get; set; }</code> → declara la propiedad <code>Id</code> de tipo <code>int</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 13	<code>public string NumeroOrden { get; set; } = string.Empty;</code> → declara la propiedad <code>NumeroOrden</code> de tipo <code>string</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 14	<code>public string Cliente { get; set; } = string.Empty;</code> → declara la propiedad <code>Cliente</code> de tipo <code>string</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 15	<code>public DateTime FechaCreacion { get; set; }</code> → declara la propiedad <code>FechaCreacion</code> de tipo <code>DateTime</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 16	<code>public DateTime? FechaEntrega { get; set; }</code> → declara la propiedad <code>FechaEntrega</code> de tipo <code>DateTime?</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 17	<code>public string Estado { get; set; } = "Pendiente";</code> → declara la propiedad <code>Estado</code> de tipo <code>string</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 18	<code>public string? Observaciones { get; set; }</code> → declara la propiedad <code>Observaciones</code> de tipo <code>string?</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 19	<code>public List<PlanchaAcero> Planchas { get; set; } = new();</code> → declara la propiedad <code>Planchas</code> de tipo <code>List<PlanchaAcero></code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 20	<code>public DetalleOrden? Detalle { get; set; }</code> → declara la propiedad <code>Detalle</code> de tipo <code>DetalleOrden?</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 21	<code>public CertificadoCalidad? Certificado { get; set; }</code> → declara la propiedad <code>Certificado</code> de tipo <code>CertificadoCalidad?</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 22	<code>public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();</code> → declara la propiedad <code>OrdenesAleaciones</code> de tipo <code>List<OrdenAleacion></code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 23	<code>}</code> → cierra el bloque de código actual.
Línea 25	<code>public class PlanchaAcero</code> → declara la clase <code>PlanchaAcero</code> que forma parte del estado acumulativo del ejercicio.
Línea 26	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 27	<code>public int Id { get; set; }</code> → declara la propiedad <code>Id</code> de tipo <code>int</code> que EF Core o la lógica de negocio utiliza en este modelo.

Línea 28	<code>public int OrdenId { get; set; } → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 29	<code>public double Espesor { get; set; } → declara la propiedad Espesor de tipo double que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 30	<code>public double Ancho { get; set; } → declara la propiedad Ancho de tipo double que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 31	<code>public double Largo { get; set; } → declara la propiedad Largo de tipo double que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 32	<code>public decimal Peso { get; set; } → declara la propiedad Peso de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 33	<code>public bool Activa { get; set; } = true; → declara la propiedad Activa de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 34	<code>public OrdenFabricacion Orden { get; set; } = null!; → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 35	<code>} → cierra el bloque de código actual.</code>
Línea 37	<code>public class Aleacion → declara la clase Aleacion que forma parte del estado acumulativo del ejercicio.</code>
Línea 38	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 39	<code>public int Id { get; set; } → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 40	<code>public string Nombre { get; set; } = string.Empty; → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 41	<code>public string Codigo { get; set; } = string.Empty; → declara la propiedad Codigo de tipo string que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 42	<code>public double PorcentajeCarbono { get; set; } → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 43	<code>public double PorcentajeManganeso { get; set; } → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 44	<code>public string? Descripcion { get; set; } → declara la propiedad Descripcion de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 45	<code>public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new(); → declara la propiedad OrdenesAleaciones de tipo List<OrdenAleacion> que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 46	<code>} → cierra el bloque de código actual.</code>
Línea 48	<code>public class EstadoOrden → declara la clase EstadoOrden que forma parte del estado acumulativo del ejercicio.</code>
Línea 49	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 50	<code>public int Id { get; set; } → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 51	<code>public string Nombre { get; set; } = string.Empty; → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.</code>

Línea 52	<code>public string Descripcion { get; set; } = string.Empty; → declara la propiedad Descripcion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 53	<code>public bool Activo { get; set; } = true; → declara la propiedad Activo de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 54	<code>}</code> → cierra el bloque de código actual.
Línea 56	<code>public class DetalleOrden → declara la clase DetalleOrden que forma parte del estado acumulativo del ejercicio.</code>
Línea 57	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 58	<code>public int Id { get; set; } → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 59	<code>public int OrdenId { get; set; } → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 60	<code>public string ComposicionQuimica { get; set; } = string.Empty; → declara la propiedad ComposicionQuimica de tipo string que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 61	<code>public double TemperaturaColada { get; set; } → declara la propiedad TemperaturaColada de tipo double que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 62	<code>public string? Notas { get; set; } → declara la propiedad Notas de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 63	<code>public OrdenFabricacion Orden { get; set; } = null!; → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 64	<code>}</code> → cierra el bloque de código actual.
Línea 66	<code>public class CertificadoCalidad → declara la clase CertificadoCalidad que forma parte del estado acumulativo del ejercicio.</code>
Línea 67	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 68	<code>public int Id { get; set; } → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 69	<code>public int OrdenId { get; set; } → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 70	<code>public string NumeroCertificado { get; set; } = string.Empty; → declara la propiedad NumeroCertificado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 71	<code>public DateTime FechaEmision { get; set; } → declara la propiedad FechaEmision de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 72	<code>public string OrganismoCertificador { get; set; } = string.Empty; → declara la propiedad OrganismoCertificador de tipo string que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 73	<code>public OrdenFabricacion Orden { get; set; } = null!; → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 74	<code>}</code> → cierra el bloque de código actual.
Línea 76	<code>public class OrdenAleacion → declara la clase OrdenAleacion que forma parte del estado acumulativo del ejercicio.</code>
Línea 77	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 78	<code>public int OrdenFabricacionId { get; set; } → declara la propiedad OrdenFabricacionId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 79	<code>public int AleacionId { get; set; } → declara la propiedad AleacionId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 80	<code>public DateTime FechaAsignacion { get; set; } = DateTime.Now; → declara la propiedad FechaAsignacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 81	<code>public decimal CantidadUtilizada { get; set; } → declara la propiedad CantidadUtilizada de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 82	<code>public string EstadoRelacion { get; set; } = "Activa"; → declara la propiedad EstadoRelacion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 83	<code>public OrdenFabricacion Orden { get; set; } = null!; → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 84	<code>public Aleacion Aleacion { get; set; } = null!; → declara la propiedad Aleacion de tipo Aleacion que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 85	<code>} → cierra el bloque de código actual.</code>
Línea 87	<code>public class AceriaDbContext : DbContext → declara la clase AceriaDbContext que forma parte del estado acumulativo del ejercicio.</code>
Línea 88	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 89	<code>public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { } → declara el constructor o método AceriaDbContext con los parámetros necesarios para este componente.</code>
Línea 91	<code>public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>(); → expone un conjunto de entidades para consultar y persistir mediante EF Core.</code>
Línea 92	<code>public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>(); → expone un conjunto de entidades para consultar y persistir mediante EF Core.</code>
Línea 93	<code>public DbSet<Aleacion> Aleaciones => Set<Aleacion>(); → expone un conjunto de entidades para consultar y persistir mediante EF Core.</code>
Línea 94	<code>public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>(); → expone un conjunto de entidades para consultar y persistir mediante EF Core.</code>
Línea 95	<code>public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>(); → expone un conjunto de entidades para consultar y persistir mediante EF Core.</code>
Línea 96	<code>public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>(); → expone un conjunto de entidades para consultar y persistir mediante EF Core.</code>
Línea 97	<code>public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>(); → expone un conjunto de entidades para consultar y persistir mediante EF Core.</code>
Línea 100	<code>protected override void OnModelCreating(ModelBuilder modelBuilder) → sobrescribe OnModelCreating para definir configuración explícita del modelo.</code>
Línea 101	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 102	<code>base.OnModelCreating(modelBuilder); → sobrescribe OnModelCreating para definir configuración explícita del modelo.</code>
Línea 103	<code>modelBuilder.Entity<OrdenFabricacion>(entity => → selecciona una entidad del modelo para aplicar su configuración Fluent API.</code>

Línea 104	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 105	entity.ToTable("OrdenesFabricacion"); → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 106	entity.HasKey(o => o.Id); → define la clave primaria de la entidad.
Línea 107	entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50); → selecciona una propiedad escalar para continuar su configuración.
Línea 108	entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200); → selecciona una propiedad escalar para continuar su configuración.
Línea 109	entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()"); → selecciona una propiedad escalar para continuar su configuración.
Línea 110	entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente"); → selecciona una propiedad escalar para continuar su configuración.
Línea 111	entity.Property(o => o.Observaciones).HasMaxLength(500); → selecciona una propiedad escalar para continuar su configuración.
Línea 112	}); → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 113	modelBuilder.Entity<PlanchaAcero>(entity => → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 114	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 115	entity.ToTable("PlanchasAcero"); → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 116	entity.HasKey(x => x.Id); → define la clave primaria de la entidad.
Línea 117	entity.Property(x => x.Peso).HasPrecision(18, 3); → selecciona una propiedad escalar para continuar su configuración.
Línea 118	entity.Property(x => x.Activa).HasDefaultValue(true); → selecciona una propiedad escalar para continuar su configuración.
Línea 119	entity.HasOne(x => x.Orden) → inicia la configuración de una navegación de referencia.
Línea 120	.WithMany(o => o.Planchas) → establece el extremo de colección de una relación uno-a-muchos.
Línea 121	.HasForeignKey(x => x.OrdenId) → declara explícitamente la propiedad o propiedades de clave foránea.
Línea 122	.OnDelete(DeleteBehavior.Cascade) → define el comportamiento de eliminación de la relación.
Línea 123	.IsRequired(); → marca la propiedad o relación como obligatoria.
Línea 124	}); → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 125	modelBuilder.Entity<Aleacion>(entity => → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 126	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 127	entity.ToTable("Aleaciones"); → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 128	<code>entity.HasKey(a => a.Id);</code> → define la clave primaria de la entidad.
Línea 129	<code>entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 130	<code>entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 131	<code>entity.Property(a => a.Descripcion).HasMaxLength(500);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 132	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 133	<code>modelBuilder.Entity<EstadoOrden>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 134	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 135	<code>entity.ToTable("EstadosOrden");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 136	<code>entity.HasKey(e => e.Id);</code> → define la clave primaria de la entidad.
Línea 137	<code>entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 138	<code>entity.Property(e => e.Descripcion).HasMaxLength(250);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 139	<code>entity.Property(e => e.Activo).HasDefaultValue(true);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 140	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 141	<code>modelBuilder.Entity<DetalleOrden>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 142	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 143	<code>entity.ToTable("DetallesOrden");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 144	<code>entity.HasKey(d => d.Id);</code> → define la clave primaria de la entidad.
Línea 145	<code>entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 146	<code>entity.Property(d => d.Notas).HasMaxLength(500);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 147	<code>entity.HasOne(d => d.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 148	<code>.WithOne(o => o.Detalle)</code> → establece el otro extremo de referencia de una relación uno-a-uno.
Línea 149	<code>.HasForeignKey<DetalleOrden>(d => d.OrdenId)</code> → declara explícitamente la propiedad o propiedades de clave foránea.
Línea 150	<code>.OnDelete(DeleteBehavior.Cascade)</code> → define el comportamiento de eliminación de la relación.
Línea 151	<code>.IsRequired();</code> → marca la propiedad o relación como obligatoria.
Línea 152	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 154	<code>modelBuilder.Entity<CertificadoCalidad>(entity => → selecciona una entidad del modelo para aplicar su configuración Fluent API.</code>
Línea 155	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 156	<code>entity.ToTable("CertificadosCalidad"); → asigna explícitamente la tabla de SQL Server usada por la entidad.</code>
Línea 157	<code>entity.HasKey(c => c.Id); → define la clave primaria de la entidad.</code>
Línea 158	<code>entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50); → selecciona una propiedad escalar para continuar su configuración.</code>
Línea 159	<code>entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100); → selecciona una propiedad escalar para continuar su configuración.</code>
Línea 160	<code>entity.HasOne(c => c.Orden) → inicia la configuración de una navegación de referencia.</code>
Línea 161	<code>.WithOne(o => o.Certificado) → establece el otro extremo de referencia de una relación uno-a-uno.</code>
Línea 162	<code>.HasForeignKey<CertificadoCalidad>(c => c.OrdenId) → declara explícitamente la propiedad o propiedades de clave foránea.</code>
Línea 163	<code>.OnDelete(DeleteBehavior.Cascade) → define el comportamiento de eliminación de la relación.</code>
Línea 164	<code>.IsRequired(); → marca la propiedad o relación como obligatoria.</code>
Línea 165	<code>}); → cierra la lambda o configuración encadenada y finaliza la llamada.</code>
Línea 166	<code>modelBuilder.Entity<OrdenAleacion>(entity => → selecciona una entidad del modelo para aplicar su configuración Fluent API.</code>
Línea 167	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 168	<code>entity.ToTable("OrdenesAleaciones"); → asigna explícitamente la tabla de SQL Server usada por la entidad.</code>
Línea 169	<code>entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId }); → define la clave primaria de la entidad.</code>
Línea 170	<code>entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()"); → selecciona una propiedad escalar para continuar su configuración.</code>
Línea 171	<code>entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3); → selecciona una propiedad escalar para continuar su configuración.</code>
Línea 172	<code>entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa"); → selecciona una propiedad escalar para continuar su configuración.</code>
Línea 173	<code>entity.HasOne(x => x.Orden) → inicia la configuración de una navegación de referencia.</code>
Línea 174	<code>.WithMany(o => o.OrdenesAleaciones) → establece el extremo de colección de una relación uno-a-muchos.</code>
Línea 175	<code>.HasForeignKey(x => x.OrdenFabricacionId) → declara explícitamente la propiedad o propiedades de clave foránea.</code>
Línea 176	<code>.OnDelete(DeleteBehavior.Cascade); → define el comportamiento de eliminación de la relación.</code>
Línea 177	<code>entity.HasOne(x => x.Aleacion) → inicia la configuración de una navegación de referencia.</code>

Línea 178	<code>.WithMany(a => a.OrdenesAleaciones) → establece el extremo de colección de una relación uno-a-muchos.</code>
Línea 179	<code>.HasForeignKey(x => x.AleacionId) → declara explícitamente la propiedad o propiedades de clave foránea.</code>
Línea 180	<code>.onDelete(DeleteBehavior.Restrict); → define el comportamiento de eliminación de la relación.</code>
Línea 181	<code>}); → cierra la lambda o configuración encadenada y finaliza la llamada.</code>
Línea 182	<code>} → cierra el bloque de código actual.</code>
Línea 183	<code>} → cierra el bloque de código actual.</code>
Línea 185	<code>public interface IOrdenRepositorio → declara el contrato IOrdenRepositorio para desacoplar consumidores e implementación.</code>
Línea 186	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 187	<code>List<OrdenFabricacion> ObtenerTodas(); → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.</code>
Línea 188	<code>OrdenFabricacion? ObtenerPorId(int id); → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.</code>
Línea 189	<code>OrdenFabricacion? ObtenerPorNumero(string numeroOrden); → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.</code>
Línea 190	<code>void Agregar(OrdenFabricacion orden); → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.</code>
Línea 191	<code>void Eliminar(OrdenFabricacion orden); → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.</code>
Línea 192	<code>int Guardar(); → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.</code>
Línea 193	<code>} → cierra el bloque de código actual.</code>
Línea 195	<code>public sealed class OrdenRepositorio : IOrdenRepositorio → declara la clase OrdenRepositorio que forma parte del estado acumulativo del ejercicio.</code>
Línea 196	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 197	<code>private readonly AceriaDbContext _context; → declara el campo privado _context de tipo AceriaDbContext que conserva una dependencia de la clase.</code>
Línea 198	<code>public OrdenRepositorio(AceriaDbContext context) => _context = context; → declara el constructor o método OrdenRepositorio con los parámetros necesarios para este componente.</code>
Línea 200	<code>public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList(); → ordena el resultado de la consulta antes de materializarlo.</code>
Línea 201	<code>public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id); → implementa la búsqueda por clave primaria mediante Find, aprovechando primero el ChangeTracker y después la base de datos.</code>
Línea 202	<code>public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden); → recupera la primera coincidencia o null cuando no existe.</code>

Línea 203	<code>public void Agregar(OrdenFabricacion orden) =></code> <code>_context.OrdenesFabricacion.Add(orden);</code> → registra una entidad o relación nueva en el <code>ChangeTracker</code> para insertarla al guardar.
Línea 204	<code>public void Eliminar(OrdenFabricacion orden) =></code> <code>_context.OrdenesFabricacion.Remove(orden);</code> → marca la entidad indicada para borrado al confirmar los cambios.
Línea 205	<code>public int Guardar() => _context.SaveChanges();</code> → persiste en SQL Server los cambios seguidos por el <code>DbContext</code> .
Línea 206	<code>}</code> → cierra el bloque de código actual.
Línea 208	<code>public interface IServicioOrdenes</code> → declara el contrato <code>IServicioOrdenes</code> para desacoplar consumidores e implementación.
Línea 209	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 210	<code>string ObtenerResumen(int ordenId);</code> → declara el servicio que construye un resumen de una orden a partir de su identificador.
Línea 211	<code>}</code> → cierra el bloque de código actual.
Línea 213	<code>public sealed class ServicioOrdenes : IServicioOrdenes</code> → declara la clase <code>ServicioOrdenes</code> que forma parte del estado acumulativo del ejercicio.
Línea 214	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 215	<code>private readonly IOrdenRepositorio _repositorio;</code> → declara el campo privado <code>_repositorio</code> de tipo <code>IOrdenRepositorio</code> que conserva una dependencia de la clase.
Línea 216	<code>private readonly AceriaDbContext _context;</code> → declara el campo privado <code>_context</code> de tipo <code>AceriaDbContext</code> que conserva una dependencia de la clase.
Línea 217	<code>public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)</code> → declara el constructor o método <code>ServicioOrdenes</code> con los parámetros necesarios para este componente.
Línea 218	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 219	<code>_repositorio = repositorio;</code> → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.
Línea 220	<code>_context = context;</code> → guarda el <code>DbContext</code> inyectado para consultar los datos necesarios durante el servicio.
Línea 221	<code>}</code> → cierra el bloque de código actual.
Línea 223	<code>public string ObtenerResumen(int ordenId)</code> → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.
Línea 224	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 225	<code>var orden = _repositorio.ObtenerPorId(ordenId);</code> → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.
Línea 226	<code>if (orden is null) return \$"Orden {ordenId} no encontrada";</code> → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.
Línea 227	<code>var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);</code> → cuenta las filas que cumplen la consulta construida.
Línea 228	<code>return \$"Orden {orden.NumeroOrden} Cliente: {orden.Cliente} Planchas: {totalPlanchas}";</code> → devuelve el resultado calculado al código que invocó el método.

Línea 229	} → cierra el bloque de código actual.
Línea 230	} → cierra el bloque de código actual.
Línea 232	public static class Program → declara la clase Program que forma parte del estado acumulativo del ejercicio.
Línea 233	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 234	public static void Main() → declara el punto de entrada ejecutable de la aplicación de consola.
Línea 235	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 236	var configuration = new ConfigurationBuilder() → inicia la construcción de la configuración de la aplicación.
Línea 237	.SetBasePath(AppContext.BaseDirectory) → establece la ruta base usada para localizar los archivos de configuración.
Línea 238	.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false) → incorpora appsettings.json a la configuración.
Línea 239	.AddEnvironmentVariables() → incorpora variables de entorno como fuente adicional de configuración.
Línea 240	.Build(); → materializa la configuración o el objeto encadenado construido en las líneas anteriores.
Línea 242	var connectionString = configuration.GetConnectionString("AceriaDB") → obtiene la cadena de conexión AceriaDB desde la configuración.
Línea 243	?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'."); → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.
Línea 245	var services = new ServiceCollection(); → crea la colección donde se registran las dependencias de la aplicación.
Línea 246	services.AddDbContext<AceriaDbContext>(options => → registra el DbContext en el contenedor de inyección de dependencias.
Línea 247	options → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.
Línea 248	.UseSqlServer(connectionString, sql => → selecciona SQL Server como proveedor de EF Core.
Línea 249	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 250	sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null); → habilita reintentos ante fallos transitorios del proveedor SQL Server.
Línea 251	sql.CommandTimeout(60); → establece el tiempo máximo permitido para los comandos SQL.
Línea 252	}) → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.
Línea 253	.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information) → configura la salida del logging de EF Core.
Línea 254	.EnableDetailedErrors(); → activa errores detallados para facilitar el diagnóstico del laboratorio.

Línea 256	<code>services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();</code> → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.
Línea 257	<code>services.AddScoped<IServicioOrdenes, ServicioOrdenes>();</code> → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.
Línea 259	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions</code> → construye el <code>ServiceProvider</code> con validación de servicios y garantiza su liberación al salir del ámbito mediante <code>using var</code> .
Línea 260	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 261	<code>ValidateScopes = true,</code> → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.
Línea 262	<code>ValidateOnBuild = true</code> → obliga al contenedor a validar las resoluciones registradas al construir el <code>ServiceProvider</code> .
Línea 263	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 265	<code>using var scope = provider.CreateScope();</code> → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.
Línea 266	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → resuelve un servicio obligatorio desde el contenedor de dependencias.
Línea 267	<code>context.Database.EnsureDeleted();</code> → elimina la base del laboratorio para reproducir el escenario desde un estado limpio.
Línea 268	<code>context.Database.Migrate();</code> → aplica las migraciones pendientes y deja <code>AceriaDB</code> en el esquema del punto.
Línea 270	<code>var orden = new OrdenFabricacion</code> → crea la entidad <code>OrdenFabricacion</code> que concentra el grafo de datos del escenario acumulativo.
Línea 271	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 272	<code>NumeroOrden = "OF-M2-0001",</code> → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.
Línea 273	<code>Cliente = "Constructora del Norte",</code> → asigna el cliente de la orden utilizada en el escenario E2E del punto.
Línea 274	<code>FechaCreacion = DateTime.UtcNow,</code> → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.
Línea 275	<code>FechaEntrega = DateTime.Today.AddDays(14),</code> → establece una fecha de entrega prevista catorce días después de la fecha local actual.
Línea 276	<code>Estado = "Pendiente",</code> → inicializa la orden con el estado <code>Pendiente</code> , coherente con el flujo del laboratorio.
Línea 277	<code>Observaciones = "Orden de validación M2",</code> → añade texto opcional para comprobar el mapeo de <code>Observaciones</code> .
Línea 278	<code>Planchas =</code> → inicia el inicializador de la colección de planchas asociadas a la orden.
Línea 279	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 280	<code>new PlanchaAcero</code> → crea una nueva plancha dentro del inicializador de la colección <code>Planchas</code> de la orden.
Línea 281	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 282	Espesor = 10.5, → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.
Línea 283	Ancho = 1500, → asigna el ancho de la plancha usada en el escenario de persistencia.
Línea 284	Largo = 3000, → asigna el largo de la plancha usada en el escenario de persistencia.
Línea 285	Peso = 371.250m, → asigna un peso decimal con tres posiciones para validar la precisión configurada.
Línea 286	Activa = true, → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.
Línea 287	} → cierra el bloque de código actual.
Línea 288	}, → continúa el inicializador o la llamada iniciada en el bloque actual.
Línea 289	Detalle = new DetalleOrden → crea el detalle uno-a-uno que se persistirá junto con la orden principal.
Línea 290	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 291	ComposicionQuimica = "C: 0.45%, Mn: 0.75%", → asigna la composición química utilizada para verificar la entidad DetalleOrden.
Línea 292	TemperaturaColada = 1550.5, → asigna la temperatura de colada del detalle de fabricación.
Línea 293	Notas = "Colada principal" → asigna las notas opcionales del detalle asociado a la orden.
Línea 294	}, → continúa el inicializador o la llamada iniciada en el bloque actual.
Línea 295	Certificado = new CertificadoCalidad → crea el certificado uno-a-uno asociado a la orden del escenario.
Línea 296	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 297	NumeroCertificado = "CERT-0001", → asigna el identificador natural del certificado de calidad.
Línea 298	FechaEmision = DateTime.Today, → asigna la fecha de emisión del certificado usado en la prueba.
Línea 299	OrganismoCertificador = "Laboratorio Aceria" → asigna el organismo que emite el certificado de calidad.
Línea 300	}, → continúa el inicializador o la llamada iniciada en el bloque actual.
Línea 301	}; → cierra el inicializador del objeto y finaliza la instrucción.
Línea 303	var aleacion = new Aleacion → crea la aleación que se enlazará con la orden mediante la entidad intermedia OrdenAleacion.
Línea 304	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 305	Nombre = "AISI 1045", → asigna el nombre comercial de la aleación utilizada en el escenario.
Línea 306	Codigo = "A1045", → asigna el código natural único de la aleación.
Línea 307	PorcentajeCarbono = 0.45, → asigna el porcentaje de carbono dentro del rango permitido por la restricción del modelo.
Línea 308	PorcentajeManganeso = 0.75, → asigna el porcentaje de manganeso dentro del rango permitido por la restricción del modelo.
Línea 309	Descripcion = "Acero medio en carbono" → asigna una descripción opcional a la aleación.

Línea 310	<code>};</code> → cierra el inicializador del objeto y finaliza la instrucción.
Línea 311	<code>orden.OrdenesAleaciones.Add(new OrdenAleacion</code> → registra una entidad o relación nueva en el <code>ChangeTracker</code> para insertarla al guardar.
Línea 312	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 313	<code>Aleacion = aleacion,</code> → conecta la entidad intermedia <code>OrdenAleacion</code> con la aleación creada para el escenario.
Línea 314	<code>CantidadUtilizada = 1500.500m,</code> → registra la cantidad utilizada con tres decimales, coherente con <code>HasPrecision(18, 3)</code> .
Línea 315	<code>EstadoRelacion = "Activa"</code> → marca como activa la relación entre la orden y la aleación.
Línea 316	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 317	<code>context.OrdenesFabricacion.Add(orden);</code> → registra una entidad o relación nueva en el <code>ChangeTracker</code> para insertarla al guardar.
Línea 318	<code>context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });</code> → registra una entidad o relación nueva en el <code>ChangeTracker</code> para insertarla al guardar.
Línea 319	<code>context.SaveChanges();</code> → persiste en SQL Server los cambios seguidos por el <code>DbContext</code> .
Línea 321	<code>var cargada = context.OrdenesFabricacion.Include(o => o.OrdenesAleaciones).ThenInclude(x => x.Aleacion).Single(o => o.NumeroOrden == "OF-M2-0001");</code> → incluye de forma eager la navegación relacionada en la consulta.
Línea 322	<code>global::System.Console.WriteLine(\$"2.5 OK Aleaciones: {cargada.OrdenesAleaciones.Count} Primera: {cargada.OrdenesAleaciones[0].Aleacion.Codigo}");</code> → escribe en consola la evidencia usada para verificar el comportamiento del estado.
Línea 323	<code>}</code> → cierra el bloque de código actual.
Línea 324	<code>}</code> → cierra el bloque de código actual.

Punto 2.6 - Configuración mediante Data Annotations

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL que trabajan con .NET 8, EF Core 8 y SQL Server LocalDB.

Proyecto: El estado 2.6 parte de 2.5 y tiene como objetivo expresar parte del mapeo con atributos sin perder la configuración acumulada.

Objetivos de aprendizaje

- Aplicar `Table`, `Key`, `Required` y `MaxLength`.
- Usar `ForeignKey` en navegaciones.
- Usar `Precision` para valores decimales.
- Definir una clave compuesta con `PrimaryKey`.
- Comprender la coexistencia con `Fluent API`.
- Comprobar metadatos del modelo resultante.

Paso 1: Abrir la solución autónoma del punto

```
cd M02/PROYECTO/2.6
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

No se abre una solución global situada en la raíz del repositorio. `AceriaData.sln` está dentro de la carpeta del punto y referencia únicamente los proyectos de ese estado.

Paso 2: Identificar el cambio respecto al estado anterior

El proyecto conserva todo lo terminado en 2.5. En 2.6 se introduce exclusivamente el contenido que corresponde a **Configuración mediante Data Annotations**. La práctica no reinicia `AceriaData` ni crea un ejemplo paralelo.

Paso 3: Implementar y comprender la configuración principal

```
[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }

    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }

    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
}
```

CSHARP

Línea 1	[Table(...)] → configura el nombre de tabla mediante atributo.
Línea 2	[Key] → identifica una clave primaria simple.
Línea 3	[Required] → refuerza la obligatoriedad de una propiedad.
Línea 4	[MaxLength(...)] → declara longitud máxima desde la clase.
Línea 5	[PrimaryKey(...)] → declara correctamente una clave primaria compuesta en EF Core 8.

Paso 4: Generar y aplicar la migración acumulativa

El repositorio contiene una migración real generada para este estado. Si se reproduce el ejercicio desde el estado anterior, los comandos son:

```
dotnet ef migrations add M2_2_6
dotnet ef database update
```

POWERSHELL

`dotnet ef migrations add` compara el modelo actual con el snapshot heredado. `dotnet ef database update` aplica únicamente los cambios pendientes.

En los estados ejecutables del curso se usa `Database.Migrate()` para aplicar el historial durante la demostración. El laboratorio puede usar `EnsureDeleted()` para empezar desde una base limpia, pero **no usa `EnsureCreated()`**, porque el esquema está gobernado por Migrations.

Paso 5: Compilar y ejecutar el estado

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

POWERSHELL

Resultado esperado: la ejecución termina sin excepción y contiene la evidencia `2.6 OK`. GitHub Actions repite esta compilación y ejecución sobre Windows y SQL Server LocalDB.

Paso 6: Verificar el estado acumulativo

Además de ejecutar el ejemplo, conviene revisar el modelo y la base:

```
dotnet ef migrations list
```

POWERSHELL

Cuando el punto modifica el esquema, la lista debe contener la migración acumulativa correspondiente. En SQL Server Object Explorer se puede comprobar que las tablas, claves, relaciones, índices o restricciones coinciden con el modelo del punto.

Errores comunes

Error	Causa	Solución
Se colocan dos [Key]	No representa correctamente una clave compuesta	Usar [PrimaryKey(...)] o HasKey.
Atributos no surten efecto	Fluent API posterior los sobrescribe	Recordar prioridad Fluent API > annotations > convenciones.
Se duplica configuración	Se repite el mismo detalle sin propósito	Mantener annotations como demostración y Fluent donde sea necesario.

Reto resuelto

Reto: Explicar en el modelo final qué configuraciones proceden de annotations y cuáles quedan sobrescritas por Fluent API.

Solución: partir del código de M02/PROYECTO/2.6, realizar únicamente el cambio descrito y volver a ejecutar build, migraciones y el programa. El reto no introduce conceptos reservados a un punto posterior.

Analogía final

Las Data Annotations son indicaciones escritas directamente sobre cada pieza del modelo; viajan con la clase.

Resultado esperado

Al terminar 2.6, la solución local compila, el proyecto se ejecuta, el esquema se actualiza mediante Migrations cuando corresponde y la salida contiene 2.6 OK. El código conserva todos los cambios válidos de los puntos anteriores.

Conexión con el siguiente punto

El siguiente estado es 2.7. Se parte del proyecto completo de 2.6; no se vuelve a crear AceriaData desde cero.

Código acumulativo completo del estado 2.6

El siguiente archivo es el Program.cs real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]

```

CSHARP

```

public string Cliente { get; set; } = string.Empty;
public DateTime FechaCreacion { get; set; }
public DateTime? FechaEntrega { get; set; }
[Required]
[MaxLength(50)]
public string Estado { get; set; } = "Pendiente";
[MaxLength(500)]
public string? Observaciones { get; set; }
public List<PlanchaAcero> Planchas { get; set; } = new();
public DetalleOrden? Detalle { get; set; }
public CertificadoCalidad? Certificado { get; set; }
public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]

```

```

public string NumeroCertificado { get; set; } = string.Empty;
public DateTime FechaEmision { get; set; }
[Required]
[MaxLength(100)]
public string OrganismoCertificador { get; set; } = string.Empty;
[ForeignKey(nameof(OrdenId))]
public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }
    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
        });
        modelBuilder.Entity<PlanchaAcero>(entity =>
        {
            entity.ToTable("PlanchasAcero");
            entity.HasKey(x => x.Id);
            entity.HasOne(x => x.Orden)
                .WithMany(o => o.Planchas)
                .HasForeignKey(x => x.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();
        });
        modelBuilder.Entity<Aleacion>(entity =>
        {
            entity.ToTable("Aleaciones");
            entity.HasKey(a => a.Id);
        });
        modelBuilder.Entity<EstadoOrden>(entity =>
        {
            entity.ToTable("EstadosOrden");
            entity.HasKey(e => e.Id);
        });
        modelBuilder.Entity<DetalleOrden>(entity =>
        {
            entity.ToTable("DetallesOrden");
            entity.HasKey(d => d.Id);
            entity.HasOne(d => d.Orden)
                .WithOne(o => o.Detalle)
                .HasForeignKey<DetalleOrden>(d => d.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();
        });
        modelBuilder.Entity<CertificadoCalidad>(entity =>
        {
            entity.ToTable("CertificadosCalidad");
            entity.HasKey(c => c.Id);
            entity.HasOne(c => c.Orden)
                .WithOne(o => o.Certificado)
                .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();
        });
    }
}

```

```

});
modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.ToTable("OrdenesAleaciones");
    entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
    entity.HasOne(x => x.Orden)
        .WithMany(o => o.OrdenesAleaciones)
        .HasForeignKey(x => x.OrdenFabricacionId)
        .OnDelete(DeleteBehavior.Cascade);
    entity.HasOne(x => x.Aleacion)
        .WithMany(a => a.OrdenesAleaciones)
        .HasForeignKey(x => x.AleacionId)
        .OnDelete(DeleteBehavior.Restrict);
});
}
}

public interface IOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB");
        ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                    {
                        sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                        sql.CommandTimeout(60);
                    }
                )
            );
    }
}

```

```

    })
    .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
LogLevel.Information)
    .EnableDetailedErrors());

services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },
    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrganizacionesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});
context.OrganizacionesFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo =
true });
context.SaveChanges();

var entity = context.Model.FindEntityType(typeof(OrdenFabricacion));
global::System.Console.WriteLine($"2.6 OK | Tabla por annotations: {entity.GetTableName()} | NumeroOrden
MaxLength: {entity.FindProperty(nameof(OrdenFabricacion.NumeroOrden)).GetMaxLength()}");
}
}

```

Explicación línea a línea del código acumulativo 2.6

Las referencias siguientes usan la numeración real de M02/PROYECTO/2.6/Program.cs. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1	<code>using System.ComponentModel.DataAnnotations;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 2	<code>using System.ComponentModel.DataAnnotations.Schema;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 3	<code>using Microsoft.EntityFrameworkCore;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 4	<code>using Microsoft.Extensions.Configuration;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 5	<code>using Microsoft.Extensions.DependencyInjection;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 6	<code>using Microsoft.Extensions.Logging;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 8	<code>namespace AceriaData.ConsoleApp;</code> → declara el espacio de nombres del estado acumulativo.
Línea 10	<code>[Table("OrdenesFabricacion")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 11	<code>public class OrdenFabricacion</code> → declara la clase OrdenFabricacion que forma parte del estado acumulativo del ejercicio.
Línea 12	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 13	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 14	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 15	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 16	<code>[MaxLength(50)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 17	<code>public string NumeroOrden { get; set; } = string.Empty;</code> → declara la propiedad NumeroOrden de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 18	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 19	<code>[MaxLength(200)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 20	<code>public string Cliente { get; set; } = string.Empty;</code> → declara la propiedad Cliente de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 21	<code>public DateTime FechaCreacion { get; set; }</code> → declara la propiedad FechaCreacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.
Línea 22	<code>public DateTime? FechaEntrega { get; set; }</code> → declara la propiedad FechaEntrega de tipo DateTime? que EF Core o la lógica de negocio utiliza en este modelo.
Línea 23	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 24	<code>[MaxLength(50)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 25	<code>public string Estado { get; set; } = "Pendiente";</code> → declara la propiedad Estado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 26	<code>[MaxLength(500)]</code> → limita la longitud máxima persistida de la propiedad siguiente.

Línea 27	<code>public string? Observaciones { get; set; } → declara la propiedad Observaciones de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 28	<code>public List<PlanchaAcero> Planchas { get; set; } = new(); → declara la propiedad Planchas de tipo List<PlanchaAcero> que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 29	<code>public DetalleOrden? Detalle { get; set; } → declara la propiedad Detalle de tipo DetalleOrden? que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 30	<code>public CertificadoCalidad? Certificado { get; set; } → declara la propiedad Certificado de tipo CertificadoCalidad? que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 31	<code>public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new(); → declara la propiedad OrdenesAleaciones de tipo List<OrdenAleacion> que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 32	<code>}</code> → cierra el bloque de código actual.
Línea 34	<code>[Table("PlanchasAcero")] → fija mediante Data Annotations la tabla relacional de la entidad siguiente.</code>
Línea 35	<code>public class PlanchaAcero → declara la clase PlanchaAcero que forma parte del estado acumulativo del ejercicio.</code>
Línea 36	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 37	<code>[Key] → marca la propiedad siguiente como clave primaria simple.</code>
Línea 38	<code>public int Id { get; set; } → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 39	<code>public int OrdenId { get; set; } → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 40	<code>public double Espesor { get; set; } → declara la propiedad Espesor de tipo double que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 41	<code>public double Ancho { get; set; } → declara la propiedad Ancho de tipo double que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 42	<code>public double Largo { get; set; } → declara la propiedad Largo de tipo double que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 43	<code>[Precision(18, 3)] → configura precisión y escala para la propiedad numérica siguiente.</code>
Línea 44	<code>public decimal Peso { get; set; } → declara la propiedad Peso de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 45	<code>public bool Activa { get; set; } = true; → declara la propiedad Activa de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 46	<code>[ForeignKey(nameof(OrdenId))] → vincula explícitamente la navegación siguiente con su clave foránea.</code>
Línea 47	<code>public OrdenFabricacion Orden { get; set; } = null!; → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.</code>
Línea 48	<code>}</code> → cierra el bloque de código actual.
Línea 50	<code>[Table("Aleaciones")] → fija mediante Data Annotations la tabla relacional de la entidad siguiente.</code>

Línea 51	<code>public class Aleacion</code> → declara la clase Aleacion que forma parte del estado acumulativo del ejercicio.
Línea 52	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 53	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 54	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 55	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 56	<code>[MaxLength(100)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 57	<code>public string Nombre { get; set; } = string.Empty;</code> → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 58	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 59	<code>[MaxLength(20)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 60	<code>public stringCodigo { get; set; } = string.Empty;</code> → declara la propiedad Codigo de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 61	<code>public double PorcentajeCarbono { get; set; }</code> → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.
Línea 62	<code>public double PorcentajeManganeso { get; set; }</code> → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.
Línea 63	<code>[MaxLength(500)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 64	<code>public string? Descripcion { get; set; }</code> → declara la propiedad Descripcion de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.
Línea 65	<code>public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();</code> → declara la propiedad OrdenesAleaciones de tipo List<OrdenAleacion> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 66	<code>}</code> → cierra el bloque de código actual.
Línea 68	<code>[Table("EstadosOrden")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 69	<code>public class EstadoOrden</code> → declara la clase EstadoOrden que forma parte del estado acumulativo del ejercicio.
Línea 70	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 71	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 72	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 73	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 74	<code>[MaxLength(50)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 75	<code>public string Nombre { get; set; } = string.Empty;</code> → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 76	<code>[MaxLength(250)]</code> → limita la longitud máxima persistida de la propiedad siguiente.

Línea 77	<code>public string Descripcion { get; set; } = string.Empty;</code> → declara la propiedad Descripcion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 78	<code>public bool Activo { get; set; } = true;</code> → declara la propiedad Activo de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.
Línea 79	<code>}</code> → cierra el bloque de código actual.
Línea 81	<code>[Table("DetallesOrden")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 82	<code>public class DetalleOrden</code> → declara la clase DetalleOrden que forma parte del estado acumulativo del ejercicio.
Línea 83	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 84	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 85	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 86	<code>public int OrdenId { get; set; }</code> → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 87	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 88	<code>[MaxLength(200)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 89	<code>public string ComposicionQuimica { get; set; } = string.Empty;</code> → declara la propiedad ComposicionQuimica de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 90	<code>public double TemperaturaColada { get; set; }</code> → declara la propiedad TemperaturaColada de tipo double que EF Core o la lógica de negocio utiliza en este modelo.
Línea 91	<code>[MaxLength(500)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 92	<code>public string? Notas { get; set; }</code> → declara la propiedad Notas de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.
Línea 93	<code>[ForeignKey(nameof(OrdenId))]</code> → vincula explícitamente la navegación siguiente con su clave foránea.
Línea 94	<code>public OrdenFabricacion Orden { get; set; } = null!;</code> → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.
Línea 95	<code>}</code> → cierra el bloque de código actual.
Línea 97	<code>[Table("CertificadosCalidad")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 98	<code>public class CertificadoCalidad</code> → declara la clase CertificadoCalidad que forma parte del estado acumulativo del ejercicio.
Línea 99	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 100	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 101	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 102	<code>public int OrdenId { get; set; }</code> → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 103	[Required] → marca la propiedad siguiente como requerida.
Línea 104	[MaxLength(50)] → limita la longitud máxima persistida de la propiedad siguiente.
Línea 105	public string NumeroCertificado { get; set; } = string.Empty; → declara la propiedad NumeroCertificado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 106	public DateTime FechaEmision { get; set; } → declara la propiedad FechaEmision de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.
Línea 107	[Required] → marca la propiedad siguiente como requerida.
Línea 108	[MaxLength(100)] → limita la longitud máxima persistida de la propiedad siguiente.
Línea 109	public string OrganismoCertificador { get; set; } = string.Empty; → declara la propiedad OrganismoCertificador de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 110	[ForeignKey(nameof(OrdenId))] → vincula explícitamente la navegación siguiente con su clave foránea.
Línea 111	public OrdenFabricacion Orden { get; set; } = null!; → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.
Línea 112	} → cierra el bloque de código actual.
Línea 114	[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))] → declara una clave primaria compuesta con las propiedades indicadas.
Línea 115	[Table("OrdenesAleaciones")] → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 116	public class OrdenAleacion → declara la clase OrdenAleacion que forma parte del estado acumulativo del ejercicio.
Línea 117	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 118	public int OrdenFabricacionId { get; set; } → declara la propiedad OrdenFabricacionId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 119	public int AleacionId { get; set; } → declara la propiedad AleacionId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 120	public DateTime FechaAsignacion { get; set; } = DateTime.Now; → declara la propiedad FechaAsignacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.
Línea 121	[Precision(18, 3)] → configura precisión y escala para la propiedad numérica siguiente.
Línea 122	public decimal CantidadUtilizada { get; set; } → declara la propiedad CantidadUtilizada de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.
Línea 123	[MaxLength(20)] → limita la longitud máxima persistida de la propiedad siguiente.
Línea 124	public string EstadoRelacion { get; set; } = "Activa"; → declara la propiedad EstadoRelacion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 125	public OrdenFabricacion Orden { get; set; } = null!; → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.
Línea 126	public Aleacion Aleacion { get; set; } = null!; → declara la propiedad Aleacion de tipo Aleacion que EF Core o la lógica de negocio utiliza en este modelo.
Línea 127	} → cierra el bloque de código actual.

Línea 129	<code>public class AceriaDbContext : DbContext</code> → declara la clase <code>AceriaDbContext</code> que forma parte del estado acumulativo del ejercicio.
Línea 130	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 131	<code>public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }</code> → declara el constructor o método <code>AceriaDbContext</code> con los parámetros necesarios para este componente.
Línea 133	<code>public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 134	<code>public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 135	<code>public DbSet<Aleacion> Aleaciones => Set<Aleacion>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 136	<code>public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 137	<code>public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 138	<code>public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 139	<code>public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 142	<code>protected override void OnModelCreating(ModelBuilder modelBuilder)</code> → sobrescribe <code>OnModelCreating</code> para definir configuración explícita del modelo.
Línea 143	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 144	<code>base.OnModelCreating(modelBuilder);</code> → sobrescribe <code>OnModelCreating</code> para definir configuración explícita del modelo.
Línea 145	<code>modelBuilder.Entity<OrdenFabricacion>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 146	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 147	<code>entity.ToTable("OrdenesFabricacion");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 148	<code>entity.HasKey(o => o.Id);</code> → define la clave primaria de la entidad.
Línea 149	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 150	<code>modelBuilder.Entity<PlanchaAcero>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 151	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 152	<code>entity.ToTable("PlanchasAcero");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 153	<code>entity.HasKey(x => x.Id);</code> → define la clave primaria de la entidad.
Línea 154	<code>entity.HasOne(x => x.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 155	<code>.WithMany(o => o.Planchas)</code> → establece el extremo de colección de una relación uno-a-muchos.

Línea 156	<code>.HasForeignKey(x => x.OrdenId)</code> → declara explícitamente la propiedad o propiedades de clave foránea.
Línea 157	<code>.OnDelete(DeleteBehavior.Cascade)</code> → define el comportamiento de eliminación de la relación.
Línea 158	<code>.IsRequired();</code> → marca la propiedad o relación como obligatoria.
Línea 159	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 160	<code>modelBuilder.Entity<Aleacion>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 161	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 162	<code>entity.ToTable("Aleaciones");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 163	<code>entity.HasKey(a => a.Id);</code> → define la clave primaria de la entidad.
Línea 164	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 165	<code>modelBuilder.Entity<EstadoOrden>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 166	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 167	<code>entity.ToTable("EstadosOrden");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 168	<code>entity.HasKey(e => e.Id);</code> → define la clave primaria de la entidad.
Línea 169	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 170	<code>modelBuilder.Entity<DetalleOrden>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 171	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 172	<code>entity.ToTable("DetallesOrden");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 173	<code>entity.HasKey(d => d.Id);</code> → define la clave primaria de la entidad.
Línea 174	<code>entity.HasOne(d => d.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 175	<code>.WithOne(o => o.Detalle)</code> → establece el otro extremo de referencia de una relación uno-a-uno.
Línea 176	<code>.HasForeignKey<DetalleOrden>(d => d.OrdenId)</code> → declara explícitamente la propiedad o propiedades de clave foránea.
Línea 177	<code>.OnDelete(DeleteBehavior.Cascade)</code> → define el comportamiento de eliminación de la relación.
Línea 178	<code>.IsRequired();</code> → marca la propiedad o relación como obligatoria.
Línea 179	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 181	<code>modelBuilder.Entity<CertificadoCalidad>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 182	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 183	<code>entity.ToTable("CertificadosCalidad");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 184	<code>entity.HasKey(c => c.Id);</code> → define la clave primaria de la entidad.
Línea 185	<code>entity.HasOne(c => c.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 186	<code>.WithOne(o => o.Certificado)</code> → establece el otro extremo de referencia de una relación uno-a-uno.
Línea 187	<code>.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)</code> → declara explícitamente la propiedad o propiedades de clave foránea.
Línea 188	<code>.OnDelete(DeleteBehavior.Cascade)</code> → define el comportamiento de eliminación de la relación.
Línea 189	<code>.IsRequired();</code> → marca la propiedad o relación como obligatoria.
Línea 190	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 191	<code>modelBuilder.Entity<OrdenAleacion>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 192	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 193	<code>entity.ToTable("OrdenesAleaciones");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 194	<code>entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });</code> → define la clave primaria de la entidad.
Línea 195	<code>entity.HasOne(x => x.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 196	<code>.WithMany(o => o.OrdenesAleaciones)</code> → establece el extremo de colección de una relación uno-a-muchos.
Línea 197	<code>.HasForeignKey(x => x.OrdenFabricacionId)</code> → declara explícitamente la propiedad o propiedades de clave foránea.
Línea 198	<code>.OnDelete(DeleteBehavior.Cascade);</code> → define el comportamiento de eliminación de la relación.
Línea 199	<code>entity.HasOne(x => x.Aleacion)</code> → inicia la configuración de una navegación de referencia.
Línea 200	<code>.WithMany(a => a.OrdenesAleaciones)</code> → establece el extremo de colección de una relación uno-a-muchos.
Línea 201	<code>.HasForeignKey(x => x.AleacionId)</code> → declara explícitamente la propiedad o propiedades de clave foránea.
Línea 202	<code>.OnDelete(DeleteBehavior.Restrict);</code> → define el comportamiento de eliminación de la relación.
Línea 203	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 204	<code>}</code> → cierra el bloque de código actual.
Línea 205	<code>}</code> → cierra el bloque de código actual.
Línea 207	<code>public interface IOrdenRepositorio</code> → declara el contrato IOrdenRepositorio para desacoplar consumidores e implementación.
Línea 208	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 209	<code>List<OrdenFabricacion> ObtenerTodas();</code> → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.
Línea 210	<code>OrdenFabricacion? ObtenerPorId(int id);</code> → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.
Línea 211	<code>OrdenFabricacion? ObtenerPorNumero(string numeroOrden);</code> → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.
Línea 212	<code>void Agregar(OrdenFabricacion orden);</code> → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.
Línea 213	<code>void Eliminar(OrdenFabricacion orden);</code> → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.
Línea 214	<code>int Guardar();</code> → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.
Línea 215	<code>}</code> → cierra el bloque de código actual.
Línea 217	<code>public sealed class OrdenRepositorio : IOrdenRepositorio</code> → declara la clase OrdenRepositorio que forma parte del estado acumulativo del ejercicio.
Línea 218	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 219	<code>private readonly AceriaDbContext _context;</code> → declara el campo privado _context de tipo AceriaDbContext que conserva una dependencia de la clase.
Línea 220	<code>public OrdenRepositorio(AceriaDbContext context) => _context = context;</code> → declara el constructor o método OrdenRepositorio con los parámetros necesarios para este componente.
Línea 222	<code>public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();</code> → ordena el resultado de la consulta antes de materializarlo.
Línea 223	<code>public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);</code> → implementa la búsqueda por clave primaria mediante Find, aprovechando primero el ChangeTracker y después la base de datos.
Línea 224	<code>public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);</code> → recupera la primera coincidencia o null cuando no existe.
Línea 225	<code>public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);</code> → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.
Línea 226	<code>public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);</code> → marca la entidad indicada para borrado al confirmar los cambios.
Línea 227	<code>public int Guardar() => _context.SaveChanges();</code> → persiste en SQL Server los cambios seguidos por el DbContext.
Línea 228	<code>}</code> → cierra el bloque de código actual.
Línea 230	<code>public interface IServicioOrdenes</code> → declara el contrato IServicioOrdenes para desacoplar consumidores e implementación.
Línea 231	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 232	<code>string ObtenerResumen(int ordenId);</code> → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 233	} → cierra el bloque de código actual.
Línea 235	public sealed class ServicioOrdenes : IServicioOrdenes → declara la clase ServicioOrdenes que forma parte del estado acumulativo del ejercicio.
Línea 236	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 237	private readonly IOrdenRepositorio _repositorio; → declara el campo privado _repositorio de tipo IOrdenRepositorio que conserva una dependencia de la clase.
Línea 238	private readonly AceriaDbContext _context; → declara el campo privado _context de tipo AceriaDbContext que conserva una dependencia de la clase.
Línea 239	public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context) → declara el constructor o método ServicioOrdenes con los parámetros necesarios para este componente.
Línea 240	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 241	_repositorio = repositorio; → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.
Línea 242	_context = context; → guarda el DbContext inyectado para consultar los datos necesarios durante el servicio.
Línea 243	} → cierra el bloque de código actual.
Línea 245	public string ObtenerResumen(int ordenId) → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.
Línea 246	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 247	var orden = _repositorio.ObtenerPorId(ordenId); → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.
Línea 248	if (orden is null) return \$"Orden {ordenId} no encontrada"; → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.
Línea 249	var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId); → cuenta las filas que cumplen la consulta construida.
Línea 250	return \$"Orden {orden.NumeroOrden} Cliente: {orden.Cliente} Planchas: {totalPlanchas}"; → devuelve el resultado calculado al código que invocó el método.
Línea 251	} → cierra el bloque de código actual.
Línea 252	} → cierra el bloque de código actual.
Línea 254	public static class Program → declara la clase Program que forma parte del estado acumulativo del ejercicio.
Línea 255	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 256	public static void Main() → declara el punto de entrada ejecutable de la aplicación de consola.
Línea 257	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 258	var configuration = new ConfigurationBuilder() → inicia la construcción de la configuración de la aplicación.
Línea 259	.SetBasePath(AppContext.BaseDirectory) → establece la ruta base usada para localizar los archivos de configuración.

Línea 260	<code>.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false) → incorpora appsettings.json a la configuración.</code>
Línea 261	<code>.AddEnvironmentVariables() → incorpora variables de entorno como fuente adicional de configuración.</code>
Línea 262	<code>.Build(); → materializa la configuración o el objeto encadenado construido en las líneas anteriores.</code>
Línea 264	<code>var connectionString = configuration.GetConnectionString("AceriaDB") → obtiene la cadena de conexión AceriaDB desde la configuración.</code>
Línea 265	<code>?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'."); → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.</code>
Línea 267	<code>var services = new ServiceCollection(); → crea la colección donde se registran las dependencias de la aplicación.</code>
Línea 268	<code>services.AddDbContext<AceriaDbContext>(options => → registra el DbContext en el contenedor de inyección de dependencias.</code>
Línea 269	<code>options → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.</code>
Línea 270	<code>.UseSqlServer(connectionString, sql => → selecciona SQL Server como proveedor de EF Core.</code>
Línea 271	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 272	<code>sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null); → habilita reintentos ante fallos transitorios del proveedor SQL Server.</code>
Línea 273	<code>sql.CommandTimeout(60); → establece el tiempo máximo permitido para los comandos SQL.</code>
Línea 274	<code>}) → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.</code>
Línea 275	<code>.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information) → configura la salida del logging de EF Core.</code>
Línea 276	<code>.EnableDetailedErrors(); → activa errores detallados para facilitar el diagnóstico del laboratorio.</code>
Línea 278	<code>services.AddScoped<IOrdenRepositorio, OrdenRepositorio>(); → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.</code>
Línea 279	<code>services.AddScoped<IServicioOrdenes, ServicioOrdenes>(); → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.</code>
Línea 281	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions → construye el ServiceProvider con validación de servicios y garantiza su liberación al salir del ámbito mediante using var.</code>
Línea 282	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 283	<code>ValidateScopes = true, → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.</code>
Línea 284	<code>ValidateOnBuild = true → obliga al contenedor a validar las resoluciones registradas al construir el ServiceProvider.</code>
Línea 285	<code>}); → cierra la lambda o configuración encadenada y finaliza la llamada.</code>

Línea 287	<code>using var scope = provider.CreateScope();</code> → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.
Línea 288	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → resuelve un servicio obligatorio desde el contenedor de dependencias.
Línea 289	<code>context.Database.EnsureDeleted();</code> → elimina la base del laboratorio para reproducir el escenario desde un estado limpio.
Línea 290	<code>context.Database.Migrate();</code> → aplica las migraciones pendientes y deja AceriaDB en el esquema del punto.
Línea 292	<code>var orden = new OrdenFabricacion</code> → crea la entidad OrdenFabricacion que concentra el grafo de datos del escenario acumulativo.
Línea 293	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 294	<code>NumeroOrden = "OF-M2-0001",</code> → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.
Línea 295	<code>Cliente = "Constructora del Norte",</code> → asigna el cliente de la orden utilizada en el escenario E2E del punto.
Línea 296	<code>FechaCreacion = DateTime.UtcNow,</code> → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.
Línea 297	<code>FechaEntrega = DateTime.Today.AddDays(14),</code> → establece una fecha de entrega prevista catorce días después de la fecha local actual.
Línea 298	<code>Estado = "Pendiente",</code> → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.
Línea 299	<code>Observaciones = "Orden de validación M2",</code> → añade texto opcional para comprobar el mapeo de Observaciones.
Línea 300	<code>Planchas =</code> → inicia el inicializador de la colección de planchas asociadas a la orden.
Línea 301	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 302	<code>new PlanchaAcero</code> → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.
Línea 303	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 304	<code>Espesor = 10.5,</code> → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.
Línea 305	<code>Ancho = 1500,</code> → asigna el ancho de la plancha usada en el escenario de persistencia.
Línea 306	<code>Largo = 3000,</code> → asigna el largo de la plancha usada en el escenario de persistencia.
Línea 307	<code>Peso = 371.250m,</code> → asigna un peso decimal con tres posiciones para validar la precisión configurada.
Línea 308	<code>Activa = true,</code> → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.
Línea 309	<code>}</code> → cierra el bloque de código actual.
Línea 310	<code>},</code> → continúa el inicializador o la llamada iniciada en el bloque actual.
Línea 311	<code>Detalle = new DetalleOrden</code> → crea el detalle uno-a-uno que se persistirá junto con la orden principal.

Línea 312	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 313	ComposicionQuimica = "C: 0.45%, Mn: 0.75%", → asigna la composición química utilizada para verificar la entidad DetalleOrden.
Línea 314	TemperaturaColada = 1550.5, → asigna la temperatura de colada del detalle de fabricación.
Línea 315	Notas = "Colada principal" → asigna las notas opcionales del detalle asociado a la orden.
Línea 316	}, → continúa el inicializador o la llamada iniciada en el bloque actual.
Línea 317	Certificado = new CertificadoCalidad → crea el certificado uno-a-uno asociado a la orden del escenario.
Línea 318	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 319	NumeroCertificado = "CERT-0001", → asigna el identificador natural del certificado de calidad.
Línea 320	FechaEmision = DateTime.Today, → asigna la fecha de emisión del certificado usado en la prueba.
Línea 321	OrganismoCertificador = "Laboratorio Aceria" → asigna el organismo que emite el certificado de calidad.
Línea 322	}, → continúa el inicializador o la llamada iniciada en el bloque actual.
Línea 323	}; → cierra el inicializador del objeto y finaliza la instrucción.
Línea 325	var aleacion = new Aleacion → crea la aleación que se enlazarán con la orden mediante la entidad intermedia OrdenAleacion.
Línea 326	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 327	Nombre = "AISI 1045", → asigna el nombre comercial de la aleación utilizada en el escenario.
Línea 328	Codigo = "A1045", → asigna el código natural único de la aleación.
Línea 329	PorcentajeCarbono = 0.45, → asigna el porcentaje de carbono dentro del rango permitido por la restricción del modelo.
Línea 330	PorcentajeManganeso = 0.75, → asigna el porcentaje de manganeso dentro del rango permitido por la restricción del modelo.
Línea 331	Descripcion = "Acero medio en carbono" → asigna una descripción opcional a la aleación.
Línea 332	}; → cierra el inicializador del objeto y finaliza la instrucción.
Línea 333	orden.OrdenesAleaciones.Add(new OrdenAleacion → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.
Línea 334	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 335	Aleacion = aleacion, → conecta la entidad intermedia OrdenAleacion con la aleación creada para el escenario.
Línea 336	CantidadUtilizada = 1500.500m, → registra la cantidad utilizada con tres decimales, coherente con HasPrecision(18, 3).
Línea 337	EstadoRelacion = "Activa" → marca como activa la relación entre la orden y la aleación.
Línea 338	}); → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 339	<code>context.OrdenesFabricacion.Add(orden);</code> → registra una entidad o relación nueva en el <code>ChangeTracker</code> para insertarla al guardar.
Línea 340	<code>context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });</code> → registra una entidad o relación nueva en el <code>ChangeTracker</code> para insertarla al guardar.
Línea 341	<code>context.SaveChanges();</code> → persiste en SQL Server los cambios seguidos por el <code>DbContext</code> .
Línea 343	<code>var entity = context.Model.FindEntityType(typeof(OrdenFabricacion))!;</code> → obtiene los metadatos EF Core de <code>OrdenFabricacion</code> para inspeccionar claves, índices o filtros del modelo construido.
Línea 344	<code>global::System.Console.WriteLine(\$"2.6 OK Tabla por annotations: {entity.GetTableName()} NumeroOrden MaxLength: {entity.FindProperty(nameof(OrdenFabricacion.NumeroOrden))?.GetMaxLength()}");</code> → escribe en consola la evidencia usada para verificar el comportamiento del estado.
Línea 345	<code>}</code> → cierra el bloque de código actual.
Línea 346	<code>}</code> → cierra el bloque de código actual.

Punto 2.7 - Configuración mediante Fluent API

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL que trabajan con .NET 8, EF Core 8 y SQL Server LocalDB.

Proyecto: El estado 2.7 parte de 2.6 y tiene como objetivo centralizar y hacer explícita la configuración con Fluent API.

Objetivos de aprendizaje

- Comprender la prioridad de Fluent API.
- Configurar propiedades desde `OnModelCreating`.
- Configurar relaciones con expresiones fuertemente tipadas.
- Usar `HasDefaultValue` y `HasDefaultValueSql`.
- Mantener Data Annotations compatibles.
- Reservar claves, índices y filtros para sus puntos específicos.

Paso 1: Abrir la solución autónoma del punto

```
cd M02/PROYECTO/2.7
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

No se abre una solución global situada en la raíz del repositorio. `AceriaData.sln` está dentro de la carpeta del punto y referencia únicamente los proyectos de ese estado.

Paso 2: Identificar el cambio respecto al estado anterior

El proyecto conserva todo lo terminado en 2.6. En 2.7 se introduce exclusivamente el contenido que corresponde a **Configuración mediante Fluent API**. La práctica no reinicia `AceriaData` ni crea un ejemplo paralelo.

Paso 3: Implementar y comprender la configuración principal

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.ToTable("OrdenesFabricacion");
    entity.HasKey(o => o.Id);
```

CSHARP

```
entity.Property(o => o.NumeroOrden)
    .IsRequired()
    .HasMaxLength(50);

entity.Property(o => o.FechaCreacion)
    .HasDefaultValueSql("GETDATE()");
});
```

Línea 1	modelBuilder.Entity<T>() → selecciona la entidad a configurar.
Línea 2	Property(...) → selecciona una propiedad escalar.
Línea 3	IsRequired() → configura obligatoriedad.
Línea 4	HasMaxLength(...) → configura el tamaño máximo.
Línea 5	HasDefaultValueSql("GETDATE()") → delega el valor por defecto de FechaCreacion en SQL Server.

Paso 4: Generar y aplicar la migración acumulativa

El repositorio contiene una migración real generada para este estado. Si se reproduce el ejercicio desde el estado anterior, los comandos son:

```
dotnet ef migrations add M2_2_7
dotnet ef database update
```

POWERSHELL

`dotnet ef migrations add` compara el modelo actual con el snapshot heredado. `dotnet ef database update` aplica únicamente los cambios pendientes.

En los estados ejecutables del curso se usa `Database.Migrate()` para aplicar el historial durante la demostración. El laboratorio puede usar `EnsureDeleted()` para empezar desde una base limpia, pero **no usa** `EnsureCreated()`, porque el esquema está gobernado por Migrations.

Paso 5: Compilar y ejecutar el estado

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

POWERSHELL

Resultado esperado: la ejecución termina sin excepción y contiene la evidencia 2.7 OK. GitHub Actions repite esta compilación y ejecución sobre Windows y SQL Server LocalDB.

Paso 6: Verificar el estado acumulativo

Además de ejecutar el ejemplo, conviene revisar el modelo y la base:

```
dotnet ef migrations list
```

POWERSHELL

Cuando el punto modifica el esquema, la lista debe contener la migración acumulativa correspondiente. En SQL Server Object Explorer se puede comprobar que las tablas, claves, relaciones, índices o restricciones coinciden con el modelo del punto.

Errores comunes

Error	Causa	Solución
Fluent contradice atributos	Dos configuraciones dan valores distintos	Definir una fuente de verdad consciente; Fluent gana.
Se introduce HasQueryFilter	Se adelantó 2.10	Reservarlo para filtros globales.

Error	Causa	Solución
Se añaden índices avanzados	Se adelantó 2.9	Mantener 2.7 en sintaxis y configuración del modelo.

Reto resuelto

Reto: Mover una regla de MaxLength desde annotation a Fluent API y comprobar que el modelo resultante conserva la misma longitud.

Solución: partir del código de `M02/PROYECTO/2.7`, realizar únicamente el cambio descrito y volver a ejecutar build, migraciones y el programa. El reto no introduce conceptos reservados a un punto posterior.

Analogía final

Fluent API es el plano central de configuración: permite expresar reglas que no caben cómodamente como atributos.

Resultado esperado

Al terminar 2.7, la solución local compila, el proyecto se ejecuta, el esquema se actualiza mediante Migrations cuando corresponde y la salida contiene `2.7 OK`. El código conserva todos los cambios válidos de los puntos anteriores.

Conexión con el siguiente punto

El siguiente estado es `2.8`. Se parte del proyecto completo de 2.7; no se vuelve a crear `AceriaData` desde cero.

Código acumulativo completo del estado 2.7

El siguiente archivo es el `Program.cs` real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

CSHARP

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
```

```

    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }
    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

```

```

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
            entity.Property(o => o.Observaciones).HasMaxLength(500);
        });
        modelBuilder.Entity<PlanchaAcero>(entity =>
        {
            entity.ToTable("PlanchasAcero");
            entity.HasKey(x => x.Id);
            entity.Property(x => x.Peso).HasPrecision(18, 3);
            entity.Property(x => x.Activa).HasDefaultValue(true);
            entity.HasOne(x => x.Orden)
                .WithMany(o => o.Planchas)
                .HasForeignKey(x => x.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();
        });
        modelBuilder.Entity<Aleacion>(entity =>
        {
            entity.ToTable("Aleaciones");
            entity.HasKey(a => a.Id);
            entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
            entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
            entity.Property(a => a.Descripcion).HasMaxLength(500);
        });
        modelBuilder.Entity<EstadoOrden>(entity =>
        {
            entity.ToTable("EstadosOrden");
            entity.HasKey(e => e.Id);
            entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
            entity.Property(e => e.Descripcion).HasMaxLength(250);
            entity.Property(e => e.Activo).HasDefaultValue(true);
        });
        modelBuilder.Entity<DetalleOrden>(entity =>
        {
            entity.ToTable("DetallesOrden");
            entity.HasKey(d => d.Id);
            entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
            entity.Property(d => d.Notas).HasMaxLength(500);
            entity.HasOne(d => d.Orden)
                .WithOne(o => o.Detalle)
                .HasForeignKey<DetalleOrden>(d => d.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();
        });
        modelBuilder.Entity<CertificadoCalidad>(entity =>
        {
            entity.ToTable("CertificadosCalidad");
            entity.HasKey(c => c.Id);
            entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
            entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
            entity.HasOne(c => c.Orden)
                .WithOne(o => o.Certificado)
                .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();
        });
        modelBuilder.Entity<OrdenAleacion>(entity =>
        {
            entity.ToTable("OrdenesAleaciones");
            entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
            entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
            entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
        });
    }
}

```

```

        entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.OrdenesAleaciones)
            .HasForeignKey(x => x.OrdenFabricacionId)
            .OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(x => x.Aleacion)
            .WithMany(a => a.OrdenesAleaciones)
            .HasForeignKey(x => x.AleacionId)
            .OnDelete(DeleteBehavior.Restrict);
    });
}

public interface IOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB");
        ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                    {
                        sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                        sql.CommandTimeout(60);
                    })
                .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
                    LogLevel.Information)
                .EnableDetailedErrors());
    }
}

```

```

services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },
    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});
context.OrdenesFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo =
true });
context.SaveChanges();

var entity = context.Model.FindEntityType(typeof(OrdenFabricacion));
global::System.Console.WriteLine($"2.7 OK | Fluent API | Estado default:
{entity.FindProperty(nameof(OrdenFabricacion.Estado))?.GetDefaultValue()}");
}
}

```

Explicación línea a línea del código acumulativo 2.7

Las referencias siguientes usan la numeración real de `M02/PROYECTO/2.7/Program.cs`. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1	<code>using System.ComponentModel.DataAnnotations;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
----------------	--

Línea 2	<code>using System.ComponentModel.DataAnnotations.Schema;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 3	<code>using Microsoft.EntityFrameworkCore;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 4	<code>using Microsoft.Extensions.Configuration;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 5	<code>using Microsoft.Extensions.DependencyInjection;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 6	<code>using Microsoft.Extensions.Logging;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 8	<code>namespace AceriaData.ConsoleApp;</code> → declara el espacio de nombres del estado acumulativo.
Línea 10	<code>[Table("OrdenesFabricacion")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 11	<code>public class OrdenFabricacion</code> → declara la clase OrdenFabricacion que forma parte del estado acumulativo del ejercicio.
Línea 12	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 13	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 14	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 15	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 16	<code>[MaxLength(50)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 17	<code>public string NumeroOrden { get; set; } = string.Empty;</code> → declara la propiedad NumeroOrden de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 18	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 19	<code>[MaxLength(200)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 20	<code>public string Cliente { get; set; } = string.Empty;</code> → declara la propiedad Cliente de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 21	<code>public DateTime FechaCreacion { get; set; }</code> → declara la propiedad FechaCreacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.
Línea 22	<code>public DateTime? FechaEntrega { get; set; }</code> → declara la propiedad FechaEntrega de tipo DateTime? que EF Core o la lógica de negocio utiliza en este modelo.
Línea 23	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 24	<code>[MaxLength(50)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 25	<code>public string Estado { get; set; } = "Pendiente";</code> → declara la propiedad Estado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 26	<code>[MaxLength(500)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 27	<code>public string? Observaciones { get; set; }</code> → declara la propiedad Observaciones de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 28	<code>public List<PlanchaAcero> Planchas { get; set; } = new();</code> → declara la propiedad Planchas de tipo <code>List<PlanchaAcero></code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 29	<code>public DetalleOrden? Detalle { get; set; }</code> → declara la propiedad Detalle de tipo <code>DetalleOrden?</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 30	<code>public CertificadoCalidad? Certificado { get; set; }</code> → declara la propiedad Certificado de tipo <code>CertificadoCalidad?</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 31	<code>public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();</code> → declara la propiedad OrdenesAleaciones de tipo <code>List<OrdenAleacion></code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 32	<code>}</code> → cierra el bloque de código actual.
Línea 34	<code>[Table("PlanchasAcero")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 35	<code>public class PlanchaAcero</code> → declara la clase PlanchaAcero que forma parte del estado acumulativo del ejercicio.
Línea 36	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 37	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 38	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo <code>int</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 39	<code>public int OrdenId { get; set; }</code> → declara la propiedad OrdenId de tipo <code>int</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 40	<code>public double Espesor { get; set; }</code> → declara la propiedad Espesor de tipo <code>double</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 41	<code>public double Ancho { get; set; }</code> → declara la propiedad Ancho de tipo <code>double</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 42	<code>public double Largo { get; set; }</code> → declara la propiedad Largo de tipo <code>double</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 43	<code>[Precision(18, 3)]</code> → configura precisión y escala para la propiedad numérica siguiente.
Línea 44	<code>public decimal Peso { get; set; }</code> → declara la propiedad Peso de tipo <code>decimal</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 45	<code>public bool Activa { get; set; } = true;</code> → declara la propiedad Activa de tipo <code>bool</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 46	<code>[ForeignKey(nameof(OrdenId))]</code> → vincula explícitamente la navegación siguiente con su clave foránea.
Línea 47	<code>public OrdenFabricacion Orden { get; set; } = null!;</code> → declara la propiedad Orden de tipo <code>OrdenFabricacion</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 48	<code>}</code> → cierra el bloque de código actual.
Línea 50	<code>[Table("Aleaciones")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 51	<code>public class Aleacion</code> → declara la clase Aleacion que forma parte del estado acumulativo del ejercicio.
Línea 52	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 53	[Key] → marca la propiedad siguiente como clave primaria simple.
Línea 54	public int Id { get; set; } → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 55	[Required] → marca la propiedad siguiente como requerida.
Línea 56	[MaxLength(100)] → limita la longitud máxima persistida de la propiedad siguiente.
Línea 57	public string Nombre { get; set; } = string.Empty; → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 58	[Required] → marca la propiedad siguiente como requerida.
Línea 59	[MaxLength(20)] → limita la longitud máxima persistida de la propiedad siguiente.
Línea 60	public stringCodigo { get; set; } = string.Empty; → declara la propiedad Codigo de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 61	public double PorcentajeCarbono { get; set; } → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.
Línea 62	public double PorcentajeManganeso { get; set; } → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.
Línea 63	[MaxLength(500)] → limita la longitud máxima persistida de la propiedad siguiente.
Línea 64	public string? Descripcion { get; set; } → declara la propiedad Descripcion de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.
Línea 65	public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new(); → declara la propiedad OrdenesAleaciones de tipo List<OrdenAleacion> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 66	} → cierra el bloque de código actual.
Línea 68	[Table("EstadosOrden")] → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 69	public class EstadoOrden → declara la clase EstadoOrden que forma parte del estado acumulativo del ejercicio.
Línea 70	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 71	[Key] → marca la propiedad siguiente como clave primaria simple.
Línea 72	public int Id { get; set; } → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 73	[Required] → marca la propiedad siguiente como requerida.
Línea 74	[MaxLength(50)] → limita la longitud máxima persistida de la propiedad siguiente.
Línea 75	public string Nombre { get; set; } = string.Empty; → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 76	[MaxLength(250)] → limita la longitud máxima persistida de la propiedad siguiente.
Línea 77	public string Descripcion { get; set; } = string.Empty; → declara la propiedad Descripcion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 78	<code>public bool Activo { get; set; } = true;</code> → declara la propiedad Activo de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.
Línea 79	<code>}</code> → cierra el bloque de código actual.
Línea 81	<code>[Table("DetallesOrden")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 82	<code>public class DetalleOrden</code> → declara la clase DetalleOrden que forma parte del estado acumulativo del ejercicio.
Línea 83	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 84	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 85	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 86	<code>public int OrdenId { get; set; }</code> → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 87	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 88	<code>[MaxLength(200)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 89	<code>public string ComposicionQuimica { get; set; } = string.Empty;</code> → declara la propiedad ComposicionQuimica de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 90	<code>public double TemperaturaColada { get; set; }</code> → declara la propiedad TemperaturaColada de tipo double que EF Core o la lógica de negocio utiliza en este modelo.
Línea 91	<code>[MaxLength(500)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 92	<code>public string? Notas { get; set; }</code> → declara la propiedad Notas de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.
Línea 93	<code>[ForeignKey(nameof(OrdenId))]</code> → vincula explícitamente la navegación siguiente con su clave foránea.
Línea 94	<code>public OrdenFabricacion Orden { get; set; } = null!;</code> → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.
Línea 95	<code>}</code> → cierra el bloque de código actual.
Línea 97	<code>[Table("CertificadosCalidad")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 98	<code>public class CertificadoCalidad</code> → declara la clase CertificadoCalidad que forma parte del estado acumulativo del ejercicio.
Línea 99	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 100	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 101	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 102	<code>public int OrdenId { get; set; }</code> → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 103	<code>[Required]</code> → marca la propiedad siguiente como requerida.

Línea 104	<code>[MaxLength(50)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 105	<code>public string NumeroCertificado { get; set; } = string.Empty;</code> → declara la propiedad <code>NumeroCertificado</code> de tipo <code>string</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 106	<code>public DateTime FechaEmision { get; set; }</code> → declara la propiedad <code>FechaEmision</code> de tipo <code>DateTime</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 107	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 108	<code>[MaxLength(100)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 109	<code>public string OrganismoCertificador { get; set; } = string.Empty;</code> → declara la propiedad <code>OrganismoCertificador</code> de tipo <code>string</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 110	<code>[ForeignKey(nameof(OrdenId))]</code> → vincula explícitamente la navegación siguiente con su clave foránea.
Línea 111	<code>public OrdenFabricacion Orden { get; set; } = null!;</code> → declara la propiedad <code>Orden</code> de tipo <code>OrdenFabricacion</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 112	<code>}</code> → cierra el bloque de código actual.
Línea 114	<code>[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]</code> → declara una clave primaria compuesta con las propiedades indicadas.
Línea 115	<code>[Table("OrdenesAleaciones")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 116	<code>public class OrdenAleacion</code> → declara la clase <code>OrdenAleacion</code> que forma parte del estado acumulativo del ejercicio.
Línea 117	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 118	<code>public int OrdenFabricacionId { get; set; }</code> → declara la propiedad <code>OrdenFabricacionId</code> de tipo <code>int</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 119	<code>public int AleacionId { get; set; }</code> → declara la propiedad <code>AleacionId</code> de tipo <code>int</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 120	<code>public DateTime FechaAsignacion { get; set; } = DateTime.Now;</code> → declara la propiedad <code>FechaAsignacion</code> de tipo <code>DateTime</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 121	<code>[Precision(18, 3)]</code> → configura precisión y escala para la propiedad numérica siguiente.
Línea 122	<code>public decimal CantidadUtilizada { get; set; }</code> → declara la propiedad <code>CantidadUtilizada</code> de tipo <code>decimal</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 123	<code>[MaxLength(20)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 124	<code>public string EstadoRelacion { get; set; } = "Activa";</code> → declara la propiedad <code>EstadoRelacion</code> de tipo <code>string</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 125	<code>public OrdenFabricacion Orden { get; set; } = null!;</code> → declara la propiedad <code>Orden</code> de tipo <code>OrdenFabricacion</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 126	<code>public Aleacion Aleacion { get; set; } = null!;</code> → declara la propiedad <code>Aleacion</code> de tipo <code>Aleacion</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 127	<code>}</code> → cierra el bloque de código actual.
Línea 129	<code>public class AceriaDbContext : DbContext</code> → declara la clase <code>AceriaDbContext</code> que forma parte del estado acumulativo del ejercicio.

Línea 130	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 131	public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { } → declara el constructor o método AceriaDbContext con los parámetros necesarios para este componente.
Línea 133	public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>(); → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 134	public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>(); → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 135	public DbSet<Aleacion> Aleaciones => Set<Aleacion>(); → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 136	public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>(); → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 137	public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>(); → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 138	public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>(); → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 139	public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>(); → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 142	protected override void OnModelCreating(ModelBuilder modelBuilder) → sobrescribe OnModelCreating para definir configuración explícita del modelo.
Línea 143	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 144	base.OnModelCreating(modelBuilder); → sobrescribe OnModelCreating para definir configuración explícita del modelo.
Línea 145	modelBuilder.Entity<OrdenFabricacion>(entity => → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 146	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 147	entity.ToTable("OrdenesFabricacion"); → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 148	entity.HasKey(o => o.Id); → define la clave primaria de la entidad.
Línea 149	entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50); → selecciona una propiedad escalar para continuar su configuración.
Línea 150	entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200); → selecciona una propiedad escalar para continuar su configuración.
Línea 151	entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()"); → selecciona una propiedad escalar para continuar su configuración.
Línea 152	entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente"); → selecciona una propiedad escalar para continuar su configuración.
Línea 153	entity.Property(o => o.Observaciones).HasMaxLength(500); → selecciona una propiedad escalar para continuar su configuración.
Línea 154	}); → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 155	<code>modelBuilder.Entity<PlanchaAcero>(entity => → selecciona una entidad del modelo para aplicar su configuración Fluent API.</code>
Línea 156	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 157	<code>entity.ToTable("PlanchasAcero"); → asigna explícitamente la tabla de SQL Server usada por la entidad.</code>
Línea 158	<code>entity.HasKey(x => x.Id); → define la clave primaria de la entidad.</code>
Línea 159	<code>entity.Property(x => x.Peso).HasPrecision(18, 3); → selecciona una propiedad escalar para continuar su configuración.</code>
Línea 160	<code>entity.Property(x => x.Activa).HasDefaultValue(true); → selecciona una propiedad escalar para continuar su configuración.</code>
Línea 161	<code>entity.HasOne(x => x.Orden) → inicia la configuración de una navegación de referencia.</code>
Línea 162	<code>.WithMany(o => o.Planchas) → establece el extremo de colección de una relación uno-a-muchos.</code>
Línea 163	<code>.HasForeignKey(x => x.OrdenId) → declara explícitamente la propiedad o propiedades de clave foránea.</code>
Línea 164	<code>.OnDelete(DeleteBehavior.Cascade) → define el comportamiento de eliminación de la relación.</code>
Línea 165	<code>.IsRequired(); → marca la propiedad o relación como obligatoria.</code>
Línea 166	<code>}); → cierra la lambda o configuración encadenada y finaliza la llamada.</code>
Línea 167	<code>modelBuilder.Entity<Aleacion>(entity => → selecciona una entidad del modelo para aplicar su configuración Fluent API.</code>
Línea 168	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 169	<code>entity.ToTable("Aleaciones"); → asigna explícitamente la tabla de SQL Server usada por la entidad.</code>
Línea 170	<code>entity.HasKey(a => a.Id); → define la clave primaria de la entidad.</code>
Línea 171	<code>entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100); → selecciona una propiedad escalar para continuar su configuración.</code>
Línea 172	<code>entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20); → selecciona una propiedad escalar para continuar su configuración.</code>
Línea 173	<code>entity.Property(a => a.Descripcion).HasMaxLength(500); → selecciona una propiedad escalar para continuar su configuración.</code>
Línea 174	<code>}); → cierra la lambda o configuración encadenada y finaliza la llamada.</code>
Línea 175	<code>modelBuilder.Entity<EstadoOrden>(entity => → selecciona una entidad del modelo para aplicar su configuración Fluent API.</code>
Línea 176	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 177	<code>entity.ToTable("EstadosOrden"); → asigna explícitamente la tabla de SQL Server usada por la entidad.</code>
Línea 178	<code>entity.HasKey(e => e.Id); → define la clave primaria de la entidad.</code>
Línea 179	<code>entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50); → selecciona una propiedad escalar para continuar su configuración.</code>

Línea 180	<code>entity.Property(e => e.Descripcion).HasMaxLength(250);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 181	<code>entity.Property(e => e.Activo).HasDefaultValue(true);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 182	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 183	<code>modelBuilder.Entity<DetalleOrden>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 184	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 185	<code>entity.ToTable("DetallesOrden");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 186	<code>entity.HasKey(d => d.Id);</code> → define la clave primaria de la entidad.
Línea 187	<code>entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 188	<code>entity.Property(d => d.Notas).HasMaxLength(500);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 189	<code>entity.HasOne(d => d.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 190	<code>.WithOne(o => o.Detalle)</code> → establece el otro extremo de referencia de una relación uno-a-uno.
Línea 191	<code>.HasForeignKey<DetalleOrden>(d => d.OrdenId)</code> → declara explícitamente la propiedad o propiedades de clave foránea.
Línea 192	<code>.OnDelete(DeleteBehavior.Cascade)</code> → define el comportamiento de eliminación de la relación.
Línea 193	<code>.IsRequired();</code> → marca la propiedad o relación como obligatoria.
Línea 194	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 196	<code>modelBuilder.Entity<CertificadoCalidad>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 197	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 198	<code>entity.ToTable("CertificadosCalidad");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 199	<code>entity.HasKey(c => c.Id);</code> → define la clave primaria de la entidad.
Línea 200	<code>entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 201	<code>entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 202	<code>entity.HasOne(c => c.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 203	<code>.WithOne(o => o.Certificado)</code> → establece el otro extremo de referencia de una relación uno-a-uno.
Línea 204	<code>.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)</code> → declara explícitamente la propiedad o propiedades de clave foránea.
Línea 205	<code>.OnDelete(DeleteBehavior.Cascade)</code> → define el comportamiento de eliminación de la relación.

Línea 206	<code>.IsRequired();</code> → marca la propiedad o relación como obligatoria.
Línea 207	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 208	<code>modelBuilder.Entity<OrdenAleacion>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 209	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 210	<code>entity.ToTable("OrdenesAleaciones");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 211	<code>entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });</code> → define la clave primaria de la entidad.
Línea 212	<code>entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 213	<code>entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 214	<code>entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 215	<code>entity.HasOne(x => x.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 216	<code>.WithMany(o => o.OrdenesAleaciones)</code> → establece el extremo de colección de una relación uno-a-muchos.
Línea 217	<code>.HasForeignKey(x => x.OrdenFabricacionId)</code> → declara explícitamente la propiedad o propiedades de clave foránea.
Línea 218	<code>.OnDelete(DeleteBehavior.Cascade);</code> → define el comportamiento de eliminación de la relación.
Línea 219	<code>entity.HasOne(x => x.Aleacion)</code> → inicia la configuración de una navegación de referencia.
Línea 220	<code>.WithMany(a => a.OrdenesAleaciones)</code> → establece el extremo de colección de una relación uno-a-muchos.
Línea 221	<code>.HasForeignKey(x => x.AleacionId)</code> → declara explícitamente la propiedad o propiedades de clave foránea.
Línea 222	<code>.OnDelete(DeleteBehavior.Restrict);</code> → define el comportamiento de eliminación de la relación.
Línea 223	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 224	<code>}</code> → cierra el bloque de código actual.
Línea 225	<code>}</code> → cierra el bloque de código actual.
Línea 227	<code>public interface IOrdenRepositorio</code> → declara el contrato IOrdenRepositorio para desacoplar consumidores e implementación.
Línea 228	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 229	<code>List<OrdenFabricacion> ObtenerTodas();</code> → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.
Línea 230	<code>OrdenFabricacion? ObtenerPorId(int id);</code> → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 231	<code>OrdenFabricacion? ObtenerPorNumero(string numeroOrden);</code> → declara en el contrato la búsqueda por el identificador de negocio <code>NumeroOrden</code> .
Línea 232	<code>void Agregar(OrdenFabricacion orden);</code> → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.
Línea 233	<code>void Eliminar(OrdenFabricacion orden);</code> → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.
Línea 234	<code>int Guardar();</code> → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.
Línea 235	<code>}</code> → cierra el bloque de código actual.
Línea 237	<code>public sealed class OrdenRepositorio : IOrdenRepositorio</code> → declara la clase <code>OrdenRepositorio</code> que forma parte del estado acumulativo del ejercicio.
Línea 238	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 239	<code>private readonly AceriaDbContext _context;</code> → declara el campo privado <code>_context</code> de tipo <code>AceriaDbContext</code> que conserva una dependencia de la clase.
Línea 240	<code>public OrdenRepositorio(AceriaDbContext context) => _context = context;</code> → declara el constructor o método <code>OrdenRepositorio</code> con los parámetros necesarios para este componente.
Línea 242	<code>public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();</code> → ordena el resultado de la consulta antes de materializarlo.
Línea 243	<code>public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);</code> → implementa la búsqueda por clave primaria mediante <code>Find</code> , aprovechando primero el <code>ChangeTracker</code> y después la base de datos.
Línea 244	<code>public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);</code> → recupera la primera coincidencia o <code>null</code> cuando no existe.
Línea 245	<code>public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);</code> → registra una entidad o relación nueva en el <code>ChangeTracker</code> para insertarla al guardar.
Línea 246	<code>public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);</code> → marca la entidad indicada para borrado al confirmar los cambios.
Línea 247	<code>public int Guardar() => _context.SaveChanges();</code> → persiste en SQL Server los cambios seguidos por el <code>DbContext</code> .
Línea 248	<code>}</code> → cierra el bloque de código actual.
Línea 250	<code>public interface IServicioOrdenes</code> → declara el contrato <code>IServicioOrdenes</code> para desacoplar consumidores e implementación.
Línea 251	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 252	<code>string ObtenerResumen(int ordenId);</code> → declara el servicio que construye un resumen de una orden a partir de su identificador.
Línea 253	<code>}</code> → cierra el bloque de código actual.
Línea 255	<code>public sealed class ServicioOrdenes : IServicioOrdenes</code> → declara la clase <code>ServicioOrdenes</code> que forma parte del estado acumulativo del ejercicio.
Línea 256	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 257	<code>private readonly IOrdenRepositorio _repositorio;</code> → declara el campo privado <code>_repositorio</code> de tipo <code>IOrdenRepositorio</code> que conserva una dependencia de la clase.
Línea 258	<code>private readonly AceriaDbContext _context;</code> → declara el campo privado <code>_context</code> de tipo <code>AceriaDbContext</code> que conserva una dependencia de la clase.
Línea 259	<code>public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)</code> → declara el constructor o método <code>ServicioOrdenes</code> con los parámetros necesarios para este componente.
Línea 260	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 261	<code>_repositorio = repositorio;</code> → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.
Línea 262	<code>_context = context;</code> → guarda el <code>DbContext</code> inyectado para consultar los datos necesarios durante el servicio.
Línea 263	<code>}</code> → cierra el bloque de código actual.
Línea 265	<code>public string ObtenerResumen(int ordenId)</code> → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.
Línea 266	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 267	<code>var orden = _repositorio.ObtenerPorId(ordenId);</code> → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.
Línea 268	<code>if (orden is null) return \$"Orden {ordenId} no encontrada";</code> → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.
Línea 269	<code>var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);</code> → cuenta las filas que cumplen la consulta construida.
Línea 270	<code>return \$"Orden {orden.NumeroOrden} Cliente: {orden.Cliente} Planchas: {totalPlanchas}";</code> → devuelve el resultado calculado al código que invocó el método.
Línea 271	<code>}</code> → cierra el bloque de código actual.
Línea 272	<code>}</code> → cierra el bloque de código actual.
Línea 274	<code>public static class Program</code> → declara la clase <code>Program</code> que forma parte del estado acumulativo del ejercicio.
Línea 275	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 276	<code>public static void Main()</code> → declara el punto de entrada ejecutable de la aplicación de consola.
Línea 277	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 278	<code>var configuration = new ConfigurationBuilder()</code> → inicia la construcción de la configuración de la aplicación.
Línea 279	<code>.SetBasePath(AppContext.BaseDirectory)</code> → establece la ruta base usada para localizar los archivos de configuración.
Línea 280	<code>.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)</code> → incorpora <code>appsettings.json</code> a la configuración.
Línea 281	<code>.AddEnvironmentVariables()</code> → incorpora variables de entorno como fuente adicional de configuración.
Línea 282	<code>.Build();</code> → materializa la configuración o el objeto encadenado construido en las líneas anteriores.

Línea 284	<code>var connectionString = configuration.GetConnectionString("AceriaDB")</code> → obtiene la cadena de conexión AceriaDB desde la configuración.
Línea 285	<code>?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");</code> → interrumpe el arranque con un error explícito si la configuración no contiene <code>ConnectionStrings:AceriaDB</code> .
Línea 287	<code>var services = new ServiceCollection();</code> → crea la colección donde se registran las dependencias de la aplicación.
Línea 288	<code>services.AddDbContext<AceriaDbContext>(options =></code> → registra el <code>DbContext</code> en el contenedor de inyección de dependencias.
Línea 289	<code>options</code> → continúa la configuración del <code>DbContext</code> sobre el parámetro <code>options</code> recibido por la lambda de <code>AddDbContext</code> .
Línea 290	<code>.UseSqlServer(connectionString, sql =></code> → selecciona SQL Server como proveedor de EF Core.
Línea 291	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 292	<code>sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);</code> → habilita reintentos ante fallos transitorios del proveedor SQL Server.
Línea 293	<code>sql.CommandTimeout(60);</code> → establece el tiempo máximo permitido para los comandos SQL.
Línea 294	<code>})</code> → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.
Línea 295	<code>.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)</code> → configura la salida del logging de EF Core.
Línea 296	<code>.EnableDetailedErrors();</code> → activa errores detallados para facilitar el diagnóstico del laboratorio.
Línea 298	<code>services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();</code> → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.
Línea 299	<code>services.AddScoped<IServicioOrdenes, ServicioOrdenes>();</code> → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.
Línea 301	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions</code> → construye el <code>ServiceProvider</code> con validación de servicios y garantiza su liberación al salir del ámbito mediante <code>using var</code> .
Línea 302	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 303	<code>ValidateScopes = true,</code> → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.
Línea 304	<code>ValidateOnBuild = true</code> → obliga al contenedor a validar las resoluciones registradas al construir el <code>ServiceProvider</code> .
Línea 305	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 307	<code>using var scope = provider.CreateScope();</code> → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.
Línea 308	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → resuelve un servicio obligatorio desde el contenedor de dependencias.
Línea 309	<code>context.Database.EnsureDeleted();</code> → elimina la base del laboratorio para reproducir el escenario desde un estado limpio.

Línea 310	<code>context.Database.Migrate();</code> → aplica las migraciones pendientes y deja AceriaDB en el esquema del punto.
Línea 312	<code>var orden = new OrdenFabricacion</code> → crea la entidad <code>OrdenFabricacion</code> que concentra el grafo de datos del escenario acumulativo.
Línea 313	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 314	<code>NumeroOrden = "OF-M2-0001",</code> → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.
Línea 315	<code>Cliente = "Constructora del Norte",</code> → asigna el cliente de la orden utilizada en el escenario E2E del punto.
Línea 316	<code>FechaCreacion = DateTime.UtcNow,</code> → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.
Línea 317	<code>FechaEntrega = DateTime.Today.AddDays(14),</code> → establece una fecha de entrega prevista catorce días después de la fecha local actual.
Línea 318	<code>Estado = "Pendiente",</code> → inicializa la orden con el estado <code>Pendiente</code> , coherente con el flujo del laboratorio.
Línea 319	<code>Observaciones = "Orden de validación M2",</code> → añade texto opcional para comprobar el mapeo de <code>Observaciones</code> .
Línea 320	<code>Planchas =</code> → inicia el inicializador de la colección de planchas asociadas a la orden.
Línea 321	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 322	<code>new PlanchaAcero</code> → crea una nueva plancha dentro del inicializador de la colección <code>Planchas</code> de la orden.
Línea 323	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 324	<code>Espesor = 10.5,</code> → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.
Línea 325	<code>Ancho = 1500,</code> → asigna el ancho de la plancha usada en el escenario de persistencia.
Línea 326	<code>Largo = 3000,</code> → asigna el largo de la plancha usada en el escenario de persistencia.
Línea 327	<code>Peso = 371.250m,</code> → asigna un peso decimal con tres posiciones para validar la precisión configurada.
Línea 328	<code>Activa = true,</code> → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.
Línea 329	<code>}</code> → cierra el bloque de código actual.
Línea 330	<code>},</code> → continúa el inicializador o la llamada iniciada en el bloque actual.
Línea 331	<code>Detalle = new DetalleOrden</code> → crea el detalle uno-a-uno que se persistirá junto con la orden principal.
Línea 332	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 333	<code>ComposicionQuimica = "C: 0.45%, Mn: 0.75%",</code> → asigna la composición química utilizada para verificar la entidad <code>DetalleOrden</code> .
Línea 334	<code>TemperaturaColada = 1550.5,</code> → asigna la temperatura de colada del detalle de fabricación.
Línea 335	<code>Notas = "Colada principal"</code> → asigna las notas opcionales del detalle asociado a la orden.

Línea 336	<code>},</code> → continúa el inicializador o la llamada iniciada en el bloque actual.
Línea 337	<code>Certificado = new CertificadoCalidad</code> → crea el certificado uno-a-uno asociado a la orden del escenario.
Línea 338	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 339	<code>NumeroCertificado = "CERT-0001",</code> → asigna el identificador natural del certificado de calidad.
Línea 340	<code>FechaEmision = DateTime.Today,</code> → asigna la fecha de emisión del certificado usado en la prueba.
Línea 341	<code>OrganismoCertificador = "Laboratorio Aceria"</code> → asigna el organismo que emite el certificado de calidad.
Línea 342	<code>},</code> → continúa el inicializador o la llamada iniciada en el bloque actual.
Línea 343	<code>};</code> → cierra el inicializador del objeto y finaliza la instrucción.
Línea 345	<code>var aleacion = new Aleacion</code> → crea la aleación que se enlazarán con la orden mediante la entidad intermedia <code>OrdenAleacion</code> .
Línea 346	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 347	<code>Nombre = "AISI 1045",</code> → asigna el nombre comercial de la aleación utilizada en el escenario.
Línea 348	<code>Codigo = "A1045",</code> → asigna el código natural único de la aleación.
Línea 349	<code>PorcentajeCarbono = 0.45,</code> → asigna el porcentaje de carbono dentro del rango permitido por la restricción del modelo.
Línea 350	<code>PorcentajeManganeso = 0.75,</code> → asigna el porcentaje de manganeso dentro del rango permitido por la restricción del modelo.
Línea 351	<code>Descripcion = "Acero medio en carbono"</code> → asigna una descripción opcional a la aleación.
Línea 352	<code>};</code> → cierra el inicializador del objeto y finaliza la instrucción.
Línea 353	<code>orden.OrdenesAleaciones.Add(new OrdenAleacion</code> → registra una entidad o relación nueva en el <code>ChangeTracker</code> para insertarla al guardar.
Línea 354	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 355	<code>Aleacion = aleacion,</code> → conecta la entidad intermedia <code>OrdenAleacion</code> con la aleación creada para el escenario.
Línea 356	<code>CantidadUtilizada = 1500.500m,</code> → registra la cantidad utilizada con tres decimales, coherente con <code>HasPrecision(18, 3)</code> .
Línea 357	<code>EstadoRelacion = "Activa"</code> → marca como activa la relación entre la orden y la aleación.
Línea 358	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 359	<code>context.OrdenesFabricacion.Add(orden);</code> → registra una entidad o relación nueva en el <code>ChangeTracker</code> para insertarla al guardar.
Línea 360	<code>context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });</code> → registra una entidad o relación nueva en el <code>ChangeTracker</code> para insertarla al guardar.
Línea 361	<code>context.SaveChanges();</code> → persiste en SQL Server los cambios seguidos por el <code>DbContext</code> .

Línea 363	<code>var entity = context.Model.FindEntityType(typeof(OrdenFabricacion))!; → obtiene los metadatos EF Core de OrdenFabricacion para inspeccionar claves, índices o filtros del modelo construido.</code>
Línea 364	<code>global::System.Console.WriteLine(\$"2.7 OK Fluent API Estado default: {entity.FindProperty(nameof(OrdenFabricacion.Estado))?.GetDefaultValue()}"); → escribe en consola la evidencia usada para verificar el comportamiento del estado.</code>
Línea 365	<code>}</code> → cierra el bloque de código actual.
Línea 366	<code>}</code> → cierra el bloque de código actual.

Punto 2.8 - Claves primarias, alternativas y compuestas

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL que trabajan con .NET 8, EF Core 8 y SQL Server LocalDB.

Proyecto: El estado 2.8 parte de 2.7 y tiene como objetivo formalizar claves simples, alternativas y compuestas.

Objetivos de aprendizaje

- Distinguir clave primaria de clave alternativa.
- Usar `HasAlternateKey`.
- Mantener la clave compuesta de `OrdenAleacion`.
- Comprender la unicidad generada por una `alternate key`.
- Inspeccionar las claves del modelo.
- Preparar el modelo para índices y restricciones.

Paso 1: Abrir la solución autónoma del punto

```
cd M02/PROYECTO/2.8
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

No se abre una solución global situada en la raíz del repositorio. `AceriaData.sln` está dentro de la carpeta del punto y referencia únicamente los proyectos de ese estado.

Paso 2: Identificar el cambio respecto al estado anterior

El proyecto conserva todo lo terminado en 2.7. En 2.8 se introduce exclusivamente el contenido que corresponde a **Claves primarias, alternativas y compuestas**. La práctica no reinicia `AceriaData` ni crea un ejemplo paralelo.

Paso 3: Implementar y comprender la configuración principal

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.HasKey(o => o.Id);
    entity.HasAlternateKey(o => o.NumeroOrden)
        .HasName("AK_OrdenesFabricacion_NumeroOrden");
});

modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
});
```

CSHARP

Línea 1	<code>HasKey(...)</code> → configura la clave primaria.
Línea 2	<code>HasAlternateKey(...)</code> → configura una clave candidata adicional con unicidad.

Línea 3	HasName(...) → da un nombre estable a la restricción en SQL Server.
Línea 4	new { x.OrdenFabricacionId, x.AleacionId } → forma una clave compuesta con dos propiedades.
Línea 5	GetKeys() → permite inspeccionar las claves configuradas en metadatos.

Paso 4: Generar y aplicar la migración acumulativa

El repositorio contiene una migración real generada para este estado. Si se reproduce el ejercicio desde el estado anterior, los comandos son:

```
dotnet ef migrations add M2_2_8
dotnet ef database update
```

POWERSHELL

dotnet ef migrations add compara el modelo actual con el snapshot heredado. dotnet ef database update aplica únicamente los cambios pendientes.

En los estados ejecutables del curso se usa Database.Migrate() para aplicar el historial durante la demostración. El laboratorio puede usar EnsureDeleted() para empezar desde una base limpia, pero **no usa EnsureCreated()**, porque el esquema está gobernado por Migrations.

Paso 5: Compilar y ejecutar el estado

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

POWERSHELL

Resultado esperado: la ejecución termina sin excepción y contiene la evidencia 2.8 OK. GitHub Actions repite esta compilación y ejecución sobre Windows y SQL Server LocalDB.

Paso 6: Verificar el estado acumulativo

Además de ejecutar el ejemplo, conviene revisar el modelo y la base:

```
dotnet ef migrations list
```

POWERSHELL

Cuando el punto modifica el esquema, la lista debe contener la migración acumulativa correspondiente. En SQL Server Object Explorer se puede comprobar que las tablas, claves, relaciones, índices o restricciones coinciden con el modelo del punto.

Errores comunes

Error	Causa	Solución
Se confunde índice único con alternate key	Tienen finalidades de modelo diferentes	Usar HasAlternateKey para una clave candidata; índices se estudian en 2.9.
PK compuesta incompleta	Falta una de las columnas	Configurar ambas propiedades en HasKey.
Valor alternativo repetido	Viola la restricción UNIQUE	Validar NumeroOrden/Codigo antes de guardar.

Reto resuelto

Reto: Intentar insertar dos órdenes con el mismo NumeroOrden y observar que la clave alternativa protege la unicidad.

Solución: partir del código de M02/PROYECTO/2.8, realizar únicamente el cambio descrito y volver a ejecutar build, migraciones y el programa. El reto no introduce conceptos reservados a un punto posterior.

Analogía final

Las claves son los identificadores y restricciones de identidad del sistema; una clave alternativa es otro identificador candidato, no sólo una ayuda de rendimiento.

Resultado esperado

Al terminar 2.8, la solución local compila, el proyecto se ejecuta, el esquema se actualiza mediante Migrations cuando corresponde y la salida contiene 2.8 OK. El código conserva todos los cambios válidos de los puntos anteriores.

Conexión con el siguiente punto

El siguiente estado es 2.9. Se parte del proyecto completo de 2.8; no se vuelve a crear AceriaData desde cero.

Código acumulativo completo del estado 2.8

El siguiente archivo es el Program.cs real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

CSHARP

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
}
```

```

    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }
    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
    }
}

```

```

modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.ToTable("OrdenesFabricacion");
    entity.HasKey(o => o.Id);
    entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
    entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
    entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
    entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
    entity.Property(o => o.Observaciones).HasMaxLength(500);
    entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");
});
modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.ToTable("PlanchasAcero");
    entity.HasKey(x => x.Id);
    entity.Property(x => x.Peso).HasPrecision(18, 3);
    entity.Property(x => x.Activa).HasDefaultValue(true);
    entity.HasOne(x => x.Orden)
        .WithMany(o => o.Planchas)
        .HasForeignKey(x => x.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
modelBuilder.Entity<Aleacion>(entity =>
{
    entity.ToTable("Aleaciones");
    entity.HasKey(a => a.Id);
    entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
    entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
    entity.Property(a => a.Descripcion).HasMaxLength(500);
    entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");
});
modelBuilder.Entity<EstadoOrden>(entity =>
{
    entity.ToTable("EstadosOrden");
    entity.HasKey(e => e.Id);
    entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
    entity.Property(e => e.Descripcion).HasMaxLength(250);
    entity.Property(e => e.Activo).HasDefaultValue(true);
});
modelBuilder.Entity<DetalleOrden>(entity =>
{
    entity.ToTable("DetallesOrden");
    entity.HasKey(d => d.Id);
    entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
    entity.Property(d => d.Notas).HasMaxLength(500);
    entity.HasOne(d => d.Orden)
        .WithOne(o => o.Detalle)
        .HasForeignKey<DetalleOrden>(d => d.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
modelBuilder.Entity<CertificadoCalidad>(entity =>
{
    entity.ToTable("CertificadosCalidad");
    entity.HasKey(c => c.Id);
    entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
    entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
    entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");
    entity.HasOne(c => c.Orden)
        .WithOne(o => o.Certificado)
        .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.ToTable("OrdenesAleaciones");
    entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
    entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
    entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
    entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
    entity.HasOne(x => x.Orden)
        .WithMany(o => o.OrdenesAleaciones)
        .HasForeignKey(x => x.OrdenFabricacionId)
        .OnDelete(DeleteBehavior.Cascade);
    entity.HasOne(x => x.Aleacion)
        .WithMany(a => a.OrdenesAleaciones)
        .HasForeignKey(x => x.AleacionId)
        .OnDelete(DeleteBehavior.Restrict);
});
}

```

```

public interface IOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                    {
                        sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                        sql.CommandTimeout(60);
                    })
                .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        context.Database.EnsureDeleted();
    }
}

```

```

context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },
    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrganizacionesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});
context.OrganizacionesFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo =
true });
context.SaveChanges();

var entity = context.Model.FindEntityType(typeof(OrdenFabricacion));
global::System.Console.WriteLine($"2.8 OK | Alternate keys: {entity.GetKeys().Count()} | PK:
{string.Join(", ", entity.FindPrimaryKey().Properties.Select(x => x.Name))}");
}
}

```

Explicación línea a línea del código acumulativo 2.8

Las referencias siguientes usan la numeración real de M02/PROYECTO/2.8/Program.cs. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1	using System.ComponentModel.DataAnnotations; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 2	using System.ComponentModel.DataAnnotations.Schema; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 3	using Microsoft.EntityFrameworkCore; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 4	using Microsoft.Extensions.Configuration; → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 5	<code>using Microsoft.Extensions.DependencyInjection;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 6	<code>using Microsoft.Extensions.Logging;</code> → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.
Línea 8	<code>namespace AceriaData.ConsoleApp;</code> → declara el espacio de nombres del estado acumulativo.
Línea 10	<code>[Table("OrdenesFabricacion")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 11	<code>public class OrdenFabricacion</code> → declara la clase OrdenFabricacion que forma parte del estado acumulativo del ejercicio.
Línea 12	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 13	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 14	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 15	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 16	<code>[MaxLength(50)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 17	<code>public string NumeroOrden { get; set; } = string.Empty;</code> → declara la propiedad NumeroOrden de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 18	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 19	<code>[MaxLength(200)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 20	<code>public string Cliente { get; set; } = string.Empty;</code> → declara la propiedad Cliente de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 21	<code>public DateTime FechaCreacion { get; set; }</code> → declara la propiedad FechaCreacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.
Línea 22	<code>public DateTime? FechaEntrega { get; set; }</code> → declara la propiedad FechaEntrega de tipo DateTime? que EF Core o la lógica de negocio utiliza en este modelo.
Línea 23	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 24	<code>[MaxLength(50)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 25	<code>public string Estado { get; set; } = "Pendiente";</code> → declara la propiedad Estado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 26	<code>[MaxLength(500)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 27	<code>public string? Observaciones { get; set; }</code> → declara la propiedad Observaciones de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.
Línea 28	<code>public List<PlanchaAcero> Planchas { get; set; } = new();</code> → declara la propiedad Planchas de tipo List<PlanchaAcero> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 29	<code>public DetalleOrden? Detalle { get; set; }</code> → declara la propiedad Detalle de tipo DetalleOrden? que EF Core o la lógica de negocio utiliza en este modelo.
Línea 30	<code>public CertificadoCalidad? Certificado { get; set; }</code> → declara la propiedad Certificado de tipo CertificadoCalidad? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 31	<code>public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();</code> → declara la propiedad <code>OrdenesAleaciones</code> de tipo <code>List<OrdenAleacion></code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 32	<code>}</code> → cierra el bloque de código actual.
Línea 34	<code>[Table("PlanchasAcero")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 35	<code>public class PlanchaAcero</code> → declara la clase <code>PlanchaAcero</code> que forma parte del estado acumulativo del ejercicio.
Línea 36	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 37	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 38	<code>public int Id { get; set; }</code> → declara la propiedad <code>Id</code> de tipo <code>int</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 39	<code>public int OrdenId { get; set; }</code> → declara la propiedad <code>OrdenId</code> de tipo <code>int</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 40	<code>public double Espesor { get; set; }</code> → declara la propiedad <code>Espesor</code> de tipo <code>double</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 41	<code>public double Ancho { get; set; }</code> → declara la propiedad <code>Ancho</code> de tipo <code>double</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 42	<code>public double Largo { get; set; }</code> → declara la propiedad <code>Largo</code> de tipo <code>double</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 43	<code>[Precision(18, 3)]</code> → configura precisión y escala para la propiedad numérica siguiente.
Línea 44	<code>public decimal Peso { get; set; }</code> → declara la propiedad <code>Peso</code> de tipo <code>decimal</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 45	<code>public bool Activa { get; set; } = true;</code> → declara la propiedad <code>Activa</code> de tipo <code>bool</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 46	<code>[ForeignKey(nameof(OrdenId))]</code> → vincula explícitamente la navegación siguiente con su clave foránea.
Línea 47	<code>public OrdenFabricacion Orden { get; set; } = null!;</code> → declara la propiedad <code>Orden</code> de tipo <code>OrdenFabricacion</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 48	<code>}</code> → cierra el bloque de código actual.
Línea 50	<code>[Table("Aleaciones")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 51	<code>public class Aleacion</code> → declara la clase <code>Aleacion</code> que forma parte del estado acumulativo del ejercicio.
Línea 52	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 53	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 54	<code>public int Id { get; set; }</code> → declara la propiedad <code>Id</code> de tipo <code>int</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 55	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 56	<code>[MaxLength(100)]</code> → limita la longitud máxima persistida de la propiedad siguiente.

Línea 57	<code>public string Nombre { get; set; } = string.Empty;</code> → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 58	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 59	<code>[MaxLength(20)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 60	<code>public string Codigo { get; set; } = string.Empty;</code> → declara la propiedad Codigo de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 61	<code>public double PorcentajeCarbono { get; set; }</code> → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.
Línea 62	<code>public double PorcentajeManganeso { get; set; }</code> → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.
Línea 63	<code>[MaxLength(500)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 64	<code>public string? Descripcion { get; set; }</code> → declara la propiedad Descripcion de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.
Línea 65	<code>public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();</code> → declara la propiedad OrdenesAleaciones de tipo List<OrdenAleacion> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 66	<code>}</code> → cierra el bloque de código actual.
Línea 68	<code>[Table("EstadosOrden")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 69	<code>public class EstadoOrden</code> → declara la clase EstadoOrden que forma parte del estado acumulativo del ejercicio.
Línea 70	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 71	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 72	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 73	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 74	<code>[MaxLength(50)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 75	<code>public string Nombre { get; set; } = string.Empty;</code> → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 76	<code>[MaxLength(250)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 77	<code>public string Descripcion { get; set; } = string.Empty;</code> → declara la propiedad Descripcion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 78	<code>public bool Activo { get; set; } = true;</code> → declara la propiedad Activo de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.
Línea 79	<code>}</code> → cierra el bloque de código actual.
Línea 81	<code>[Table("DetallesOrden")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 82	<code>public class DetalleOrden</code> → declara la clase DetalleOrden que forma parte del estado acumulativo del ejercicio.

Línea 83	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 84	[Key] → marca la propiedad siguiente como clave primaria simple.
Línea 85	public int Id { get; set; } → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 86	public int OrdenId { get; set; } → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 87	[Required] → marca la propiedad siguiente como requerida.
Línea 88	[MaxLength(200)] → limita la longitud máxima persistida de la propiedad siguiente.
Línea 89	public string ComposicionQuimica { get; set; } = string.Empty; → declara la propiedad ComposicionQuimica de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 90	public double TemperaturaColada { get; set; } → declara la propiedad TemperaturaColada de tipo double que EF Core o la lógica de negocio utiliza en este modelo.
Línea 91	[MaxLength(500)] → limita la longitud máxima persistida de la propiedad siguiente.
Línea 92	public string? Notas { get; set; } → declara la propiedad Notas de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.
Línea 93	[ForeignKey(nameof(OrdenId))] → vincula explícitamente la navegación siguiente con su clave foránea.
Línea 94	public OrdenFabricacion Orden { get; set; } = null!; → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.
Línea 95	} → cierra el bloque de código actual.
Línea 97	[Table("CertificadosCalidad")] → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 98	public class CertificadoCalidad → declara la clase CertificadoCalidad que forma parte del estado acumulativo del ejercicio.
Línea 99	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 100	[Key] → marca la propiedad siguiente como clave primaria simple.
Línea 101	public int Id { get; set; } → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 102	public int OrdenId { get; set; } → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.
Línea 103	[Required] → marca la propiedad siguiente como requerida.
Línea 104	[MaxLength(50)] → limita la longitud máxima persistida de la propiedad siguiente.
Línea 105	public string NumeroCertificado { get; set; } = string.Empty; → declara la propiedad NumeroCertificado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.
Línea 106	public DateTime FechaEmision { get; set; } → declara la propiedad FechaEmision de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.
Línea 107	[Required] → marca la propiedad siguiente como requerida.
Línea 108	[MaxLength(100)] → limita la longitud máxima persistida de la propiedad siguiente.

Línea 109	<code>public string OrganismoCertificador { get; set; } = string.Empty;</code> → declara la propiedad <code>OrganismoCertificador</code> de tipo <code>string</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 110	<code>[ForeignKey(nameof(OrdenId))]</code> → vincula explícitamente la navegación siguiente con su clave foránea.
Línea 111	<code>public OrdenFabricacion Orden { get; set; } = null!;</code> → declara la propiedad <code>Orden</code> de tipo <code>OrdenFabricacion</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 112	<code>}</code> → cierra el bloque de código actual.
Línea 114	<code>[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]</code> → declara una clave primaria compuesta con las propiedades indicadas.
Línea 115	<code>[Table("OrdenesAleaciones")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 116	<code>public class OrdenAleacion</code> → declara la clase <code>OrdenAleacion</code> que forma parte del estado acumulativo del ejercicio.
Línea 117	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 118	<code>public int OrdenFabricacionId { get; set; }</code> → declara la propiedad <code>OrdenFabricacionId</code> de tipo <code>int</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 119	<code>public int AleacionId { get; set; }</code> → declara la propiedad <code>AleacionId</code> de tipo <code>int</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 120	<code>public DateTime FechaAsignacion { get; set; } = DateTime.Now;</code> → declara la propiedad <code>FechaAsignacion</code> de tipo <code>DateTime</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 121	<code>[Precision(18, 3)]</code> → configura precisión y escala para la propiedad numérica siguiente.
Línea 122	<code>public decimal CantidadUtilizada { get; set; }</code> → declara la propiedad <code>CantidadUtilizada</code> de tipo <code>decimal</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 123	<code>[MaxLength(20)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 124	<code>public string EstadoRelacion { get; set; } = "Activa";</code> → declara la propiedad <code>EstadoRelacion</code> de tipo <code>string</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 125	<code>public OrdenFabricacion Orden { get; set; } = null!;</code> → declara la propiedad <code>Orden</code> de tipo <code>OrdenFabricacion</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 126	<code>public Aleacion Aleacion { get; set; } = null!;</code> → declara la propiedad <code>Aleacion</code> de tipo <code>Aleacion</code> que EF Core o la lógica de negocio utiliza en este modelo.
Línea 127	<code>}</code> → cierra el bloque de código actual.
Línea 129	<code>public class AceriaDbContext : DbContext</code> → declara la clase <code>AceriaDbContext</code> que forma parte del estado acumulativo del ejercicio.
Línea 130	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 131	<code>public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }</code> → declara el constructor o método <code>AceriaDbContext</code> con los parámetros necesarios para este componente.
Línea 133	<code>public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 134	<code>public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 135	<code>public DbSet<Aleacion> Aleaciones => Set<Aleacion>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 136	<code>public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 137	<code>public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 138	<code>public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 139	<code>public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();</code> → expone un conjunto de entidades para consultar y persistir mediante EF Core.
Línea 142	<code>protected override void OnModelCreating(ModelBuilder modelBuilder)</code> → sobrescribe <code>OnModelCreating</code> para definir configuración explícita del modelo.
Línea 143	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 144	<code>base.OnModelCreating(modelBuilder);</code> → sobrescribe <code>OnModelCreating</code> para definir configuración explícita del modelo.
Línea 145	<code>modelBuilder.Entity<OrdenFabricacion>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 146	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 147	<code>entity.ToTable("OrdenesFabricacion");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 148	<code>entity.HasKey(o => o.Id);</code> → define la clave primaria de la entidad.
Línea 149	<code>entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 150	<code>entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 151	<code>entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 152	<code>entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 153	<code>entity.Property(o => o.Observaciones).HasMaxLength(500);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 154	<code>entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");</code> → define una clave alternativa que también debe ser única.
Línea 155	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 156	<code>modelBuilder.Entity<PlanchaAcero>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 157	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 158	<code>entity.ToTable("PlanchasAcero");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 159	<code>entity.HasKey(x => x.Id);</code> → define la clave primaria de la entidad.
Línea 160	<code>entity.Property(x => x.Peso).HasPrecision(18, 3);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 161	<code>entity.Property(x => x.Activa).HasDefaultValue(true);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 162	<code>entity.HasOne(x => x.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 163	<code>.WithMany(o => o.Planchas)</code> → establece el extremo de colección de una relación uno-a-muchos.
Línea 164	<code>.HasForeignKey(x => x.OrdenId)</code> → declara explícitamente la propiedad o propiedades de clave foránea.
Línea 165	<code>.OnDelete(DeleteBehavior.Cascade)</code> → define el comportamiento de eliminación de la relación.
Línea 166	<code>.IsRequired();</code> → marca la propiedad o relación como obligatoria.
Línea 167	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 168	<code>modelBuilder.Entity<Aleacion>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 169	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 170	<code>entity.ToTable("Aleaciones");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 171	<code>entity.HasKey(a => a.Id);</code> → define la clave primaria de la entidad.
Línea 172	<code>entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 173	<code>entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 174	<code>entity.Property(a => a.Descripcion).HasMaxLength(500);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 175	<code>entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");</code> → define una clave alternativa que también debe ser única.
Línea 176	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 177	<code>modelBuilder.Entity<EstadoOrden>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 178	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 179	<code>entity.ToTable("EstadosOrden");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 180	<code>entity.HasKey(e => e.Id);</code> → define la clave primaria de la entidad.
Línea 181	<code>entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 182	<code>entity.Property(e => e.Descripcion).HasMaxLength(250);</code> → selecciona una propiedad escalar para continuar su configuración.

Línea 183	<code>entity.Property(e => e.Activo).HasDefaultValue(true);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 184	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 185	<code>modelBuilder.Entity<DetalleOrden>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 186	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 187	<code>entity.ToTable("DetallesOrden");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 188	<code>entity.HasKey(d => d.Id);</code> → define la clave primaria de la entidad.
Línea 189	<code>entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 190	<code>entity.Property(d => d.Notas).HasMaxLength(500);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 191	<code>entity.HasOne(d => d.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 192	<code>.WithOne(o => o.Detalle)</code> → establece el otro extremo de referencia de una relación uno-a-uno.
Línea 193	<code>.HasForeignKey<DetalleOrden>(d => d.OrdenId)</code> → declara explícitamente la propiedad o propiedades de clave foránea.
Línea 194	<code>.OnDelete(DeleteBehavior.Cascade)</code> → define el comportamiento de eliminación de la relación.
Línea 195	<code>.IsRequired();</code> → marca la propiedad o relación como obligatoria.
Línea 196	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 198	<code>modelBuilder.Entity<CertificadoCalidad>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 199	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 200	<code>entity.ToTable("CertificadosCalidad");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 201	<code>entity.HasKey(c => c.Id);</code> → define la clave primaria de la entidad.
Línea 202	<code>entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 203	<code>entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 204	<code>entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");</code> → define una clave alternativa que también debe ser única.
Línea 205	<code>entity.HasOne(c => c.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 206	<code>.WithOne(o => o.Certificado)</code> → establece el otro extremo de referencia de una relación uno-a-uno.
Línea 207	<code>.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)</code> → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 208	<code>.OnDelete(DeleteBehavior.Cascade)</code> → define el comportamiento de eliminación de la relación.
Línea 209	<code>.IsRequired();</code> → marca la propiedad o relación como obligatoria.
Línea 210	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 211	<code>modelBuilder.Entity<OrdenAleacion>(entity =></code> → selecciona una entidad del modelo para aplicar su configuración Fluent API.
Línea 212	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 213	<code>entity.ToTable("OrdenesAleaciones");</code> → asigna explícitamente la tabla de SQL Server usada por la entidad.
Línea 214	<code>entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });</code> → define la clave primaria de la entidad.
Línea 215	<code>entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 216	<code>entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 217	<code>entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 218	<code>entity.HasOne(x => x.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 219	<code>.WithMany(o => o.OrdenesAleaciones)</code> → establece el extremo de colección de una relación uno-a-muchos.
Línea 220	<code>.HasForeignKey(x => x.OrdenFabricacionId)</code> → declara explícitamente la propiedad o propiedades de clave foránea.
Línea 221	<code>.OnDelete(DeleteBehavior.Cascade);</code> → define el comportamiento de eliminación de la relación.
Línea 222	<code>entity.HasOne(x => x.Aleacion)</code> → inicia la configuración de una navegación de referencia.
Línea 223	<code>.WithMany(a => a.OrdenesAleaciones)</code> → establece el extremo de colección de una relación uno-a-muchos.
Línea 224	<code>.HasForeignKey(x => x.AleacionId)</code> → declara explícitamente la propiedad o propiedades de clave foránea.
Línea 225	<code>.OnDelete(DeleteBehavior.Restrict);</code> → define el comportamiento de eliminación de la relación.
Línea 226	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 227	<code>}</code> → cierra el bloque de código actual.
Línea 228	<code>}</code> → cierra el bloque de código actual.
Línea 230	<code>public interface IOrdenRepositorio</code> → declara el contrato IOrdenRepositorio para desacoplar consumidores e implementación.
Línea 231	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 232	<code>List<OrdenFabricacion> ObtenerTodas();</code> → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 233	<code>OrdenFabricacion? ObtenerPorId(int id);</code> → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.
Línea 234	<code>OrdenFabricacion? ObtenerPorNumero(string numeroOrden);</code> → declara en el contrato la búsqueda por el identificador de negocio <code>NumeroOrden</code> .
Línea 235	<code>void Agregar(OrdenFabricacion orden);</code> → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.
Línea 236	<code>void Eliminar(OrdenFabricacion orden);</code> → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.
Línea 237	<code>int Guardar();</code> → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.
Línea 238	<code>}</code> → cierra el bloque de código actual.
Línea 240	<code>public sealed class OrdenRepositorio : IOrdenRepositorio</code> → declara la clase <code>OrdenRepositorio</code> que forma parte del estado acumulativo del ejercicio.
Línea 241	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 242	<code>private readonly AceriaDbContext _context;</code> → declara el campo privado <code>_context</code> de tipo <code>AceriaDbContext</code> que conserva una dependencia de la clase.
Línea 243	<code>public OrdenRepositorio(AceriaDbContext context) => _context = context;</code> → declara el constructor o método <code>OrdenRepositorio</code> con los parámetros necesarios para este componente.
Línea 245	<code>public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();</code> → ordena el resultado de la consulta antes de materializarlo.
Línea 246	<code>public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);</code> → implementa la búsqueda por clave primaria mediante <code>Find</code> , aprovechando primero el <code>ChangeTracker</code> y después la base de datos.
Línea 247	<code>public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);</code> → recupera la primera coincidencia o null cuando no existe.
Línea 248	<code>public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);</code> → registra una entidad o relación nueva en el <code>ChangeTracker</code> para insertarla al guardar.
Línea 249	<code>public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);</code> → marca la entidad indicada para borrado al confirmar los cambios.
Línea 250	<code>public int Guardar() => _context.SaveChanges();</code> → persiste en SQL Server los cambios seguidos por el <code>DbContext</code> .
Línea 251	<code>}</code> → cierra el bloque de código actual.
Línea 253	<code>public interface IServicioOrdenes</code> → declara el contrato <code>IServicioOrdenes</code> para desacoplar consumidores e implementación.
Línea 254	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 255	<code>string ObtenerResumen(int ordenId);</code> → declara el servicio que construye un resumen de una orden a partir de su identificador.
Línea 256	<code>}</code> → cierra el bloque de código actual.

Línea 258	<code>public sealed class ServicioOrdenes : IServicioOrdenes</code> → declara la clase <code>ServicioOrdenes</code> que forma parte del estado acumulativo del ejercicio.
Línea 259	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 260	<code>private readonly IOrdenRepositorio _repositorio;</code> → declara el campo privado <code>_repositorio</code> de tipo <code>IOrdenRepositorio</code> que conserva una dependencia de la clase.
Línea 261	<code>private readonly AceriaDbContext _context;</code> → declara el campo privado <code>_context</code> de tipo <code>AceriaDbContext</code> que conserva una dependencia de la clase.
Línea 262	<code>public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)</code> → declara el constructor o método <code>ServicioOrdenes</code> con los parámetros necesarios para este componente.
Línea 263	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 264	<code>_repositorio = repositorio;</code> → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.
Línea 265	<code>_context = context;</code> → guarda el <code>DbContext</code> inyectado para consultar los datos necesarios durante el servicio.
Línea 266	<code>}</code> → cierra el bloque de código actual.
Línea 268	<code>public string ObtenerResumen(int ordenId)</code> → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.
Línea 269	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 270	<code>var orden = _repositorio.ObtenerPorId(ordenId);</code> → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.
Línea 271	<code>if (orden is null) return \$"Orden {ordenId} no encontrada";</code> → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.
Línea 272	<code>var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);</code> → cuenta las filas que cumplen la consulta construida.
Línea 273	<code>return \$"Orden {orden.NumeroOrden} Cliente: {orden.Cliente} Planchas: {totalPlanchas}";</code> → devuelve el resultado calculado al código que invocó el método.
Línea 274	<code>}</code> → cierra el bloque de código actual.
Línea 275	<code>}</code> → cierra el bloque de código actual.
Línea 277	<code>public static class Program</code> → declara la clase <code>Program</code> que forma parte del estado acumulativo del ejercicio.
Línea 278	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 279	<code>public static void Main()</code> → declara el punto de entrada ejecutable de la aplicación de consola.
Línea 280	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 281	<code>var configuration = new ConfigurationBuilder()</code> → inicia la construcción de la configuración de la aplicación.
Línea 282	<code>.SetBasePath(AppContext.BaseDirectory)</code> → establece la ruta base usada para localizar los archivos de configuración.
Línea 283	<code>.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)</code> → incorpora <code>appsettings.json</code> a la configuración.

Línea 284	<code>.AddEnvironmentVariables()</code> → incorpora variables de entorno como fuente adicional de configuración.
Línea 285	<code>.Build();</code> → materializa la configuración o el objeto encadenado construido en las líneas anteriores.
Línea 287	<code>var connectionString = configuration.GetConnectionString("AceriaDB")</code> → obtiene la cadena de conexión AceriaDB desde la configuración.
Línea 288	<code>?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");</code> → interrumpe el arranque con un error explícito si la configuración no contiene <code>ConnectionStrings:AceriaDB</code> .
Línea 290	<code>var services = new ServiceCollection();</code> → crea la colección donde se registran las dependencias de la aplicación.
Línea 291	<code>services.AddDbContext<AceriaDbContext>(options =></code> → registra el <code>DbContext</code> en el contenedor de inyección de dependencias.
Línea 292	<code>options</code> → continúa la configuración del <code>DbContext</code> sobre el parámetro <code>options</code> recibido por la lambda de <code>AddDbContext</code> .
Línea 293	<code>.UseSqlServer(connectionString, sql =></code> → selecciona SQL Server como proveedor de EF Core.
Línea 294	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 295	<code>sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);</code> → habilita reintentos ante fallos transitorios del proveedor SQL Server.
Línea 296	<code>sql.CommandTimeout(60);</code> → establece el tiempo máximo permitido para los comandos SQL.
Línea 297	<code>})</code> → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.
Línea 298	<code>.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)</code> → configura la salida del logging de EF Core.
Línea 299	<code>.EnableDetailedErrors();</code> → activa errores detallados para facilitar el diagnóstico del laboratorio.
Línea 301	<code>services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();</code> → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.
Línea 302	<code>services.AddScoped<IServicioOrdenes, ServicioOrdenes>();</code> → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.
Línea 304	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions</code> → construye el <code>ServiceProvider</code> con validación de servicios y garantiza su liberación al salir del ámbito mediante <code>using var</code> .
Línea 305	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 306	<code>ValidateScopes = true,</code> → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.
Línea 307	<code>ValidateOnBuild = true</code> → obliga al contenedor a validar las resoluciones registradas al construir el <code>ServiceProvider</code> .
Línea 308	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 310	<code>using var scope = provider.CreateScope();</code> → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 311	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → resuelve un servicio obligatorio desde el contenedor de dependencias.
Línea 312	<code>context.Database.EnsureDeleted();</code> → elimina la base del laboratorio para reproducir el escenario desde un estado limpio.
Línea 313	<code>context.Database.Migrate();</code> → aplica las migraciones pendientes y deja AceriaDB en el esquema del punto.
Línea 315	<code>var orden = new OrdenFabricacion</code> → crea la entidad OrdenFabricacion que concentra el grafo de datos del escenario acumulativo.
Línea 316	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 317	<code>NumeroOrden = "OF-M2-0001",</code> → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.
Línea 318	<code>Cliente = "Constructora del Norte",</code> → asigna el cliente de la orden utilizada en el escenario E2E del punto.
Línea 319	<code>FechaCreacion = DateTime.UtcNow,</code> → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.
Línea 320	<code>FechaEntrega = DateTime.Today.AddDays(14),</code> → establece una fecha de entrega prevista catorce días después de la fecha local actual.
Línea 321	<code>Estado = "Pendiente",</code> → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.
Línea 322	<code>Observaciones = "Orden de validación M2",</code> → añade texto opcional para comprobar el mapeo de Observaciones.
Línea 323	<code>Planchas =</code> → inicia el inicializador de la colección de planchas asociadas a la orden.
Línea 324	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 325	<code>new PlanchaAcero</code> → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.
Línea 326	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 327	<code>Espesor = 10.5,</code> → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.
Línea 328	<code>Ancho = 1500,</code> → asigna el ancho de la plancha usada en el escenario de persistencia.
Línea 329	<code>Largo = 3000,</code> → asigna el largo de la plancha usada en el escenario de persistencia.
Línea 330	<code>Peso = 371.250m,</code> → asigna un peso decimal con tres posiciones para validar la precisión configurada.
Línea 331	<code>Activa = true,</code> → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.
Línea 332	<code>}</code> → cierra el bloque de código actual.
Línea 333	<code>},</code> → continúa el inicializador o la llamada iniciada en el bloque actual.
Línea 334	<code>Detalle = new DetalleOrden</code> → crea el detalle uno-a-uno que se persistirá junto con la orden principal.
Línea 335	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 336	<code>ComposicionQuimica = "C: 0.45%, Mn: 0.75%",</code> → asigna la composición química utilizada para verificar la entidad <code>DetalleOrden</code> .
Línea 337	<code>TemperaturaColada = 1550.5,</code> → asigna la temperatura de colada del detalle de fabricación.
Línea 338	<code>Notas = "Colada principal"</code> → asigna las notas opcionales del detalle asociado a la orden.
Línea 339	<code>},</code> → continúa el inicializador o la llamada iniciada en el bloque actual.
Línea 340	<code>Certificado = new CertificadoCalidad</code> → crea el certificado uno-a-uno asociado a la orden del escenario.
Línea 341	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 342	<code>NumeroCertificado = "CERT-0001",</code> → asigna el identificador natural del certificado de calidad.
Línea 343	<code>FechaEmision = DateTime.Today,</code> → asigna la fecha de emisión del certificado usado en la prueba.
Línea 344	<code>OrganismoCertificador = "Laboratorio Aceria"</code> → asigna el organismo que emite el certificado de calidad.
Línea 345	<code>},</code> → continúa el inicializador o la llamada iniciada en el bloque actual.
Línea 346	<code>};</code> → cierra el inicializador del objeto y finaliza la instrucción.
Línea 348	<code>var aleacion = new Aleacion</code> → crea la aleación que se enlazarán con la orden mediante la entidad intermedia <code>OrdenAleacion</code> .
Línea 349	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 350	<code>Nombre = "AISI 1045",</code> → asigna el nombre comercial de la aleación utilizada en el escenario.
Línea 351	<code>Codigo = "A1045",</code> → asigna el código natural único de la aleación.
Línea 352	<code>PorcentajeCarbono = 0.45,</code> → asigna el porcentaje de carbono dentro del rango permitido por la restricción del modelo.
Línea 353	<code>PorcentajeManganeso = 0.75,</code> → asigna el porcentaje de manganeso dentro del rango permitido por la restricción del modelo.
Línea 354	<code>Descripcion = "Acero medio en carbono"</code> → asigna una descripción opcional a la aleación.
Línea 355	<code>};</code> → cierra el inicializador del objeto y finaliza la instrucción.
Línea 356	<code>orden.OrdenesAleaciones.Add(new OrdenAleacion</code> → registra una entidad o relación nueva en el <code>ChangeTracker</code> para insertarla al guardar.
Línea 357	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 358	<code>Aleacion = aleacion,</code> → conecta la entidad intermedia <code>OrdenAleacion</code> con la aleación creada para el escenario.
Línea 359	<code>CantidadUtilizada = 1500.500m,</code> → registra la cantidad utilizada con tres decimales, coherente con <code>HasPrecision(18, 3)</code> .
Línea 360	<code>EstadoRelacion = "Activa"</code> → marca como activa la relación entre la orden y la aleación.
Línea 361	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 362	<code>context.OrdenesFabricacion.Add(orden);</code> → registra una entidad o relación nueva en el <code>ChangeTracker</code> para insertarla al guardar.

Línea 363	<code>context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });</code> → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.
Línea 364	<code>context.SaveChanges();</code> → persiste en SQL Server los cambios seguidos por el DbContext.
Línea 366	<code>var entity = context.Model.FindEntityType(typeof(OrdenFabricacion))!;</code> → obtiene los metadatos EF Core de OrdenFabricacion para inspeccionar claves, índices o filtros del modelo construido.
Línea 367	<code>global::System.Console.WriteLine(\$"2.8 OK Alternate keys: {entity.GetKeys().Count()} PK: {string.Join(", ", entity.FindPrimaryKey().Properties.Select(x => x.Name))}");</code> → imprime el marcador E2E de 2.8, el número de claves del modelo y las propiedades de la clave primaria.
Línea 368	<code>}</code> → cierra el bloque de código actual.
Línea 369	<code>}</code> → cierra el bloque de código actual.

Punto 2.9 - Índices y restricciones

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL que trabajan con .NET 8, EF Core 8 y SQL Server LocalDB.

Proyecto: El estado 2.9 parte de 2.8 y tiene como objetivo añadir índices simples, compuestos, filtrados y cubrientes, además de restricciones CHECK.

Objetivos de aprendizaje

- Crear índices con HasIndex.
- Configurar índices compuestos.
- Configurar índices filtrados.
- Usar IncludeProperties en SQL Server.
- Añadir restricciones CHECK.
- Verificar que migraciones y modelo permanecen sincronizados.

Paso 1: Abrir la solución autónoma del punto

```
cd M02/PROYECTO/2.9
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

No se abre una solución global situada en la raíz del repositorio. `AceriaData.sln` está dentro de la carpeta del punto y referencia únicamente los proyectos de ese estado.

Paso 2: Identificar el cambio respecto al estado anterior

El proyecto conserva todo lo terminado en 2.8. En 2.9 se introduce exclusivamente el contenido que corresponde a **Índices y restricciones**. La práctica no reinicia `AceriaData` ni crea un ejemplo paralelo.

Paso 3: Implementar y comprender la configuración principal

```
entity.HasIndex(o => new { o.Cliente, o.FechaCreacion })
    .HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");

entity.HasIndex(o => o.FechaEntrega)
    .HasFilter("[Estado] = 'Pendiente'")
    .HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");

entity.HasIndex(o => o.Estado)
```

CSHARP

```
.IncludeProperties(o => new { o.NumeroOrden, o.Cliente, o.FechaCreacion })
.HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");
```

Línea 1	HasIndex(...) → crea un índice para las propiedades seleccionadas.
Línea 2	new { o.Cliente, o.FechaCreacion } → define un índice compuesto respetando el orden de columnas.
Línea 3	HasFilter(...) → limita el índice a las filas que cumplen una condición SQL.
Línea 4	IncludeProperties(...) → añade columnas incluidas al índice de SQL Server.
Línea 5	HasCheckConstraint(...) → traslada una regla de integridad al propio motor de base de datos.

Paso 4: Generar y aplicar la migración acumulativa

El repositorio contiene una migración real generada para este estado. Si se reproduce el ejercicio desde el estado anterior, los comandos son:

```
dotnet ef migrations add M2_2_9
dotnet ef database update
```

POWERSHELL

`dotnet ef migrations add` compara el modelo actual con el snapshot heredado. `dotnet ef database update` aplica únicamente los cambios pendientes.

En los estados ejecutables del curso se usa `Database.Migrate()` para aplicar el historial durante la demostración. El laboratorio puede usar `EnsureDeleted()` para empezar desde una base limpia, pero **no usa** `EnsureCreated()`, porque el esquema está gobernado por Migrations.

Paso 5: Compilar y ejecutar el estado

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

POWERSHELL

Resultado esperado: la ejecución termina sin excepción y contiene la evidencia 2.9 OK. GitHub Actions repite esta compilación y ejecución sobre Windows y SQL Server LocalDB.

Paso 6: Verificar el estado acumulativo

Además de ejecutar el ejemplo, conviene revisar el modelo y la base:

```
dotnet ef migrations list
```

POWERSHELL

Cuando el punto modifica el esquema, la lista debe contener la migración acumulativa correspondiente. En SQL Server Object Explorer se puede comprobar que las tablas, claves, relaciones, índices o restricciones coinciden con el modelo del punto.

Errores comunes

Error	Causa	Solución
Índice redundante	Se indexan las mismas columnas sin necesidad	Relacionar cada índice con consultas reales.
CHECK falla al migrar	Existen datos incompatibles	Corregir datos antes de aplicar la restricción.
Filtro de índice no coincide con SQL Server	Expresión SQL inválida	Usar sintaxis válida para el proveedor real.

Reto resuelto

Reto: Intentar guardar una plancha con Espesor negativo y comprobar que SQL Server rechaza la operación por CHECK.

Solución: partir del código de M02/PROYECTO/2.9, realizar únicamente el cambio descrito y volver a ejecutar build, migraciones y el programa. El reto no introduce conceptos reservados a un punto posterior.

Analogía final

Los índices son caminos de acceso y las restricciones son controles de calidad en la propia base de datos.

Resultado esperado

Al terminar 2.9, la solución local compila, el proyecto se ejecuta, el esquema se actualiza mediante Migrations cuando corresponde y la salida contiene 2.9 OK. El código conserva todos los cambios válidos de los puntos anteriores.

Conexión con el siguiente punto

El siguiente estado es 2.10. Se parte del proyecto completo de 2.9; no se vuelve a crear AceriaData desde cero.

Código acumulativo completo del estado 2.9

El siguiente archivo es el Program.cs real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

CSHARP

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("Aleaciones")]
public class Aleacion
```

```

{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }
    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
}

```

```

public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
    modelBuilder.Entity<OrdenFabricacion>(entity =>
    {
        entity.ToTable("OrdenesFabricacion");
        entity.HasKey(o => o.Id);
        entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
        entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
        entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
        entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
        entity.Property(o => o.Observaciones).HasMaxLength(500);
        entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");
        entity.HasIndex(o => o.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");
        entity.HasIndex(o => new { o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");
        entity.HasIndex(o => o.FechaEntrega).HasFilter("[Estado] = 'Pendiente'");
        entity.HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");
        entity.HasIndex(o => o.Estado).IncludeProperties(o => new { o.NumeroOrden, o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");
    });
    modelBuilder.Entity<PlanchaAcero>(entity =>
    {
        entity.ToTable("PlanchasAcero", t =>
        {
            t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");
            t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");
            t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");
            t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");
        });
        entity.HasKey(x => x.Id);
        entity.Property(x => x.Peso).HasPrecision(18, 3);
        entity.Property(x => x.Activa).HasDefaultValue(true);
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.Planchas)
            .HasForeignKey(x => x.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
        entity.HasIndex(x => new { x.OrdenId, x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");
        entity.HasIndex(x => x.Espesor).HasFilter("[Activa] = 1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");
    });
    modelBuilder.Entity<Aleacion>(entity =>
    {
        entity.ToTable("Aleaciones", t =>
        {
            t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND [PorcentajeCarbono] <= 2");
            t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND [PorcentajeManganeso] <= 5");
        });
        entity.HasKey(a => a.Id);
        entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
        entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
        entity.Property(a => a.Descripcion).HasMaxLength(500);
        entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");
        entity.HasIndex(a => a.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");
        entity.HasIndex(a => new { a.PorcentajeCarbono,
a.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");
    });
    modelBuilder.Entity<EstadoOrden>(entity =>
    {
        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
        entity.Property(e => e.Descripcion).HasMaxLength(250);
        entity.Property(e => e.Activo).HasDefaultValue(true);
    });
    modelBuilder.Entity<DetalleOrden>(entity =>
    {
        entity.ToTable("DetallesOrden");
        entity.HasKey(d => d.Id);
        entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
        entity.Property(d => d.Notas).HasMaxLength(500);
        entity.HasOne(d => d.Orden)
            .WithOne(o => o.Detalle)
            .HasForeignKey<DetalleOrden>(d => d.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
}

```

```

modelBuilder.Entity<CertificadoCalidad>(entity =>
{
    entity.ToTable("CertificadosCalidad");
    entity.HasKey(c => c.Id);
    entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
    entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
    entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");
    entity.HasIndex(c => c.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");
    entity.HasOne(c => c.Orden)
        .WithOne(o => o.Certificado)
        .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.ToTable("OrdenesAleaciones");
    entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
    entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
    entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
    entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
    entity.HasOne(x => x.Orden)
        .WithMany(o => o.OrdenesAleaciones)
        .HasForeignKey(x => x.OrdenFabricacionId)
        .OnDelete(DeleteBehavior.Cascade);
    entity.HasOne(x => x.Aleacion)
        .WithMany(a => a.OrdenesAleaciones)
        .HasForeignKey(x => x.AleacionId)
        .OnDelete(DeleteBehavior.Restrict);
    entity.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");
    entity.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] = 'Activa'");
    entity.HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");
});
}
}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

```

```

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                {
                    sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                    sql.CommandTimeout(60);
                })
                .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
                    LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        context.Database.EnsureDeleted();
        context.Database.Migrate();

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-M2-0001",
            Cliente = "Constructora del Norte",
            FechaCreacion = DateTime.UtcNow,
            FechaEntrega = DateTime.Today.AddDays(14),
            Estado = "Pendiente",
            Observaciones = "Orden de validación M2",
            Planchas =
            {
                new PlanchaAcero
                {
                    Espesor = 10.5,
                    Ancho = 1500,
                    Largo = 3000,
                    Peso = 371.250m,
                    Activa = true,
                }
            },
            Detalle = new DetalleOrden
            {
                ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
                TemperaturaColada = 1550.5,
                Notas = "Colada principal"
            },
            Certificado = new CertificadoCalidad
            {
                NumeroCertificado = "CERT-0001",
                FechaEmision = DateTime.Today,
                OrganismoCertificador = "Laboratorio Aceria"
            },
        };

        var aleacion = new Aleacion
        {
            Nombre = "AISI 1045",
           Codigo = "A1045",
            PorcentajeCarbono = 0.45,
            PorcentajeManganeso = 0.75,
            Descripcion = "Acero medio en carbono"
        };
        orden.OrdenesAleaciones.Add(new OrdenAleacion
        {
            Aleacion = aleacion,
            CantidadUtilizada = 1500.500m,
            EstadoRelacion = "Activa"
        });
    }
}

```

```

    });
    context.OrdenesFabricacion.Add(orden);
    context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo =
true });
    context.SaveChanges();

    var entity = context.Model.FindEntityType(typeof(OrdenFabricacion));
    global::System.Console.WriteLine($"2.9 OK | Índices: {string.Join(", ", entity.GetIndexes().Select(i =>
i.GetDatabaseName()))}");
    }
}

```

Explicación línea a línea del código acumulativo 2.9

Las referencias usan la numeración real de M02/PROYECTO/2.9/Program.cs. Se explican todas las líneas no vacías del archivo que compila.

Línea 1	using System.ComponentModel.DataAnnotations; → importa el espacio de nombres necesario para este archivo.
Línea 2	using System.ComponentModel.DataAnnotations.Schema; → importa el espacio de nombres necesario para este archivo.
Línea 3	using Microsoft.EntityFrameworkCore; → importa el espacio de nombres necesario para este archivo.
Línea 4	using Microsoft.Extensions.Configuration; → importa el espacio de nombres necesario para este archivo.
Línea 5	using Microsoft.Extensions.DependencyInjection; → importa el espacio de nombres necesario para este archivo.
Línea 6	using Microsoft.Extensions.Logging; → importa el espacio de nombres necesario para este archivo.
Línea 8	namespace AceriaData.ConsoleApp; → declara el espacio de nombres del estado acumulativo.
Línea 10	[Table("OrdenesFabricacion")] → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 11	public class OrdenFabricacion → declara la clase OrdenFabricacion que forma parte del estado acumulativo.
Línea 12	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 13	[Key] → marca la propiedad siguiente como clave primaria simple.
Línea 14	public int Id { get; set; } → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.
Línea 15	[Required] → marca la propiedad siguiente como requerida.
Línea 16	[MaxLength(50)] → limita la longitud máxima persistida de la propiedad siguiente.
Línea 17	public string NumeroOrden { get; set; } = string.Empty; → declara la propiedad NumeroOrden de tipo string usada por el modelo o la lógica de negocio.
Línea 18	[Required] → marca la propiedad siguiente como requerida.
Línea 19	[MaxLength(200)] → limita la longitud máxima persistida de la propiedad siguiente.
Línea 20	public string Cliente { get; set; } = string.Empty; → declara la propiedad Cliente de tipo string usada por el modelo o la lógica de negocio.

Línea 21	<code>public DateTime FechaCreacion { get; set; } → declara la propiedad FechaCreacion de tipo DateTime usada por el modelo o la lógica de negocio.</code>
Línea 22	<code>public DateTime? FechaEntrega { get; set; } → declara la propiedad FechaEntrega de tipo DateTime? usada por el modelo o la lógica de negocio.</code>
Línea 23	<code>[Required] → marca la propiedad siguiente como requerida.</code>
Línea 24	<code>[MaxLength(50)] → limita la longitud máxima persistida de la propiedad siguiente.</code>
Línea 25	<code>public string Estado { get; set; } = "Pendiente"; → declara la propiedad Estado de tipo string usada por el modelo o la lógica de negocio.</code>
Línea 26	<code>[MaxLength(500)] → limita la longitud máxima persistida de la propiedad siguiente.</code>
Línea 27	<code>public string? Observaciones { get; set; } → declara la propiedad Observaciones de tipo string? usada por el modelo o la lógica de negocio.</code>
Línea 28	<code>public List<PlanchaAcero> Planchas { get; set; } = new(); → declara la propiedad Planchas de tipo List<PlanchaAcero> usada por el modelo o la lógica de negocio.</code>
Línea 29	<code>public DetalleOrden? Detalle { get; set; } → declara la propiedad Detalle de tipo DetalleOrden? usada por el modelo o la lógica de negocio.</code>
Línea 30	<code>public CertificadoCalidad? Certificado { get; set; } → declara la propiedad Certificado de tipo CertificadoCalidad? usada por el modelo o la lógica de negocio.</code>
Línea 31	<code>public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new(); → declara la propiedad OrdenesAleaciones de tipo List<OrdenAleacion> usada por el modelo o la lógica de negocio.</code>
Línea 32	<code>} → cierra el bloque de código actual.</code>
Línea 34	<code>[Table("PlanchasAcero")] → fija mediante Data Annotations la tabla relacional de la entidad siguiente.</code>
Línea 35	<code>public class PlanchaAcero → declara la clase PlanchaAcero que forma parte del estado acumulativo.</code>
Línea 36	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 37	<code>[Key] → marca la propiedad siguiente como clave primaria simple.</code>
Línea 38	<code>public int Id { get; set; } → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.</code>
Línea 39	<code>public int OrdenId { get; set; } → declara la propiedad OrdenId de tipo int usada por el modelo o la lógica de negocio.</code>
Línea 40	<code>public double Espesor { get; set; } → declara la propiedad Espesor de tipo double usada por el modelo o la lógica de negocio.</code>
Línea 41	<code>public double Ancho { get; set; } → declara la propiedad Ancho de tipo double usada por el modelo o la lógica de negocio.</code>
Línea 42	<code>public double Largo { get; set; } → declara la propiedad Largo de tipo double usada por el modelo o la lógica de negocio.</code>
Línea 43	<code>[Precision(18, 3)] → configura precisión y escala para la propiedad numérica siguiente.</code>
Línea 44	<code>public decimal Peso { get; set; } → declara la propiedad Peso de tipo decimal usada por el modelo o la lógica de negocio.</code>

Línea 45	<code>public bool Activa { get; set; } = true;</code> → declara la propiedad Activa de tipo bool usada por el modelo o la lógica de negocio.
Línea 46	<code>[ForeignKey(nameof(OrdenId))]</code> → vincula la navegación siguiente con su clave foránea.
Línea 47	<code>public OrdenFabricacion Orden { get; set; } = null!;</code> → declara la propiedad Orden de tipo OrdenFabricacion usada por el modelo o la lógica de negocio.
Línea 48	<code>}</code> → cierra el bloque de código actual.
Línea 50	<code>[Table("Aleaciones")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 51	<code>public class Aleacion</code> → declara la clase Aleacion que forma parte del estado acumulativo.
Línea 52	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 53	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 54	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.
Línea 55	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 56	<code>[MaxLength(100)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 57	<code>public string Nombre { get; set; } = string.Empty;</code> → declara la propiedad Nombre de tipo string usada por el modelo o la lógica de negocio.
Línea 58	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 59	<code>[MaxLength(20)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 60	<code>public string Codigo { get; set; } = string.Empty;</code> → declara la propiedad Codigo de tipo string usada por el modelo o la lógica de negocio.
Línea 61	<code>public double PorcentajeCarbono { get; set; }</code> → declara la propiedad PorcentajeCarbono de tipo double usada por el modelo o la lógica de negocio.
Línea 62	<code>public double PorcentajeManganeso { get; set; }</code> → declara la propiedad PorcentajeManganeso de tipo double usada por el modelo o la lógica de negocio.
Línea 63	<code>[MaxLength(500)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 64	<code>public string? Descripcion { get; set; }</code> → declara la propiedad Descripcion de tipo string? usada por el modelo o la lógica de negocio.
Línea 65	<code>public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();</code> → declara la propiedad OrdenesAleaciones de tipo List<OrdenAleacion> usada por el modelo o la lógica de negocio.
Línea 66	<code>}</code> → cierra el bloque de código actual.
Línea 68	<code>[Table("EstadosOrden")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 69	<code>public class EstadoOrden</code> → declara la clase EstadoOrden que forma parte del estado acumulativo.
Línea 70	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 71	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.

Línea 72	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.
Línea 73	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 74	<code>[MaxLength(50)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 75	<code>public string Nombre { get; set; } = string.Empty;</code> → declara la propiedad Nombre de tipo string usada por el modelo o la lógica de negocio.
Línea 76	<code>[MaxLength(250)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 77	<code>public string Descripcion { get; set; } = string.Empty;</code> → declara la propiedad Descripcion de tipo string usada por el modelo o la lógica de negocio.
Línea 78	<code>public bool Activo { get; set; } = true;</code> → declara la propiedad Activo de tipo bool usada por el modelo o la lógica de negocio.
Línea 79	<code>}</code> → cierra el bloque de código actual.
Línea 81	<code>[Table("DetallesOrden")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 82	<code>public class DetalleOrden</code> → declara la clase DetalleOrden que forma parte del estado acumulativo.
Línea 83	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 84	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 85	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.
Línea 86	<code>public int OrdenId { get; set; }</code> → declara la propiedad OrdenId de tipo int usada por el modelo o la lógica de negocio.
Línea 87	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 88	<code>[MaxLength(200)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 89	<code>public string ComposicionQuimica { get; set; } = string.Empty;</code> → declara la propiedad ComposicionQuimica de tipo string usada por el modelo o la lógica de negocio.
Línea 90	<code>public double TemperaturaColada { get; set; }</code> → declara la propiedad TemperaturaColada de tipo double usada por el modelo o la lógica de negocio.
Línea 91	<code>[MaxLength(500)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 92	<code>public string? Notas { get; set; }</code> → declara la propiedad Notas de tipo string? usada por el modelo o la lógica de negocio.
Línea 93	<code>[ForeignKey(nameof(OrdenId))]</code> → vincula la navegación siguiente con su clave foránea.
Línea 94	<code>public OrdenFabricacion Orden { get; set; } = null!;</code> → declara la propiedad Orden de tipo OrdenFabricacion usada por el modelo o la lógica de negocio.
Línea 95	<code>}</code> → cierra el bloque de código actual.
Línea 97	<code>[Table("CertificadosCalidad")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 98	<code>public class CertificadoCalidad</code> → declara la clase <code>CertificadoCalidad</code> que forma parte del estado acumulativo.
Línea 99	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 100	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 101	<code>public int Id { get; set; }</code> → declara la propiedad <code>Id</code> de tipo <code>int</code> usada por el modelo o la lógica de negocio.
Línea 102	<code>public int OrdenId { get; set; }</code> → declara la propiedad <code>OrdenId</code> de tipo <code>int</code> usada por el modelo o la lógica de negocio.
Línea 103	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 104	<code>[MaxLength(50)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 105	<code>public string NumeroCertificado { get; set; } = string.Empty;</code> → declara la propiedad <code>NumeroCertificado</code> de tipo <code>string</code> usada por el modelo o la lógica de negocio.
Línea 106	<code>public DateTime FechaEmision { get; set; }</code> → declara la propiedad <code>FechaEmision</code> de tipo <code>DateTime</code> usada por el modelo o la lógica de negocio.
Línea 107	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 108	<code>[MaxLength(100)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 109	<code>public string OrganismoCertificador { get; set; } = string.Empty;</code> → declara la propiedad <code>OrganismoCertificador</code> de tipo <code>string</code> usada por el modelo o la lógica de negocio.
Línea 110	<code>[ForeignKey(nameof(OrdenId))]</code> → vincula la navegación siguiente con su clave foránea.
Línea 111	<code>public OrdenFabricacion Orden { get; set; } = null!;</code> → declara la propiedad <code>Orden</code> de tipo <code>OrdenFabricacion</code> usada por el modelo o la lógica de negocio.
Línea 112	<code>}</code> → cierra el bloque de código actual.
Línea 114	<code>[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]</code> → declara una clave primaria compuesta.
Línea 115	<code>[Table("OrdenesAleaciones")]</code> → fija mediante <code>Data Annotations</code> la tabla relacional de la entidad siguiente.
Línea 116	<code>public class OrdenAleacion</code> → declara la clase <code>OrdenAleacion</code> que forma parte del estado acumulativo.
Línea 117	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 118	<code>public int OrdenFabricacionId { get; set; }</code> → declara la propiedad <code>OrdenFabricacionId</code> de tipo <code>int</code> usada por el modelo o la lógica de negocio.
Línea 119	<code>public int AleacionId { get; set; }</code> → declara la propiedad <code>AleacionId</code> de tipo <code>int</code> usada por el modelo o la lógica de negocio.
Línea 120	<code>public DateTime FechaAsignacion { get; set; } = DateTime.Now;</code> → declara la propiedad <code>FechaAsignacion</code> de tipo <code>DateTime</code> usada por el modelo o la lógica de negocio.
Línea 121	<code>[Precision(18, 3)]</code> → configura precisión y escala para la propiedad numérica siguiente.
Línea 122	<code>public decimal CantidadUtilizada { get; set; }</code> → declara la propiedad <code>CantidadUtilizada</code> de tipo <code>decimal</code> usada por el modelo o la lógica de negocio.

Línea 123	<code>[MaxLength(20)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 124	<code>public string EstadoRelacion { get; set; } = "Activa";</code> → declara la propiedad EstadoRelacion de tipo string usada por el modelo o la lógica de negocio.
Línea 125	<code>public OrdenFabricacion Orden { get; set; } = null!;</code> → declara la propiedad Orden de tipo OrdenFabricacion usada por el modelo o la lógica de negocio.
Línea 126	<code>public Aleacion Aleacion { get; set; } = null!;</code> → declara la propiedad Aleacion de tipo Aleacion usada por el modelo o la lógica de negocio.
Línea 127	<code>}</code> → cierra el bloque de código actual.
Línea 129	<code>public class AceriaDbContext : DbContext</code> → declara la clase AceriaDbContext que forma parte del estado acumulativo.
Línea 130	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 131	<code>public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }</code> → recibe las opciones del DbContext y las pasa a la clase base.
Línea 133	<code>public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();</code> → expone un conjunto de entidades para consultas y persistencia.
Línea 134	<code>public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();</code> → expone un conjunto de entidades para consultas y persistencia.
Línea 135	<code>public DbSet<Aleacion> Aleaciones => Set<Aleacion>();</code> → expone un conjunto de entidades para consultas y persistencia.
Línea 136	<code>public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();</code> → expone un conjunto de entidades para consultas y persistencia.
Línea 137	<code>public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();</code> → expone un conjunto de entidades para consultas y persistencia.
Línea 138	<code>public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();</code> → expone un conjunto de entidades para consultas y persistencia.
Línea 139	<code>public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();</code> → expone un conjunto de entidades para consultas y persistencia.
Línea 142	<code>protected override void OnModelCreating(ModelBuilder modelBuilder)</code> → entra en la configuración explícita del modelo.
Línea 143	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 144	<code>base.OnModelCreating(modelBuilder);</code> → entra en la configuración explícita del modelo.
Línea 145	<code>modelBuilder.Entity<OrdenFabricacion>(entity =></code> → selecciona una entidad para configurarla con Fluent API.
Línea 146	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 147	<code>entity.ToTable("OrdenesFabricacion");</code> → asigna explícitamente la tabla usada por la entidad.
Línea 148	<code>entity.HasKey(o => o.Id);</code> → define la clave primaria.
Línea 149	<code>entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 150	<code>entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);</code> → selecciona una propiedad escalar para continuar su configuración.

Línea 151	<code>entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 152	<code>entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 153	<code>entity.Property(o => o.Observaciones).HasMaxLength(500);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 154	<code>entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");</code> → define una clave alternativa única.
Línea 155	<code>entity.HasIndex(o => o.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");</code> → define un índice sobre las propiedades indicadas.
Línea 156	<code>entity.HasIndex(o => new { o.Cliente, o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");</code> → define un índice sobre las propiedades indicadas.
Línea 157	<code>entity.HasIndex(o => o.FechaEntrega).HasFilter("[Estado] = 'Pendiente'").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");</code> → define un índice sobre las propiedades indicadas.
Línea 158	<code>entity.HasIndex(o => o.Estado).IncludeProperties(o => new { o.NumeroOrden, o.Cliente, o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");</code> → define un índice sobre las propiedades indicadas.
Línea 159	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 160	<code>modelBuilder.Entity<PlanchaAcero>(entity =></code> → selecciona una entidad para configurarla con Fluent API.
Línea 161	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 162	<code>entity.ToTable("PlanchasAcero", t =></code> → asigna explícitamente la tabla usada por la entidad.
Línea 163	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 164	<code>t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");</code> → añade una restricción CHECK en SQL Server.
Línea 165	<code>t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");</code> → añade una restricción CHECK en SQL Server.
Línea 166	<code>t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");</code> → añade una restricción CHECK en SQL Server.
Línea 167	<code>t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");</code> → añade una restricción CHECK en SQL Server.
Línea 168	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 169	<code>entity.HasKey(x => x.Id);</code> → define la clave primaria.
Línea 170	<code>entity.Property(x => x.Peso).HasPrecision(18, 3);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 171	<code>entity.Property(x => x.Activa).HasDefaultValue(true);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 172	<code>entity.HasOne(x => x.Orden)</code> → inicia la configuración de una navegación de referencia.

Línea 173	<code>.WithMany(o => o.Planchas) → establece el extremo de colección de una relación uno-a-muchos.</code>
Línea 174	<code>.HasForeignKey(x => x.OrdenId) → declara explícitamente la clave foránea.</code>
Línea 175	<code>.OnDelete(DeleteBehavior.Cascade) → define el comportamiento de eliminación de la relación.</code>
Línea 176	<code>.IsRequired(); → marca la propiedad o relación como obligatoria.</code>
Línea 177	<code>entity.HasIndex(x => new { x.OrdenId, x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa"); → define un índice sobre las propiedades indicadas.</code>
Línea 178	<code>entity.HasIndex(x => x.Espesor).HasFilter("[Activa] = 1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas"); → define un índice sobre las propiedades indicadas.</code>
Línea 179	<code>}); → cierra la lambda o configuración encadenada y finaliza la llamada.</code>
Línea 180	<code>modelBuilder.Entity<Aleacion>(entity => → selecciona una entidad para configurarla con Fluent API.</code>
Línea 181	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 182	<code>entity.ToTable("Aleaciones", t => → asigna explícitamente la tabla usada por la entidad.</code>
Línea 183	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 184	<code>t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND [PorcentajeCarbono] <= 2"); → añade una restricción CHECK en SQL Server.</code>
Línea 185	<code>t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND [PorcentajeManganeso] <= 5"); → añade una restricción CHECK en SQL Server.</code>
Línea 186	<code>}); → cierra la lambda o configuración encadenada y finaliza la llamada.</code>
Línea 187	<code>entity.HasKey(a => a.Id); → define la clave primaria.</code>
Línea 188	<code>entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100); → selecciona una propiedad escalar para continuar su configuración.</code>
Línea 189	<code>entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20); → selecciona una propiedad escalar para continuar su configuración.</code>
Línea 190	<code>entity.Property(a => a.Descripcion).HasMaxLength(500); → selecciona una propiedad escalar para continuar su configuración.</code>
Línea 191	<code>entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo"); → define una clave alternativa única.</code>
Línea 192	<code>entity.HasIndex(a => a.Nombre).HasDatabaseName("IX_Aleaciones_Nombre"); → define un índice sobre las propiedades indicadas.</code>
Línea 193	<code>entity.HasIndex(a => new { a.PorcentajeCarbono, a.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes"); → define un índice sobre las propiedades indicadas.</code>
Línea 194	<code>}); → cierra la lambda o configuración encadenada y finaliza la llamada.</code>
Línea 195	<code>modelBuilder.Entity<EstadoOrden>(entity => → selecciona una entidad para configurarla con Fluent API.</code>
Línea 196	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>

Línea 197	<code>entity.ToTable("EstadosOrden");</code> → asigna explícitamente la tabla usada por la entidad.
Línea 198	<code>entity.HasKey(e => e.Id);</code> → define la clave primaria.
Línea 199	<code>entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 200	<code>entity.Property(e => e.Descripcion).HasMaxLength(250);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 201	<code>entity.Property(e => e.Activo).HasDefaultValue(true);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 202	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 203	<code>modelBuilder.Entity<DetalleOrden>(entity =></code> → selecciona una entidad para configurarla con Fluent API.
Línea 204	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 205	<code>entity.ToTable("DetallesOrden");</code> → asigna explícitamente la tabla usada por la entidad.
Línea 206	<code>entity.HasKey(d => d.Id);</code> → define la clave primaria.
Línea 207	<code>entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 208	<code>entity.Property(d => d.Notas).HasMaxLength(500);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 209	<code>entity.HasOne(d => d.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 210	<code>.WithOne(o => o.Detalle)</code> → establece el extremo de referencia de una relación uno-a-uno.
Línea 211	<code>.HasForeignKey<DetalleOrden>(d => d.OrdenId)</code> → declara explícitamente la clave foránea.
Línea 212	<code>.OnDelete(DeleteBehavior.Cascade)</code> → define el comportamiento de eliminación de la relación.
Línea 213	<code>.IsRequired();</code> → marca la propiedad o relación como obligatoria.
Línea 214	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 216	<code>modelBuilder.Entity<CertificadoCalidad>(entity =></code> → selecciona una entidad para configurarla con Fluent API.
Línea 217	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 218	<code>entity.ToTable("CertificadosCalidad");</code> → asigna explícitamente la tabla usada por la entidad.
Línea 219	<code>entity.HasKey(c => c.Id);</code> → define la clave primaria.
Línea 220	<code>entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 221	<code>entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 222	<code>entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");</code> → define una clave alternativa única.

Línea 223	<code>entity.HasIndex(c => c.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");</code> → define un índice sobre las propiedades indicadas.
Línea 224	<code>entity.HasOne(c => c.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 225	<code>.WithOne(o => o.Certificado)</code> → establece el extremo de referencia de una relación uno-a-uno.
Línea 226	<code>.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)</code> → declara explícitamente la clave foránea.
Línea 227	<code>.onDelete(DeleteBehavior.Cascade)</code> → define el comportamiento de eliminación de la relación.
Línea 228	<code>.IsRequired();</code> → marca la propiedad o relación como obligatoria.
Línea 229	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 230	<code>modelBuilder.Entity<OrdenAleacion>(entity =></code> → selecciona una entidad para configurarla con Fluent API.
Línea 231	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 232	<code>entity.ToTable("OrdenesAleaciones");</code> → asigna explícitamente la tabla usada por la entidad.
Línea 233	<code>entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });</code> → define la clave primaria.
Línea 234	<code>entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 235	<code>entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 236	<code>entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 237	<code>entity.HasOne(x => x.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 238	<code>.WithMany(o => o.OrdenesAleaciones)</code> → establece el extremo de colección de una relación uno-a-muchos.
Línea 239	<code>.HasForeignKey(x => x.OrdenFabricacionId)</code> → declara explícitamente la clave foránea.
Línea 240	<code>.onDelete(DeleteBehavior.Cascade);</code> → define el comportamiento de eliminación de la relación.
Línea 241	<code>entity.HasOne(x => x.Aleacion)</code> → inicia la configuración de una navegación de referencia.
Línea 242	<code>.WithMany(a => a.OrdenesAleaciones)</code> → establece el extremo de colección de una relación uno-a-muchos.
Línea 243	<code>.HasForeignKey(x => x.AleacionId)</code> → declara explícitamente la clave foránea.
Línea 244	<code>.onDelete(DeleteBehavior.Restrict);</code> → define el comportamiento de eliminación de la relación.
Línea 245	<code>entity.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");</code> → define un índice sobre las propiedades indicadas.
Línea 246	<code>entity.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] = 'Activa'");</code> → define un índice sobre las propiedades indicadas.
Línea 247	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 248	} → cierra el bloque de código actual.
Línea 249	} → cierra el bloque de código actual.
Línea 251	public interface IOrdenRepositorio → declara el contrato IOrdenRepositorio usado por la aplicación.
Línea 252	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 253	List<OrdenFabricacion> ObtenerTodas(); → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.
Línea 254	OrdenFabricacion? ObtenerPorId(int id); → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.
Línea 255	OrdenFabricacion? ObtenerPorNumero(string numeroOrden); → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.
Línea 256	void Agregar(OrdenFabricacion orden); → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.
Línea 257	void Eliminar(OrdenFabricacion orden); → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.
Línea 258	int Guardar(); → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.
Línea 259	} → cierra el bloque de código actual.
Línea 261	public sealed class OrdenRepositorio : IOrdenRepositorio → declara la clase OrdenRepositorio que forma parte del estado acumulativo.
Línea 262	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 263	private readonly AceriaDbContext _context; → declara el campo privado _context de tipo AceriaDbContext.
Línea 264	public OrdenRepositorio(AceriaDbContext context) => _context = context; → inyecta AceriaDbContext en el repositorio y lo conserva como dependencia de persistencia.
Línea 266	public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList(); → ordena el resultado antes de materializarlo.
Línea 267	public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id); → implementa la búsqueda por clave primaria mediante Find, aprovechando primero el ChangeTracker y después la base de datos.
Línea 268	public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden); → recupera la primera coincidencia o null.
Línea 269	public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden); → registra una entidad nueva para insertarla al guardar.
Línea 270	public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden); → marca una entidad para borrado al guardar.
Línea 271	public int Guardar() => _context.SaveChanges(); → confirma en SQL Server los cambios del ChangeTracker.
Línea 272	} → cierra el bloque de código actual.

Línea 274	<code>public interface IServicio0rdenes → declara el contrato IServicioOrdenes usado por la aplicación.</code>
Línea 275	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 276	<code>string ObtenerResumen(int ordenId); → declara el servicio que construye un resumen de una orden a partir de su identificador.</code>
Línea 277	<code>} → cierra el bloque de código actual.</code>
Línea 279	<code>public sealed class ServicioOrdenes : IServicio0rdenes → declara la clase ServicioOrdenes que forma parte del estado acumulativo.</code>
Línea 280	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 281	<code>private readonly IOrdenRepositorio _repositorio; → declara el campo privado _repositorio de tipo IOrdenRepositorio.</code>
Línea 282	<code>private readonly AceriaDbContext _context; → declara el campo privado _context de tipo AceriaDbContext.</code>
Línea 283	<code>public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context) → declara el constructor que recibe por inyección el repositorio de órdenes y el DbContext.</code>
Línea 284	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 285	<code>_repositorio = repositorio; → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.</code>
Línea 286	<code>_context = context; → guarda el DbContext inyectado para consultar los datos necesarios durante el servicio.</code>
Línea 287	<code>} → cierra el bloque de código actual.</code>
Línea 289	<code>public string ObtenerResumen(int ordenId) → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.</code>
Línea 290	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 291	<code>var orden = _repositorio.ObtenerPorId(ordenId); → declara una variable local y almacena el resultado de la expresión.</code>
Línea 292	<code>if (orden is null) return \$"Orden {ordenId} no encontrada"; → evalúa una condición antes de ejecutar el bloque asociado.</code>
Línea 293	<code>var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId); → cuenta las filas que cumplen la consulta.</code>
Línea 294	<code>return \$"Orden {orden.NumeroOrden} Cliente: {orden.Cliente} Planchas: {totalPlanchas}"; → devuelve el resultado al código llamador.</code>
Línea 295	<code>} → cierra el bloque de código actual.</code>
Línea 296	<code>} → cierra el bloque de código actual.</code>
Línea 298	<code>public static class Program → declara la clase Program que forma parte del estado acumulativo.</code>
Línea 299	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 300	<code>public static void Main() → declara el punto de entrada ejecutable de la aplicación de consola.</code>
Línea 301	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>

Línea 302	<code>var configuration = new ConfigurationBuilder()</code> → inicia la construcción de la configuración de la aplicación.
Línea 303	<code>.SetBasePath(AppContext.BaseDirectory)</code> → establece la ruta base para los archivos de configuración.
Línea 304	<code>.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)</code> → incorpora <code>appsettings.json</code> .
Línea 305	<code>.AddEnvironmentVariables()</code> → incorpora variables de entorno.
Línea 306	<code>.Build();</code> → materializa el objeto construido por la cadena anterior.
Línea 308	<code>var connectionString = configuration.GetConnectionString("AceriaDB")</code> → obtiene la cadena <code>AceriaDB</code> desde configuración.
Línea 309	<code>?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");</code> → interrumpe el arranque con un error explícito si la configuración no contiene <code>ConnectionStrings:AceriaDB</code> .
Línea 311	<code>var services = new ServiceCollection();</code> → crea la colección de servicios.
Línea 312	<code>services.AddDbContext<AceriaDbContext>(options =></code> → registra el <code>DbContext</code> en inyección de dependencias.
Línea 313	<code>options</code> → continúa la configuración del <code>DbContext</code> sobre el parámetro <code>options</code> recibido por la lambda de <code>AddDbContext</code> .
Línea 314	<code>.UseSqlServer(connectionString, sql =></code> → selecciona <code>SQL Server</code> como proveedor de <code>EF Core</code> .
Línea 315	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 316	<code>sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);</code> → habilita reintentos ante fallos transitorios.
Línea 317	<code>sql.CommandTimeout(60);</code> → establece el <code>timeout</code> de los comandos <code>SQL</code> .
Línea 318	<code>})</code> → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.
Línea 319	<code>.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)</code> → configura el logging de <code>EF Core</code> .
Línea 320	<code>.EnableDetailedErrors();</code> → activa errores detallados.
Línea 322	<code>services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();</code> → registra un servicio con ciclo de vida <code>scoped</code> .
Línea 323	<code>services.AddScoped<IServicioOrdenes, ServicioOrdenes>();</code> → registra un servicio con ciclo de vida <code>scoped</code> .
Línea 325	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions</code> → construye el <code>ServiceProvider</code> con validación de servicios y garantiza su liberación al salir del ámbito mediante <code>using var</code> .
Línea 326	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 327	<code>ValidateScopes = true,</code> → activa la validación de ámbitos para detectar dependencias <code>scoped</code> usadas desde ámbitos incorrectos.
Línea 328	<code>ValidateOnBuild = true</code> → obliga al contenedor a validar las resoluciones registradas al construir el <code>ServiceProvider</code> .

Línea 329	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 331	<code>using var scope = provider.CreateScope();</code> → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.
Línea 332	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → resuelve un servicio obligatorio del contenedor.
Línea 333	<code>context.Database.EnsureDeleted();</code> → reinicia la base del laboratorio para una ejecución reproducible.
Línea 334	<code>context.Database.Migrate();</code> → aplica el historial de migraciones pendiente.
Línea 336	<code>var orden = new OrdenFabricacion</code> → declara una variable local y almacena el resultado de la expresión.
Línea 337	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 338	<code>NumeroOrden = "OF-M2-0001",</code> → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.
Línea 339	<code>Cliente = "Constructora del Norte",</code> → asigna el cliente de la orden utilizada en el escenario E2E del punto.
Línea 340	<code>FechaCreacion = DateTime.UtcNow,</code> → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.
Línea 341	<code>FechaEntrega = DateTime.Today.AddDays(14),</code> → establece una fecha de entrega prevista catorce días después de la fecha local actual.
Línea 342	<code>Estado = "Pendiente",</code> → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.
Línea 343	<code>Observaciones = "Orden de validación M2",</code> → añade texto opcional para comprobar el mapeo de Observaciones.
Línea 344	<code>Planchas =</code> → inicia el inicializador de la colección de planchas asociadas a la orden.
Línea 345	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 346	<code>new PlanchaAcero</code> → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.
Línea 347	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 348	<code>Espesor = 10.5,</code> → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.
Línea 349	<code>Ancho = 1500,</code> → asigna el ancho de la plancha usada en el escenario de persistencia.
Línea 350	<code>Largo = 3000,</code> → asigna el largo de la plancha usada en el escenario de persistencia.
Línea 351	<code>Peso = 371.250m,</code> → asigna un peso decimal con tres posiciones para validar la precisión configurada.
Línea 352	<code>Activa = true,</code> → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.
Línea 353	<code>}</code> → cierra el bloque de código actual.
Línea 354	<code>},</code> → continúa el inicializador o la llamada del bloque actual.

Línea 355	Detalle = new DetalleOrden → crea el detalle uno-a-uno que se persistirá junto con la orden principal.
Línea 356	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 357	ComposicionQuimica = "C: 0.45%, Mn: 0.75%", → asigna la composición química utilizada para verificar la entidad DetalleOrden.
Línea 358	TemperaturaColada = 1550.5, → asigna la temperatura de colada del detalle de fabricación.
Línea 359	Notas = "Colada principal" → asigna las notas opcionales del detalle asociado a la orden.
Línea 360	}, → continúa el inicializador o la llamada del bloque actual.
Línea 361	Certificado = new CertificadoCalidad → crea el certificado uno-a-uno asociado a la orden del escenario.
Línea 362	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 363	NumeroCertificado = "CERT-0001", → asigna el identificador natural del certificado de calidad.
Línea 364	FechaEmision = DateTime.Today, → asigna la fecha de emisión del certificado usado en la prueba.
Línea 365	OrganismoCertificador = "Laboratorio Aceria" → asigna el organismo que emite el certificado de calidad.
Línea 366	}, → continúa el inicializador o la llamada del bloque actual.
Línea 367	}; → cierra el inicializador del objeto y finaliza la instrucción.
Línea 369	var aleacion = new Aleacion → declara una variable local y almacena el resultado de la expresión.
Línea 370	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 371	Nombre = "AISI 1045", → asigna el nombre comercial de la aleación utilizada en el escenario.
Línea 372	Codigo = "A1045", → asigna el código natural único de la aleación.
Línea 373	PorcentajeCarbono = 0.45, → asigna el porcentaje de carbono dentro del rango permitido por la restricción del modelo.
Línea 374	PorcentajeManganeso = 0.75, → asigna el porcentaje de manganeso dentro del rango permitido por la restricción del modelo.
Línea 375	Descripcion = "Acero medio en carbono" → asigna una descripción opcional a la aleación.
Línea 376	}; → cierra el inicializador del objeto y finaliza la instrucción.
Línea 377	orden.OrdenesAleaciones.Add(new OrdenAleacion → registra una entidad nueva para insertarla al guardar.
Línea 378	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 379	Aleacion = aleacion, → conecta la entidad intermedia OrdenAleacion con la aleación creada para el escenario.
Línea 380	CantidadUtilizada = 1500.500m, → registra la cantidad utilizada con tres decimales, coherente con HasPrecision(18, 3).
Línea 381	EstadoRelacion = "Activa" → marca como activa la relación entre la orden y la aleación.
Línea 382	}); → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 383	<code>context.OrdenesFabricacion.Add(orden);</code> → registra una entidad nueva para insertarla al guardar.
Línea 384	<code>context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });</code> → registra una entidad nueva para insertarla al guardar.
Línea 385	<code>context.SaveChanges();</code> → confirma en SQL Server los cambios del ChangeTracker.
Línea 387	<code>var entity = context.Model.FindEntityType(typeof(OrdenFabricacion));</code> → declara una variable local y almacena el resultado de la expresión.
Línea 388	<code>global::System.Console.WriteLine(\$"2.9 OK Índices: {string.Join(", ", entity.GetIndexes().Select(i => i.GetDatabaseName()))}");</code> → escribe la evidencia de ejecución usada para validar este estado.
Línea 389	<code>}</code> → cierra el bloque de código actual.
Línea 390	<code>}</code> → cierra el bloque de código actual.

Punto 2.10 - Filtros globales de consulta

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL que trabajan con .NET 8, EF Core 8 y SQL Server LocalDB.

Proyecto: El estado 2.10 parte de 2.9 y tiene como objetivo añadir `HasQueryFilter` sin introducir todavía las propiedades de `Soft Delete`.

Objetivos de aprendizaje

- Configurar `HasQueryFilter`.
- Filtrar órdenes canceladas.
- Filtrar planchas inactivas.
- Filtrar relaciones `OrdenAleacion` no activas.
- Usar `IgnoreQueryFilters` de forma explícita.
- Demostrar que el registro sigue existiendo en la base.

Paso 1: Abrir la solución autónoma del punto

```
cd M02/PROYECTO/2.10
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

No se abre una solución global situada en la raíz del repositorio. `AceriaData.sln` está dentro de la carpeta del punto y referencia únicamente los proyectos de ese estado.

Paso 2: Identificar el cambio respecto al estado anterior

El proyecto conserva todo lo terminado en 2.9. En 2.10 se introduce exclusivamente el contenido que corresponde a **Filtros globales de consulta**. La práctica no reinicia `AceriaData` ni crea un ejemplo paralelo.

Paso 3: Implementar y comprender la configuración principal

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.HasQueryFilter(o => o.Estado != "Cancelada");
});

modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.HasQueryFilter(p => p.Activa);
});
```

CSHARP

```
var visibles = context.OrdenesFabricacion.Count();
var todas = context.OrdenesFabricacion
    .IgnoreQueryFilters()
    .Count();
```

Línea 1	HasQueryFilter(...) → añade una condición transversal a todas las consultas normales de la entidad.
Línea 2	o.Estado != "Cancelada" → excluye las órdenes canceladas sin borrarlas.
Línea 3	p.Activa → limita las planchas visibles a las activas.
Línea 4	IgnoreQueryFilters() → desactiva expresamente los filtros para una consulta concreta.
Línea 5	Count() → permite comparar de manera reproducible la vista filtrada con la vista completa.

Paso 4: Generar y aplicar la migración acumulativa

El repositorio contiene una migración real generada para este estado. Si se reproduce el ejercicio desde el estado anterior, los comandos son:

```
dotnet ef migrations add M2_2_10
dotnet ef database update
```

POWERSHELL

`dotnet ef migrations add` compara el modelo actual con el snapshot heredado. `dotnet ef database update` aplica únicamente los cambios pendientes.

En los estados ejecutables del curso se usa `Database.Migrate()` para aplicar el historial durante la demostración. El laboratorio puede usar `EnsureDeleted()` para empezar desde una base limpia, pero **no usa `EnsureCreated()`**, porque el esquema está gobernado por Migrations.

Paso 5: Compilar y ejecutar el estado

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

POWERSHELL

Resultado esperado: la ejecución termina sin excepción y contiene la evidencia **2.10 OK**. GitHub Actions repite esta compilación y ejecución sobre Windows y SQL Server LocalDB.

Paso 6: Verificar el estado acumulativo

Además de ejecutar el ejemplo, conviene revisar el modelo y la base:

```
dotnet ef migrations list
```

POWERSHELL

Cuando el punto modifica el esquema, la lista debe contener la migración acumulativa correspondiente. En SQL Server Object Explorer se puede comprobar que las tablas, claves, relaciones, índices o restricciones coinciden con el modelo del punto.

Errores comunes

Error	Causa	Solución
Registro filtrado parece borrado	HasQueryFilter lo oculta	Comprobar con IgnoreQueryFilters.
IgnoreQueryFilters se usa indiscriminadamente	Se desactiva la regla transversal	Reservarlo para diagnósticos/administración.
Se añade IsDeleted	Se adelanta 2.11	En 2.10 usar sólo filtros de negocio ya existentes.

Reto resuelto

Reto: Insertar una orden Cancelada y comparar Count() con Count() después de IgnoreQueryFilters().

Solución: partir del código de M02/PROYECTO/2.10, realizar únicamente el cambio descrito y volver a ejecutar build, migraciones y el programa. El reto no introduce conceptos reservados a un punto posterior.

Analogía final

Un filtro global actúa como una regla automática de visibilidad que EF Core adjunta a cada consulta normal.

Resultado esperado

Al terminar 2.10, la solución local compila, el proyecto se ejecuta, el esquema se actualiza mediante Migrations cuando corresponde y la salida contiene 2.10 OK. El código conserva todos los cambios válidos de los puntos anteriores.

Conexión con el siguiente punto

El siguiente estado es 2.11. Se parte del proyecto completo de 2.10; no se vuelve a crear AceriaData desde cero.

Código acumulativo completo del estado 2.10

El siguiente archivo es el Program.cs real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

CSHARP

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("Aleaciones")]
public class Aleacion
```

```

{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }
    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
}

```

```

public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
    modelBuilder.Entity<OrdenFabricacion>(entity =>
    {
        entity.ToTable("OrdenesFabricacion");
        entity.HasKey(o => o.Id);
        entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
        entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
        entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
        entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
        entity.Property(o => o.Observaciones).HasMaxLength(500);
        entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");
        entity.HasIndex(o => o.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");
        entity.HasIndex(o => new { o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");
        entity.HasIndex(o => o.FechaEntrega).HasFilter("[Estado] =
'Pendiente'").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");
        entity.HasIndex(o => o.Estado).IncludeProperties(o => new { o.NumeroOrden, o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");
        entity.HasQueryFilter(o => o.Estado != "Cancelada");
    });
    modelBuilder.Entity<PlanchaAcero>(entity =>
    {
        entity.ToTable("PlanchasAcero", t =>
        {
            t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");
            t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");
            t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");
            t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");
        });
        entity.HasKey(x => x.Id);
        entity.Property(x => x.Peso).HasPrecision(18, 3);
        entity.Property(x => x.Activa).HasDefaultValue(true);
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.Planchas)
            .HasForeignKey(x => x.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
        entity.HasIndex(x => new { x.OrdenId, x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");
        entity.HasIndex(x => x.Espesor).HasFilter("[Activa] =
1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");
        entity.HasQueryFilter(x => x.Activa);
    });
    modelBuilder.Entity<Aleacion>(entity =>
    {
        entity.ToTable("Aleaciones", t =>
        {
            t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND
[PorcentajeCarbono] <= 2");
            t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND
[PorcentajeManganeso] <= 5");
        });
        entity.HasKey(a => a.Id);
        entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
        entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
        entity.Property(a => a.Descripcion).HasMaxLength(500);
        entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");
        entity.HasIndex(a => a.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");
        entity.HasIndex(a => new { a.PorcentajeCarbono,
a.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");
    });
    modelBuilder.Entity<EstadoOrden>(entity =>
    {
        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
        entity.Property(e => e.Descripcion).HasMaxLength(250);
        entity.Property(e => e.Activo).HasDefaultValue(true);
        entity.HasQueryFilter(e => e.Activo);
    });
    modelBuilder.Entity<DetalleOrden>(entity =>
    {
        entity.ToTable("DetallesOrden");
        entity.HasKey(d => d.Id);
        entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
        entity.Property(d => d.Notas).HasMaxLength(500);
        entity.HasOne(d => d.Orden)
            .WithOne(o => o.Detalle)
            .HasForeignKey<DetalleOrden>(d => d.OrdenId)

```

```

        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
    });

    modelBuilder.Entity<CertificadoCalidad>(entity =>
    {
        entity.ToTable("CertificadosCalidad");
        entity.HasKey(c => c.Id);
        entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
        entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
        entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");
        entity.HasIndex(c => c.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");
        entity.HasOne(c => c.Orden)
            .WithOne(o => o.Certificado)
            .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });

    modelBuilder.Entity<OrdenAleacion>(entity =>
    {
        entity.ToTable("OrdenesAleaciones");
        entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
        entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
        entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.OrdenesAleaciones)
            .HasForeignKey(x => x.OrdenFabricacionId)
            .OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(x => x.Aleacion)
            .WithMany(a => a.OrdenesAleaciones)
            .HasForeignKey(x => x.AleacionId)
            .OnDelete(DeleteBehavior.Restrict);
        entity.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");
        entity.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] = 'Activa'");
        entity.HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");
        entity.HasQueryFilter(x => x.EstadoRelacion == "Activa");
    });
}

}

public interface IOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
    }
}

```

```

        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                {
                    sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                    sql.CommandTimeout(60);
                })
                .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
                    LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        context.Database.EnsureDeleted();
        context.Database.Migrate();

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-M2-0001",
            Cliente = "Constructora del Norte",
            FechaCreacion = DateTime.UtcNow,
            FechaEntrega = DateTime.Today.AddDays(14),
            Estado = "Pendiente",
            Observaciones = "Orden de validación M2",
            Planchas =
            {
                new PlanchaAcero
                {
                    Espesor = 10.5,
                    Ancho = 1500,
                    Largo = 3000,
                    Peso = 371.250m,
                    Activa = true,
                }
            },
            Detalle = new DetalleOrden
            {
                ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
                TemperaturaColada = 1550.5,
                Notas = "Colada principal"
            },
            Certificado = new CertificadoCalidad
            {
                NumeroCertificado = "CERT-0001",
                FechaEmision = DateTime.Today,
                OrganismoCertificador = "Laboratorio Aceria"
            },
        };

        var aleacion = new Aleacion
        {
            Nombre = "AISI 1045",
           Codigo = "A1045",
            PorcentajeCarbono = 0.45,
            PorcentajeManganeso = 0.75,
            Descripcion = "Acero medio en carbono"
        };
        orden.OrdenesAleaciones.Add(new OrdenAleacion

```

```

    {
        Aleacion = aleacion,
        CantidadUtilizada = 1500.500m,
        EstadoRelacion = "Activa"
    });
    context.OrdenesFabricacion.Add(orden);
    context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo =
true });
    context.SaveChanges();

    var cancelada = new OrdenFabricacion
    {
        NumeroOrden = "OF-M2-CANCELADA",
        Cliente = "Cliente Histórico",
        FechaCreacion = DateTime.UtcNow,
        Estado = "Cancelada"
    };
    context.OrdenesFabricacion.Add(cancelada);
    context.SaveChanges();
    var visibles = context.OrdenesFabricacion.Count();
    var todas = context.OrdenesFabricacion.IgnoreQueryFilters().Count();
    global::System.Console.WriteLine($"2.10 OK | Visibles: {visibles} | Sin filtro: {todas}");
    }
}

```

Explicación línea a línea del código acumulativo 2.10

Las referencias usan la numeración real de M02/PROYECTO/2.10/Program.cs. Se explican todas las líneas no vacías del archivo que compila.

Línea 1	using System.ComponentModel.DataAnnotations; → importa el espacio de nombres necesario para este archivo.
Línea 2	using System.ComponentModel.DataAnnotations.Schema; → importa el espacio de nombres necesario para este archivo.
Línea 3	using Microsoft.EntityFrameworkCore; → importa el espacio de nombres necesario para este archivo.
Línea 4	using Microsoft.Extensions.Configuration; → importa el espacio de nombres necesario para este archivo.
Línea 5	using Microsoft.Extensions.DependencyInjection; → importa el espacio de nombres necesario para este archivo.
Línea 6	using Microsoft.Extensions.Logging; → importa el espacio de nombres necesario para este archivo.
Línea 8	namespace AceriaData.ConsoleApp; → declara el espacio de nombres del estado acumulativo.
Línea 10	[Table("OrdenesFabricacion")] → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 11	public class OrdenFabricacion → declara la clase OrdenFabricacion que forma parte del estado acumulativo.
Línea 12	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 13	[Key] → marca la propiedad siguiente como clave primaria simple.
Línea 14	public int Id { get; set; } → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.
Línea 15	[Required] → marca la propiedad siguiente como requerida.
Línea 16	[MaxLength(50)] → limita la longitud máxima persistida de la propiedad siguiente.

Línea 17	<code>public string NumeroOrden { get; set; } = string.Empty;</code> → declara la propiedad NumeroOrden de tipo string usada por el modelo o la lógica de negocio.
Línea 18	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 19	<code>[MaxLength(200)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 20	<code>public string Cliente { get; set; } = string.Empty;</code> → declara la propiedad Cliente de tipo string usada por el modelo o la lógica de negocio.
Línea 21	<code>public DateTime FechaCreacion { get; set; }</code> → declara la propiedad FechaCreacion de tipo DateTime usada por el modelo o la lógica de negocio.
Línea 22	<code>public DateTime? FechaEntrega { get; set; }</code> → declara la propiedad FechaEntrega de tipo DateTime? usada por el modelo o la lógica de negocio.
Línea 23	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 24	<code>[MaxLength(50)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 25	<code>public string Estado { get; set; } = "Pendiente";</code> → declara la propiedad Estado de tipo string usada por el modelo o la lógica de negocio.
Línea 26	<code>[MaxLength(500)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 27	<code>public string? Observaciones { get; set; }</code> → declara la propiedad Observaciones de tipo string? usada por el modelo o la lógica de negocio.
Línea 28	<code>public List<PlanchaAcero> Planchas { get; set; } = new();</code> → declara la propiedad Planchas de tipo List<PlanchaAcero> usada por el modelo o la lógica de negocio.
Línea 29	<code>public DetalleOrden? Detalle { get; set; }</code> → declara la propiedad Detalle de tipo DetalleOrden? usada por el modelo o la lógica de negocio.
Línea 30	<code>public CertificadoCalidad? Certificado { get; set; }</code> → declara la propiedad Certificado de tipo CertificadoCalidad? usada por el modelo o la lógica de negocio.
Línea 31	<code>public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();</code> → declara la propiedad OrdenesAleaciones de tipo List<OrdenAleacion> usada por el modelo o la lógica de negocio.
Línea 32	<code>}</code> → cierra el bloque de código actual.
Línea 34	<code>[Table("PlanchasAcero")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 35	<code>public class PlanchaAcero</code> → declara la clase PlanchaAcero que forma parte del estado acumulativo.
Línea 36	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 37	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 38	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.
Línea 39	<code>public int OrdenId { get; set; }</code> → declara la propiedad OrdenId de tipo int usada por el modelo o la lógica de negocio.
Línea 40	<code>public double Espesor { get; set; }</code> → declara la propiedad Espesor de tipo double usada por el modelo o la lógica de negocio.
Línea 41	<code>public double Ancho { get; set; }</code> → declara la propiedad Ancho de tipo double usada por el modelo o la lógica de negocio.

Línea 42	<code>public double Largo { get; set; } → declara la propiedad Largo de tipo double usada por el modelo o la lógica de negocio.</code>
Línea 43	<code>[Precision(18, 3)] → configura precisión y escala para la propiedad numérica siguiente.</code>
Línea 44	<code>public decimal Peso { get; set; } → declara la propiedad Peso de tipo decimal usada por el modelo o la lógica de negocio.</code>
Línea 45	<code>public bool Activa { get; set; } = true; → declara la propiedad Activa de tipo bool usada por el modelo o la lógica de negocio.</code>
Línea 46	<code>[ForeignKey(nameof(OrdenId))] → vincula la navegación siguiente con su clave foránea.</code>
Línea 47	<code>public OrdenFabricacion Orden { get; set; } = null!; → declara la propiedad Orden de tipo OrdenFabricacion usada por el modelo o la lógica de negocio.</code>
Línea 48	<code>} → cierra el bloque de código actual.</code>
Línea 50	<code>[Table("Aleaciones")] → fija mediante Data Annotations la tabla relacional de la entidad siguiente.</code>
Línea 51	<code>public class Aleacion → declara la clase Aleacion que forma parte del estado acumulativo.</code>
Línea 52	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 53	<code>[Key] → marca la propiedad siguiente como clave primaria simple.</code>
Línea 54	<code>public int Id { get; set; } → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.</code>
Línea 55	<code>[Required] → marca la propiedad siguiente como requerida.</code>
Línea 56	<code>[MaxLength(100)] → limita la longitud máxima persistida de la propiedad siguiente.</code>
Línea 57	<code>public string Nombre { get; set; } = string.Empty; → declara la propiedad Nombre de tipo string usada por el modelo o la lógica de negocio.</code>
Línea 58	<code>[Required] → marca la propiedad siguiente como requerida.</code>
Línea 59	<code>[MaxLength(20)] → limita la longitud máxima persistida de la propiedad siguiente.</code>
Línea 60	<code>public string Codigo { get; set; } = string.Empty; → declara la propiedad Codigo de tipo string usada por el modelo o la lógica de negocio.</code>
Línea 61	<code>public double PorcentajeCarbono { get; set; } → declara la propiedad PorcentajeCarbono de tipo double usada por el modelo o la lógica de negocio.</code>
Línea 62	<code>public double PorcentajeManganeso { get; set; } → declara la propiedad PorcentajeManganeso de tipo double usada por el modelo o la lógica de negocio.</code>
Línea 63	<code>[MaxLength(500)] → limita la longitud máxima persistida de la propiedad siguiente.</code>
Línea 64	<code>public string? Descripcion { get; set; } → declara la propiedad Descripcion de tipo string? usada por el modelo o la lógica de negocio.</code>
Línea 65	<code>public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new(); → declara la propiedad OrdenesAleaciones de tipo List<OrdenAleacion> usada por el modelo o la lógica de negocio.</code>
Línea 66	<code>} → cierra el bloque de código actual.</code>
Línea 68	<code>[Table("EstadosOrden")] → fija mediante Data Annotations la tabla relacional de la entidad siguiente.</code>

Línea 69	<code>public class EstadoOrden</code> → declara la clase EstadoOrden que forma parte del estado acumulativo.
Línea 70	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 71	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 72	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.
Línea 73	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 74	<code>[MaxLength(50)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 75	<code>public string Nombre { get; set; } = string.Empty;</code> → declara la propiedad Nombre de tipo string usada por el modelo o la lógica de negocio.
Línea 76	<code>[MaxLength(250)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 77	<code>public string Descripcion { get; set; } = string.Empty;</code> → declara la propiedad Descripcion de tipo string usada por el modelo o la lógica de negocio.
Línea 78	<code>public bool Activo { get; set; } = true;</code> → declara la propiedad Activo de tipo bool usada por el modelo o la lógica de negocio.
Línea 79	<code>}</code> → cierra el bloque de código actual.
Línea 81	<code>[Table("DetallesOrden")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 82	<code>public class DetalleOrden</code> → declara la clase DetalleOrden que forma parte del estado acumulativo.
Línea 83	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 84	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 85	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.
Línea 86	<code>public int OrdenId { get; set; }</code> → declara la propiedad OrdenId de tipo int usada por el modelo o la lógica de negocio.
Línea 87	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 88	<code>[MaxLength(200)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 89	<code>public string ComposicionQuimica { get; set; } = string.Empty;</code> → declara la propiedad ComposicionQuimica de tipo string usada por el modelo o la lógica de negocio.
Línea 90	<code>public double TemperaturaColada { get; set; }</code> → declara la propiedad TemperaturaColada de tipo double usada por el modelo o la lógica de negocio.
Línea 91	<code>[MaxLength(500)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 92	<code>public string? Notas { get; set; }</code> → declara la propiedad Notas de tipo string? usada por el modelo o la lógica de negocio.
Línea 93	<code>[ForeignKey(nameof(OrdenId))]</code> → vincula la navegación siguiente con su clave foránea.
Línea 94	<code>public OrdenFabricacion Orden { get; set; } = null!;</code> → declara la propiedad Orden de tipo OrdenFabricacion usada por el modelo o la lógica de negocio.

Línea 95	} → cierra el bloque de código actual.
Línea 97	[Table("CertificadosCalidad")] → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 98	public class CertificadoCalidad → declara la clase CertificadoCalidad que forma parte del estado acumulativo.
Línea 99	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 100	[Key] → marca la propiedad siguiente como clave primaria simple.
Línea 101	public int Id { get; set; } → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.
Línea 102	public int OrdenId { get; set; } → declara la propiedad OrdenId de tipo int usada por el modelo o la lógica de negocio.
Línea 103	[Required] → marca la propiedad siguiente como requerida.
Línea 104	[MaxLength(50)] → limita la longitud máxima persistida de la propiedad siguiente.
Línea 105	public string NumeroCertificado { get; set; } = string.Empty; → declara la propiedad NumeroCertificado de tipo string usada por el modelo o la lógica de negocio.
Línea 106	public DateTime FechaEmision { get; set; } → declara la propiedad FechaEmision de tipo DateTime usada por el modelo o la lógica de negocio.
Línea 107	[Required] → marca la propiedad siguiente como requerida.
Línea 108	[MaxLength(100)] → limita la longitud máxima persistida de la propiedad siguiente.
Línea 109	public string OrganismoCertificador { get; set; } = string.Empty; → declara la propiedad OrganismoCertificador de tipo string usada por el modelo o la lógica de negocio.
Línea 110	[ForeignKey(nameof(OrdenId))] → vincula la navegación siguiente con su clave foránea.
Línea 111	public OrdenFabricacion Orden { get; set; } = null!; → declara la propiedad Orden de tipo OrdenFabricacion usada por el modelo o la lógica de negocio.
Línea 112	} → cierra el bloque de código actual.
Línea 114	[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))] → declara una clave primaria compuesta.
Línea 115	[Table("OrdenesAleaciones")] → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 116	public class OrdenAleacion → declara la clase OrdenAleacion que forma parte del estado acumulativo.
Línea 117	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 118	public int OrdenFabricacionId { get; set; } → declara la propiedad OrdenFabricacionId de tipo int usada por el modelo o la lógica de negocio.
Línea 119	public int AleacionId { get; set; } → declara la propiedad AleacionId de tipo int usada por el modelo o la lógica de negocio.
Línea 120	public DateTime FechaAsignacion { get; set; } = DateTime.Now; → declara la propiedad FechaAsignacion de tipo DateTime usada por el modelo o la lógica de negocio.

Línea 121	<code>[Precision(18, 3)]</code> → configura precisión y escala para la propiedad numérica siguiente.
Línea 122	<code>public decimal CantidadUtilizada { get; set; }</code> → declara la propiedad CantidadUtilizada de tipo decimal usada por el modelo o la lógica de negocio.
Línea 123	<code>[MaxLength(20)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 124	<code>public string EstadoRelacion { get; set; } = "Activa";</code> → declara la propiedad EstadoRelacion de tipo string usada por el modelo o la lógica de negocio.
Línea 125	<code>public OrdenFabricacion Orden { get; set; } = null!;</code> → declara la propiedad Orden de tipo OrdenFabricacion usada por el modelo o la lógica de negocio.
Línea 126	<code>public Aleacion Aleacion { get; set; } = null!;</code> → declara la propiedad Aleacion de tipo Aleacion usada por el modelo o la lógica de negocio.
Línea 127	<code>}</code> → cierra el bloque de código actual.
Línea 129	<code>public class AceriaDbContext : DbContext</code> → declara la clase AceriaDbContext que forma parte del estado acumulativo.
Línea 130	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 131	<code>public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }</code> → recibe las opciones del DbContext y las pasa a la clase base.
Línea 133	<code>public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();</code> → expone un conjunto de entidades para consultas y persistencia.
Línea 134	<code>public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();</code> → expone un conjunto de entidades para consultas y persistencia.
Línea 135	<code>public DbSet<Aleacion> Aleaciones => Set<Aleacion>();</code> → expone un conjunto de entidades para consultas y persistencia.
Línea 136	<code>public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();</code> → expone un conjunto de entidades para consultas y persistencia.
Línea 137	<code>public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();</code> → expone un conjunto de entidades para consultas y persistencia.
Línea 138	<code>public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();</code> → expone un conjunto de entidades para consultas y persistencia.
Línea 139	<code>public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();</code> → expone un conjunto de entidades para consultas y persistencia.
Línea 142	<code>protected override void OnModelCreating(ModelBuilder modelBuilder)</code> → entra en la configuración explícita del modelo.
Línea 143	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 144	<code>base.OnModelCreating(modelBuilder);</code> → entra en la configuración explícita del modelo.
Línea 145	<code>modelBuilder.Entity<OrdenFabricacion>(entity =></code> → selecciona una entidad para configurarla con Fluent API.
Línea 146	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 147	<code>entity.ToTable("OrdenesFabricacion");</code> → asigna explícitamente la tabla usada por la entidad.
Línea 148	<code>entity.HasKey(o => o.Id);</code> → define la clave primaria.

Línea 149	<code>entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 150	<code>entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 151	<code>entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 152	<code>entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 153	<code>entity.Property(o => o.Observaciones).HasMaxLength(500);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 154	<code>entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");</code> → define una clave alternativa única.
Línea 155	<code>entity.HasIndex(o => o.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");</code> → define un índice sobre las propiedades indicadas.
Línea 156	<code>entity.HasIndex(o => new { o.Cliente, o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");</code> → define un índice sobre las propiedades indicadas.
Línea 157	<code>entity.HasIndex(o => o.FechaEntrega).HasFilter("[Estado] = 'Pendiente'").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");</code> → define un índice sobre las propiedades indicadas.
Línea 158	<code>entity.HasIndex(o => o.Estado).IncludeProperties(o => new { o.NumeroOrden, o.Cliente, o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");</code> → define un índice sobre las propiedades indicadas.
Línea 159	<code>entity.HasQueryFilter(o => o.Estado != "Cancelada");</code> → aplica un filtro global de consulta.
Línea 160	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 161	<code>modelBuilder.Entity<PlanchaAcero>(entity =></code> → selecciona una entidad para configurarla con Fluent API.
Línea 162	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 163	<code>entity.ToTable("PlanchasAcero", t =></code> → asigna explícitamente la tabla usada por la entidad.
Línea 164	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 165	<code>t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");</code> → añade una restricción CHECK en SQL Server.
Línea 166	<code>t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");</code> → añade una restricción CHECK en SQL Server.
Línea 167	<code>t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");</code> → añade una restricción CHECK en SQL Server.
Línea 168	<code>t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");</code> → añade una restricción CHECK en SQL Server.
Línea 169	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 170	<code>entity.HasKey(x => x.Id);</code> → define la clave primaria.

Línea 171	<code>entity.Property(x => x.Peso).HasPrecision(18, 3);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 172	<code>entity.Property(x => x.Activa).HasDefaultValue(true);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 173	<code>entity.HasOne(x => x.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 174	<code>.WithMany(o => o.Planchas)</code> → establece el extremo de colección de una relación uno-a-muchos.
Línea 175	<code>.HasForeignKey(x => x.OrdenId)</code> → declara explícitamente la clave foránea.
Línea 176	<code>.OnDelete(DeleteBehavior.Cascade)</code> → define el comportamiento de eliminación de la relación.
Línea 177	<code>.IsRequired();</code> → marca la propiedad o relación como obligatoria.
Línea 178	<code>entity.HasIndex(x => new { x.OrdenId, x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");</code> → define un índice sobre las propiedades indicadas.
Línea 179	<code>entity.HasIndex(x => x.Espesor).HasFilter("[Activa] = 1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");</code> → define un índice sobre las propiedades indicadas.
Línea 180	<code>entity.HasQueryFilter(x => x.Activa);</code> → aplica un filtro global de consulta.
Línea 181	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 182	<code>modelBuilder.Entity<Aleacion>(entity =></code> → selecciona una entidad para configurarla con Fluent API.
Línea 183	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 184	<code>entity.ToTable("Aleaciones", t =></code> → asigna explícitamente la tabla usada por la entidad.
Línea 185	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 186	<code>t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND [PorcentajeCarbono] <= 2");</code> → añade una restricción CHECK en SQL Server.
Línea 187	<code>t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND [PorcentajeManganeso] <= 5");</code> → añade una restricción CHECK en SQL Server.
Línea 188	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 189	<code>entity.HasKey(a => a.Id);</code> → define la clave primaria.
Línea 190	<code>entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 191	<code>entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 192	<code>entity.Property(a => a.Descripcion).HasMaxLength(500);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 193	<code>entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");</code> → define una clave alternativa única.
Línea 194	<code>entity.HasIndex(a => a.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");</code> → define un índice sobre las propiedades indicadas.

Línea 195	<code>entity.HasIndex(a => new { a.PorcentajeCarbono, a.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");</code> → define un índice sobre las propiedades indicadas.
Línea 196	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 197	<code>modelBuilder.Entity<EstadoOrden>(entity =></code> → selecciona una entidad para configurarla con Fluent API.
Línea 198	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 199	<code>entity.ToTable("EstadosOrden");</code> → asigna explícitamente la tabla usada por la entidad.
Línea 200	<code>entity.HasKey(e => e.Id);</code> → define la clave primaria.
Línea 201	<code>entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 202	<code>entity.Property(e => e.Descripcion).HasMaxLength(250);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 203	<code>entity.Property(e => e.Activo).HasDefaultValue(true);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 204	<code>entity.HasQueryFilter(e => e.Activo);</code> → aplica un filtro global de consulta.
Línea 205	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 206	<code>modelBuilder.Entity<DetalleOrden>(entity =></code> → selecciona una entidad para configurarla con Fluent API.
Línea 207	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 208	<code>entity.ToTable("DetallesOrden");</code> → asigna explícitamente la tabla usada por la entidad.
Línea 209	<code>entity.HasKey(d => d.Id);</code> → define la clave primaria.
Línea 210	<code>entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 211	<code>entity.Property(d => d.Notas).HasMaxLength(500);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 212	<code>entity.HasOne(d => d.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 213	<code>.WithOne(o => o.Detalle)</code> → establece el extremo de referencia de una relación uno-a-uno.
Línea 214	<code>.HasForeignKey<DetalleOrden>(d => d.OrdenId)</code> → declara explícitamente la clave foránea.
Línea 215	<code>.OnDelete(DeleteBehavior.Cascade)</code> → define el comportamiento de eliminación de la relación.
Línea 216	<code>.IsRequired();</code> → marca la propiedad o relación como obligatoria.
Línea 217	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 219	<code>modelBuilder.Entity<CertificadoCalidad>(entity =></code> → selecciona una entidad para configurarla con Fluent API.
Línea 220	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 221	<code>entity.ToTable("CertificadosCalidad");</code> → asigna explícitamente la tabla usada por la entidad.

Línea 222	<code>entity.HasKey(c => c.Id);</code> → define la clave primaria.
Línea 223	<code>entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 224	<code>entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 225	<code>entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");</code> → define una clave alternativa única.
Línea 226	<code>entity.HasIndex(c => c.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");</code> → define un índice sobre las propiedades indicadas.
Línea 227	<code>entity.HasOne(c => c.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 228	<code>.WithOne(o => o.Certificado)</code> → establece el extremo de referencia de una relación uno-a-uno.
Línea 229	<code>.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)</code> → declara explícitamente la clave foránea.
Línea 230	<code>.OnDelete(DeleteBehavior.Cascade)</code> → define el comportamiento de eliminación de la relación.
Línea 231	<code>.IsRequired();</code> → marca la propiedad o relación como obligatoria.
Línea 232	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 233	<code>modelBuilder.Entity<OrdenAleacion>(entity =></code> → selecciona una entidad para configurarla con Fluent API.
Línea 234	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 235	<code>entity.ToTable("OrdenesAleaciones");</code> → asigna explícitamente la tabla usada por la entidad.
Línea 236	<code>entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });</code> → define la clave primaria.
Línea 237	<code>entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 238	<code>entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 239	<code>entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 240	<code>entity.HasOne(x => x.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 241	<code>.WithMany(o => o.OrdenesAleaciones)</code> → establece el extremo de colección de una relación uno-a-muchos.
Línea 242	<code>.HasForeignKey(x => x.OrdenFabricacionId)</code> → declara explícitamente la clave foránea.
Línea 243	<code>.OnDelete(DeleteBehavior.Cascade);</code> → define el comportamiento de eliminación de la relación.
Línea 244	<code>entity.HasOne(x => x.Aleacion)</code> → inicia la configuración de una navegación de referencia.
Línea 245	<code>.WithMany(a => a.OrdenesAleaciones)</code> → establece el extremo de colección de una relación uno-a-muchos.
Línea 246	<code>.HasForeignKey(x => x.AleacionId)</code> → declara explícitamente la clave foránea.

Línea 247	<code>.OnDelete(DeleteBehavior.Restrict);</code> → define el comportamiento de eliminación de la relación.
Línea 248	<code>entity.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");</code> → define un índice sobre las propiedades indicadas.
Línea 249	<code>entity.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] = 'Activa'").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");</code> → define un índice sobre las propiedades indicadas.
Línea 250	<code>entity.HasQueryFilter(x => x.EstadoRelacion == "Activa");</code> → aplica un filtro global de consulta.
Línea 251	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 252	<code>}</code> → cierra el bloque de código actual.
Línea 253	<code>}</code> → cierra el bloque de código actual.
Línea 255	<code>public interface IOrdenRepositorio</code> → declara el contrato IOrdenRepositorio usado por la aplicación.
Línea 256	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 257	<code>List<OrdenFabricacion> ObtenerTodas();</code> → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.
Línea 258	<code>OrdenFabricacion? ObtenerPorId(int id);</code> → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.
Línea 259	<code>OrdenFabricacion? ObtenerPorNumero(string numeroOrden);</code> → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.
Línea 260	<code>void Agregar(OrdenFabricacion orden);</code> → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.
Línea 261	<code>void Eliminar(OrdenFabricacion orden);</code> → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.
Línea 262	<code>int Guardar();</code> → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.
Línea 263	<code>}</code> → cierra el bloque de código actual.
Línea 265	<code>public sealed class OrdenRepositorio : IOrdenRepositorio</code> → declara la clase OrdenRepositorio que forma parte del estado acumulativo.
Línea 266	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 267	<code>private readonly AceriaDbContext _context;</code> → declara el campo privado _context de tipo AceriaDbContext.
Línea 268	<code>public OrdenRepositorio(AceriaDbContext context) => _context = context;</code> → inyecta AceriaDbContext en el repositorio y lo conserva como dependencia de persistencia.
Línea 270	<code>public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();</code> → ordena el resultado antes de materializarlo.
Línea 271	<code>public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);</code> → implementa la búsqueda por clave primaria mediante Find, aprovechando primero el ChangeTracker y después la base de datos.

Línea 272	<code>public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);</code> → recupera la primera coincidencia o null.
Línea 273	<code>public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);</code> → registra una entidad nueva para insertarla al guardar.
Línea 274	<code>public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);</code> → marca una entidad para borrado al guardar.
Línea 275	<code>public int Guardar() => _context.SaveChanges();</code> → confirma en SQL Server los cambios del ChangeTracker.
Línea 276	<code>}</code> → cierra el bloque de código actual.
Línea 278	<code>public interface IServicioOrdenes</code> → declara el contrato IServicioOrdenes usado por la aplicación.
Línea 279	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 280	<code>string ObtenerResumen(int ordenId);</code> → declara el servicio que construye un resumen de una orden a partir de su identificador.
Línea 281	<code>}</code> → cierra el bloque de código actual.
Línea 283	<code>public sealed class ServicioOrdenes : IServicioOrdenes</code> → declara la clase ServicioOrdenes que forma parte del estado acumulativo.
Línea 284	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 285	<code>private readonly IOrdenRepositorio _repositorio;</code> → declara el campo privado _repositorio de tipo IOrdenRepositorio.
Línea 286	<code>private readonly AceriaDbContext _context;</code> → declara el campo privado _context de tipo AceriaDbContext.
Línea 287	<code>public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)</code> → declara el constructor que recibe por inyección el repositorio de órdenes y el DbContext.
Línea 288	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 289	<code>_repositorio = repositorio;</code> → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.
Línea 290	<code>_context = context;</code> → guarda el DbContext inyectado para consultar los datos necesarios durante el servicio.
Línea 291	<code>}</code> → cierra el bloque de código actual.
Línea 293	<code>public string ObtenerResumen(int ordenId)</code> → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.
Línea 294	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 295	<code>var orden = _repositorio.ObtenerPorId(ordenId);</code> → declara una variable local y almacena el resultado de la expresión.
Línea 296	<code>if (orden is null) return \$"Orden {ordenId} no encontrada";</code> → evalúa una condición antes de ejecutar el bloque asociado.
Línea 297	<code>var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);</code> → cuenta las filas que cumplen la consulta.

Línea 298	<code>return \$"Orden {orden.NumeroOrden} Cliente: {orden.Cliente} Planchas: {totalPlanchas}";</code> → devuelve el resultado al código llamador.
Línea 299	<code>}</code> → cierra el bloque de código actual.
Línea 300	<code>}</code> → cierra el bloque de código actual.
Línea 302	<code>public static class Program</code> → declara la clase Program que forma parte del estado acumulativo.
Línea 303	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 304	<code>public static void Main()</code> → declara el punto de entrada ejecutable de la aplicación de consola.
Línea 305	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 306	<code>var configuration = new ConfigurationBuilder()</code> → inicia la construcción de la configuración de la aplicación.
Línea 307	<code>.SetBasePath(AppContext.BaseDirectory)</code> → establece la ruta base para los archivos de configuración.
Línea 308	<code>.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)</code> → incorpora appsettings.json.
Línea 309	<code>.AddEnvironmentVariables()</code> → incorpora variables de entorno.
Línea 310	<code>.Build();</code> → materializa el objeto construido por la cadena anterior.
Línea 312	<code>var connectionString = configuration.GetConnectionString("AceriaDB")</code> → obtiene la cadena AceriaDB desde configuración.
Línea 313	<code>?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");</code> → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.
Línea 315	<code>var services = new ServiceCollection();</code> → crea la colección de servicios.
Línea 316	<code>services.AddDbContext<AceriaDbContext>(options =></code> → registra el DbContext en inyección de dependencias.
Línea 317	<code>options</code> → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.
Línea 318	<code>.UseSqlServer(connectionString, sql =></code> → selecciona SQL Server como proveedor de EF Core.
Línea 319	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 320	<code>sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);</code> → habilita reintentos ante fallos transitorios.
Línea 321	<code>sql.CommandTimeout(60);</code> → establece el timeout de los comandos SQL.
Línea 322	<code>})</code> → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.
Línea 323	<code>.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)</code> → configura el logging de EF Core.
Línea 324	<code>.EnableDetailedErrors();</code> → activa errores detallados.

Línea 326	<code>services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();</code> → registra un servicio con ciclo de vida scoped.
Línea 327	<code>services.AddScoped<IServicioOrdenes, ServicioOrdenes>();</code> → registra un servicio con ciclo de vida scoped.
Línea 329	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions</code> → construye el <code>ServiceProvider</code> con validación de servicios y garantiza su liberación al salir del ámbito mediante <code>using var</code> .
Línea 330	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 331	<code>ValidateScopes = true,</code> → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.
Línea 332	<code>ValidateOnBuild = true</code> → obliga al contenedor a validar las resoluciones registradas al construir el <code>ServiceProvider</code> .
Línea 333	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 335	<code>using var scope = provider.CreateScope();</code> → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.
Línea 336	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → resuelve un servicio obligatorio del contenedor.
Línea 337	<code>context.Database.EnsureDeleted();</code> → reinicia la base del laboratorio para una ejecución reproducible.
Línea 338	<code>context.Database.Migrate();</code> → aplica el historial de migraciones pendiente.
Línea 340	<code>var orden = new OrdenFabricacion</code> → declara una variable local y almacena el resultado de la expresión.
Línea 341	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 342	<code>NumeroOrden = "OF-M2-0001",</code> → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.
Línea 343	<code>Cliente = "Constructora del Norte",</code> → asigna el cliente de la orden utilizada en el escenario E2E del punto.
Línea 344	<code>FechaCreacion = DateTime.UtcNow,</code> → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.
Línea 345	<code>FechaEntrega = DateTime.Today.AddDays(14),</code> → establece una fecha de entrega prevista catorce días después de la fecha local actual.
Línea 346	<code>Estado = "Pendiente",</code> → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.
Línea 347	<code>Observaciones = "Orden de validación M2",</code> → añade texto opcional para comprobar el mapeo de Observaciones.
Línea 348	<code>Planchas =</code> → inicia el inicializador de la colección de planchas asociadas a la orden.
Línea 349	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 350	<code>new PlanchaAcero</code> → crea una nueva plancha dentro del inicializador de la colección <code>Planchas</code> de la orden.
Línea 351	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 352	Espesor = 10.5, → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.
Línea 353	Ancho = 1500, → asigna el ancho de la plancha usada en el escenario de persistencia.
Línea 354	Largo = 3000, → asigna el largo de la plancha usada en el escenario de persistencia.
Línea 355	Peso = 371.250m, → asigna un peso decimal con tres posiciones para validar la precisión configurada.
Línea 356	Activa = true, → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.
Línea 357	} → cierra el bloque de código actual.
Línea 358	}, → continúa el inicializador o la llamada del bloque actual.
Línea 359	Detalle = new DetalleOrden → crea el detalle uno-a-uno que se persistirá junto con la orden principal.
Línea 360	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 361	ComposicionQuimica = "C: 0.45%, Mn: 0.75%", → asigna la composición química utilizada para verificar la entidad DetalleOrden.
Línea 362	TemperaturaColada = 1550.5, → asigna la temperatura de colada del detalle de fabricación.
Línea 363	Notas = "Colada principal" → asigna las notas opcionales del detalle asociado a la orden.
Línea 364	}, → continúa el inicializador o la llamada del bloque actual.
Línea 365	Certificado = new CertificadoCalidad → crea el certificado uno-a-uno asociado a la orden del escenario.
Línea 366	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 367	NumeroCertificado = "CERT-0001", → asigna el identificador natural del certificado de calidad.
Línea 368	FechaEmision = DateTime.Today, → asigna la fecha de emisión del certificado usado en la prueba.
Línea 369	OrganismoCertificador = "Laboratorio Aceria" → asigna el organismo que emite el certificado de calidad.
Línea 370	}, → continúa el inicializador o la llamada del bloque actual.
Línea 371	}; → cierra el inicializador del objeto y finaliza la instrucción.
Línea 373	var aleacion = new Aleacion → declara una variable local y almacena el resultado de la expresión.
Línea 374	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 375	Nombre = "AISI 1045", → asigna el nombre comercial de la aleación utilizada en el escenario.
Línea 376	Codigo = "A1045", → asigna el código natural único de la aleación.
Línea 377	PorcentajeCarbono = 0.45, → asigna el porcentaje de carbono dentro del rango permitido por la restricción del modelo.
Línea 378	PorcentajeManganeso = 0.75, → asigna el porcentaje de manganeso dentro del rango permitido por la restricción del modelo.
Línea 379	Descripcion = "Acero medio en carbono" → asigna una descripción opcional a la aleación.

Línea 380	<code>};</code> → cierra el inicializador del objeto y finaliza la instrucción.
Línea 381	<code>orden.OrdenesAleaciones.Add(new OrdenAleacion</code> → registra una entidad nueva para insertarla al guardar.
Línea 382	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 383	<code>Aleacion = aleacion,</code> → conecta la entidad intermedia <code>OrdenAleacion</code> con la aleación creada para el escenario.
Línea 384	<code>CantidadUtilizada = 1500.500m,</code> → registra la cantidad utilizada con tres decimales, coherente con <code>HasPrecision(18, 3)</code> .
Línea 385	<code>EstadoRelacion = "Activa"</code> → marca como activa la relación entre la orden y la aleación.
Línea 386	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 387	<code>context.OrdenesFabricacion.Add(orden);</code> → registra una entidad nueva para insertarla al guardar.
Línea 388	<code>context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });</code> → registra una entidad nueva para insertarla al guardar.
Línea 389	<code>context.SaveChanges();</code> → confirma en SQL Server los cambios del <code>ChangeTracker</code> .
Línea 391	<code>var cancelada = new OrdenFabricacion</code> → declara una variable local y almacena el resultado de la expresión.
Línea 392	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 393	<code>NumeroOrden = "OF-M2-CANCELADA",</code> → asigna el número de la orden de control que se crea con estado Cancelada.
Línea 394	<code>Cliente = "Cliente Histórico",</code> → asigna el cliente de la orden histórica utilizada para comprobar el filtro global.
Línea 395	<code>FechaCreacion = DateTime.UtcNow,</code> → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.
Línea 396	<code>Estado = "Cancelada"</code> → establece el estado Cancelada para verificar que <code>HasQueryFilter</code> excluye la orden de las consultas normales.
Línea 397	<code>};</code> → cierra el inicializador del objeto y finaliza la instrucción.
Línea 398	<code>context.OrdenesFabricacion.Add(cancelada);</code> → registra una entidad nueva para insertarla al guardar.
Línea 399	<code>context.SaveChanges();</code> → confirma en SQL Server los cambios del <code>ChangeTracker</code> .
Línea 400	<code>var visibles = context.OrdenesFabricacion.Count();</code> → cuenta las filas que cumplen la consulta.
Línea 401	<code>var todas = context.OrdenesFabricacion.IgnoreQueryFilters().Count();</code> → omite de forma explícita los filtros globales para esta consulta.
Línea 402	<code>global::System.Console.WriteLine(\$"2.10 OK Visibles: {visibles} Sin filtro: {todas}");</code> → escribe la evidencia de ejecución usada para validar este estado.
Línea 403	<code>}</code> → cierra el bloque de código actual.
Línea 404	<code>}</code> → cierra el bloque de código actual.

Punto 2.11 - Migraciones en el modelado: ciclo completo con Soft Delete

Audiencia: Desarrolladores que ya han completado los puntos 2.1 a 2.10.

Proyecto: Se utiliza el cambio real de Soft Delete para aprender el ciclo completo de una migración incremental: generar, revisar, aplicar, inspeccionar el historial, revertir, reaplicar, retirar una migración local y generar scripts SQL normales e idempotentes.

Objetivos de aprendizaje

Generar una migración incremental sobre el estado 2.10.

Revisar línea a línea la migración `M2_2_11`.

Comprobar el snapshot y `__EFMigrationsHistory`.

Aplicar y revertir el esquema hasta una migración concreta.

Entender el uso seguro de `migrations remove`.

Generar scripts SQL e idempotentes.

Validar que el esquema resultante soporta el Soft Delete y su restauración.

Paso 1: Abrir la solución autónoma del punto

```
cd M02/PROYECTO/2.11
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

BASH

`cd` → entra en el estado acumulativo 2.11.

`dotnet restore` → restaura EF Core 8 y el proveedor SQL Server.

`dotnet build` → comprueba que modelo, migraciones y fábrica de diseño compilan juntos.

Paso 2: Comparar el modelo 2.10 con el cambio funcional 2.11

En 2.10 existen filtros globales, pero todavía no existen las columnas persistentes del borrado lógico. El estado 2.11 añade:

```
public bool IsDeleted { get; set; }
public DateTime? DeletedAt { get; set; }
```

CSHARP

Estas propiedades aparecen en `OrdenFabricacion`, `PlanchaAcero`, `Aleacion` y `EstadoOrden`. Después se combinan con `HasQueryFilter` para que las consultas normales excluyan registros eliminados.

Paso 3: Generar la migración incremental al construir el estado 2.11

Al partir físicamente del estado 2.10, después de introducir las propiedades y filtros de 2.11, el comando es:

```
dotnet ef migrations add M2_2_11
```

BASH

`dotnet ef migrations add` → compara el modelo modificado con el snapshot de 2.10.

`M2_2_11` → identifica el checkpoint que introduce Soft Delete.

En el repositorio entregado la migración ya está generada porque 2.11 es un estado completo y reproducible. No se debe volver a ejecutar `migrations add M2_2_11` sobre el estado final: el delta original ya está registrado.

Paso 4: Revisar la migración real antes de aplicarla

Abrir `Migrations/20260927204841_M2_2_11.cs`:

```
using System;
using Microsoft.EntityFrameworkCore.Migrations;

#nullable disable

namespace AceriaData.ConsoleApp.Migrations
{
```

CSHARP

```

/// <inheritdoc />
public partial class M2_2_11 : Migration
{
    /// <inheritdoc />
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.AddColumn<DateTime>(
            name: "DeletedAt",
            table: "PlanchasAcero",
            type: "datetime2",
            nullable: true);

        migrationBuilder.AddColumn<bool>(
            name: "IsDeleted",
            table: "PlanchasAcero",
            type: "bit",
            nullable: false,
            defaultValue: false);

        migrationBuilder.AddColumn<DateTime>(
            name: "DeletedAt",
            table: "OrdenesFabricacion",
            type: "datetime2",
            nullable: true);

        migrationBuilder.AddColumn<bool>(
            name: "IsDeleted",
            table: "OrdenesFabricacion",
            type: "bit",
            nullable: false,
            defaultValue: false);

        migrationBuilder.AddColumn<DateTime>(
            name: "DeletedAt",
            table: "EstadosOrden",
            type: "datetime2",
            nullable: true);

        migrationBuilder.AddColumn<bool>(
            name: "IsDeleted",
            table: "EstadosOrden",
            type: "bit",
            nullable: false,
            defaultValue: false);

        migrationBuilder.AddColumn<DateTime>(
            name: "DeletedAt",
            table: "Aleaciones",
            type: "datetime2",
            nullable: true);

        migrationBuilder.AddColumn<bool>(
            name: "IsDeleted",
            table: "Aleaciones",
            type: "bit",
            nullable: false,
            defaultValue: false);
    }

    /// <inheritdoc />
    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropColumn(
            name: "DeletedAt",
            table: "PlanchasAcero");

        migrationBuilder.DropColumn(
            name: "IsDeleted",
            table: "PlanchasAcero");

        migrationBuilder.DropColumn(
            name: "DeletedAt",
            table: "OrdenesFabricacion");

        migrationBuilder.DropColumn(
            name: "IsDeleted",
            table: "OrdenesFabricacion");

        migrationBuilder.DropColumn(
            name: "DeletedAt",
            table: "EstadosOrden");

        migrationBuilder.DropColumn(
            name: "IsDeleted",
            table: "EstadosOrden");
    }
}

```

```

        migrationBuilder.DropColumn(
            name: "DeletedAt",
            table: "Aleaciones");

        migrationBuilder.DropColumn(
            name: "IsDeleted",
            table: "Aleaciones");
    }
}
}

```

Explicación línea a línea de la migración M2_2_11

Línea 1	<code>using System;</code> → importa los tipos base de .NET necesarios para las columnas DateTime.
Línea 2	<code>using Microsoft.EntityFrameworkCore.Migrations;</code> → importa Migration y MigrationBuilder, la API que expresa cambios de esquema.
Línea 4	<code>#nullable disable</code> → mantiene el contexto de nulabilidad generado por EF Core para este archivo.
Línea 6	<code>namespace AceriaData.ConsoleApp.Migrations</code> → declara el espacio de nombres del historial de migraciones del contexto.
Línea 7	<code>{</code> → abre el bloque de la declaración o método anterior.
Línea 8	<code>/// <inheritdoc /></code> → comentario XML generado automáticamente para documentar el miembro siguiente.
Línea 9	<code>public partial class M2_2_11 : Migration</code> → declara la migración incremental correspondiente al checkpoint 2.11.
Línea 10	<code>{</code> → abre el bloque de la declaración o método anterior.
Línea 11	<code>/// <inheritdoc /></code> → comentario XML generado automáticamente para documentar el miembro siguiente.
Línea 12	<code>protected override void Up(MigrationBuilder migrationBuilder)</code> → define las operaciones que llevan el esquema desde 2.10 hasta 2.11.
Línea 13	<code>{</code> → abre el bloque de la declaración o método anterior.
Línea 14	<code>migrationBuilder.AddColumn<DateTime>(</code> → inicia la adición de una columna DeletedAt de fecha/hora.
Línea 15	<code>name: "DeletedAt",</code> → identifica la columna afectada como DeletedAt.
Línea 16	<code>table: "PlanchasAcero",</code> → dirige la operación a la tabla PlanchasAcero.
Línea 17	<code>type: "datetime2",</code> → fija el tipo SQL Server datetime2.
Línea 18	<code>nullable: true);</code> → DeletedAt admite NULL mientras el registro no esté eliminado.
Línea 20	<code>migrationBuilder.AddColumn<bool>(</code> → inicia la adición de una columna IsDeleted booleana.
Línea 21	<code>name: "IsDeleted",</code> → identifica la columna afectada como IsDeleted.
Línea 22	<code>table: "PlanchasAcero",</code> → dirige la operación a la tabla PlanchasAcero.
Línea 23	<code>type: "bit",</code> → fija el tipo SQL Server bit.
Línea 24	<code>nullable: false,</code> → IsDeleted es obligatorio para todas las filas.

Línea 25	<code>defaultValue: false);</code> → rellena las filas existentes con false para mantenerlas activas al aplicar el cambio.
Línea 27	<code>migrationBuilder.AddColumn<DateTime>(</code> → inicia la adición de una columna DeletedAt de fecha/hora.
Línea 28	<code>name: "DeletedAt",</code> → identifica la columna afectada como DeletedAt .
Línea 29	<code>table: "OrdenesFabricacion",</code> → dirige la operación a la tabla OrdenesFabricacion .
Línea 30	<code>type: "datetime2",</code> → fija el tipo SQL Server datetime2 .
Línea 31	<code>nullable: true);</code> → DeletedAt admite NULL mientras el registro no esté eliminado.
Línea 33	<code>migrationBuilder.AddColumn<bool>(</code> → inicia la adición de una columna IsDeleted booleana.
Línea 34	<code>name: "IsDeleted",</code> → identifica la columna afectada como IsDeleted .
Línea 35	<code>table: "OrdenesFabricacion",</code> → dirige la operación a la tabla OrdenesFabricacion .
Línea 36	<code>type: "bit",</code> → fija el tipo SQL Server bit .
Línea 37	<code>nullable: false,</code> → IsDeleted es obligatorio para todas las filas.
Línea 38	<code>defaultValue: false);</code> → rellena las filas existentes con false para mantenerlas activas al aplicar el cambio.
Línea 40	<code>migrationBuilder.AddColumn<DateTime>(</code> → inicia la adición de una columna DeletedAt de fecha/hora.
Línea 41	<code>name: "DeletedAt",</code> → identifica la columna afectada como DeletedAt .
Línea 42	<code>table: "EstadosOrden",</code> → dirige la operación a la tabla EstadosOrden .
Línea 43	<code>type: "datetime2",</code> → fija el tipo SQL Server datetime2 .
Línea 44	<code>nullable: true);</code> → DeletedAt admite NULL mientras el registro no esté eliminado.
Línea 46	<code>migrationBuilder.AddColumn<bool>(</code> → inicia la adición de una columna IsDeleted booleana.
Línea 47	<code>name: "IsDeleted",</code> → identifica la columna afectada como IsDeleted .
Línea 48	<code>table: "EstadosOrden",</code> → dirige la operación a la tabla EstadosOrden .
Línea 49	<code>type: "bit",</code> → fija el tipo SQL Server bit .
Línea 50	<code>nullable: false,</code> → IsDeleted es obligatorio para todas las filas.
Línea 51	<code>defaultValue: false);</code> → rellena las filas existentes con false para mantenerlas activas al aplicar el cambio.
Línea 53	<code>migrationBuilder.AddColumn<DateTime>(</code> → inicia la adición de una columna DeletedAt de fecha/hora.
Línea 54	<code>name: "DeletedAt",</code> → identifica la columna afectada como DeletedAt .
Línea 55	<code>table: "Aleaciones",</code> → dirige la operación a la tabla Aleaciones .
Línea 56	<code>type: "datetime2",</code> → fija el tipo SQL Server datetime2 .
Línea 57	<code>nullable: true);</code> → DeletedAt admite NULL mientras el registro no esté eliminado.

Línea 59	<code>migrationBuilder.AddColumn<bool>(</code> → inicia la adición de una columna <code>IsDeleted</code> booleana.
Línea 60	<code>name: "IsDeleted",</code> → identifica la columna afectada como <code>IsDeleted</code> .
Línea 61	<code>table: "Aleaciones",</code> → dirige la operación a la tabla <code>Aleaciones</code> .
Línea 62	<code>type: "bit",</code> → fija el tipo SQL Server <code>bit</code> .
Línea 63	<code>nullable: false,</code> → <code>IsDeleted</code> es obligatorio para todas las filas.
Línea 64	<code>defaultValue: false);</code> → rellena las filas existentes con <code>false</code> para mantenerlas activas al aplicar el cambio.
Línea 65	<code>}</code> → cierra el bloque de la declaración o método anterior.
Línea 67	<code>/// <inheritdoc /></code> → comentario XML generado automáticamente para documentar el miembro siguiente.
Línea 68	<code>protected override void Down(MigrationBuilder migrationBuilder)</code> → define las operaciones inversas que devuelven el esquema desde 2.11 hasta 2.10.
Línea 69	<code>{</code> → abre el bloque de la declaración o método anterior.
Línea 70	<code>migrationBuilder.DropColumn(</code> → inicia la eliminación de una de las columnas añadidas por esta migración.
Línea 71	<code>name: "DeletedAt",</code> → identifica la columna afectada como <code>DeletedAt</code> .
Línea 72	<code>table: "PlanchasAcero");</code> → dirige la operación a la tabla <code>PlanchasAcero</code> .
Línea 74	<code>migrationBuilder.DropColumn(</code> → inicia la eliminación de una de las columnas añadidas por esta migración.
Línea 75	<code>name: "IsDeleted",</code> → identifica la columna afectada como <code>IsDeleted</code> .
Línea 76	<code>table: "PlanchasAcero");</code> → dirige la operación a la tabla <code>PlanchasAcero</code> .
Línea 78	<code>migrationBuilder.DropColumn(</code> → inicia la eliminación de una de las columnas añadidas por esta migración.
Línea 79	<code>name: "DeletedAt",</code> → identifica la columna afectada como <code>DeletedAt</code> .
Línea 80	<code>table: "OrdenesFabricacion");</code> → dirige la operación a la tabla <code>OrdenesFabricacion</code> .
Línea 82	<code>migrationBuilder.DropColumn(</code> → inicia la eliminación de una de las columnas añadidas por esta migración.
Línea 83	<code>name: "IsDeleted",</code> → identifica la columna afectada como <code>IsDeleted</code> .
Línea 84	<code>table: "OrdenesFabricacion");</code> → dirige la operación a la tabla <code>OrdenesFabricacion</code> .
Línea 86	<code>migrationBuilder.DropColumn(</code> → inicia la eliminación de una de las columnas añadidas por esta migración.
Línea 87	<code>name: "DeletedAt",</code> → identifica la columna afectada como <code>DeletedAt</code> .
Línea 88	<code>table: "EstadosOrden");</code> → dirige la operación a la tabla <code>EstadosOrden</code> .
Línea 90	<code>migrationBuilder.DropColumn(</code> → inicia la eliminación de una de las columnas añadidas por esta migración.
Línea 91	<code>name: "IsDeleted",</code> → identifica la columna afectada como <code>IsDeleted</code> .

Línea 92	table: "EstadosOrden"); → dirige la operación a la tabla EstadosOrden .
Línea 94	migrationBuilder.DropColumn(→ inicia la eliminación de una de las columnas añadidas por esta migración.
Línea 95	name: "DeletedAt", → identifica la columna afectada como DeletedAt .
Línea 96	table: "Aleaciones"); → dirige la operación a la tabla Aleaciones .
Línea 98	migrationBuilder.DropColumn(→ inicia la eliminación de una de las columnas añadidas por esta migración.
Línea 99	name: "IsDeleted", → identifica la columna afectada como IsDeleted .
Línea 100	table: "Aleaciones"); → dirige la operación a la tabla Aleaciones .
Línea 101	} → cierra el bloque de la declaración o método anterior.
Línea 102	} → cierra el bloque de la declaración o método anterior.
Línea 103	} → cierra el bloque de la declaración o método anterior.

Las columnas `IsDeleted` usan `defaultValue: false`. Así, las filas existentes permanecen activas después del cambio de esquema.

Paso 5: Aplicar la migración

```
dotnet ef database update --configuration Release
```

BASH

Resultado esperado: EF Core aplica las migraciones pendientes hasta `M2_2_11`. Las cuatro tablas afectadas contienen `IsDeleted` y `DeletedAt`.

Paso 6: Verificar la cadena y la tabla de historial

```
dotnet ef migrations list --configuration Release
```

BASH

Consultar además en SQL Server LocalDB:

```
SELECT MigrationId, ProductVersion
FROM __EFMigrationsHistory
ORDER BY MigrationId;
```

SQL

Resultado esperado: `M2_2_11` aparece como última migración aplicada.

Paso 7: Revertir exactamente hasta 2.10

```
dotnet ef database update 20260927204833_M2_2_10 --configuration Release
```

BASH

Resultado esperado: EF Core ejecuta el `Down` de `M2_2_11`, elimina las columnas añadidas en este punto y retira la migración de `__EFMigrationsHistory`.

Paso 8: Reaplicar 2.11

```
dotnet ef database update --configuration Release
```

BASH

Resultado esperado: `Up` vuelve a crear las columnas y el historial vuelve a registrar `M2_2_11`.

Paso 9: Comprender migrations remove sin dañar el repositorio canónico

`migrations remove` modifica archivos fuente y snapshot; por eso no se ejecuta directamente sobre la copia canónica ya validada. Para practicarlo, trabajar sobre una copia desechable de 2.11, revertir primero la base de laboratorio a 2.10 y ejecutar:

```
dotnet ef database update 20260927204833_M2_2_10 --configuration Release
dotnet ef migrations remove --configuration Release
```

BASH

Resultado esperado en la copia: desaparecen los archivos de la última migración y el snapshot vuelve al modelo anterior. Después se descarta la copia. Si una migración ya se ha compartido o desplegado, se crea una migración correctiva nueva en lugar de reescribir la historia.

Paso 10: Revisar el ModelSnapshot

Localizar en `Migrations/AceriaDbContextModelSnapshot.cs` las propiedades nuevas. Un fragmento real del snapshot es:

```
b.Property<DateTime?>("DeletedAt")
    .HasColumnType("datetime2");

b.Property<string>("Descripcion")
    .HasMaxLength(500)
    .HasColumnType("nvarchar(500)");

b.Property<bool>("IsDeleted")
    .HasColumnType("bit");

b.Property<string>("Nombre")
    .IsRequired()
    .HasMaxLength(100)
    .HasColumnType("nvarchar(100)");

b.Property<double>("PorcentajeCarbono")
    .HasColumnType("float");

b.Property<double>("PorcentajeManganeso")
    .HasColumnType("float");

b.HasKey("Id");
```

CSHARP

El snapshot no es la base ni el historial aplicado: es la referencia de modelo que la siguiente ejecución de `migrations add` comparará con el nuevo modelo.

Paso 11: Generar scripts SQL normal e idempotente

```
dotnet ef migrations script 20260927204833_M2_2_10 20260927204841_M2_2_11 --configuration Release --output
softdelete.sql
dotnet ef migrations script --idempotent --configuration Release --output migraciones_idempotentes.sql
```

BASH

`softdelete.sql` → contiene sólo el tramo 2.10 → 2.11.

`migraciones_idempotentes.sql` → comprueba `EFMigrationsHistory` antes de ejecutar cada migración.

Paso 12: Ejecutar el E2E funcional del Soft Delete

```
dotnet run --project AceriaData.Console.csproj --configuration Release --no-build
```

BASH

Resultado esperado:

```
2.11 OK | Tras borrar visibles: 0 | Totales: 1 | Restaurada: True
```

TEXT

El E2E demuestra que el cambio de esquema es funcional: el filtro oculta la entidad marcada, `IgnoreQueryFilters()` permite recuperarla y la restauración vuelve a hacerla visible.

Errores comunes

Error	Causa	Solución
Migración vacía	El modelo y el snapshot ya representan el mismo estado	Generarla durante la transición real 2.10 → 2.11
Columna no anulable incompatible con datos existentes	No existe valor válido para las filas previas	Diseñar default, backfill o transición nullable
<code>database update</code> no cambia nada	La migración ya figura en <code>__EFMigrationsHistory</code>	Comprobar el historial y la base objetivo
La reversión pierde información	<code>Down</code> elimina columnas o tablas	Revisar impacto de datos y disponer de backup
<code>migrations remove</code> no es apropiado	La migración ya se compartió o desplegó	Crear una migración correctiva en vez de reescribir la historia
Script idempotente incompleto	Se está usando otro contexto/proyecto	Verificar proyecto, contexto y cadena

Reto resuelto

Generar únicamente el SQL del delta 2.10 → 2.11 y localizar en el script las cuatro parejas `DeletedAt/IsDeleted`. Después comprobar que el script idempotente consulta `__EFMigrationsHistory` antes de ejecutar los bloques. La solución es usar el rango explícito para el primer script y `--idempotent` para el segundo.

Analogía final

El modelo es el plano actual de una línea de producción; el snapshot es el plano archivado después de la última reforma; `M2_2_11` es la orden de obra que transforma 2.10 en 2.11; y `__EFMigrationsHistory` es el libro de reformas realmente ejecutadas. Soft Delete es la reforma concreta con la que se estudia todo el ciclo.

Resultado esperado

Al terminar el punto se sabe generar y revisar una migración incremental, comprobar `Up` y `Down`, leer el snapshot, aplicar y revertir hasta un checkpoint, entender `migrations remove`, producir SQL e idempotentes y validar funcionalmente el Soft Delete.

Conexión con el siguiente punto

2.12 reorganiza el mismo sistema en Domain, Application, Infrastructure y Console. Las migraciones pasan a Infrastructure y el contexto de diseño se resuelve mediante `IDesignTimeDbContextFactory`, pero el historial y el esquema continúan siendo los validados en 2.11.

Código acumulativo completo del estado 2.11

El siguiente archivo es el `Program.cs` real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]

```

CSHARP

```

public string Cliente { get; set; } = string.Empty;
public DateTime FechaCreacion { get; set; }
public DateTime? FechaEntrega { get; set; }
[Required]
[MaxLength(50)]
public string Estado { get; set; } = "Pendiente";
[MaxLength(500)]
public string? Observaciones { get; set; }
public List<PlanchaAcero> Planchas { get; set; } = new();
public DetalleOrden? Detalle { get; set; }
public CertificadoCalidad? Certificado { get; set; }
public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
public bool IsDeleted { get; set; }
public DateTime? DeletedAt { get; set; }
}

[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

```

```

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }
    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
            entity.Property(o => o.Observaciones).HasMaxLength(500);
            entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");
            entity.HasIndex(o => o.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");
            entity.HasIndex(o => new { o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");
            entity.HasIndex(o => o.FechaEntrega).HasFilter("[Estado] = 'Pendiente'").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");
            entity.HasIndex(o => o.Estado).IncludeProperties(o => new { o.NumeroOrden, o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");
            entity.HasQueryFilter(o => !o.IsDeleted && o.Estado != "Cancelada");
        });
        modelBuilder.Entity<PlanchaAcero>(entity =>
        {
            entity.ToTable("PlanchasAcero", t =>
            {
                t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");
                t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");
                t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");
                t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");
            });
            entity.HasKey(x => x.Id);
            entity.Property(x => x.Peso).HasPrecision(18, 3);
            entity.Property(x => x.Activa).HasDefaultValue(true);
            entity.HasOne(x => x.Orden)
                .WithMany(o => o.Planchas)
                .HasForeignKey(x => x.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();
            entity.HasIndex(x => new { x.OrdenId, x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");
        });
    }
}

```

```

        entity.HasIndex(x => x.Espesor).HasFilter("[Activa] = 1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");
        entity.HasQueryFilter(x => !x.IsDeleted && x.Activa);
    });
    modelBuilder.Entity<Aleacion>(entity =>
    {
        entity.ToTable("Aleaciones", t =>
        {
            t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND [PorcentajeCarbono] <= 2");
            t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND [PorcentajeManganeso] <= 5");
        });
        entity.HasKey(a => a.Id);
        entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
        entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
        entity.Property(a => a.Descripcion).HasMaxLength(500);
        entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");
        entity.HasIndex(a => a.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");
        entity.HasIndex(a => new { a.PorcentajeCarbono, a.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");
        entity.HasQueryFilter(a => !a.IsDeleted);
    });
    modelBuilder.Entity<EstadoOrden>(entity =>
    {
        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
        entity.Property(e => e.Descripcion).HasMaxLength(250);
        entity.Property(e => e.Activo).HasDefaultValue(true);
        entity.HasQueryFilter(e => !e.IsDeleted && e.Activo);
    });
    modelBuilder.Entity<DetalleOrden>(entity =>
    {
        entity.ToTable("DetallesOrden");
        entity.HasKey(d => d.Id);
        entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
        entity.Property(d => d.Notas).HasMaxLength(500);
        entity.HasOne(d => d.Orden)
            .WithOne(o => o.Detalle)
            .HasForeignKey<DetalleOrden>(d => d.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
        entity.HasQueryFilter(d => !d.Orden.IsDeleted && d.Orden.Estado != "Cancelada");
    });

    modelBuilder.Entity<CertificadoCalidad>(entity =>
    {
        entity.ToTable("CertificadosCalidad");
        entity.HasKey(c => c.Id);
        entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
        entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
        entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");
        entity.HasIndex(c => c.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");
        entity.HasOne(c => c.Orden)
            .WithOne(o => o.Certificado)
            .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
        entity.HasQueryFilter(c => !c.Orden.IsDeleted && c.Orden.Estado != "Cancelada");
    });

    modelBuilder.Entity<OrdenAleacion>(entity =>
    {
        entity.ToTable("OrdenesAleaciones");
        entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
        entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
        entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.OrdenesAleaciones)
            .HasForeignKey(x => x.OrdenFabricacionId)
            .OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(x => x.Aleacion)
            .WithMany(a => a.OrdenesAleaciones)
            .HasForeignKey(x => x.AleacionId)
            .OnDelete(DeleteBehavior.Restrict);
        entity.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");
        entity.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] = 'Activa'").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");
        entity.HasQueryFilter(x => x.EstadoRelacion == "Activa" && !x.Orden.IsDeleted && !x.Aleacion.IsDeleted);
    });
}

}

public interface IOordenRepositorio

```

```

{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    void EliminarLogicamente(int id);
    void Restaurar(int id);

    int Guardar();
}

public sealed class OrdenRepositorio : IOOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public void EliminarLogicamente(int id)
    {
        var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == id);
        if (orden is null) return;
        orden.IsDeleted = true;
        orden.DeletedAt = DateTime.UtcNow;
    }

    public void Restaurar(int id)
    {
        var orden = _context.OrdenesFabricacion.IgnoreQueryFilters().FirstOrDefault(o => o.Id == id);
        if (orden is null) return;
        orden.IsDeleted = false;
        orden.DeletedAt = null;
    }

    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                    {
                        sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                    }
                )
        );
    }
}

```

```

        sql.CommandTimeout(60);
    })
    .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
LogLevel.Information)
    .EnableDetailedErrors());

services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    IsDeleted = false,
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },
    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});
context.OrdenesFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo =
true, IsDeleted = false });
context.SaveChanges();

var id = orden.Id;
orden.IsDeleted = true;
orden.DeletedAt = DateTime.UtcNow;
context.SaveChanges();
var visibles = context.OrdenesFabricacion.Count();
var todas = context.OrdenesFabricacion.IgnoreQueryFilters().Count();
var restaurable = context.OrdenesFabricacion.IgnoreQueryFilters().Single(o => o.Id == id);
restaurable.IsDeleted = false;
restaurable.DeletedAt = null;
context.SaveChanges();
global::System.Console.WriteLine($"2.11 OK | Tras borrar visibles: {visibles} | Totales: {todas} |
Restaurada: {context.OrdenesFabricacion.Any(o => o.Id == id)}");

```

```
}
}
```

Explicación línea a línea del código acumulativo 2.11

Las referencias usan la numeración real de M02/PROYECTO/2.11/Program.cs. Se explican todas las líneas no vacías del archivo que compila.

Línea 1	<code>using System.ComponentModel.DataAnnotations;</code> → importa el espacio de nombres necesario para este archivo.
Línea 2	<code>using System.ComponentModel.DataAnnotations.Schema;</code> → importa el espacio de nombres necesario para este archivo.
Línea 3	<code>using Microsoft.EntityFrameworkCore;</code> → importa el espacio de nombres necesario para este archivo.
Línea 4	<code>using Microsoft.Extensions.Configuration;</code> → importa el espacio de nombres necesario para este archivo.
Línea 5	<code>using Microsoft.Extensions.DependencyInjection;</code> → importa el espacio de nombres necesario para este archivo.
Línea 6	<code>using Microsoft.Extensions.Logging;</code> → importa el espacio de nombres necesario para este archivo.
Línea 8	<code>namespace AceriaData.ConsoleApp;</code> → declara el espacio de nombres del estado acumulativo.
Línea 10	<code>[Table("OrdenesFabricacion")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 11	<code>public class OrdenFabricacion</code> → declara la clase OrdenFabricacion que forma parte del estado acumulativo.
Línea 12	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 13	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 14	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.
Línea 15	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 16	<code>[MaxLength(50)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 17	<code>public string NumeroOrden { get; set; } = string.Empty;</code> → declara la propiedad NumeroOrden de tipo string usada por el modelo o la lógica de negocio.
Línea 18	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 19	<code>[MaxLength(200)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 20	<code>public string Cliente { get; set; } = string.Empty;</code> → declara la propiedad Cliente de tipo string usada por el modelo o la lógica de negocio.
Línea 21	<code>public DateTime FechaCreacion { get; set; }</code> → declara la propiedad FechaCreacion de tipo DateTime usada por el modelo o la lógica de negocio.
Línea 22	<code>public DateTime? FechaEntrega { get; set; }</code> → declara la propiedad FechaEntrega de tipo DateTime? usada por el modelo o la lógica de negocio.
Línea 23	<code>[Required]</code> → marca la propiedad siguiente como requerida.

Línea 24	<code>[MaxLength(50)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 25	<code>public string Estado { get; set; } = "Pendiente";</code> → declara la propiedad Estado de tipo string usada por el modelo o la lógica de negocio.
Línea 26	<code>[MaxLength(500)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 27	<code>public string? Observaciones { get; set; }</code> → declara la propiedad Observaciones de tipo string? usada por el modelo o la lógica de negocio.
Línea 28	<code>public List<PlanchaAcero> Planchas { get; set; } = new();</code> → declara la propiedad Planchas de tipo List<PlanchaAcero> usada por el modelo o la lógica de negocio.
Línea 29	<code>public DetalleOrden? Detalle { get; set; }</code> → declara la propiedad Detalle de tipo DetalleOrden? usada por el modelo o la lógica de negocio.
Línea 30	<code>public CertificadoCalidad? Certificado { get; set; }</code> → declara la propiedad Certificado de tipo CertificadoCalidad? usada por el modelo o la lógica de negocio.
Línea 31	<code>public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();</code> → declara la propiedad OrdenesAleaciones de tipo List<OrdenAleacion> usada por el modelo o la lógica de negocio.
Línea 32	<code>public bool IsDeleted { get; set; }</code> → declara la propiedad IsDeleted de tipo bool usada por el modelo o la lógica de negocio.
Línea 33	<code>public DateTime? DeletedAt { get; set; }</code> → declara la propiedad DeletedAt de tipo DateTime? usada por el modelo o la lógica de negocio.
Línea 34	<code>}</code> → cierra el bloque de código actual.
Línea 36	<code>[Table("PlanchasAcero")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 37	<code>public class PlanchaAcero</code> → declara la clase PlanchaAcero que forma parte del estado acumulativo.
Línea 38	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 39	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 40	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.
Línea 41	<code>public int OrdenId { get; set; }</code> → declara la propiedad OrdenId de tipo int usada por el modelo o la lógica de negocio.
Línea 42	<code>public double Espesor { get; set; }</code> → declara la propiedad Espesor de tipo double usada por el modelo o la lógica de negocio.
Línea 43	<code>public double Ancho { get; set; }</code> → declara la propiedad Ancho de tipo double usada por el modelo o la lógica de negocio.
Línea 44	<code>public double Largo { get; set; }</code> → declara la propiedad Largo de tipo double usada por el modelo o la lógica de negocio.
Línea 45	<code>[Precision(18, 3)]</code> → configura precisión y escala para la propiedad numérica siguiente.
Línea 46	<code>public decimal Peso { get; set; }</code> → declara la propiedad Peso de tipo decimal usada por el modelo o la lógica de negocio.
Línea 47	<code>public bool Activa { get; set; } = true;</code> → declara la propiedad Activa de tipo bool usada por el modelo o la lógica de negocio.

Línea 48	<code>[ForeignKey(nameof(OrdenId))]</code> → vincula la navegación siguiente con su clave foránea.
Línea 49	<code>public OrdenFabricacion Orden { get; set; } = null!;</code> → declara la propiedad Orden de tipo OrdenFabricacion usada por el modelo o la lógica de negocio.
Línea 50	<code>public bool IsDeleted { get; set; }</code> → declara la propiedad IsDeleted de tipo bool usada por el modelo o la lógica de negocio.
Línea 51	<code>public DateTime? DeletedAt { get; set; }</code> → declara la propiedad DeletedAt de tipo DateTime? usada por el modelo o la lógica de negocio.
Línea 52	<code>}</code> → cierra el bloque de código actual.
Línea 54	<code>[Table("Aleaciones")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 55	<code>public class Aleacion</code> → declara la clase Aleacion que forma parte del estado acumulativo.
Línea 56	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 57	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 58	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.
Línea 59	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 60	<code>[MaxLength(100)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 61	<code>public string Nombre { get; set; } = string.Empty;</code> → declara la propiedad Nombre de tipo string usada por el modelo o la lógica de negocio.
Línea 62	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 63	<code>[MaxLength(20)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 64	<code>public string Codigo { get; set; } = string.Empty;</code> → declara la propiedad Codigo de tipo string usada por el modelo o la lógica de negocio.
Línea 65	<code>public double PorcentajeCarbono { get; set; }</code> → declara la propiedad PorcentajeCarbono de tipo double usada por el modelo o la lógica de negocio.
Línea 66	<code>public double PorcentajeManganeso { get; set; }</code> → declara la propiedad PorcentajeManganeso de tipo double usada por el modelo o la lógica de negocio.
Línea 67	<code>[MaxLength(500)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 68	<code>public string? Descripcion { get; set; }</code> → declara la propiedad Descripcion de tipo string? usada por el modelo o la lógica de negocio.
Línea 69	<code>public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();</code> → declara la propiedad OrdenesAleaciones de tipo List<OrdenAleacion> usada por el modelo o la lógica de negocio.
Línea 70	<code>public bool IsDeleted { get; set; }</code> → declara la propiedad IsDeleted de tipo bool usada por el modelo o la lógica de negocio.
Línea 71	<code>public DateTime? DeletedAt { get; set; }</code> → declara la propiedad DeletedAt de tipo DateTime? usada por el modelo o la lógica de negocio.
Línea 72	<code>}</code> → cierra el bloque de código actual.

Línea 74	<code>[Table("EstadosOrden")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 75	<code>public class EstadoOrden</code> → declara la clase EstadoOrden que forma parte del estado acumulativo.
Línea 76	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 77	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 78	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.
Línea 79	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 80	<code>[MaxLength(50)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 81	<code>public string Nombre { get; set; } = string.Empty;</code> → declara la propiedad Nombre de tipo string usada por el modelo o la lógica de negocio.
Línea 82	<code>[MaxLength(250)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 83	<code>public string Descripcion { get; set; } = string.Empty;</code> → declara la propiedad Descripcion de tipo string usada por el modelo o la lógica de negocio.
Línea 84	<code>public bool Activo { get; set; } = true;</code> → declara la propiedad Activo de tipo bool usada por el modelo o la lógica de negocio.
Línea 85	<code>public bool IsDeleted { get; set; }</code> → declara la propiedad IsDeleted de tipo bool usada por el modelo o la lógica de negocio.
Línea 86	<code>public DateTime? DeletedAt { get; set; }</code> → declara la propiedad DeletedAt de tipo DateTime? usada por el modelo o la lógica de negocio.
Línea 87	<code>}</code> → cierra el bloque de código actual.
Línea 89	<code>[Table("DetallesOrden")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 90	<code>public class DetalleOrden</code> → declara la clase DetalleOrden que forma parte del estado acumulativo.
Línea 91	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 92	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 93	<code>public int Id { get; set; }</code> → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.
Línea 94	<code>public int OrdenId { get; set; }</code> → declara la propiedad OrdenId de tipo int usada por el modelo o la lógica de negocio.
Línea 95	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 96	<code>[MaxLength(200)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 97	<code>public string ComposicionQuimica { get; set; } = string.Empty;</code> → declara la propiedad ComposicionQuimica de tipo string usada por el modelo o la lógica de negocio.
Línea 98	<code>public double TemperaturaColada { get; set; }</code> → declara la propiedad TemperaturaColada de tipo double usada por el modelo o la lógica de negocio.

Línea 99	<code>[MaxLength(500)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 100	<code>public string? Notas { get; set; }</code> → declara la propiedad <code>Notas</code> de tipo <code>string?</code> usada por el modelo o la lógica de negocio.
Línea 101	<code>[ForeignKey(nameof(OrdenId))]</code> → vincula la navegación siguiente con su clave foránea.
Línea 102	<code>public OrdenFabricacion Orden { get; set; } = null!</code> → declara la propiedad <code>Orden</code> de tipo <code>OrdenFabricacion</code> usada por el modelo o la lógica de negocio.
Línea 103	<code>}</code> → cierra el bloque de código actual.
Línea 105	<code>[Table("CertificadosCalidad")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 106	<code>public class CertificadoCalidad</code> → declara la clase <code>CertificadoCalidad</code> que forma parte del estado acumulativo.
Línea 107	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 108	<code>[Key]</code> → marca la propiedad siguiente como clave primaria simple.
Línea 109	<code>public int Id { get; set; }</code> → declara la propiedad <code>Id</code> de tipo <code>int</code> usada por el modelo o la lógica de negocio.
Línea 110	<code>public int OrdenId { get; set; }</code> → declara la propiedad <code>OrdenId</code> de tipo <code>int</code> usada por el modelo o la lógica de negocio.
Línea 111	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 112	<code>[MaxLength(50)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 113	<code>public string NumeroCertificado { get; set; } = string.Empty;</code> → declara la propiedad <code>NumeroCertificado</code> de tipo <code>string</code> usada por el modelo o la lógica de negocio.
Línea 114	<code>public DateTime FechaEmision { get; set; }</code> → declara la propiedad <code>FechaEmision</code> de tipo <code>DateTime</code> usada por el modelo o la lógica de negocio.
Línea 115	<code>[Required]</code> → marca la propiedad siguiente como requerida.
Línea 116	<code>[MaxLength(100)]</code> → limita la longitud máxima persistida de la propiedad siguiente.
Línea 117	<code>public string OrganismoCertificador { get; set; } = string.Empty;</code> → declara la propiedad <code>OrganismoCertificador</code> de tipo <code>string</code> usada por el modelo o la lógica de negocio.
Línea 118	<code>[ForeignKey(nameof(OrdenId))]</code> → vincula la navegación siguiente con su clave foránea.
Línea 119	<code>public OrdenFabricacion Orden { get; set; } = null!</code> → declara la propiedad <code>Orden</code> de tipo <code>OrdenFabricacion</code> usada por el modelo o la lógica de negocio.
Línea 120	<code>}</code> → cierra el bloque de código actual.
Línea 122	<code>[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]</code> → declara una clave primaria compuesta.
Línea 123	<code>[Table("OrdenesAleaciones")]</code> → fija mediante Data Annotations la tabla relacional de la entidad siguiente.
Línea 124	<code>public class OrdenAleacion</code> → declara la clase <code>OrdenAleacion</code> que forma parte del estado acumulativo.
Línea 125	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 126	<code>public int OrdenFabricacionId { get; set; } → declara la propiedad OrdenFabricacionId de tipo int usada por el modelo o la lógica de negocio.</code>
Línea 127	<code>public int AleacionId { get; set; } → declara la propiedad AleacionId de tipo int usada por el modelo o la lógica de negocio.</code>
Línea 128	<code>public DateTime FechaAsignacion { get; set; } = DateTime.Now; → declara la propiedad FechaAsignacion de tipo DateTime usada por el modelo o la lógica de negocio.</code>
Línea 129	<code>[Precision(18, 3)] → configura precisión y escala para la propiedad numérica siguiente.</code>
Línea 130	<code>public decimal CantidadUtilizada { get; set; } → declara la propiedad CantidadUtilizada de tipo decimal usada por el modelo o la lógica de negocio.</code>
Línea 131	<code>[MaxLength(20)] → limita la longitud máxima persistida de la propiedad siguiente.</code>
Línea 132	<code>public string EstadoRelacion { get; set; } = "Activa"; → declara la propiedad EstadoRelacion de tipo string usada por el modelo o la lógica de negocio.</code>
Línea 133	<code>public OrdenFabricacion Orden { get; set; } = null!; → declara la propiedad Orden de tipo OrdenFabricacion usada por el modelo o la lógica de negocio.</code>
Línea 134	<code>public Aleacion Aleacion { get; set; } = null!; → declara la propiedad Aleacion de tipo Aleacion usada por el modelo o la lógica de negocio.</code>
Línea 135	<code>} → cierra el bloque de código actual.</code>
Línea 137	<code>public class AceriaDbContext : DbContext → declara la clase AceriaDbContext que forma parte del estado acumulativo.</code>
Línea 138	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 139	<code>public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { } → recibe las opciones del DbContext y las pasa a la clase base.</code>
Línea 141	<code>public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>(); → expone un conjunto de entidades para consultas y persistencia.</code>
Línea 142	<code>public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>(); → expone un conjunto de entidades para consultas y persistencia.</code>
Línea 143	<code>public DbSet<Aleacion> Aleaciones => Set<Aleacion>(); → expone un conjunto de entidades para consultas y persistencia.</code>
Línea 144	<code>public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>(); → expone un conjunto de entidades para consultas y persistencia.</code>
Línea 145	<code>public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>(); → expone un conjunto de entidades para consultas y persistencia.</code>
Línea 146	<code>public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>(); → expone un conjunto de entidades para consultas y persistencia.</code>
Línea 147	<code>public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>(); → expone un conjunto de entidades para consultas y persistencia.</code>
Línea 150	<code>protected override void OnModelCreating(ModelBuilder modelBuilder) → entra en la configuración explícita del modelo.</code>
Línea 151	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 152	<code>base.OnModelCreating(modelBuilder); → entra en la configuración explícita del modelo.</code>

Línea 153	<code>modelBuilder.Entity<OrdenFabricacion>(entity => → selecciona una entidad para configurarla con Fluent API.</code>
Línea 154	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 155	<code>entity.ToTable("OrdenesFabricacion"); → asigna explícitamente la tabla usada por la entidad.</code>
Línea 156	<code>entity.HasKey(o => o.Id); → define la clave primaria.</code>
Línea 157	<code>entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50); → selecciona una propiedad escalar para continuar su configuración.</code>
Línea 158	<code>entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200); → selecciona una propiedad escalar para continuar su configuración.</code>
Línea 159	<code>entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()"); → selecciona una propiedad escalar para continuar su configuración.</code>
Línea 160	<code>entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente"); → selecciona una propiedad escalar para continuar su configuración.</code>
Línea 161	<code>entity.Property(o => o.Observaciones).HasMaxLength(500); → selecciona una propiedad escalar para continuar su configuración.</code>
Línea 162	<code>entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden"); → define una clave alternativa única.</code>
Línea 163	<code>entity.HasIndex(o => o.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente"); → define un índice sobre las propiedades indicadas.</code>
Línea 164	<code>entity.HasIndex(o => new { o.Cliente, o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion"); → define un índice sobre las propiedades indicadas.</code>
Línea 165	<code>entity.HasIndex(o => o.FechaEntrega).HasFilter("[Estado] = 'Pendiente'").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes"); → define un índice sobre las propiedades indicadas.</code>
Línea 166	<code>entity.HasIndex(o => o.Estado).IncludeProperties(o => new { o.NumeroOrden, o.Cliente, o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye"); → define un índice sobre las propiedades indicadas.</code>
Línea 167	<code>entity.HasQueryFilter(o => !o.IsDeleted && o.Estado != "Cancelada"); → aplica un filtro global de consulta.</code>
Línea 168	<code>}); → cierra la lambda o configuración encadenada y finaliza la llamada.</code>
Línea 169	<code>modelBuilder.Entity<PlanchaAcero>(entity => → selecciona una entidad para configurarla con Fluent API.</code>
Línea 170	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 171	<code>entity.ToTable("PlanchasAcero", t => → asigna explícitamente la tabla usada por la entidad.</code>
Línea 172	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 173	<code>t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0"); → añade una restricción CHECK en SQL Server.</code>
Línea 174	<code>t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0"); → añade una restricción CHECK en SQL Server.</code>

Línea 175	<code>t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");</code> → añade una restricción CHECK en SQL Server.
Línea 176	<code>t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");</code> → añade una restricción CHECK en SQL Server.
Línea 177	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 178	<code>entity.HasKey(x => x.Id);</code> → define la clave primaria.
Línea 179	<code>entity.Property(x => x.Peso).HasPrecision(18, 3);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 180	<code>entity.Property(x => x.Activa).HasDefaultValue(true);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 181	<code>entity.HasOne(x => x.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 182	<code>.WithMany(o => o.Planchas)</code> → establece el extremo de colección de una relación uno-a-muchos.
Línea 183	<code>.HasForeignKey(x => x.OrdenId)</code> → declara explícitamente la clave foránea.
Línea 184	<code>.OnDelete(DeleteBehavior.Cascade)</code> → define el comportamiento de eliminación de la relación.
Línea 185	<code>.IsRequired();</code> → marca la propiedad o relación como obligatoria.
Línea 186	<code>entity.HasIndex(x => new { x.OrdenId, x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");</code> → define un índice sobre las propiedades indicadas.
Línea 187	<code>entity.HasIndex(x => x.Espesor).HasFilter("[Activa] = 1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");</code> → define un índice sobre las propiedades indicadas.
Línea 188	<code>entity.HasQueryFilter(x => !x.IsDeleted && x.Activa);</code> → aplica un filtro global de consulta.
Línea 189	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 190	<code>modelBuilder.Entity<Aleacion>(entity =></code> → selecciona una entidad para configurarla con Fluent API.
Línea 191	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 192	<code>entity.ToTable("Aleaciones", t =></code> → asigna explícitamente la tabla usada por la entidad.
Línea 193	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 194	<code>t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND [PorcentajeCarbono] <= 2");</code> → añade una restricción CHECK en SQL Server.
Línea 195	<code>t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND [PorcentajeManganeso] <= 5");</code> → añade una restricción CHECK en SQL Server.
Línea 196	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 197	<code>entity.HasKey(a => a.Id);</code> → define la clave primaria.
Línea 198	<code>entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 199	<code>entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);</code> → selecciona una propiedad escalar para continuar su configuración.

Línea 200	<code>entity.Property(a => a.Descripcion).HasMaxLength(500);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 201	<code>entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");</code> → define una clave alternativa única.
Línea 202	<code>entity.HasIndex(a => a.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");</code> → define un índice sobre las propiedades indicadas.
Línea 203	<code>entity.HasIndex(a => new { a.PorcentajeCarbono, a.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");</code> → define un índice sobre las propiedades indicadas.
Línea 204	<code>entity.HasQueryFilter(a => !a.IsDeleted);</code> → aplica un filtro global de consulta.
Línea 205	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 206	<code>modelBuilder.Entity<EstadoOrden>(entity =></code> → selecciona una entidad para configurarla con Fluent API.
Línea 207	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 208	<code>entity.ToTable("EstadosOrden");</code> → asigna explícitamente la tabla usada por la entidad.
Línea 209	<code>entity.HasKey(e => e.Id);</code> → define la clave primaria.
Línea 210	<code>entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 211	<code>entity.Property(e => e.Descripcion).HasMaxLength(250);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 212	<code>entity.Property(e => e.Activo).HasDefaultValue(true);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 213	<code>entity.HasQueryFilter(e => !e.IsDeleted && e.Activo);</code> → aplica un filtro global de consulta.
Línea 214	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 215	<code>modelBuilder.Entity<DetalleOrden>(entity =></code> → selecciona una entidad para configurarla con Fluent API.
Línea 216	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 217	<code>entity.ToTable("DetallesOrden");</code> → asigna explícitamente la tabla usada por la entidad.
Línea 218	<code>entity.HasKey(d => d.Id);</code> → define la clave primaria.
Línea 219	<code>entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 220	<code>entity.Property(d => d.Notas).HasMaxLength(500);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 221	<code>entity.HasOne(d => d.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 222	<code>.WithOne(o => o.Detalle)</code> → establece el extremo de referencia de una relación uno-a-uno.
Línea 223	<code>.HasForeignKey<DetalleOrden>(d => d.OrdenId)</code> → declara explícitamente la clave foránea.
Línea 224	<code>.OnDelete(DeleteBehavior.Cascade)</code> → define el comportamiento de eliminación de la relación.

Línea 225	<code>.IsRequired();</code> → marca la propiedad o relación como obligatoria.
Línea 226	<code>entity.HasQueryFilter(d => !d.Orden.IsDeleted && d.Orden.Estado != "Cancelada");</code> → aplica un filtro global de consulta.
Línea 227	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 229	<code>modelBuilder.Entity<CertificadoCalidad>(entity =></code> → selecciona una entidad para configurarla con Fluent API.
Línea 230	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 231	<code>entity.ToTable("CertificadosCalidad");</code> → asigna explícitamente la tabla usada por la entidad.
Línea 232	<code>entity.HasKey(c => c.Id);</code> → define la clave primaria.
Línea 233	<code>entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 234	<code>entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 235	<code>entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");</code> → define una clave alternativa única.
Línea 236	<code>entity.HasIndex(c => c.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");</code> → define un índice sobre las propiedades indicadas.
Línea 237	<code>entity.HasOne(c => c.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 238	<code>.WithOne(o => o.Certificado)</code> → establece el extremo de referencia de una relación uno-a-uno.
Línea 239	<code>.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)</code> → declara explícitamente la clave foránea.
Línea 240	<code>.OnDelete(DeleteBehavior.Cascade)</code> → define el comportamiento de eliminación de la relación.
Línea 241	<code>.IsRequired();</code> → marca la propiedad o relación como obligatoria.
Línea 242	<code>entity.HasQueryFilter(c => !c.Orden.IsDeleted && c.Orden.Estado != "Cancelada");</code> → aplica un filtro global de consulta.
Línea 243	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 244	<code>modelBuilder.Entity<OrdenAleacion>(entity =></code> → selecciona una entidad para configurarla con Fluent API.
Línea 245	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 246	<code>entity.ToTable("OrdenesAleaciones");</code> → asigna explícitamente la tabla usada por la entidad.
Línea 247	<code>entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });</code> → define la clave primaria.
Línea 248	<code>entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 249	<code>entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);</code> → selecciona una propiedad escalar para continuar su configuración.

Línea 250	<code>entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");</code> → selecciona una propiedad escalar para continuar su configuración.
Línea 251	<code>entity.HasOne(x => x.Orden)</code> → inicia la configuración de una navegación de referencia.
Línea 252	<code>.WithMany(o => o.OrdenesAleaciones)</code> → establece el extremo de colección de una relación uno-a-muchos.
Línea 253	<code>.HasForeignKey(x => x.OrdenFabricacionId)</code> → declara explícitamente la clave foránea.
Línea 254	<code>.OnDelete(DeleteBehavior.Cascade);</code> → define el comportamiento de eliminación de la relación.
Línea 255	<code>entity.HasOne(x => x.Aleacion)</code> → inicia la configuración de una navegación de referencia.
Línea 256	<code>.WithMany(a => a.OrdenesAleaciones)</code> → establece el extremo de colección de una relación uno-a-muchos.
Línea 257	<code>.HasForeignKey(x => x.AleacionId)</code> → declara explícitamente la clave foránea.
Línea 258	<code>.OnDelete(DeleteBehavior.Restrict);</code> → define el comportamiento de eliminación de la relación.
Línea 259	<code>entity.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");</code> → define un índice sobre las propiedades indicadas.
Línea 260	<code>entity.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] = 'Activa'").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");</code> → define un índice sobre las propiedades indicadas.
Línea 261	<code>entity.HasQueryFilter(x => x.EstadoRelacion == "Activa" && !x.Orden.IsDeleted && !x.Aleacion.IsDeleted);</code> → aplica un filtro global de consulta.
Línea 262	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 263	<code>}</code> → cierra el bloque de código actual.
Línea 264	<code>}</code> → cierra el bloque de código actual.
Línea 266	<code>public interface IOrdenRepositorio</code> → declara el contrato IOrdenRepositorio usado por la aplicación.
Línea 267	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 268	<code>List<OrdenFabricacion> ObtenerTodas();</code> → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.
Línea 269	<code>OrdenFabricacion? ObtenerPorId(int id);</code> → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.
Línea 270	<code>OrdenFabricacion? ObtenerPorNumero(string numeroOrden);</code> → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.
Línea 271	<code>void Agregar(OrdenFabricacion orden);</code> → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.
Línea 272	<code>void Eliminar(OrdenFabricacion orden);</code> → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.
Línea 273	<code>void EliminarLogicamente(int id);</code> → declara en el contrato del repositorio la operación de Soft Delete por identificador.

Línea 274	<code>void Restaurar(int id);</code> → declara en el contrato del repositorio la operación que revierte un Soft Delete.
Línea 276	<code>int Guardar();</code> → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.
Línea 277	<code>}</code> → cierra el bloque de código actual.
Línea 279	<code>public sealed class OrdenRepositorio : IOrdenRepositorio</code> → declara la clase OrdenRepositorio que forma parte del estado acumulativo.
Línea 280	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 281	<code>private readonly AceriaDbContext _context;</code> → declara el campo privado _context de tipo AceriaDbContext.
Línea 282	<code>public OrdenRepositorio(AceriaDbContext context) => _context = context;</code> → inyecta AceriaDbContext en el repositorio y lo conserva como dependencia de persistencia.
Línea 284	<code>public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();</code> → ordena el resultado antes de materializarlo.
Línea 285	<code>public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);</code> → implementa la búsqueda por clave primaria mediante Find, aprovechando primero el ChangeTracker y después la base de datos.
Línea 286	<code>public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);</code> → recupera la primera coincidencia o null.
Línea 287	<code>public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);</code> → registra una entidad nueva para insertarla al guardar.
Línea 288	<code>public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);</code> → marca una entidad para borrado al guardar.
Línea 289	<code>public void EliminarLogicamente(int id)</code> → implementa la operación que localiza una orden y cambia sus campos de borrado lógico.
Línea 290	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 291	<code>var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == id);</code> → recupera la primera coincidencia o null.
Línea 292	<code>if (orden is null) return;</code> → evalúa una condición antes de ejecutar el bloque asociado.
Línea 293	<code>orden.IsDeleted = true;</code> → marca la orden como eliminada lógicamente sin borrar su fila física.
Línea 294	<code>orden.DeletedAt = DateTime.UtcNow;</code> → registra en UTC el instante en que se realizó el borrado lógico.
Línea 295	<code>}</code> → cierra el bloque de código actual.
Línea 297	<code>public void Restaurar(int id)</code> → implementa la restauración cargando también entidades ocultas por el filtro global.
Línea 298	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 299	<code>var orden = _context.OrdenesFabricacion.IgnoreQueryFilters().FirstOrDefault(o => o.Id == id);</code> → omite de forma explícita los filtros globales para esta consulta.
Línea 300	<code>if (orden is null) return;</code> → evalúa una condición antes de ejecutar el bloque asociado.

Línea 301	<code>orden.IsDeleted = false;</code> → desactiva la marca de borrado lógico para volver a hacer visible la orden.
Línea 302	<code>orden.DeletedAt = null;</code> → elimina la fecha de borrado porque la orden vuelve al estado activo.
Línea 303	<code>}</code> → cierra el bloque de código actual.
Línea 305	<code>public int Guardar() => _context.SaveChanges();</code> → confirma en SQL Server los cambios del <code>ChangeTracker</code> .
Línea 306	<code>}</code> → cierra el bloque de código actual.
Línea 308	<code>public interface IServicioOrdenes</code> → declara el contrato <code>IServicioOrdenes</code> usado por la aplicación.
Línea 309	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 310	<code>string ObtenerResumen(int ordenId);</code> → declara el servicio que construye un resumen de una orden a partir de su identificador.
Línea 311	<code>}</code> → cierra el bloque de código actual.
Línea 313	<code>public sealed class ServicioOrdenes : IServicioOrdenes</code> → declara la clase <code>ServicioOrdenes</code> que forma parte del estado acumulativo.
Línea 314	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 315	<code>private readonly IOrdenRepositorio _repositorio;</code> → declara el campo privado <code>_repositorio</code> de tipo <code>IOrdenRepositorio</code> .
Línea 316	<code>private readonly AceriaDbContext _context;</code> → declara el campo privado <code>_context</code> de tipo <code>AceriaDbContext</code> .
Línea 317	<code>public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)</code> → declara el constructor que recibe por inyección el repositorio de órdenes y el <code>DbContext</code> .
Línea 318	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 319	<code>_repositorio = repositorio;</code> → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.
Línea 320	<code>_context = context;</code> → guarda el <code>DbContext</code> inyectado para consultar los datos necesarios durante el servicio.
Línea 321	<code>}</code> → cierra el bloque de código actual.
Línea 323	<code>public string ObtenerResumen(int ordenId)</code> → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.
Línea 324	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 325	<code>var orden = _repositorio.ObtenerPorId(ordenId);</code> → declara una variable local y almacena el resultado de la expresión.
Línea 326	<code>if (orden is null) return \$"Orden {ordenId} no encontrada";</code> → evalúa una condición antes de ejecutar el bloque asociado.
Línea 327	<code>var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);</code> → cuenta las filas que cumplen la consulta.
Línea 328	<code>return \$"Orden {orden.NumeroOrden} Cliente: {orden.Cliente} Planchas: {totalPlanchas}";</code> → devuelve el resultado al código llamador.

Línea 329	<code>}</code> → cierra el bloque de código actual.
Línea 330	<code>}</code> → cierra el bloque de código actual.
Línea 332	<code>public static class Program</code> → declara la clase Program que forma parte del estado acumulativo.
Línea 333	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 334	<code>public static void Main()</code> → declara el punto de entrada ejecutable de la aplicación de consola.
Línea 335	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 336	<code>var configuration = new ConfigurationBuilder()</code> → inicia la construcción de la configuración de la aplicación.
Línea 337	<code>.SetBasePath(AppContext.BaseDirectory)</code> → establece la ruta base para los archivos de configuración.
Línea 338	<code>.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)</code> → incorpora appsettings.json.
Línea 339	<code>.AddEnvironmentVariables()</code> → incorpora variables de entorno.
Línea 340	<code>.Build();</code> → materializa el objeto construido por la cadena anterior.
Línea 342	<code>var connectionString = configuration.GetConnectionString("AceriaDB")</code> → obtiene la cadena AceriaDB desde configuración.
Línea 343	<code>?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");</code> → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.
Línea 345	<code>var services = new ServiceCollection();</code> → crea la colección de servicios.
Línea 346	<code>services.AddDbContext<AceriaDbContext>(options =></code> → registra el DbContext en inyección de dependencias.
Línea 347	<code>options</code> → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.
Línea 348	<code>.UseSqlServer(connectionString, sql =></code> → selecciona SQL Server como proveedor de EF Core.
Línea 349	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 350	<code>sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);</code> → habilita reintentos ante fallos transitorios.
Línea 351	<code>sql.CommandTimeout(60);</code> → establece el timeout de los comandos SQL.
Línea 352	<code>})</code> → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.
Línea 353	<code>.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)</code> → configura el logging de EF Core.
Línea 354	<code>.EnableDetailedErrors();</code> → activa errores detallados.
Línea 356	<code>services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();</code> → registra un servicio con ciclo de vida scoped.

Línea 357	<code>services.AddScoped<IServicioOrdenes, ServicioOrdenes>();</code> → registra un servicio con ciclo de vida scoped.
Línea 359	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions</code> → construye el <code>ServiceProvider</code> con validación de servicios y garantiza su liberación al salir del ámbito mediante <code>using var</code> .
Línea 360	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 361	<code>ValidateScopes = true,</code> → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.
Línea 362	<code>ValidateOnBuild = true</code> → obliga al contenedor a validar las resoluciones registradas al construir el <code>ServiceProvider</code> .
Línea 363	<code>});</code> → cierra la lambda o configuración encadenada y finaliza la llamada.
Línea 365	<code>using var scope = provider.CreateScope();</code> → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.
Línea 366	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → resuelve un servicio obligatorio del contenedor.
Línea 367	<code>context.Database.EnsureDeleted();</code> → reinicia la base del laboratorio para una ejecución reproducible.
Línea 368	<code>context.Database.Migrate();</code> → aplica el historial de migraciones pendiente.
Línea 370	<code>var orden = new OrdenFabricacion</code> → declara una variable local y almacena el resultado de la expresión.
Línea 371	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 372	<code>NumeroOrden = "0F-M2-0001",</code> → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.
Línea 373	<code>Cliente = "Constructora del Norte",</code> → asigna el cliente de la orden utilizada en el escenario E2E del punto.
Línea 374	<code>FechaCreacion = DateTime.UtcNow,</code> → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.
Línea 375	<code>FechaEntrega = DateTime.Today.AddDays(14),</code> → establece una fecha de entrega prevista catorce días después de la fecha local actual.
Línea 376	<code>Estado = "Pendiente",</code> → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.
Línea 377	<code>Observaciones = "Orden de validación M2",</code> → añade texto opcional para comprobar el mapeo de Observaciones.
Línea 378	<code>IsDeleted = false,</code> → inicializa explícitamente la orden del escenario como no eliminada.
Línea 379	<code>Planchas =</code> → inicia el inicializador de la colección de planchas asociadas a la orden.
Línea 380	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 381	<code>new PlanchaAcero</code> → crea una nueva plancha dentro del inicializador de la colección <code>Planchas</code> de la orden.
Línea 382	<code>{</code> → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 383	<code>Espesor = 10.5,</code> → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.

Línea 384	Ancho = 1500, → asigna el ancho de la plancha usada en el escenario de persistencia.
Línea 385	Largo = 3000, → asigna el largo de la plancha usada en el escenario de persistencia.
Línea 386	Peso = 371.250m, → asigna un peso decimal con tres posiciones para validar la precisión configurada.
Línea 387	Activa = true, → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.
Línea 388	} → cierra el bloque de código actual.
Línea 389	}, → continúa el inicializador o la llamada del bloque actual.
Línea 390	Detalle = new DetalleOrden → crea el detalle uno-a-uno que se persistirá junto con la orden principal.
Línea 391	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 392	ComposicionQuimica = "C: 0.45%, Mn: 0.75%", → asigna la composición química utilizada para verificar la entidad DetalleOrden.
Línea 393	TemperaturaColada = 1550.5, → asigna la temperatura de colada del detalle de fabricación.
Línea 394	Notas = "Colada principal" → asigna las notas opcionales del detalle asociado a la orden.
Línea 395	}, → continúa el inicializador o la llamada del bloque actual.
Línea 396	Certificado = new CertificadoCalidad → crea el certificado uno-a-uno asociado a la orden del escenario.
Línea 397	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 398	NumeroCertificado = "CERT-0001", → asigna el identificador natural del certificado de calidad.
Línea 399	FechaEmision = DateTime.Today, → asigna la fecha de emisión del certificado usado en la prueba.
Línea 400	OrganismoCertificador = "Laboratorio Aceria" → asigna el organismo que emite el certificado de calidad.
Línea 401	}, → continúa el inicializador o la llamada del bloque actual.
Línea 402	}; → cierra el inicializador del objeto y finaliza la instrucción.
Línea 404	var aleacion = new Aleacion → declara una variable local y almacena el resultado de la expresión.
Línea 405	{ → abre el bloque de código asociado a la declaración o instrucción anterior.
Línea 406	Nombre = "AISI 1045", → asigna el nombre comercial de la aleación utilizada en el escenario.
Línea 407	Codigo = "A1045", → asigna el código natural único de la aleación.
Línea 408	PorcentajeCarbono = 0.45, → asigna el porcentaje de carbono dentro del rango permitido por la restricción del modelo.
Línea 409	PorcentajeManganeso = 0.75, → asigna el porcentaje de manganeso dentro del rango permitido por la restricción del modelo.
Línea 410	Descripcion = "Acero medio en carbono" → asigna una descripción opcional a la aleación.
Línea 411	}; → cierra el inicializador del objeto y finaliza la instrucción.

Línea 412	<code>orden.OrdenesAleaciones.Add(new OrdenAleacion → registra una entidad nueva para insertarla al guardar.</code>
Línea 413	<code>{ → abre el bloque de código asociado a la declaración o instrucción anterior.</code>
Línea 414	<code>Aleacion = aleacion, → conecta la entidad intermedia OrdenAleacion con la aleación creada para el escenario.</code>
Línea 415	<code>CantidadUtilizada = 1500.500m, → registra la cantidad utilizada con tres decimales, coherente con HasPrecision(18, 3).</code>
Línea 416	<code>EstadoRelacion = "Activa" → marca como activa la relación entre la orden y la aleación.</code>
Línea 417	<code>}); → cierra la lambda o configuración encadenada y finaliza la llamada.</code>
Línea 418	<code>context.OrdenesFabricacion.Add(orden); → registra una entidad nueva para insertarla al guardar.</code>
Línea 419	<code>context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true, IsDeleted = false }); → registra una entidad nueva para insertarla al guardar.</code>
Línea 420	<code>context.SaveChanges(); → confirma en SQL Server los cambios del ChangeTracker.</code>
Línea 422	<code>var id = orden.Id; → declara una variable local y almacena el resultado de la expresión.</code>
Línea 423	<code>orden.IsDeleted = true; → marca la orden como eliminada lógicamente sin borrar su fila física.</code>
Línea 424	<code>orden.DeletedAt = DateTime.UtcNow; → registra en UTC el instante en que se realizó el borrado lógico.</code>
Línea 425	<code>context.SaveChanges(); → confirma en SQL Server los cambios del ChangeTracker.</code>
Línea 426	<code>var visibles = context.OrdenesFabricacion.Count(); → cuenta las filas que cumplen la consulta.</code>
Línea 427	<code>var todas = context.OrdenesFabricacion.IgnoreQueryFilters().Count(); → omite de forma explícita los filtros globales para esta consulta.</code>
Línea 428	<code>var restaurable = context.OrdenesFabricacion.IgnoreQueryFilters().Single(o => o.Id == id); → omite de forma explícita los filtros globales para esta consulta.</code>
Línea 429	<code>restaurable.IsDeleted = false; → restaura la entidad cargada ignorando filtros al desactivar su marca IsDeleted.</code>
Línea 430	<code>restaurable.DeletedAt = null; → limpia la fecha de eliminación de la entidad restaurada.</code>
Línea 431	<code>context.SaveChanges(); → confirma en SQL Server los cambios del ChangeTracker.</code>
Línea 432	<code>global::System.Console.WriteLine(\$"2.11 OK Tras borrar visibles: {visibles} Totales: {todas} Restaurada: {context.OrdenesFabricacion.Any(o => o.Id == id)}"); → escribe la evidencia de ejecución usada para validar este estado.</code>
Línea 433	<code>} → cierra el bloque de código actual.</code>
Línea 434	<code>} → cierra el bloque de código actual.</code>

Punto 2.12 - Clean Architecture y Arquitectura Hexagonal

Audiencia: Desarrolladores que ya han completado el modelado acumulativo 2.1-2.11.

Proyecto: 2.12 parte de 2.11 y refactoriza el mismo sistema en Domain, Application, Infrastructure y Console. El refactor cambia la organización del código, no el dominio ni el objetivo de persistencia.

Objetivos de aprendizaje

- Separar dominio y persistencia.
- Mantener EF Core fuera de Domain y Application.
- Definir puertos de repositorio y unidad de trabajo.
- Implementar adaptadores en Infrastructure.
- Registrar dependencias desde una extensión de IServiceCollection.
- Conservar el historial de migraciones y el comportamiento del modelo.

Paso 1: Abrir la solución multiproyecto

```
cd M02/PROYECTO/2.12
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

La solución contiene cuatro proyectos:

```
src/
├─ AceriaData.Domain
├─ AceriaData.Application
├─ AceriaData.Infrastructure
└─ AceriaData.Console
```

TEXT

Paso 2: Mantener el dominio independiente de EF Core

```
namespace AceriaData.Domain.Entities;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public stringCodigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

public class EstadoOrden
{

```

CSHARP

```

public int Id { get; set; }
public string Nombre { get; set; } = string.Empty;
public string Descripcion { get; set; } = string.Empty;
public bool Activo { get; set; } = true;
public bool IsDeleted { get; set; }
public DateTime? DeletedAt { get; set; }
}

public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    public string? Notas { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class CertificadoCalidad
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    public string OrganismoCertificador { get; set; } = string.Empty;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    public decimal CantidadUtilizada { get; set; }
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

```

Las entidades están en Domain y no necesitan DbContext, DbSet, IEntityTypeConfiguration ni referencias a paquetes de EF Core.

Paso 3: Definir los puertos de aplicación

```

using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    List<OrdenFabricacion> ObtenerTodas();
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}

public interface IUnidadDeTrabajo : IDisposable
{
    IOrdenRepositorio Ordenes { get; }
    int Guardar();
}

```

CSHARP

Línea 1 IOrdenRepositorio → expresa operaciones que necesita la aplicación sin exponer EF Core.

Línea 2 IUnidadDeTrabajo → coordina repositorios y el guardado como abstracción de aplicación.

Paso 4: Crear un caso de uso

```

using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.UseCases;

public sealed class CrearOrdenUseCase

```

CSHARP

```

{
    private readonly IUnidadDeTrabajo _unidad;
    public CrearOrdenUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public int Ejecutar(string numeroOrden, string cliente)
    {
        _unidad.Ordenes.Agregar(new OrdenFabricacion
        {
            NumeroOrden = numeroOrden,
            Cliente = cliente,
            FechaCreacion = DateTime.UtcNow,
            Estado = "Pendiente"
        });
        return _unidad.Guardar();
    }
}

```

El caso de uso depende de interfaces de Application y de entidades de Domain. No referencia Microsoft.EntityFrameworkCore.

Paso 5: Colocar EF Core y toda la configuración acumulada en Infrastructure

El `AceriaDbContext` real de 2.12 contiene los `DbSet` y descubre las configuraciones de Infrastructure mediante `ApplyConfigurationsFromAssembly`:

```

using AceriaData.Domain.Entities;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Persistence;

public sealed class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(AceriaDbContext).Assembly);
    }
}

```

CSHARP

Línea 1	<code>AceriaDbContext : DbContext</code> → mantiene EF Core exclusivamente en Infrastructure.
Línea 2	<code>DbSet<...></code> → expone las siete entidades acumuladas del modelo 2.1-2.11.
Línea 3	<code>OnModelCreating</code> → centraliza la construcción del modelo relacional.
Línea 4	<code>ApplyConfigurationsFromAssembly(typeof(AceriaDbContext).Assembly)</code> → descubre automáticamente todas las clases <code>IEntityTypeConfiguration<T></code> del ensamblado.
Línea 5	<code>base.OnModelCreating(modelBuilder)</code> → conserva el comportamiento base antes de aplicar la configuración del dominio.

Paso 5.1: Conservar las configuraciones Fluent API acumuladas

Mover el `DbContext` no basta: para que 2.12 represente realmente el mismo modelo que 2.11 hay que conservar relaciones, claves alternativas, índices, restricciones, valores por defecto y filtros globales. Estos son los archivos reales del estado 2.12.

OrdenFabricacionConfiguration.cs

```

using AceriaData.Domain.Entities;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;

```

CSHARP

```
namespace AceriaData.Infrastructure.Persistence.Configurations;

public sealed class OrdenFabricacionConfiguration : IEntityTypeConfiguration<OrdenFabricacion>
{
    public void Configure(EntityTypeBuilder<OrdenFabricacion> b)
    {
        b.ToTable("OrdenesFabricacion");
        b.HasKey(x => x.Id);
        b.HasAlternateKey(x => x.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");
        b.Property(x => x.NumeroOrden).IsRequired().HasMaxLength(50);
        b.Property(x => x.Cliente).IsRequired().HasMaxLength(200);
        b.Property(x => x.FechaCreacion).HasDefaultValueSql("GETDATE()");
        b.Property(x => x.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
        b.Property(x => x.Observaciones).HasMaxLength(500);
        b.HasQueryFilter(x => !x.IsDeleted && x.Estado != "Cancelada");
        b.HasIndex(x => x.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");
        b.HasIndex(x => new { x.Cliente,
            x.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");
        b.HasIndex(x => x.FechaEntrega).HasFilter("[Estado] = 'Pendiente'").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");
        b.HasIndex(x => x.Estado).IncludeProperties(x => new { x.NumeroOrden, x.Cliente,
            x.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");
    }
}
```

PlanchaAceroConfiguration.cs

```
using AceriaData.Domain.Entities;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;

namespace AceriaData.Infrastructure.Persistence.Configurations;

public sealed class PlanchaAceroConfiguration : IEntityTypeConfiguration<PlanchaAcero>
{
    public void Configure(EntityTypeBuilder<PlanchaAcero> b)
    {
        b.ToTable("PlanchasAcero", t =>
        {
            t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");
            t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");
            t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");
            t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");
        });
        b.HasKey(x => x.Id);
        b.Property(x => x.Peso).HasPrecision(18, 3);
        b.Property(x => x.Activa).HasDefaultValue(true);
        b.HasOne(x => x.Orden).WithMany(x => x.Planchas).HasForeignKey(x =>
            x.OrdenId).OnDelete(DeleteBehavior.Cascade).IsRequired();
        b.HasIndex(x => new { x.OrdenId, x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");
        b.HasIndex(x => x.Espesor).HasFilter("[Activa] = 1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");
        b.HasQueryFilter(x => !x.IsDeleted && x.Activa);
    }
}
```

ModeloConfiguration.cs

```
using AceriaData.Domain.Entities;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;

namespace AceriaData.Infrastructure.Persistence.Configurations;

public sealed class AleacionConfiguration : IEntityTypeConfiguration<Aleacion>
{
    public void Configure(EntityTypeBuilder<Aleacion> b)
    {
        b.ToTable("Aleaciones", t =>
        {
            t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono",
                "[PorcentajeCarbono] >= 0 AND [PorcentajeCarbono] <= 2");
            t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND
                [PorcentajeManganeso] <= 5");
        });
        b.HasKey(x => x.Id);
        b.HasAlternateKey(x => x.Codigo).HasName("AK_Aleaciones_Codigo");
        b.Property(x => x.Nombre).IsRequired().HasMaxLength(100);
        b.Property(x => x.Codigo).IsRequired().HasMaxLength(20);
        b.Property(x => x.Descripcion).HasMaxLength(500);
    }
}
```

```

        b.HasIndex(x => x.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");
        b.HasIndex(x => new { x.PorcentajeCarbono,
x.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");
        b.HasQueryFilter(x => !x.IsDeleted);
    }
}

public sealed class EstadoOrdenConfiguration : IEntityTypeConfiguration<EstadoOrden>
{
    public void Configure(EntityTypeBuilder<EstadoOrden> b)
    {
        b.ToTable("EstadosOrden");
        b.HasKey(x => x.Id);
        b.Property(x => x.Nombre).IsRequired().HasMaxLength(50);
        b.Property(x => x.Descripcion).HasMaxLength(250);
        b.Property(x => x.Activo).HasDefaultValue(true);
        b.HasQueryFilter(x => !x.IsDeleted && x.Activo);
    }
}

public sealed class DetalleOrdenConfiguration : IEntityTypeConfiguration<DetalleOrden>
{
    public void Configure(EntityTypeBuilder<DetalleOrden> b)
    {
        b.ToTable("DetallesOrden");
        b.HasKey(x => x.Id);
        b.Property(x => x.ComposicionQuimica).IsRequired().HasMaxLength(200);
        b.Property(x => x.Notas).HasMaxLength(500);
        b.HasOne(x => x.Orden).WithOne(x => x.Detalle).HasForeignKey<DetalleOrden>(x =>
x.OrdenId).OnDelete(DeleteBehavior.Cascade).IsRequired();
        b.HasQueryFilter(x => !x.Orden.IsDeleted && x.Orden.Estado != "Cancelada");
    }
}

public sealed class CertificadoCalidadConfiguration : IEntityTypeConfiguration<CertificadoCalidad>
{
    public void Configure(EntityTypeBuilder<CertificadoCalidad> b)
    {
        b.ToTable("CertificadosCalidad");
        b.HasKey(x => x.Id);
        b.HasAlternateKey(x => x.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");
        b.Property(x => x.NumeroCertificado).IsRequired().HasMaxLength(50);
        b.Property(x => x.OrganismoCertificador).IsRequired().HasMaxLength(100);
        b.HasIndex(x => x.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");
        b.HasOne(x => x.Orden).WithOne(x => x.Certificado).HasForeignKey<CertificadoCalidad>(x =>
x.OrdenId).OnDelete(DeleteBehavior.Cascade).IsRequired();
        b.HasQueryFilter(x => !x.Orden.IsDeleted && x.Orden.Estado != "Cancelada");
    }
}

public sealed class OrdenAleacionConfiguration : IEntityTypeConfiguration<OrdenAleacion>
{
    public void Configure(EntityTypeBuilder<OrdenAleacion> b)
    {
        b.ToTable("OrdenesAleaciones");
        b.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        b.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
        b.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
        b.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
        b.HasOne(x => x.Orden).WithMany(x => x.OrdenesAleaciones).HasForeignKey(x =>
x.OrdenFabricacionId).OnDelete(DeleteBehavior.Cascade);
        b.HasOne(x => x.Aleacion).WithMany(x => x.OrdenesAleaciones).HasForeignKey(x =>
x.AleacionId).OnDelete(DeleteBehavior.Restrict);
        b.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");
        b.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] =
'Activa'").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");
        b.HasQueryFilter(x => x.EstadoRelacion == "Activa" && !x.Orden.IsDeleted && !x.Aleacion.IsDeleted);
    }
}

```

Línea 1	HasAlternateKey → conserva los identificadores naturales configurados en 2.8.
Línea 2	HasIndex / HasFilter / IncludeProperties → conserva los índices simples, compuestos, filtrados y con columnas incluidas introducidos en 2.9.
Línea 3	HasCheckConstraint → mantiene las reglas de integridad que deben cumplirse también fuera de la aplicación.
Línea 4	HasQueryFilter → conserva los filtros globales y el Soft Delete de 2.10-2.11.

Línea 5

HasOne / WithMany / WithOne → conserva las relaciones uno-a-muchos, uno-a-uno y muchos-a-muchos acumuladas.

Paso 6: Implementar y registrar los adaptadores de Infrastructure

Antes de registrar los puertos hay que implementar sus adaptadores. El archivo real `Repositories.cs` contiene tanto `OrdenRepositorio` como `UnidadDeTrabajo`:

CSHARP

```
using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(x =>
x.NumeroOrden == numeroOrden);
    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(x => x.Id).ToList();
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
}

public sealed class UnidadDeTrabajo : IUnidadDeTrabajo
{
    private readonly AceriaDbContext _context;
    private IOrdenRepositorio? _ordenes;
    public UnidadDeTrabajo(AceriaDbContext context) => _context = context;
    public IOrdenRepositorio Ordenes => _ordenes ??= new OrdenRepositorio(_context);
    public int Guardar() => _context.SaveChanges();
    public void Dispose() => _context.Dispose();
}
```

Línea 1

`OrdenRepositorio : IOrdenRepositorio` → implementa en Infrastructure el puerto definido por Application.

Línea 2

`ObtenerPorNumero` → traduce una operación de aplicación a una consulta de EF Core.

Línea 3

`Agregar / Eliminar` → modifican el estado del DbContext sin llamar por sí mismos a `SaveChanges`.

Línea 4

`UnidadDeTrabajo : IUnidadDeTrabajo` → coordina el repositorio y el DbContext compartido.

Línea 5

`Guardar() => _context.SaveChanges()` → concentra la confirmación de cambios en la unidad de trabajo.

Después se registran SQL Server y los adaptadores con la extensión real `AddAceriaInfrastructure`:

CSHARP

```
using AceriaData.Application.Interfaces;
using AceriaData.Infrastructure.Persistence;
using AceriaData.Infrastructure.Repositories;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;

namespace AceriaData.Infrastructure;

public static class DependencyInjection
{
    public static IServiceCollection AddAceriaInfrastructure(this IServiceCollection services, string
connectionString)
    {
        services.AddDbContext<AceriaDbContext>(o => o.UseSqlServer(connectionString));
        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>();
        return services;
    }
}
```

Línea 1	AddDbContext<AceriaDbContext> → registra el contexto con el proveedor de SQL Server.
Línea 2	AddScoped<IOrdenRepositorio, OrdenRepositorio>() → enlaza el puerto de Application con su adaptador de Infrastructure.
Línea 3	AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>() → registra la unidad de trabajo con el mismo ciclo de vida scoped.

Paso 7: Ejecutar desde la capa de entrada

CSHARP

```
using AceriaData.Application.UseCases;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var cs = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(cs);
services.AddScoped<CrearOrdenUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true,
    ValidateScopes = true });
using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var crear = scope.ServiceProvider.GetRequiredService<CrearOrdenUseCase>();
crear.Ejecutar("OF-M2-HEX-0001", "Cliente Arquitectura");
var orden = context.OrdenesFabricacion.Single();
Console.WriteLine($"2.12 OK | {orden.NumeroOrden} | {orden.Cliente}");
```

La consola construye el contenedor, resuelve el caso de uso y ejecuta la aplicación. El resultado esperado contiene 2.12 OK.

Paso 8: Validar migraciones desde la nueva ubicación

Para que las herramientas de EF Core puedan crear el contexto de forma reproducible en tiempo de diseño, Infrastructure contiene la factory real:

CSHARP

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Design;

namespace AceriaData.Infrastructure.Persistence;

public sealed class AceriaDesignTimeDbContextFactory : IDesignTimeDbContextFactory<AceriaDbContext>
{
    public AceriaDbContext CreateDbContext(string[] args)
    {
        var options = new DbContextOptionsBuilder<AceriaDbContext>()
            .UseSqlServer("Server=(localdb)\
\MSSQLLocalDB;Database=AceriaDB;Trusted_Connection=True;TrustServerCertificate=True;")
            .Options;
        return new AceriaDbContext(options);
    }
}
```

Línea 1	IDesignTimeDbContextFactory<AceriaDbContext> → ofrece a dotnet ef una forma explícita de construir el contexto.
Línea 2	UseSqlServer(...) → usa la misma instancia LocalDB y la misma base AceriaDB del curso.

Línea 3 `return new AceriaDbContext(options) → devuelve el contexto configurado sin depender del flujo interactivo de Console.`

Con la factory y las migraciones trasladadas a Infrastructure, se valida el historial y se ejecuta el estado final:

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

El historial de migraciones de 2.1-2.11 se conserva y se añade el estado de arquitectura 2.12. El segundo comando debe terminar mostrando el marcador 2.12 OK.

Explicación línea a línea del estado 2.12 por capas

A diferencia de 2.1-2.11, el estado 2.12 está repartido en varios proyectos y archivos. Las líneas siguientes explican **cada línea no vacía** de los once archivos fuente que la práctica muestra y que el repositorio ejecutable utiliza.

src/AceriaData.Domain/Entities.cs

Línea 1	<code>namespace AceriaData.Domain.Entities; → declara el espacio de nombres AceriaData.Domain.Entities y sitúa el archivo en su capa arquitectónica.</code>
Línea 3	<code>public class OrdenFabricacion → declara la entidad raíz de la orden de fabricación dentro de Domain.</code>
Línea 4	<code>{ → abre el bloque correspondiente a la declaración o instrucción anterior.</code>
Línea 5	<code>public int Id { get; set; } → define la clave técnica de la entidad; su configuración concreta se mantiene en Infrastructure.</code>
Línea 6	<code>public string NumeroOrden { get; set; } = string.Empty; → define el identificador de negocio de la orden.</code>
Línea 7	<code>public string Cliente { get; set; } = string.Empty; → almacena el cliente asociado a la orden.</code>
Línea 8	<code>public DateTime FechaCreacion { get; set; } → almacena la fecha de creación de la orden.</code>
Línea 9	<code>public DateTime? FechaEntrega { get; set; } → almacena una fecha de entrega opcional.</code>
Línea 10	<code>public string Estado { get; set; } = "Pendiente"; → almacena el estado de la orden y parte con el valor de dominio Pendiente.</code>
Línea 11	<code>public string? Observaciones { get; set; } → almacena observaciones opcionales.</code>
Línea 12	<code>public bool IsDeleted { get; set; } → mantiene la marca utilizada por Soft Delete.</code>
Línea 13	<code>public DateTime? DeletedAt { get; set; } → registra opcionalmente cuándo se produjo el borrado lógico.</code>
Línea 14	<code>public List<PlanchaAcero> Planchas { get; set; } = new(); → expone la navegación de colección hacia las planchas de la orden.</code>
Línea 15	<code>public DetalleOrden? Detalle { get; set; } → expone la navegación opcional uno-a-uno hacia DetalleOrden.</code>
Línea 16	<code>public CertificadoCalidad? Certificado { get; set; } → expone la navegación opcional uno-a-uno hacia CertificadoCalidad.</code>
Línea 17	<code>public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new(); → expone la colección de filas de unión de la relación muchos-a-muchos.</code>
Línea 18	<code>} → cierra el bloque de código actual.</code>

Línea 20	<code>public class PlanchaAcero</code> → declara la entidad de plancha relacionada con una orden.
Línea 21	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 22	<code>public int Id { get; set; }</code> → define la clave técnica de la entidad; su configuración concreta se mantiene en Infrastructure.
Línea 23	<code>public int OrdenId { get; set; }</code> → almacena la clave foránea hacia OrdenFabricacion.
Línea 24	<code>public double Espesor { get; set; }</code> → almacena el espesor de la plancha.
Línea 25	<code>public double Ancho { get; set; }</code> → almacena el ancho de la plancha.
Línea 26	<code>public double Largo { get; set; }</code> → almacena el largo de la plancha.
Línea 27	<code>public decimal Peso { get; set; }</code> → almacena el peso con precisión configurada posteriormente mediante Fluent API.
Línea 28	<code>public bool Activa { get; set; } = true;</code> → indica si la plancha está activa.
Línea 29	<code>public bool IsDeleted { get; set; }</code> → mantiene la marca utilizada por Soft Delete.
Línea 30	<code>public DateTime? DeletedAt { get; set; }</code> → registra opcionalmente cuándo se produjo el borrado lógico.
Línea 31	<code>public OrdenFabricacion Orden { get; set; } = null!</code> ; → expone la navegación hacia la orden principal.
Línea 32	<code>}</code> → cierra el bloque de código actual.
Línea 34	<code>public class Aleacion</code> → declara la entidad de aleación utilizada por la relación muchos-a-muchos explícita.
Línea 35	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 36	<code>public int Id { get; set; }</code> → define la clave técnica de la entidad; su configuración concreta se mantiene en Infrastructure.
Línea 37	<code>public string Nombre { get; set; } = string.Empty;</code> → almacena el nombre funcional de la entidad.
Línea 38	<code>public string Codigo { get; set; } = string.Empty;</code> → almacena el código natural de la aleación.
Línea 39	<code>public double PorcentajeCarbono { get; set; }</code> → almacena el porcentaje de carbono validado por una restricción CHECK.
Línea 40	<code>public double PorcentajeManganeso { get; set; }</code> → almacena el porcentaje de manganeso validado por una restricción CHECK.
Línea 41	<code>public string? Descripcion { get; set; }</code> → almacena una descripción de negocio.
Línea 42	<code>public bool IsDeleted { get; set; }</code> → mantiene la marca utilizada por Soft Delete.
Línea 43	<code>public DateTime? DeletedAt { get; set; }</code> → registra opcionalmente cuándo se produjo el borrado lógico.
Línea 44	<code>public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();</code> → expone la colección de filas de unión de la relación muchos-a-muchos.
Línea 45	<code>}</code> → cierra el bloque de código actual.
Línea 47	<code>public class EstadoOrden</code> → declara la entidad que representa un estado de negocio de la orden.

Línea 48	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 49	<code>public int Id { get; set; }</code> → define la clave técnica de la entidad; su configuración concreta se mantiene en Infrastructure.
Línea 50	<code>public string Nombre { get; set; } = string.Empty;</code> → almacena el nombre funcional de la entidad.
Línea 51	<code>public string Descripcion { get; set; } = string.Empty;</code> → almacena una descripción de negocio.
Línea 52	<code>public bool Activo { get; set; } = true;</code> → indica si el estado está activo.
Línea 53	<code>public bool IsDeleted { get; set; }</code> → mantiene la marca utilizada por Soft Delete.
Línea 54	<code>public DateTime? DeletedAt { get; set; }</code> → registra opcionalmente cuándo se produjo el borrado lógico.
Línea 55	<code>}</code> → cierra el bloque de código actual.
Línea 57	<code>public class DetalleOrden</code> → declara la entidad dependiente de la relación uno-a-uno con OrdenFabricacion.
Línea 58	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 59	<code>public int Id { get; set; }</code> → define la clave técnica de la entidad; su configuración concreta se mantiene en Infrastructure.
Línea 60	<code>public int OrdenId { get; set; }</code> → almacena la clave foránea hacia OrdenFabricacion.
Línea 61	<code>public string ComposicionQuimica { get; set; } = string.Empty;</code> → almacena la composición química del detalle.
Línea 62	<code>public double TemperaturaColada { get; set; }</code> → almacena la temperatura de colada.
Línea 63	<code>public string? Notas { get; set; }</code> → almacena notas opcionales del detalle.
Línea 64	<code>public OrdenFabricacion Orden { get; set; } = null!;</code> → expone la navegación hacia la orden principal.
Línea 65	<code>}</code> → cierra el bloque de código actual.
Línea 67	<code>public class CertificadoCalidad</code> → declara la entidad dependiente que representa el certificado de calidad de una orden.
Línea 68	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 69	<code>public int Id { get; set; }</code> → define la clave técnica de la entidad; su configuración concreta se mantiene en Infrastructure.
Línea 70	<code>public int OrdenId { get; set; }</code> → almacena la clave foránea hacia OrdenFabricacion.
Línea 71	<code>public string NumeroCertificado { get; set; } = string.Empty;</code> → almacena el identificador natural del certificado.
Línea 72	<code>public DateTime FechaEmision { get; set; }</code> → almacena la fecha de emisión del certificado.
Línea 73	<code>public string OrganismoCertificador { get; set; } = string.Empty;</code> → almacena el organismo que emitió el certificado.
Línea 74	<code>public OrdenFabricacion Orden { get; set; } = null!;</code> → expone la navegación hacia la orden principal.

Línea 75	} → cierra el bloque de código actual.
Línea 77	public class OrdenAleacion → declara la entidad de unión explícita entre OrdenFabricacion y Aleacion.
Línea 78	{ → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 79	public int OrdenFabricacionId { get; set; } → almacena la parte de la clave compuesta que referencia a OrdenFabricacion.
Línea 80	public int AleacionId { get; set; } → almacena la parte de la clave compuesta que referencia a Aleacion.
Línea 81	public DateTime FechaAsignacion { get; set; } = DateTime.Now; → almacena cuándo se creó la relación orden-aleación.
Línea 82	public decimal CantidadUtilizada { get; set; } → almacena la cantidad de aleación utilizada.
Línea 83	public string EstadoRelacion { get; set; } = "Activa"; → almacena el estado lógico de la relación orden-aleación.
Línea 84	public OrdenFabricacion Orden { get; set; } = null!; → expone la navegación hacia la orden principal.
Línea 85	public Aleacion Aleacion { get; set; } = null!; → expone la navegación hacia Aleacion.
Línea 86	} → cierra el bloque de código actual.

src/AceriaData.Application/Interfaces.cs

Línea 1	using AceriaData.Domain.Entities; → importa las entidades del dominio sin introducir dependencias de EF Core en Domain.
Línea 3	namespace AceriaData.Application.Interfaces; → declara el espacio de nombres AceriaData.Application.Interfaces y sitúa el archivo en su capa arquitectónica.
Línea 5	public interface IOrdenRepositorio → declara el puerto de persistencia de órdenes que Application necesita y que Infrastructure implementará.
Línea 6	{ → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 7	OrdenFabricacion? ObtenerPorId(int id); → declara la operación para recuperar una orden por su clave técnica.
Línea 8	OrdenFabricacion? ObtenerPorNumero(string numeroOrden); → declara la operación para recuperar una orden por su número de negocio.
Línea 9	List<OrdenFabricacion> ObtenerTodas(); → declara la operación que devuelve las órdenes visibles para el repositorio.
Línea 10	void Agregar(OrdenFabricacion orden); → declara la operación que registra una nueva orden.
Línea 11	void Eliminar(OrdenFabricacion orden); → declara la operación que marca una orden para eliminación física cuando el caso de uso lo requiera.
Línea 12	} → cierra el bloque de código actual.
Línea 14	public interface IUnidadDeTrabajo : IDisposable → declara el puerto de unidad de trabajo que coordina el repositorio y la confirmación de cambios.
Línea 15	{ → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 16	<code>IOrdenRepositorio Ordenes { get; } →</code> expone el repositorio de órdenes a través del puerto de unidad de trabajo.
Línea 17	<code>int Guardar(); →</code> declara la operación que confirma de forma coordinada los cambios pendientes.
Línea 18	<code>} →</code> cierra el bloque de código actual.

src/AceriaData.Application/CrearOrdenUseCase.cs

Línea 1	<code>using AceriaData.Application.Interfaces; →</code> importa los puertos definidos por Application para que Infrastructure pueda implementarlos.
Línea 2	<code>using AceriaData.Domain.Entities; →</code> importa las entidades del dominio sin introducir dependencias de EF Core en Domain.
Línea 4	<code>namespace AceriaData.Application.UseCases; →</code> declara el espacio de nombres AceriaData.Application.UseCases y sitúa el archivo en su capa arquitectónica.
Línea 6	<code>public sealed class CrearOrdenUseCase →</code> declara el caso de uso de Application encargado de crear una orden sin depender de EF Core.
Línea 7	<code>{ →</code> abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 8	<code>private readonly IUnidadDeTrabajo _unidad; →</code> guarda la dependencia del caso de uso contra la abstracción IUnidadDeTrabajo.
Línea 9	<code>public CrearOrdenUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad; →</code> inyecta IUnidadDeTrabajo en el caso de uso sin acoplar Application a Infrastructure.
Línea 11	<code>public int Ejecutar(string numeroOrden, string cliente) →</code> define la operación de aplicación que crea una orden y devuelve el resultado del guardado.
Línea 12	<code>{ →</code> abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 13	<code>_unidad.Ordenes.Agregar(new OrdenFabricacion →</code> crea una orden a través del puerto IOrdenRepositorio expuesto por la unidad de trabajo.
Línea 14	<code>{ →</code> abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 15	<code>NumeroOrden = numeroOrden, →</code> copia al dominio el número de orden recibido por el caso de uso.
Línea 16	<code>Cliente = cliente, →</code> copia al dominio el cliente recibido por el caso de uso.
Línea 17	<code>FechaCreacion = DateTime.UtcNow, →</code> asigna una fecha de creación UTC desde la aplicación.
Línea 18	<code>Estado = "Pendiente" →</code> inicializa la nueva orden con el estado Pendiente.
Línea 19	<code>}); →</code> cierra el inicializador o la llamada iniciada en las líneas anteriores.
Línea 20	<code>return _unidad.Guardar(); →</code> confirma los cambios una única vez mediante la unidad de trabajo y devuelve las filas afectadas.
Línea 21	<code>} →</code> cierra el bloque de código actual.
Línea 22	<code>} →</code> cierra el bloque de código actual.

src/AceriaData.Infrastructure/Persistence/AceriaDbContext.cs

Línea 1	<code>using AceriaData.Domain.Entities; →</code> importa las entidades del dominio sin introducir dependencias de EF Core en Domain.
----------------	--

Línea 2	<code>using Microsoft.EntityFrameworkCore;</code> → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.
Línea 4	<code>namespace AceriaData.Infrastructure.Persistence;</code> → declara el espacio de nombres <code>AceriaData.Infrastructure.Persistence</code> y sitúa el archivo en su capa arquitectónica.
Línea 6	<code>public sealed class AceriaDbContext : DbContext</code> → declara el <code>DbContext</code> de Infrastructure y concentra el modelo EF Core.
Línea 7	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 8	<code>public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }</code> → recibe <code>DbContextOptions</code> por inyección y los pasa al constructor base de <code>DbContext</code> .
Línea 10	<code>public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();</code> → expone el <code>DbSet</code> <code>OrdenesFabricacion</code> para consultar y persistir esa entidad.
Línea 11	<code>public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();</code> → expone el <code>DbSet</code> <code>PlanchasAcero</code> para consultar y persistir esa entidad.
Línea 12	<code>public DbSet<Aleacion> Aleaciones => Set<Aleacion>();</code> → expone el <code>DbSet</code> <code>Aleaciones</code> para consultar y persistir esa entidad.
Línea 13	<code>public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();</code> → expone el <code>DbSet</code> <code>EstadosOrden</code> para consultar y persistir esa entidad.
Línea 14	<code>public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();</code> → expone el <code>DbSet</code> <code>DetallesOrden</code> para consultar y persistir esa entidad.
Línea 15	<code>public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();</code> → expone el <code>DbSet</code> <code>CertificadosCalidad</code> para consultar y persistir esa entidad.
Línea 16	<code>public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();</code> → expone el <code>DbSet</code> <code>OrdenesAleaciones</code> para consultar y persistir esa entidad.
Línea 18	<code>protected override void OnModelCreating(ModelBuilder modelBuilder)</code> → sobrescribe <code>OnModelCreating</code> para aplicar la configuración relacional del modelo.
Línea 19	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 20	<code>base.OnModelCreating(modelBuilder);</code> → ejecuta primero la configuración base de <code>DbContext</code> .
Línea 21	<code>modelBuilder.ApplyConfigurationsFromAssembly(typeof(AceriaDbContext).Assembly);</code> → descubre y aplica automáticamente todas las implementaciones <code>IEntityTypeConfiguration</code> del ensamblado de Infrastructure.
Línea 22	<code>}</code> → cierra el bloque de código actual.
Línea 23	<code>}</code> → cierra el bloque de código actual.

src/AceriaData.Infrastructure/Persistence/Configurations/OrdenFabricacionConfiguration.cs

Línea 1	<code>using AceriaData.Domain.Entities;</code> → importa las entidades del dominio sin introducir dependencias de EF Core en Domain.
Línea 2	<code>using Microsoft.EntityFrameworkCore;</code> → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.
Línea 3	<code>using Microsoft.EntityFrameworkCore.Metadata.Builders;</code> → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 5	<code>namespace AceriaData.Infrastructure.Persistence.Configurations;</code> → declara el espacio de nombres <code>AceriaData.Infrastructure.Persistence.Configurations</code> y sitúa el archivo en su capa arquitectónica.
Línea 7	<code>public sealed class OrdenFabricacionConfiguration :</code> <code>IEntityTypeConfiguration<OrdenFabricacion></code> → declara la configuración Fluent API de <code>OrdenFabricacion</code> .
Línea 8	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 9	<code>public void Configure(EntityTypeBuilder<OrdenFabricacion> b)</code> → implementa el método <code>Configure</code> de <code>IEntityTypeConfiguration</code> para describir el mapeo de la entidad.
Línea 10	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 11	<code>b.ToTable("OrdenesFabricacion");</code> → asigna la entidad a la tabla SQL indicada; cuando incluye lambda, configura además restricciones de tabla.
Línea 12	<code>b.HasKey(x => x.Id);</code> → define explícitamente la clave primaria de la entidad.
Línea 13	<code>b.HasAlternateKey(x => x.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");</code> → define una clave alternativa para el identificador natural y permite referenciarlo como principal.
Línea 14	<code>b.Property(x => x.NumeroOrden).IsRequired().HasMaxLength(50);</code> → marca la propiedad como requerida y limita su longitud en el esquema.
Línea 15	<code>b.Property(x => x.Cliente).IsRequired().HasMaxLength(200);</code> → marca la propiedad como requerida y limita su longitud en el esquema.
Línea 16	<code>b.Property(x => x.FechaCreacion).HasDefaultValueSql("GETDATE()");</code> → configura una propiedad cuyo valor por defecto se obtiene mediante una expresión SQL del servidor.
Línea 17	<code>b.Property(x => x.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");</code> → configura un valor por defecto de base de datos para la propiedad.
Línea 18	<code>b.Property(x => x.Observaciones).HasMaxLength(500);</code> → limita la longitud máxima de la columna correspondiente.
Línea 19	<code>b.HasQueryFilter(x => !x.IsDeleted && x.Estado != "Cancelada");</code> → aplica el filtro global que oculta registros eliminados o inactivos en las consultas normales.
Línea 20	<code>b.HasIndex(x => x.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");</code> → crea y nombra un índice sobre la propiedad indicada.
Línea 21	<code>b.HasIndex(x => new { x.Cliente, x.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");</code> → crea un índice compuesto sobre las propiedades indicadas.
Línea 22	<code>b.HasIndex(x => x.FechaEntrega).HasFilter("[Estado] = 'Pendiente'");</code> <code>HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");</code> → crea un índice filtrado para el subconjunto de filas indicado.
Línea 23	<code>b.HasIndex(x => x.Estado).IncludeProperties(x => new { x.NumeroOrden, x.Cliente, x.FechaCreacion });</code> <code>HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");</code> → crea un índice con columnas incluidas para cubrir consultas frecuentes.
Línea 24	<code>}</code> → cierra el bloque de código actual.
Línea 25	<code>}</code> → cierra el bloque de código actual.

src/AceriaData.Infrastructure/Persistence/Configurations/PlanchaAceroConfiguration.cs

Línea 1	<code>using AceriaData.Domain.Entities;</code> → importa las entidades del dominio sin introducir dependencias de EF Core en Domain.
Línea 2	<code>using Microsoft.EntityFrameworkCore;</code> → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.
Línea 3	<code>using Microsoft.EntityFrameworkCore.Metadata.Builders;</code> → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.
Línea 5	<code>namespace AceriaData.Infrastructure.Persistence.Configurations;</code> → declara el espacio de nombres <code>AceriaData.Infrastructure.Persistence.Configurations</code> y sitúa el archivo en su capa arquitectónica.
Línea 7	<code>public sealed class PlanchaAceroConfiguration :</code> <code>IEntityTypeConfiguration<PlanchaAcero></code> → declara la configuración Fluent API de <code>PlanchaAcero</code> .
Línea 8	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 9	<code>public void Configure(EntityTypeBuilder<PlanchaAcero> b)</code> → implementa el método <code>Configure</code> de <code>IEntityTypeConfiguration</code> para describir el mapeo de la entidad.
Línea 10	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 11	<code>b.ToTable("PlanchasAcero", t =></code> → asigna la entidad a la tabla SQL indicada; cuando incluye lambda, configura además restricciones de tabla.
Línea 12	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 13	<code>t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");</code> → añade una restricción CHECK en SQL Server para reforzar una regla de integridad en la base de datos.
Línea 14	<code>t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");</code> → añade una restricción CHECK en SQL Server para reforzar una regla de integridad en la base de datos.
Línea 15	<code>t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");</code> → añade una restricción CHECK en SQL Server para reforzar una regla de integridad en la base de datos.
Línea 16	<code>t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");</code> → añade una restricción CHECK en SQL Server para reforzar una regla de integridad en la base de datos.
Línea 17	<code>});</code> → cierra el inicializador o la llamada iniciada en las líneas anteriores.
Línea 18	<code>b.HasKey(x => x.Id);</code> → define explícitamente la clave primaria de la entidad.
Línea 19	<code>b.Property(x => x.Peso).HasPrecision(18, 3);</code> → configura precisión y escala para el valor decimal.
Línea 20	<code>b.Property(x => x.Activa).HasDefaultValue(true);</code> → configura un valor por defecto de base de datos para la propiedad.
Línea 21	<code>b.HasOne(x => x.Orden).WithMany(x => x.Planchas).HasForeignKey(x =></code> <code>x.OrdenId).OnDelete(DeleteBehavior.Cascade).IsRequired();</code> → configura una relación uno-a-muchos y encadena la clave foránea y el comportamiento de borrado.
Línea 22	<code>b.HasIndex(x => new { x.OrdenId,</code> <code>x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");</code> → crea un índice compuesto sobre las propiedades indicadas.
Línea 23	<code>b.HasIndex(x => x.Espesor).HasFilter("[Activa] = 1");</code> → crea un índice filtrado para el subconjunto de filas indicado.

Línea 24	<code>b.HasQueryFilter(x => !x.IsDeleted && x.Activa);</code> → aplica el filtro global que oculta registros eliminados o inactivos en las consultas normales.
Línea 25	<code>}</code> → cierra el bloque de código actual.
Línea 26	<code>}</code> → cierra el bloque de código actual.

src/AceriaData.Infrastructure/Persistence/Configurations/ModeloConfiguration.cs

Línea 1	<code>using AceriaData.Domain.Entities;</code> → importa las entidades del dominio sin introducir dependencias de EF Core en Domain.
Línea 2	<code>using Microsoft.EntityFrameworkCore;</code> → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.
Línea 3	<code>using Microsoft.EntityFrameworkCore.Metadata.Builders;</code> → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.
Línea 5	<code>namespace AceriaData.Infrastructure.Persistence.Configurations;</code> → declara el espacio de nombres <code>AceriaData.Infrastructure.Persistence.Configurations</code> y sitúa el archivo en su capa arquitectónica.
Línea 7	<code>public sealed class AleacionConfiguration : IEntityTypeConfiguration<Aleacion></code> → declara la configuración Fluent API de Aleacion.
Línea 8	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 9	<code>public void Configure(EntityTypeBuilder<Aleacion> b)</code> → implementa el método <code>Configure</code> de <code>IEntityTypeConfiguration</code> para describir el mapeo de la entidad.
Línea 10	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 11	<code>b.ToTable("Aleaciones", t =></code> → asigna la entidad a la tabla SQL indicada; cuando incluye lambda, configura además restricciones de tabla.
Línea 12	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 13	<code>t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND [PorcentajeCarbono] <= 2");</code> → añade una restricción CHECK en SQL Server para reforzar una regla de integridad en la base de datos.
Línea 14	<code>t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND [PorcentajeManganeso] <= 5");</code> → añade una restricción CHECK en SQL Server para reforzar una regla de integridad en la base de datos.
Línea 15	<code>});</code> → cierra el inicializador o la llamada iniciada en las líneas anteriores.
Línea 16	<code>b.HasKey(x => x.Id);</code> → define explícitamente la clave primaria de la entidad.
Línea 17	<code>b.HasAlternateKey(x => x.Codigo).HasName("AK_Aleaciones_Codigo");</code> → define una clave alternativa para el identificador natural y permite referenciarlo como principal.
Línea 18	<code>b.Property(x => x.Nombre).IsRequired().HasMaxLength(100);</code> → marca la propiedad como requerida y limita su longitud en el esquema.
Línea 19	<code>b.Property(x => x.Codigo).IsRequired().HasMaxLength(20);</code> → marca la propiedad como requerida y limita su longitud en el esquema.
Línea 20	<code>b.Property(x => x.Descripcion).HasMaxLength(500);</code> → limita la longitud máxima de la columna correspondiente.

Línea 21	<code>b.HasIndex(x => x.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");</code> → crea y nombra un índice sobre la propiedad indicada.
Línea 22	<code>b.HasIndex(x => new { x.PorcentajeCarbono, x.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");</code> → crea un índice compuesto sobre las propiedades indicadas.
Línea 23	<code>b.HasQueryFilter(x => !x.IsDeleted);</code> → aplica el filtro global que oculta registros eliminados o inactivos en las consultas normales.
Línea 24	<code>}</code> → cierra el bloque de código actual.
Línea 25	<code>}</code> → cierra el bloque de código actual.
Línea 27	<code>public sealed class EstadoOrdenConfiguration :</code> <code>IEntityTypeConfiguration<EstadoOrden></code> → declara la configuración Fluent API de EstadoOrden.
Línea 28	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 29	<code>public void Configure(EntityTypeBuilder<EstadoOrden> b)</code> → implementa el método Configure de <code>IEntityTypeConfiguration</code> para describir el mapeo de la entidad.
Línea 30	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 31	<code>b.ToTable("EstadosOrden");</code> → asigna la entidad a la tabla SQL indicada; cuando incluye lambda, configura además restricciones de tabla.
Línea 32	<code>b.HasKey(x => x.Id);</code> → define explícitamente la clave primaria de la entidad.
Línea 33	<code>b.Property(x => x.Nombre).IsRequired().HasMaxLength(50);</code> → marca la propiedad como requerida y limita su longitud en el esquema.
Línea 34	<code>b.Property(x => x.Descripcion).HasMaxLength(250);</code> → limita la longitud máxima de la columna correspondiente.
Línea 35	<code>b.Property(x => x.Activo).HasDefaultValue(true);</code> → configura un valor por defecto de base de datos para la propiedad.
Línea 36	<code>b.HasQueryFilter(x => !x.IsDeleted && x.Activo);</code> → aplica el filtro global que oculta registros eliminados o inactivos en las consultas normales.
Línea 37	<code>}</code> → cierra el bloque de código actual.
Línea 38	<code>}</code> → cierra el bloque de código actual.
Línea 40	<code>public sealed class DetalleOrdenConfiguration :</code> <code>IEntityTypeConfiguration<DetalleOrden></code> → declara la configuración Fluent API de DetalleOrden.
Línea 41	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 42	<code>public void Configure(EntityTypeBuilder<DetalleOrden> b)</code> → implementa el método Configure de <code>IEntityTypeConfiguration</code> para describir el mapeo de la entidad.
Línea 43	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 44	<code>b.ToTable("DetallesOrden");</code> → asigna la entidad a la tabla SQL indicada; cuando incluye lambda, configura además restricciones de tabla.
Línea 45	<code>b.HasKey(x => x.Id);</code> → define explícitamente la clave primaria de la entidad.
Línea 46	<code>b.Property(x => x.ComposicionQuimica).IsRequired().HasMaxLength(200);</code> → marca la propiedad como requerida y limita su longitud en el esquema.

Línea 47	<code>b.Property(x => x.Notas).HasMaxLength(500);</code> → limita la longitud máxima de la columna correspondiente.
Línea 48	<code>b.HasOne(x => x.Orden).WithOne(x => x.Detalle).HasForeignKey<DetalleOrden>(x => x.OrdenId).OnDelete(DeleteBehavior.Cascade).IsRequired();</code> → configura una relación uno-a-uno y encadena su clave foránea y comportamiento de borrado.
Línea 49	<code>b.HasQueryFilter(x => !x.Orden.IsDeleted && x.Orden.Estado != "Cancelada");</code> → aplica el filtro global que oculta registros eliminados o inactivos en las consultas normales.
Línea 50	<code>}</code> → cierra el bloque de código actual.
Línea 51	<code>}</code> → cierra el bloque de código actual.
Línea 53	<code>public sealed class CertificadoCalidadConfiguration :</code> <code>IEntityTypeConfiguration<CertificadoCalidad></code> → declara la configuración Fluent API de CertificadoCalidad.
Línea 54	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 55	<code>public void Configure(EntityTypeBuilder<CertificadoCalidad> b)</code> → implementa el método Configure de IEntityTypeConfiguration para describir el mapeo de la entidad.
Línea 56	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 57	<code>b.ToTable("CertificadosCalidad");</code> → asigna la entidad a la tabla SQL indicada; cuando incluye lambda, configura además restricciones de tabla.
Línea 58	<code>b.HasKey(x => x.Id);</code> → define explícitamente la clave primaria de la entidad.
Línea 59	<code>b.HasAlternateKey(x => x.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");</code> → define una clave alternativa para el identificador natural y permite referenciarlo como principal.
Línea 60	<code>b.Property(x => x.NumeroCertificado).IsRequired().HasMaxLength(50);</code> → marca la propiedad como requerida y limita su longitud en el esquema.
Línea 61	<code>b.Property(x => x.OrganismoCertificador).IsRequired().HasMaxLength(100);</code> → marca la propiedad como requerida y limita su longitud en el esquema.
Línea 62	<code>b.HasIndex(x => x.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");</code> → crea y nombra un índice sobre la propiedad indicada.
Línea 63	<code>b.HasOne(x => x.Orden).WithOne(x => x.Certificado).HasForeignKey<CertificadoCalidad>(x => x.OrdenId).OnDelete(DeleteBehavior.Cascade).IsRequired();</code> → configura una relación uno-a-uno y encadena su clave foránea y comportamiento de borrado.
Línea 64	<code>b.HasQueryFilter(x => !x.Orden.IsDeleted && x.Orden.Estado != "Cancelada");</code> → aplica el filtro global que oculta registros eliminados o inactivos en las consultas normales.
Línea 65	<code>}</code> → cierra el bloque de código actual.
Línea 66	<code>}</code> → cierra el bloque de código actual.
Línea 68	<code>public sealed class OrdenAleacionConfiguration :</code> <code>IEntityTypeConfiguration<OrdenAleacion></code> → declara la configuración Fluent API de OrdenAleacion.
Línea 69	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 70	<code>public void Configure(EntityTypeBuilder<OrdenAleacion> b)</code> → implementa el método Configure de IEntityTypeConfiguration para describir el mapeo de la entidad.

Línea 71	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 72	<code>b.ToTable("OrdenesAleaciones");</code> → asigna la entidad a la tabla SQL indicada; cuando incluye lambda, configura además restricciones de tabla.
Línea 73	<code>b.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });</code> → define la clave primaria compuesta de la entidad de unión con ambas claves foráneas.
Línea 74	<code>b.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");</code> → configura una propiedad cuyo valor por defecto se obtiene mediante una expresión SQL del servidor.
Línea 75	<code>b.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);</code> → configura precisión y escala para el valor decimal.
Línea 76	<code>b.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");</code> → configura un valor por defecto de base de datos para la propiedad.
Línea 77	<code>b.HasOne(x => x.Orden).WithMany(x => x.OrdenesAleaciones).HasForeignKey(x => x.OrdenFabricacionId).OnDelete(DeleteBehavior.Cascade);</code> → configura una relación uno-a-muchos y encadena la clave foránea y el comportamiento de borrado.
Línea 78	<code>b.HasOne(x => x.Aleacion).WithMany(x => x.OrdenesAleaciones).HasForeignKey(x => x.AleacionId).OnDelete(DeleteBehavior.Restrict);</code> → configura una relación uno-a-muchos y encadena la clave foránea y el comportamiento de borrado.
Línea 79	<code>b.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");</code> → crea y nombra un índice sobre la propiedad indicada.
Línea 80	<code>b.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] = 'Activa'").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");</code> → crea un índice filtrado para el subconjunto de filas indicado.
Línea 81	<code>b.HasQueryFilter(x => x.EstadoRelacion == "Activa" && !x.Orden.IsDeleted && !x.Aleacion.IsDeleted);</code> → aplica el filtro global que oculta registros eliminados o inactivos en las consultas normales.
Línea 82	<code>}</code> → cierra el bloque de código actual.
Línea 83	<code>}</code> → cierra el bloque de código actual.

src/AceriaData.Infrastructure/Repositories/Repositories.cs

Línea 1	<code>using AceriaData.Application.Interfaces;</code> → importa los puertos definidos por Application para que Infrastructure pueda implementarlos.
Línea 2	<code>using AceriaData.Domain.Entities;</code> → importa las entidades del dominio sin introducir dependencias de EF Core en Domain.
Línea 3	<code>using AceriaData.Infrastructure.Persistence;</code> → importa el DbContext y los componentes de persistencia de Infrastructure.
Línea 4	<code>using Microsoft.EntityFrameworkCore;</code> → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.
Línea 6	<code>namespace AceriaData.Infrastructure.Repositories;</code> → declara el espacio de nombres AceriaData.Infrastructure.Repositories y sitúa el archivo en su capa arquitectónica.
Línea 8	<code>public sealed class OrdenRepositorio : IOrdenRepositorio</code> → declara el adaptador de Infrastructure que implementa IOrdenRepositorio.
Línea 9	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 10	<code>private readonly AceriaDbContext _context;</code> → mantiene la referencia al DbContext compartido por el adaptador dentro del ámbito DI.
Línea 11	<code>public OrdenRepositorio(AceriaDbContext context) => _context = context;</code> → inyecta <code>AceriaDbContext</code> en el adaptador de repositorio.
Línea 12	<code>public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);</code> → implementa la búsqueda por clave mediante <code>DbSet.Find</code> .
Línea 13	<code>public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(x => x.NumeroOrden == numeroOrden);</code> → implementa la búsqueda por número de orden mediante una consulta LINQ traducida a SQL.
Línea 14	<code>public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(x => x.Id).ToList();</code> → materializa las órdenes ordenadas por Id.
Línea 15	<code>public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);</code> → registra la orden como <code>Added</code> en el <code>ChangeTracker</code> sin ejecutar todavía <code>SaveChanges</code> .
Línea 16	<code>public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);</code> → marca la orden como <code>Deleted</code> en el <code>ChangeTracker</code> ; la escritura se confirma al guardar.
Línea 17	<code>}</code> → cierra el bloque de código actual.
Línea 19	<code>public sealed class UnidadDeTrabajo : IUnidadDeTrabajo</code> → declara el adaptador que implementa <code>IUnidadDeTrabajo</code> y concentra <code>SaveChanges</code> .
Línea 20	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 21	<code>private readonly AceriaDbContext _context;</code> → mantiene la referencia al DbContext compartido por el adaptador dentro del ámbito DI.
Línea 22	<code>private IOrdenRepositorio? _ordenes;</code> → mantiene la instancia perezosa del repositorio de órdenes.
Línea 23	<code>public UnidadDeTrabajo(AceriaDbContext context) => _context = context;</code> → inyecta el mismo <code>AceriaDbContext</code> en la unidad de trabajo.
Línea 24	<code>public IOrdenRepositorio Ordenes => _ordenes ??= new OrdenRepositorio(_context);</code> → crea perezosamente <code>OrdenRepositorio</code> y reutiliza el <code>DbContext</code> de la unidad de trabajo.
Línea 25	<code>public int Guardar() => _context.SaveChanges();</code> → confirma en SQL Server todos los cambios pendientes del DbContext.
Línea 26	<code>public void Dispose() => _context.Dispose();</code> → libera el DbContext cuando termina la unidad de trabajo.
Línea 27	<code>}</code> → cierra el bloque de código actual.

src/AceriaData.Infrastructure/DependencyInjection.cs

Línea 1	<code>using AceriaData.Application.Interfaces;</code> → importa los puertos definidos por <code>Application</code> para que <code>Infrastructure</code> pueda implementarlos.
Línea 2	<code>using AceriaData.Infrastructure.Persistence;</code> → importa el <code>DbContext</code> y los componentes de persistencia de <code>Infrastructure</code> .
Línea 3	<code>using AceriaData.Infrastructure.Repositories;</code> → importa los adaptadores concretos que implementan los puertos de <code>Application</code> .

Línea 4	<code>using Microsoft.EntityFrameworkCore;</code> → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.
Línea 5	<code>using Microsoft.Extensions.DependencyInjection;</code> → importa el contenedor de dependencias y sus métodos de registro/resolución.
Línea 7	<code>namespace AceriaData.Infrastructure;</code> → declara el espacio de nombres AceriaData.Infrastructure y sitúa el archivo en su capa arquitectónica.
Línea 9	<code>public static class DependencyInjection</code> → declara una clase estática usada como contenedor de extensiones de registro.
Línea 10	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 11	<code>public static IServiceCollection AddAceriaInfrastructure(this IServiceCollection services, string connectionString)</code> → define el método de extensión que encapsula todos los registros de Infrastructure.
Línea 12	<code>{</code> → abre el bloque correspondiente a la declaración o instrucción anterior.
Línea 13	<code>services.AddDbContext<AceriaDbContext>(o => o.UseSqlServer(connectionString));</code> → registra AceriaDbContext con SQL Server usando la cadena recibida por el composition root.
Línea 14	<code>services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();</code> → vincula IOrdenRepositorio con OrdenRepositorio mediante ciclo de vida scoped.
Línea 15	<code>services.AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>();</code> → vincula IUnidadDeTrabajo con UnidadDeTrabajo mediante ciclo de vida scoped.
Línea 16	<code>return services;</code> → devuelve la colección para permitir encadenar el registro de servicios.
Línea 17	<code>}</code> → cierra el bloque de código actual.
Línea 18	<code>}</code> → cierra el bloque de código actual.

src/AceriaData.Console/Program.cs

Línea 1	<code>using AceriaData.Application.UseCases;</code> → importa el caso de uso que la capa de entrada va a resolver y ejecutar.
Línea 2	<code>using AceriaData.Infrastructure;</code> → importa el espacio de nombres requerido por los tipos usados en este archivo.
Línea 3	<code>using AceriaData.Infrastructure.Persistence;</code> → importa el DbContext y los componentes de persistencia de Infrastructure.
Línea 4	<code>using Microsoft.EntityFrameworkCore;</code> → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.
Línea 5	<code>using Microsoft.Extensions.Configuration;</code> → importa la API de configuración usada para leer appsettings.json y variables de entorno.
Línea 6	<code>using Microsoft.Extensions.DependencyInjection;</code> → importa el contenedor de dependencias y sus métodos de registro/resolución.
Línea 8	<code>var configuration = new ConfigurationBuilder()</code> → inicia la configuración de la aplicación de consola.
Línea 9	<code>.SetBasePath(AppContext.BaseDirectory)</code> → establece la carpeta base desde la que se localizará appsettings.json.

Línea 10	<code>.AddJsonFile("appsettings.json", optional: false) → carga appsettings.json como fuente obligatoria de configuración.</code>
Línea 11	<code>.AddEnvironmentVariables() → añade variables de entorno, que pueden complementar o sobrescribir la configuración.</code>
Línea 12	<code>.Build(); → construye el objeto IConfiguration definitivo.</code>
Línea 14	<code>var cs = configuration.GetConnectionString("AceriaDB") → obtiene la cadena AceriaDB desde la configuración.</code>
Línea 15	<code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB"); → falla de forma explícita si la cadena de conexión no está configurada.</code>
Línea 17	<code>var services = new ServiceCollection(); → crea la colección de servicios del composition root.</code>
Línea 18	<code>services.AddAceriaInfrastructure(cs); → registra en Console todos los adaptadores de Infrastructure mediante su extensión.</code>
Línea 19	<code>services.AddScoped<CrearOrdenUseCase>(); → registra el caso de uso para resolverlo dentro del mismo ámbito.</code>
Línea 21	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true, ValidateScopes = true }); → construye el proveedor de inyección de dependencias con validación y garantiza su liberación automática al terminar.</code>
Línea 22	<code>using var scope = provider.CreateScope(); → crea un ámbito scoped para resolver DbContext, repositorios y casos de uso, y lo libera automáticamente al terminar.</code>
Línea 23	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>(); → resuelve el DbContext registrado en Infrastructure.</code>
Línea 24	<code>context.Database.EnsureDeleted(); → elimina la base del laboratorio para que la ejecución E2E sea reproducible; no sustituye a Migrations.</code>
Línea 25	<code>context.Database.Migrate(); → aplica el historial real de migraciones hasta el esquema 2.12.</code>
Línea 27	<code>var crear = scope.ServiceProvider.GetRequiredService<CrearOrdenUseCase>(); → resuelve el caso de uso desde el contenedor.</code>
Línea 28	<code>crear.Ejecutar("0F-M2-HEX-0001", "Cliente Arquitectura"); → ejecuta el caso de uso y persiste la orden a través de los puertos de Application.</code>
Línea 29	<code>var orden = context.OrdenesFabricacion.Single(); → consulta la única orden creada para verificar el resultado del escenario E2E.</code>
Línea 30	<code>Console.WriteLine(\$"2.12 OK {orden.NumeroOrden} {orden.Cliente}"); → imprime el marcador 2.12 OK y los datos persistidos que usa la CI como evidencia E2E.</code>

src/AceriaData.Infrastructure/Persistence/AceriaDesignTimeDbContextFactory.cs

Línea 1	<code>using Microsoft.EntityFrameworkCore; → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.</code>
Línea 2	<code>using Microsoft.EntityFrameworkCore.Design; → importa el contrato de factory de diseño utilizado por las herramientas dotnet ef.</code>
Línea 4	<code>namespace AceriaData.Infrastructure.Persistence; → declara el espacio de nombres AceriaData.Infrastructure.Persistence y sitúa el archivo en su capa arquitectónica.</code>

Línea 6	<code>public sealed class AceriaDesignTimeDbContextFactory : IDesignTimeDbContextFactory<AceriaDbContext> → declara la factory de diseño que permite a dotnet ef construir AceriaDbContext sin arrancar Console.</code>
Línea 7	<code>{ → abre el bloque correspondiente a la declaración o instrucción anterior.</code>
Línea 8	<code>public AceriaDbContext CreateDbContext(string[] args) → implementa el contrato de diseño que usan las herramientas EF Core.</code>
Línea 9	<code>{ → abre el bloque correspondiente a la declaración o instrucción anterior.</code>
Línea 10	<code>var options = new DbContextOptionsBuilder<AceriaDbContext>() → inicia la construcción de DbContextOptions para el contexto de diseño.</code>
Línea 11	<code>.UseSqlServer("Server=(localdb)\ \MSSQLLocalDB;Database=AceriaDB;Trusted_Connection=True;TrustServerCertificate=True;") → configura SQL Server LocalDB y la base AceriaDB para las operaciones de diseño.</code>
Línea 12	<code>.Options; → extrae las opciones ya configuradas del builder.</code>
Línea 13	<code>return new AceriaDbContext(options); → crea y devuelve AceriaDbContext con esas opciones para dotnet ef.</code>
Línea 14	<code>} → cierra el bloque de código actual.</code>
Línea 15	<code>} → cierra el bloque de código actual.</code>

Errores comunes

Error	Causa	Solución
Domain referencia EF Core	Se movieron entidades junto con tipos de persistencia	Mantener Domain como clases de negocio sin paquete EF Core.
Application usa DbContext	Se filtró una dependencia de infraestructura	Depender de IOrdenRepositorio/IUnidadDeTrabajo.
EF Tools no encuentra el contexto	Startup o project incorrectos	Indicar <code>--project Infrastructure --startup-project Console</code> .
La consola no resuelve servicios	Falta AddAceriaInfrastructure	Registrar la infraestructura antes de BuildServiceProvider.

Reto resuelto

Reto: Crear una orden desde CrearOrdenUseCase y comprobar que Application no referencia Microsoft.EntityFrameworkCore.

Solución: revisar los `.csproj`: Domain y Application no contienen referencias a `Microsoft.EntityFrameworkCore`; Infrastructure sí contiene `Microsoft.EntityFrameworkCore.SqlServer` y `Design`. Ejecutar después la solución y comprobar el marcador 2.12 OK.

Analogía final

Clean Architecture separa el conocimiento del negocio de la tecnología de persistencia: EF Core pasa a ser un adaptador reemplazable alrededor del núcleo.

Resultado esperado

M2 termina con una solución multiproyecto compilable y ejecutable, el mismo modelo acumulado, EF Core aislado en Infrastructure y una aplicación que accede a persistencia mediante puertos definidos fuera de la infraestructura.

Conexión con el siguiente módulo

M03/PROYECTO0/3.1 deberá partir de este estado completo de 2.12.